

[7] Classes and objects

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [7]:

- [\[7.1\] What is a class?](#)
 - [\[7.2\] What is an object?](#)
 - [\[7.3\] When is an interface "good"?](#)
 - [\[7.4\] What is encapsulation?](#)
 - [\[7.5\] How does C++ help with the tradeoff of safety vs. usability?](#)
 - [\[7.6\] How can I prevent other programmers from violating encapsulation by seeing the private parts of my class?](#)
 - [\[7.7\] Is Encapsulation a Security device?](#)
 - [\[7.8\] What's the difference between the keywords `struct` and `class`?](#)
-

[7.1] What is a class?

The fundamental building block of OO software.

A `class` defines a data type, much like a `struct` would be in C. In a computer science sense, a type consists of both a set of states *and* a set of operations which transition between those states. Thus `int` is a type because it has both a set of states *and* it has operations like `i + j` or `i++`, etc. In exactly the same way, a `class` provides a set of (usually `public`) operations, and a set of (usually non-`public`) data bits representing the abstract values that instances of the type can have.

You can imagine that `int` is a `class` that has member functions called `operator++`, etc. (`int` isn't really a `class`, but the basic analogy is this: a `class` is a type, much like `int` is a type.)

Note: a C programmer can think of a `class` as a C `struct` whose members default to `private`. But if that's all you think of a `class`, then you probably need to experience a personal paradigm shift.

[7.2] What is an object?

A region of storage with associated semantics.

After the declaration `int i;` we say that "`i` is an object of type `int`." In OO/C++, "object" usually means "an instance of a class." Thus a `class` defines the behavior of possibly many objects (instances).

[7.3] When is an interface "good"?

When it provides a *simplified view* of a chunk of software, and it is expressed in the *vocabulary of a user* (where a "chunk" is normally a class or a [tight group of classes](#), and a "user" is another developer rather than the ultimate customer).

- The "simplified view" means unnecessary details are intentionally hidden. This reduces the user's defect-rate.
- The "vocabulary of users" means users don't need to learn a new set of words and concepts. This reduces the user's learning curve.

[7.4] What is encapsulation?

Preventing unauthorized access to some piece of information or functionality.

The key money-saving insight is to separate the volatile part of some chunk of software from the stable part. Encapsulation puts a firewall around the chunk, which prevents other chunks from accessing the volatile parts; other chunks can only access the stable parts. This prevents the other chunks from breaking if (when!) the volatile parts are changed. In context of OO software, a "chunk" is normally a class or a [tight group of classes](#).

The "volatile parts" are the implementation details. If the chunk is a single class, the volatile part is normally encapsulated using the [private and/or protected keywords](#). If the chunk is a [tight group of classes](#), encapsulation can be used to deny access to entire classes in that group. [Inheritance](#) can also be used as [a form of encapsulation](#).

The "stable parts" are the interfaces. A good interface provides a [simplified view in the vocabulary of a user](#), and is designed [from the outside-in](#) (here a "user" means another developer, not the end-user who buys the completed application). If the chunk is a single class, the interface is simply the class's `public` member functions and [friend](#) functions. If the chunk is a [tight group of classes](#), the interface can include several of the classes in the chunk.

Designing a clean interface and [separating that interface from its implementation](#) merely *allows* users to use the interface. But encapsulating (putting "in a capsule") the implementation *forces* users to use the interface.

[7.5] How does C++ help with the tradeoff of safety vs. usability?

In C, [encapsulation](#) was accomplished by making things `static` in a compilation unit or module. This prevented another module from accessing the `static` stuff. (By the way, `static` data at file-scope is now deprecated in C++: don't do that.)

Unfortunately this approach doesn't support multiple instances of the data, since there is no direct support for making multiple instances of a module's `static` data. If multiple instances were needed in C, programmers typically used a `struct`. But unfortunately C `structs` don't support

[encapsulation](#). This exacerbates the tradeoff between safety (information hiding) and usability (multiple instances).

In C++, you can have both multiple instances and encapsulation via a class. The `public` part of a class contains the class's interface, which normally consists of the class's `public` member functions and its `friend` functions. The [private and/or protected](#) parts of a class contain the class's implementation, which is typically where the data lives.

The end result is like an "encapsulated `struct`." This reduces the tradeoff between safety (information hiding) and usability (multiple instances).

[7.6] How can I prevent other programmers from violating encapsulation by seeing the `private` parts of my class?

Not worth the effort — encapsulation is for code, not people.

It doesn't violate encapsulation for a *programmer* to see the [private and/or protected](#) parts of your class, so long as they don't write code that somehow depends on what they saw. In other words, encapsulation doesn't prevent *people* from knowing about the inside of a class; it prevents the *code* they write from becoming dependent on the insides of the class. Your company doesn't have to pay a "maintenance cost" to maintain the gray matter between your ears; but it does have to pay a maintenance cost to maintain the code that comes out of your finger tips. What you know as a person doesn't increase maintenance cost, provided the code you write depends on the interface rather than the implementation.

Besides, this is rarely if ever a problem. I don't know any programmers who have intentionally tried to access the `private` parts of a class. "My recommendation in such cases would be to change the programmer, not the code" [[James Kanze](#); used with permission].

[7.7] Is Encapsulation a Security device?

No.

Encapsulation != security.

Encapsulation prevents mistakes, not espionage.

[7.8] What's the difference between the keywords `struct` and `class`?

The members and base classes of a `struct` are `public` by default, while in `class`, they default to `private`. Note: you should make your base classes *explicitly* `public`, `private`, or `protected`, rather than relying on the defaults.

`struct` and `class` are otherwise functionally equivalent.

OK, enough of that squeaky clean techno talk. Emotionally, most developers make a strong distinction between a `class` and a `struct`. A `struct` simply *feels* like an open pile of bits with very little in the way of encapsulation or functionality. A `class` *feels* like a living and responsible member of society with intelligent services, a strong encapsulation barrier, and a well defined interface. Since that's the connotation most people already have, you should probably use the `struct` keyword if you have a class that has very few methods and has `public` data (such things *do* exist in well designed systems!), but otherwise you should probably use the `class` keyword.

  [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[8] References

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [8]:

- [\[8.1\] What is a reference?](#)
 - [\[8.2\] What happens if you assign to a reference?](#)
 - [\[8.3\] What happens if you return a reference?](#)
 - [\[8.4\] What does `object.method1\(\).method2\(\)` mean?](#)
 - [\[8.5\] How can you reseat a reference to make it refer to a different object?](#)
 - [\[8.6\] When should I use references, and when should I use pointers?](#)
 - [\[8.7\] What is a *handle* to an object? Is it a pointer? Is it a reference? Is it a pointer -to-a-pointer? What is it?](#)
-

[8.1] What is a reference?

An alias (an alternate name) for an object.

References are frequently used for pass-by-reference:

```
void swap(int& i, int& j)
{
    int tmp = i;
    i = j;
    j = tmp;
}

int main()
{
    int x, y;
```

```
...
swap(x, y);
...
}
```

Here `i` and `j` are aliases for main's `x` and `y` respectively. In other words, `i` *is* `x` — not a pointer to `x`, nor a copy of `x`, but `x` itself. Anything you do to `i` gets done to `x`, and vice versa.

OK. That's how you should think of references as a programmer. Now, at the risk of confusing you by giving you a different perspective, here's how references are implemented. Underneath it all, a reference `i` to object `x` is typically the machine address of the object `x`. But when the programmer says `i++`, the compiler generates code that increments `x`. In particular, the address bits that the compiler uses to find `x` are not changed. A C programmer will think of this as if you used the C style pass-by-pointer, with the syntactic variant of (1) moving the `&` from the caller into the callee, and (2) eliminating the `*`s. In other words, a C programmer will think of `i` as a macro for `(*p)`, where `p` is a pointer to `x` (e.g., the compiler automatically dereferences the underlying pointer; `i++` is changed to `(*p)++`; `i = 7` is automatically changed to `*p = 7`).

Important note: Even though a reference is often implemented using an address in the underlying assembly language, please do *not* think of a reference as a funny looking pointer to an object. A reference *is* the object. It is not a pointer to the object, nor a copy of the object. It *is* the object.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[8.2] What happens if you assign to a reference?

You change the state of the referent (the referent is the object to which the reference refers).

Remember: the reference *is* the referent, so changing the reference changes the state of the referent. In compiler writer lingo, a reference is an "lvalue" (something that can appear on the left hand side of an assignment operator).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[8.3] What happens if you return a reference?

The function call can appear on the left hand side of an assignment operator.

This ability may seem strange at first. For example, no one thinks the expression `f() = 7` makes sense. Yet, if `a` is an object of class `Array`, most people think that `a[i] = 7` makes sense even though `a[i]` is really just a function call in disguise (it calls `Array::operator[](int)`, which is the subscript operator for class `Array`).

```
class Array {
public:
    int size() const;
    float& operator[] (int index);
};
```

```

...
};

int main()
{
    Array a;
    for (int i = 0; i < a.size(); ++i)
        a[i] = 7;    // This line invokes Array::operator[](int)
    ...
}

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[8.4] What does `object.method1().method2()` mean?

It *chains* these method calls, which is why this is called *method chaining*.

The first thing that gets executed is `object.method1()`. This returns some object, which might be a reference to `object` (i.e., `method1()` might end with `return *this;`), or it might be some other object. Let's call the returned object `objectB`. Then `objectB` becomes the `this` object of `method2()`.

The most common use of method chaining is in the `iostream` library. E.g., `cout << x << y` works because `cout << x` is a function that returns `cout`.

A less common, but still rather slick, use for method chaining is in [the Named Parameter Idiom](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[8.5] How can you reseal a reference to make it refer to a different object?

No way.

You can't separate the reference from the referent.

Unlike a pointer, once a reference is bound to an object, it can *not* be "reseated" to another object. The reference itself isn't an object (it has no identity; taking the address of a reference gives you the address of the referent; remember: the reference *is* its referent).

In that sense, a reference is similar to a [const pointer](#) such as `int* const p` (as opposed to a [pointer to const](#) such as `const int* p`). In spite of the gross similarity, please don't confuse references with pointers; they're not *at all* the same.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[8.6] When should I use references, and when should I use pointers?

Use references when you can, and pointers when you have to.

References are usually preferred over pointers when you don't need ["reseating"](#). This usually means that references are most useful in a class's public interface. References typically appear on the skin of an object, and pointers on the inside.

The exception to the above is where a function's parameter or return value needs a "sentinel" reference. This is usually best done by returning/ taking a pointer, and giving the `NULL` pointer this special significance (references should always alias objects, not a dereferenced `NULL` pointer).

Note: Old line C programmers sometimes don't like references since they provide reference semantics that isn't explicit in the caller's code. After some C++ experience, however, one quickly realizes this is a form of information hiding, which is an asset rather than a liability. E.g., programmers should write code in the language of the problem rather than the language of the machine.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[8.7] What is a *handle* to an object? Is it a pointer? Is it a reference? Is it a pointer-to-a-pointer? What is it?

The term *handle* is used to mean any technique that lets you get to another object — a generalized pseudo-pointer. The term is (intentionally) ambiguous and vague.

Ambiguity is actually an asset in certain cases. For example, during early design you might not be ready to commit to a specific representation for the handles. You might not be sure whether you'll want simple pointers vs. references vs. pointers-to-pointers vs. references-to-pointers vs. integer indices into an array vs. strings (or other key) that can be looked up in a hash-table (or other data structure) vs. database keys vs. some other technique. If you merely know that you'll need some sort of thingy that will uniquely identify and get to an object, you call the thingy a Handle.

So if your ultimate goal is to enable a glob of code to uniquely identify/look-up a specific object of some class `Fred`, you need to pass a `Fred` handle into that glob of code. The handle might be a string that can be used as a key in some well-known lookup table (e.g., a key in a [std::map<std::string, Fred> Or a std::map<std::string, Fred*>](#)), or it might be an integer that would be an index into some well-known array (e.g., [Fred* array = new Fred\[maxNumFreds\]](#)), or it might be a simple `Fred*`, or it might be something else.

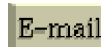
Novices often think in terms of pointers, but in reality there are downside risks to using raw pointers. E.g., what if the `Fred` object needs to move? How do we know when it's safe to delete the `Fred` objects? What if the `Fred` object needs to (temporarily) get serialized on disk? etc., etc. Most of the time we add more layers of indirection to manage situations like these. For example, the handles might be `Fred**`, where the pointed-to `Fred*` pointers are guaranteed to never move but when the `Fred` objects need to move, you just update the pointed-to `Fred*` pointers. Or you make the handle an integer then have the `Fred` objects (or pointers to the `Fred` objects) looked up in a table/array/whatever. Or whatever.

The point is that we use the word Handle when we don't yet know the details of what we're going to do.

Another time we use the word Handle is when we want to be vague about what we've already done (sometimes the term *magic cookie* is used for this as well, as in, "The software passes around a *magic cookie* that is used to uniquely identify and locate the appropriate Fred object"). The reason we (sometimes) want to be vague about what we've already done is to minimize the ripple effect if/when the specific details/representation of the handle change. E.g., if/when someone changes the handle from a string that is used in a lookup table to an integer that is looked up in an array, we don't want to go and update a zillion lines of code.

To further ease maintenance if/when the details/representation of a handle changes (or to generally make the code easier to read/write), we often encapsulate the handle in a class. This class often [overloads operators operator-> and operator*](#) (since the handle acts like a pointer, it might as well look like a pointer).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[9] Inline functions

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
[cline@parashift.com](#))

FAQs in section [9]:

- [\[9.1\] What's the deal with inline functions?](#)
 - [\[9.2\] What's a simple example of procedural integration?](#)
 - [\[9.3\] Do inline functions improve performance?](#) **UPDATED!**
 - [\[9.4\] How can inline functions help with the tradeoff of safety vs. speed?](#)
 - [\[9.5\] Why should I use inline functions instead of plain old #define macros?](#)
 - [\[9.6\] How do you tell the compiler to make a non-member function inline?](#)
 - [\[9.7\] How do you tell the compiler to make a member function inline?](#)
 - [\[9.8\] Is there another way to tell the compiler to make a member function inline?](#)
 - [\[9.9\] With inline member functions that are defined outside the class, is it best to put the inline keyword next to the declaration within the class body, next to the definition outside the class body, or both?](#) **NEW!**
-

[9.1] What's the deal with `inline` functions?

When the compiler inline-expands a function call, the function's code gets inserted into the caller's code stream (conceptually similar to what happens with a [`#define` macro](#)). This can, [depending on a zillion other things](#), improve performance, because the optimizer can [procedurally integrate](#) the called code — optimize the called code into the caller.

There are several ways to designate that a function is `inline`, some of which [involve the `inline` keyword](#), others [do not](#). No matter how you designate a function as `inline`, it is a request that the compiler is allowed to ignore: it might inline-expand some, all, or none of the calls to an `inline` function. (Don't get discouraged if that seems hopelessly vague. The flexibility of the above is actually a huge advantage: it lets the compiler treat large functions differently from small ones, plus it lets the compiler generate code that is easy to debug if you select the right compiler options.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.2] What's a simple example of procedural integration?

Consider the following call to function `g()`:

```
void f()
{
    int x = /*...*/;
    int y = /*...*/;
    int z = /*...*/;
    ...code that uses X, y and Z...
    g(x, y, z);
    ...more code that uses X, y and Z...
}
```

Assuming a typical C++ implementation that has registers and a stack, the registers and parameters get written to the stack just before the call to `g()`, then the parameters get read from the stack inside `g()` and read again to restore the registers while `g()` returns to `f()`. But that's a lot of unnecessary reading and writing, especially in cases when the compiler is able to use registers for variables `x`, `y` and `z`: each variable could get written twice (as a register and also as a parameter) and read twice (when used within `g()` and to restore the registers during the return to `f()`).

```
void g(int x, int y, int z)
{
    ...code that uses X, y and Z...
}
```

If the compiler inline-expands the call to `g()`, all those memory operations could vanish. The registers wouldn't need to get written or read since there wouldn't be a function call, and the parameters wouldn't need to get written or read since the optimizer would know they're already in registers.

Naturally your mileage may vary, and there are a zillion variables that are outside the scope of this particular FAQ, but the above serves as an example of the sorts of things that can happen with procedural integration.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.3] Do `inline` functions improve performance? UPDATED!

[Recently added two bullets on cache misses thanks to a suggestion from [Tom Brown](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Yes and no. Sometimes. Maybe.

There are no simple answers. `inline` functions might make the code faster, they might make it slower. They might make the executable larger, they might make it smaller. They might cause thrashing, they might prevent thrashing. And they might be, and often are, totally irrelevant to speed.

`inline` functions might make it *faster*: As shown [above](#), procedural integration might remove a bunch of unnecessary instructions, which might make things run faster.

`inline` functions might make it *slower*: Too much inlining might cause code bloat, which might cause "thrashing" on demand-paged virtual-memory systems. In other words, if the executable size is too big, the system might spend most of its time going out to disk to fetch the next chunk of code.

`inline` functions might make it *larger*: This is the notion of code bloat, as described above. For example, if a system has 100 `inline` functions each of which expands to 100 bytes of executable code and is called in 100 places, that's an increase of 1MB. Is that 1MB going to cause problems? Who knows, but it is possible that that last 1MB could cause the system to "thrash," and that could slow things down.

`inline` functions might make it *smaller*: The compiler often generates more code to push/pop registers/parameters than it would by `inline`-expanding the function's body. This happens with very small functions, and it also happens with large functions when the optimizer is able to remove a lot of redundant code through procedural integration — that is, when the optimizer is able to make the large function small.

`inline` functions might *cause thrashing*: Inlining might increase the size of the binary executable, and that might cause thrashing.

`inline` functions might *prevent thrashing*: The working set size (number of pages that need to be in memory at once) might go down even if the executable size goes up. When `f()` calls `g()`, the code is often on two distinct pages; when the compiler procedurally integrates the code of `g()` into `f()`, the code is often on the same page.

`inline` functions might *increase the number of cache misses*: Inlining might cause an inner loop to span across multiple lines of the memory cache, and that might cause thrashing of the memory-cache.

inline functions might decrease the number of cache misses : Inlining usually improves locality of reference within the binary code, which might decrease the number of cache lines needed to store the code of an inner loop. This ultimately could cause a CPU -bound application to run faster.

inline functions might be irrelevant to speed: Most systems are not CPU-bound. Most systems are I/O-bound, database-bound or network-bound, meaning the bottleneck in the system's overall performance is the file system, the database or the network. Unless your "CPU meter" is pegged at 100%, inline functions probably won't make your system faster. (Even in CPU-bound systems, inline will help only when used within the bottleneck itself, and the bottleneck is typically in only a small percentage of the code.)

There are no simple answers: You have to play with it to see what is best. Do *not* settle for simplistic answers like, "Never use inline functions" or "Always use inline functions" or "Use inline functions if and only if the function is less than *N* lines of code." These one-size-fits-all rules may be easy to write down, but they will produce sub-optimal results.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.4] How can inline functions help with the tradeoff of safety vs. speed?

In straight C, you can achieve "encapsulated structs" by putting a void* in a struct, in which case the void* points to the real data that is unknown to users of the struct. Therefore users of the struct don't know how to interpret the stuff pointed to by the void*, but the access functions cast the void* to the appropriate hidden type. This gives a form of encapsulation.

Unfortunately it forfeits type safety, and also imposes a function call to access even trivial fields of the struct (if you allowed direct access to the struct's fields, anyone and everyone would be able to get direct access since they would of necessity know how to interpret the stuff pointed to by the void*; this would make it difficult to change the underlying data structure).

Function call overhead is small, but can add up. C++ classes allow function calls to be expanded inline. This lets you have the safety of encapsulation along with the speed of direct access. Furthermore the parameter types of these inline functions are checked by the compiler, an improvement over C's #define macros.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.5] Why should I use inline functions instead of plain old #define macros?

Because #define macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#). Sometimes you should [use them anyway](#), but they're still evil.

Unlike #define macros, inline functions avoid infamous macro errors since inline functions always evaluate every argument exactly once. In other words, invoking an inline function is semantically just like invoking a regular function, only faster:

```

// A macro that returns the absolute value of i
#define unsafe(i) \
    ( (i) >= 0 ? (i) : -(i) )

// An inline function that returns the absolute value of i
inline
int safe(int i)
{
    return i >= 0 ? i : -i;
}

int f();

void userCode(int x)
{
    int ans;

    ans = unsafe(x++);    // Error! X is incremented twice
    ans = unsafe(f());    // Danger! f() is called twice

    ans = safe(x++);      // Correct! X is incremented once
    ans = safe(f());      // Correct! f() is called once
}

```

Also unlike macros, argument types are checked, and necessary conversions are performed correctly.

Macros are bad for your health; don't use them [unless you have to](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.6] How do you tell the compiler to make a non-member function `inline`?

When you declare an `inline` function, it looks just like a normal function:

```
void f(int i, char c);
```

But when you define an `inline` function, you prepend the function's definition with the keyword `inline`, and you put the definition into a header file:

```

inline
void f(int i, char c)
{
    ...
}

```

Note: It's imperative that the function's definition (the part between the `{...}`) be placed in a header file, unless the function is used only in a single `.cpp` file. In particular, if you put the `inline` function's definition into a `.cpp` file and you call it from some other `.cpp` file, you'll get an "unresolved external" error from the linker.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.7] How do you tell the compiler to make a member function `inline`?

When you declare an `inline` member function, it looks just like a normal member function:

```
class Fred {
public:
    void f(int i, char c);
};
```

But when you define an `inline` member function, you prepend the member function's definition with the keyword `inline`, and you put the definition into a header file:

```
inline
void Fred::f(int i, char c)
{
    ...
}
```

It's usually imperative that the function's definition (the part between the `{...}`) be placed in a header file. If you put the `inline` function's definition into a `.cpp` file, and if it is called from some other `.cpp` file, you'll get an "unresolved external" error from the linker.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.8] Is there another way to tell the compiler to make a member function `inline`?

Yep: define the member function in the class body itself:

```
class Fred {
public:
    void f(int i, char c)
    {
        ...
    }
};
```

Although this is easier on the person who writes the class, it's harder on all the readers since it mixes "what" a class does with "how" it does them. Because of this mixture, we normally prefer to define member functions [outside the class body with the `inline` keyword](#). The insight that makes sense of this: in a reuse-oriented world, there will usually be many people who use your class, but there is only one person who builds it (yourself); therefore you should do things that favor the many rather than the few. This approach is further exploited in [the next FAQ](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[9.9] With inline member functions that are defined outside the class, is it best to put the `inline` keyword next to the declaration within the class body, next to the definition outside the class body, or both? NEW!

[Recently created (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Best practice: only in the definition outside the class body.

```
class Foo {
public:
    void method();    ← best practice: don't put the inline keyword here
    ...
};

inline void Foo::method()    ← best practice: put the inline keyword here
{ ... }
```

Here's the basic idea:

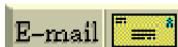
- The `public:` part of the class body is where you describe the *observable semantics* of a class, its public member functions, its friend functions, and anything else exported by the class. Try not to provide any inklings of anything that can't be observed from the caller's code.
- The other parts of the class, including non-`public:` part of the class body, the definitions of your member and friend functions, etc. are pure implementation. Try not to describe any observable semantics that were not already described in the class's `public:` part.

From a practical standpoint, this separation makes life easier and safer for your users. Say Chuck wants to simply "use" your class. Because you read this FAQ and used the above separation, Chuck can read your class's `public:` part and see everything he needs to see and nothing he doesn't need to see. His life is easier because he needs to look in only one spot, and his life is safer because his pure mind isn't polluted by implementation minutiae.

Back to inline-ness: the decision of whether a function is or is not `inline` is an implementation detail that does not change the observable semantics (the "meaning") of a call. Therefore the `inline` keyword should go next to the function's definition, not within the class's `public:` part.

NOTE: most people use the terms "declaration" and "definition" to differentiate the above two places. For example, they might say, "Should I put the `inline` keyword next to the declaration or the definition?" Unfortunately that usage is sloppy and somebody out there will eventually gig you for it. The people who gig you are probably insecure, pathetic wannabes who know they're not good enough to actually accomplish something with their lives, nonetheless you might as well learn the correct terminology to avoid getting gigged. Here it is: every definition is also a declaration. This means using the two as if they are mutually exclusive would be like asking which is heavier, steel or metal? Almost everybody will know what you mean if you use "definition" as if it is the opposite of "declaration," and only the worst of the techie weenies will gig you for it, but at least you now know how to use the terms correctly.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | ©
| [Download your own copy](#)]

Revised Feb 15, 2005

[10] Constructors

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [10]:

- [\[10.1\] What's the deal with constructors?](#)
 - [\[10.2\] Is there any difference between `List x;` and `List x\(\);`?](#)
 - [\[10.3\] Can one constructor of a class call another constructor of the same class to initialize the `this` object?](#)
 - [\[10.4\] Is the default constructor for `Fred` always `Fred::Fred\(\)`?](#)
 - [\[10.5\] Which constructor gets called when I create an array of `Fred` objects?](#)
 - [\[10.6\] Should my constructors use "initialization lists" or "assignment"?](#) **UPDATED!**
 - [\[10.7\] Should you use the `this` pointer in the constructor?](#)
 - [\[10.8\] What is the "Named Constructor Idiom"?](#) **UPDATED!**
 - [\[10.9\] Does return-by-value mean extra copies and extra overhead?](#) **NEW!**
 - [\[10.10\] Why can't I initialize my `static` member data in my constructor's initialization list?](#)
 - [\[10.11\] Why are classes with `static` data members getting linker errors?](#)
 - [\[10.12\] What's the "`static` initialization order fiasco"?](#)
 - [\[10.13\] How do I prevent the "`static` initialization order fiasco"?](#)
 - [\[10.14\] Why doesn't the construct-on-first-use idiom use a `static` object instead of a `static` pointer?](#)
 - [\[10.15\] How do I prevent the "`static` initialization order fiasco" for my `static` data members?](#)
 - [\[10.16\] Do I need to worry about the "`static` initialization order fiasco" for variables of built-in/intrinsic types?](#)
 - [\[10.17\] How can I handle a constructor that fails?](#)
 - [\[10.18\] What is the "Named Parameter Idiom"?](#) **UPDATED!**
 - [\[10.19\] Why am I getting an error after declaring a `Foo` object via `Foo x\(Bar\(\)\)`?](#) **NEW!**
-

[10.1] What's the deal with constructors?

Constructors build objects from dust.



Constructors are like "init functions". They turn a pile of arbitrary bits into a living object. Minimally they initialize internally used fields. They may also allocate resources (memory, files, semaphores, sockets, etc).

"ctor" is a typical abbreviation for constructor.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.2] Is there any difference between `List x;` and `List x();`?

A *big* difference!

Suppose that `List` is the name of some class. Then function `f()` declares a local `List` object called `x`:

```
void f()
{
    List x;      // Local object named x (of class List)
    ...
}
```

But function `g()` declares a function called `x()` that returns a `List`:

```
void g()
{
    List x();    // Function named x (that returns a List)
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.3] Can one constructor of a class call another constructor of the same class to initialize the `this` object?

Nope.

Let's work an example. Suppose you want your constructor `Foo::Foo(char)` to call another constructor of the same class, say `Foo::Foo(char, int)`, in order that `Foo::Foo(char, int)` would help initialize the `this` object. Unfortunately there's no way to do this in C++.

Some people do it anyway. Unfortunately it doesn't do what they want. For example, the line `Foo(x, 0);` does *not* call `Foo::Foo(char, int)` on the `this` object. Instead it calls `Foo::Foo(char, int)` to initialize a temporary, local object (*not this*), then it immediately destructs that temporary when control flows over the `;`.

```
class Foo {
public:
    Foo(char x);
```

```

    Foo(char x, int y);
    ...
};

Foo::Foo(char x)
{
    ...
    Foo(x, 0); // this line does NOT help initialize the this object!!
    ...
}

```

You can sometimes combine two constructors via a default parameter:

```

class Foo {
public:
    Foo(char x, int y=0); // this line combines the two constructors
    ...
};

```

If that doesn't work, e.g., if there isn't an appropriate default parameter that combines the two constructors, sometimes you can share their common code in a private `init()` member function:

```

class Foo {
public:
    Foo(char x);
    Foo(char x, int y);
    ...
private:
    void init(char x, int y);
};

Foo::Foo(char x)
{
    init(x, int(x) + 7);
    ...
}

Foo::Foo(char x, int y)
{
    init(x, y);
    ...
}

void Foo::init(char x, int y)
{
    ...
}

```

BTW do *NOT* try to achieve this via [placement new](#). Some people think they can say `new(this) Foo(x, int(x)+7)` within the body of `Foo::Foo(char)`. However that is bad, bad, bad. Please don't write me and tell me that it *seems* to work on your particular version of your particular compiler; it's bad. Constructors do a bunch of little magical things behind the scenes, but that bad technique steps on those partially constructed bits. Just say no.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.4] Is the default constructor for `Fred` always `Fred::Fred()`?

No. A "default constructor" is a constructor that *can be called* with no arguments. One example of this is a constructor that takes no parameters:

```
class Fred {
public:
    Fred();    // Default constructor: can be called with no args
    ...
};
```

Another example of a "default constructor" is one that can take arguments, provided they are given default values:

```
class Fred {
public:
    Fred(int i=3, int j=5);    // Default constructor: can be called with no args
    ...
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.5] Which constructor gets called when I create an array of `Fred` objects?

`Fred`'s [default constructor](#) (except as discussed below).

There is no way to tell the compiler to call a different constructor (except as discussed below). If your `class Fred` doesn't have a [default constructor](#), attempting to create an array of `Fred` objects is trapped as an error at compile time.

```
class Fred {
public:
    Fred(int i, int j);
    ...assume there is no default constructor in class Fred...
};

int main()
{
    Fred a[10];                // ERROR: Fred doesn't have a default constructor
    Fred* p = new Fred[10];    // ERROR: Fred doesn't have a default constructor
    ...
}
```

However if you are constructing an object of [the standard `std::vector<Fred>`](#) rather than an array of `Fred` (which you probably should be doing anyway since [arrays are evil](#)), you don't have to have a default constructor in `class Fred`, since you can give the `std::vector` a `Fred` object to be used to initialize the elements:

```
#include <vector>

int main()
{
    std::vector<Fred> a(10, Fred(5,7));
```

```
// The 10 Fred objects in std::vector a will be initialized with Fred(5, 7).
...
}
```

Even though you *ought* to use a `std::vector` rather than an array, there are times when an array might be the right thing to do, and for those, there is the "explicit initialization of arrays" syntax. Here's how it looks:

```
class Fred {
public:
    Fred(int i, int j);
    ...assume there is no default constructor in class Fred...
};

int main()
{
    Fred a[10] = {
        Fred(5,7), Fred(5,7), Fred(5,7), Fred(5,7), Fred(5,7),
        Fred(5,7), Fred(5,7), Fred(5,7), Fred(5,7), Fred(5,7)
    };

    // The 10 Fred objects in array a will be initialized with Fred(5, 7).
    ...
}
```

Of course you don't have to do `Fred(5, 7)` for every entry — you can put in any numbers you want, even parameters or other variables. The point is that this syntax is (a) doable but (b) not as nice as the `std::vector` syntax. Remember this: [arrays are evil](#) — unless there is a compelling reason to use an array, use a `std::vector` instead.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.6] Should my constructors use "initialization lists" or "assignment"?

UPDATED!

[Recently added an "or when" and a cross reference to the last paragraph, plus fixed some grammar, thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Initialization lists. In fact, constructors should initialize as a rule *all* member objects in the initialization list. One exception is discussed further down.

Consider the following constructor that initializes member object `x_` using an initialization list: `Fred::Fred() : x_(whatever) { }`. The most common benefit of doing this is improved performance. For example, if the expression *whatever* is the same type as as member variable `x_`, the result of the *whatever* expression is constructed directly inside `x_` — the compiler does not make a separate copy of the object. Even if the types are not the same, the compiler is usually able to do a better job with initialization lists than with assignments.

The other (inefficient) way to build constructors is via assignment, such as: `Fred::Fred() { x_ = whatever; }`. In this case the expression *whatever* causes a separate, temporary object to be created, and this temporary object is passed into the `x_` object's assignment operator. Then that temporary object is destructed at the `;`. That's inefficient.

As if that wasn't bad enough, there's another source of inefficiency when using assignment in a constructor: the member object will get fully constructed by its default constructor, and this might, for example, allocate some default amount of memory or open some default file. All this work could be for naught if the *whatever* expression and/or assignment operator causes the object to close that file and/or release that memory (e.g., if the default constructor didn't allocate a large enough pool of memory or if it opened the wrong file).

Conclusion: All other things being equal, your code will run faster if you use initialization lists rather than assignment.

Note: There is no performance difference if the type of `x_` is some built-in/intrinsic type, such as `int` or `char*` or `float`. But even in these cases, my personal preference is to set those data members in the initialization list rather than via assignment for consistency. Another symmetry argument in favor of using initialization lists even for built-in/intrinsic types: non-static const data members *can't* be assigned a value in the constructor, so for symmetry it makes sense to initialize everything in the initialization list.

Now for the exceptions. Every rule has exceptions (hmmm; does "every rule has exceptions" have exceptions? reminds me of Gödel's Incompleteness Theorems), and there are a couple of exceptions to the "use initialization lists" rule. Bottom line is to use common sense: if it's cheaper, better, faster, etc. to not use them, then by all means, don't use them. This might happen when your class has two constructors that need to initialize the `this` object's data members in different orders. Or it might happen when two data members are self-referential. Or when a data-member needs a reference to the `this` object, and you want to avoid a compiler warning about using the `this` keyword prior to the `{` that begins the constructor's body (when your particular compiler happens to issue that particular warning). Or when you need to do [an if/throw test on a variable](#) (parameter, global, etc.) prior to using that variable to initialize one of your `this` members. This list is not exhaustive; please don't write me asking me to add another "Or when...". The point is simply this: use common sense.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.7] Should you use the `this` pointer in the constructor?

Some people feel you should not use the `this` pointer in a constructor because the object is not fully formed yet. However you can use `this` in the constructor (in the `{body}` and even in the [initialization list](#)) if you are careful.

Here is something that *always* works: the `{body}` of a constructor (or a function called from the constructor) can reliably access the data members declared in a base class and/or the data members declared in the constructor's own class. This is because all those data members are guaranteed to have been fully constructed by the time the constructor's `{body}` starts executing.

Here is something that *never* works: the `{body}` of a constructor (or a function called from the constructor) *cannot* get down to a derived class by calling a `virtual` member function that is overridden in the derived class. If your goal was to get to the overridden function in the derived class, [you won't get what you want](#). Note that you won't get to the override in the derived class independent of *how* you call the `virtual` member function: explicitly using the `this` pointer (e.g.,

`this->method()`), implicitly using the `this` pointer (e.g., `method()`), or even calling some other function that calls the `virtual` member function on your `this` object. The bottom line is this: even if the caller is constructing an object of a derived class, during the constructor of the base class, [your object is not yet of that derived class](#). You have been warned.

Here is something that *sometimes* works: if you pass any of the data members in `this` object to another data member's [initializer](#), you must make sure that the other data member has already been initialized. The good news is that you can determine whether the other data member has (or has not) been initialized using some straightforward language rules that are independent of the particular compiler you're using. The bad news is that you have to know those language rules (e.g., base class sub-objects are initialized first (look up the order if you have multiple and/or virtual inheritance!), then data members defined in the class are initialized in the order in which they appear in the class declaration). If you don't know these rules, then don't pass any data member from the `this` object (regardless of whether or not you explicitly use the `this` keyword) to any other data member's [initializer](#)! And if you do know the rules, please be careful.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.8] What is the "Named Constructor Idiom"? UPDATED!

[Recently added the last paragraph to discuss the return-by-value optimization thanks to a question from [Can Isin](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes](#).]

A technique that provides more intuitive and/or safer construction operations for users of your class.

The problem is that constructors always have the same name as the class. Therefore the only way to differentiate between the various constructors of a class is by the parameter list. But if there are lots of constructors, the differences between them become somewhat subtle and error prone.

With the Named Constructor Idiom, you declare all the class's constructors in the `private` or `protected` sections, and you provide `public static` methods that return an object. These `static` methods are the so-called "Named Constructors." In general there is one such `static` method for each different way to construct an object.

For example, suppose we are building a `Point` class that represents a position on the X-Y plane. Turns out there are two common ways to specify a 2-space coordinate: rectangular coordinates (X+Y), polar coordinates (Radius+Angle). (Don't worry if you can't remember these; the point isn't the particulars of coordinate systems; the point is that there are several ways to create a `Point` object.) Unfortunately the parameters for these two coordinate systems are the same: two floats. This would create an ambiguity error in the overloaded constructors:

```
class Point {
public:
    Point(float x, float y);        // Rectangular coordinates
    Point(float r, float a);        // Polar coordinates (radius and angle)
    // ERROR: Overload is Ambiguous: Point::Point(float, float)
};
```

```
int main()
{
    Point p = Point(5.7, 1.2);    // Ambiguous: Which coordinate system?
    ...
}
```

One way to solve this ambiguity is to use the Named Constructor Idiom:

```
#include <cmath>                // To get sin() and cos()

class Point {
public:
    static Point rectangular(float x, float y);    // Rectangular coord's
    static Point polar(float radius, float angle); // Polar coordinates
    // These static methods are the so-called "named constructors"
    ...
private:
    Point(float x, float y);    // Rectangular coordinates
    float x_, y_;
};

inline Point::Point(float x, float y)
: x_(x), y_(y) { }

inline Point Point::rectangular(float x, float y)
{ return Point(x, y); }

inline Point Point::polar(float radius, float angle)
{ return Point(radius*cos(angle), radius*sin(angle)); }
```

Now the users of `Point` have a clear and unambiguous syntax for creating `Points` in either coordinate system:

```
int main()
{
    Point p1 = Point::rectangular(5.7, 1.2);    // Obviously rectangular
    Point p2 = Point::polar(5.7, 1.2);          // Obviously polar
    ...
}
```

Make sure your constructors are in the protected section if you expect `Point` to have derived classes.

The Named Constructor Idiom can also be used to [make sure your objects are always created via new](#).

Note that the Named Constructor Idiom, at least as implemented above, is just as fast as directly calling a constructor — [modern compilers will not make any extra copies of your object](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.9] Does return-by-value mean extra copies and extra overhead? NEW!

[Recently created (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Not necessarily.

All(?) commercial-grade compilers optimize away the extra copy, at least in cases as illustrated in [the previous FAQ](#).

To keep the example clean, let's strip things down to the bare essentials. Suppose `yourCode()` calls `rbv()` ("rbv" stands for "return by value") which returns a `Foo` object by value:

```
class Foo { ... };

Foo rbv();

void yourCode()
{
    Foo x = rbv(); ← the return-value of rbv() goes into x
    ...
}
```

Now the question is, How many `Foo` objects will there be? Will `rbv()` create a temporary `Foo` object that gets copy-constructed into `x`? How many temporaries? Said another way, does return-by-value necessarily degrade performance?

The point of this FAQ is that the answer is No, commercial-grade C++ compilers implement return-by-value in a way that lets them eliminate the overhead, at least in simple cases like those shown in the previous FAQ. In particular, all(?) commercial-grade C++ compilers will optimize this case:

```
Foo rbv()
{
    ...
    return Foo(42, 73); ← suppose Foo has a ctor Foo::Foo(int a, int b)
}
```

Certainly the compiler is *allowed* to create a temporary, local `Foo` object, then copy-construct that temporary into variable `x` within `yourCode()`, then destruct the temporary. But all(?) commercial-grade C++ compilers won't do that: the `return` statement will directly construct `x` itself. Not a copy of `x`, not a pointer to `x`, not a reference to `x`, but `x` itself.

You can stop here if you don't want to genuinely understand the previous paragraph, but if you want to know the secret sauce (so you can, for example, reliably predict when the compiler can and cannot provide that optimization for you), the key is to know that compilers usually implement return-by-value using pass-by-pointer. When `yourCode()` calls `rbv()`, the compiler secretly passes a pointer to the location where `rbv()` is supposed to construct the "returned" object. It might look something like this (it's shown as a `void*` rather than a `Foo*` since the `Foo` object has not yet been constructed):

```
// Pseudo-code
void rbv(void* put_result_here) ← Original C++ code: Foo rbv()
{
    ...
}

// Pseudo-code
```

```

void yourCode()
{
    struct Foo x;
    rbv(&x); ← Original C++ code: Foo x = rbv()
    ...
}

```

So the first ingredient in the secret sauce is that the compiler (usually) transforms return -by-value into pass-by-pointer. This means that commercial-grade compilers don't bother creating a temporary: they directly construct the returned object in the location pointed to by `put_result_here`.

The second ingredient in the secret sauce is that compilers typically implement constructors using a similar technique. This is compiler-dependent and somewhat idealized (I'm intentionally ignoring how to handle `new` and overloading), but compilers typically implement `Foo::Foo(int a, int b)` using something like this:

```

// Pseudo-code
void Foo_ctor(Foo* this, int a, int b) ← Original C++ code: Foo::Foo(int a, int b)
{
    ...
}

```

Putting these together, the compiler might implement the `return` statement in `rbv()` by simply passing `put_result_here` as the constructor's `this` pointer:

```

// Pseudo-code
void rbv(void* put_result_here) ← Original C++ code: Foo rbv()
{
    ...
    Foo_ctor((Foo*)put_result_here, 42, 73); ← Original C++ code: return Foo(42, 73);
    return;
}

```

So `yourCode()` passes `&x` to `rbv()`, and `rbv()` in turn passes `&x` to the constructor (as the `this` pointer). That means constructor *directly* constructs `x`.

In the early 90s I did a seminar for IBM's compiler group in Toronto, and one of their engineers told me that they found this return-by-value optimization to be so fast that you get it even if you don't compile with optimization turned on. Because the return-by-value optimization causes the compiler to generate less code, it actually improves compile-times in addition to making your generated code smaller and faster. The point is that the return-by-value optimization is almost universally implemented, at least in code cases like those shown above.

Some compilers also provide the return-by-value optimization your function returns a local variable by value, provided *all* the function's `return` statements return the same local variable. This requires a little more work on the part of the compiler writers, so it isn't universally implemented; for example, GNU g++ 3.3.3 does it but Microsoft Visual C++.NET 2003 does not :

```

// Actual C++ code for rbv()
Foo rbv()
{
    ...
}

```

```

    Foo ans = Foo(42, 73);
    ...
    do_something_with(ans);
    ...
    return ans;
}

```

The compiler might construct `ans` in a local object, then in the `return` statement copy-construct `ans` into the location pointed to by `put_result_here` and destruct `ans`. But if *all* `return` statements return the same local object, in this case `ans`, the compiler is also allowed to construct `ans` in the location pointed to by `put_result_here`:

```

// Pseudo-code
void rbv(void* put_result_here) ← Original C++ code: Foo rbv()
{
    ...
    Foo_ctor((Foo*)put_result_here, 42, 73); ← Original C++ code: Foo ans = Foo(42, 73)
;
    ...
    do_something_with(*(Foo*)put_result_here); ← Original C++ code: do_something_with(
ans);
    ...
    return; ← Original C++ code: return ans;
}

```

Final thought: this discussion was limited to whether there will be any extra copies of the returned object in a return-by-value call. Don't confuse that with other things that could happen in `yourCode()`. For example, if you changed `yourCode()` from `Foo x = rbv();` to `Foo x; x = rbv();` (note the `;` after the declaration), the compiler is required to use `Foo`'s assignment operator, and unless the compiler can prove that `Foo`'s default constructor followed by assignment operator is exactly the same as its copy constructor, the compiler is required by the language to put the returned object into an unnamed temporary within `yourCode()`, use the assignment operator to copy the temporary into `x`, then destruct the temporary. The return-by-value optimization still plays its part since there will be only one temporary, but by changing `Foo x = rbv();` to `Foo x; x = rbv();`, you have prevented the compiler from eliminating that last temporary.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.10] Why can't I initialize my `static` member data in my constructor's initialization list?

Because you must *explicitly* define your class's `static` data members.

Fred.h:

```

class Fred {
public:
    Fred();
    ...
private:

```

```
int i_;
static int j_;
};
```

Fred.cpp (or Fred.c or whatever):

```
Fred::Fred()
: i_(10) // OK: you can (and should) initialize member data this way
, j_(42) // Error: you cannot initialize static member data like this
{
    ...
}

// You must define static data members this way:
int Fred::j_ = 42;
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.11] Why are classes with `static` data members getting linker errors?

Because [static data members must be explicitly defined in exactly one compilation unit](#). If you didn't do this, you'll probably get an "undefined external" linker error. For example:

```
// Fred.h

class Fred {
public:
    ...
private:
    static int j_; // Declares static data member Fred::j_
    ...
};
```

The linker will holler at you ("Fred::j_ is not defined") unless you define (as opposed to merely declare) Fred::j_ in (exactly) one of your source files:

```
// Fred.cpp

#include "Fred.h"

int Fred::j_ = some_expression_evaluating_to_an_int;

// Alternatively, if you wish to use the implicit 0 value for static ints:
// int Fred::j_;
```

The usual place to define `static` data members of `class Fred` is file Fred.cpp (or Fred.c or whatever source file extension you use).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.12] What's the "static initialization order fiasco"?

A subtle way to crash your program.

The *static initialization order fiasco* is a very subtle and commonly misunderstood aspect of C++. Unfortunately it's very hard to detect — the errors occur before `main()` begins.

In short, suppose you have two `static` objects `x` and `y` which exist in separate source files, say `x.cpp` and `y.cpp`. Suppose further that the initialization for the `y` object (typically the `y` object's constructor) calls some method on the `x` object.

That's it. It's that simple.

The tragedy is that you have a 50%-50% chance of dying. If the compilation unit for `x.cpp` happens to get initialized first, all is well. But if the compilation unit for `y.cpp` get initialized first, then `y`'s initialization will get run before `x`'s initialization, and you're toast. E.g., `y`'s constructor could call a method on the `x` object, yet the `x` object hasn't yet been constructed.

I hear they're hiring down at McDonalds. Enjoy your new job flipping burgers.

If you think it's "exciting" to play Russian Roulette with live rounds in half the chambers, you can stop reading here. On the other hand if you like to improve your chances of survival by preventing disasters in a systematic way, you probably want to read [the next FAQ](#).

Note: The static initialization order fiasco can also, [in some cases](#), apply to built-in/intrinsic types.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.13] How do I prevent the "static initialization order fiasco"?

Use the "construct on first use" idiom, which simply means to wrap your `static` object inside a function.

For example, suppose you have two classes, `Fred` and `Barney`. There is a global `Fred` object called `x`, and a global `Barney` object called `y`. `Barney`'s constructor invokes the `goBowling()` method on the `x` object. The file `x.cpp` defines the `x` object:

```
// File x.cpp
#include "Fred.hpp"
Fred x;
```

The file `y.cpp` defines the `y` object:

```
// File y.cpp
#include "Barney.hpp"
Barney y;
```

For completeness the `Barney` constructor might look something like this:

```
// File Barney.cpp
#include "Barney.hpp"
```

```
Barney::Barney()
{
    ...
    x.goBowling();
    ...
}
```

As described [above](#), the disaster occurs if `y` is constructed before `x`, which happens 50% of the time since they're in different source files.

There are many solutions to this problem, but a very simple and completely portable solution is to replace the global `Fred` object, `x`, with a global function, `x()`, that returns the `Fred` object by reference.

```
// File x.cpp

#include "Fred.hpp"

Fred& x()
{
    static Fred* ans = new Fred();
    return *ans;
}
```

Since static local objects are constructed the first time control flows over their declaration (only), the above `new Fred()` statement will only happen once: the first time `x()` is called. Every subsequent call will return the same `Fred` object (the one pointed to by `ans`). Then all you do is change your usages of `x` to `x()`:

```
// File Barney.cpp
#include "Barney.hpp"

Barney::Barney()
{
    ...
    x().goBowling();
    ...
}
```

This is called the *Construct On First Use Idiom* because it does just that: the global `Fred` object is constructed on its first use.

The downside of this approach is that the `Fred` object is never destructed. There is [another technique](#) that answers this concern, but it needs to be used with care since it creates the possibility of another (equally nasty) problem.

Note: The static initialization order fiasco can also, [in some cases](#), apply to built-in/intrinsic types.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.14] Why doesn't the construct-on-first-use idiom use a static object instead of a static pointer?

Short answer: it's possible to use a static object [rather than a static pointer](#), but doing so opens up another (equally subtle, equally nasty) problem.

Long answer: sometimes people worry about the fact that [the previous solution](#) "leaks." In many cases, this is not a problem, but it is a problem in some cases. Note: even though the object pointed to by `ans` in the previous FAQ is never deleted, the memory doesn't actually "leak" when the program exits since the operating system automatically reclaims all the memory in a program's heap when that program exits. In other words, the only time you'd need to worry about this is when the destructor for the `Fred` object performs some important action (such as writing something to a file) that must occur sometime while the program is exiting.

In those cases where the construct-on-first-use object (the `Fred`, in this case) needs to eventually get destructed, you might consider changing function `x()` as follows:

```
// File x.cpp

#include "Fred.hpp"

Fred& x()
{
    static Fred ans; // was static Fred* ans = new Fred();
    return ans;      // was return *ans;
}
```

However there is (or rather, may be) a rather subtle problem with this change. To understand this potential problem, let's remember why we're doing all this in the first place: we need to make 100% sure our static object (a) gets constructed prior to its first use and (b) doesn't get destructed until after its last use. Obviously it would be a disaster if any static object got used either before construction or after destruction. The message here is that you need to worry about two situations (static initialization and static deinitialization), not just one.

By changing the declaration from `static Fred* ans = new Fred();` to `static Fred ans;`, we still correctly handle the initialization situation but we no longer handle the deinitialization situation. For example, if there are 3 static objects, say `a`, `b` and `c`, that use `ans` during their destructors, the only way to avoid a static deinitialization disaster is if `ans` is destructed after all three.

The point is simple: if there are any other static objects whose destructors might use `ans` after `ans` is destructed, bang, you're dead. If the *constructors* of `a`, `b` and `c` use `ans`, you *should* normally be okay since the runtime system will, during static deinitialization, destruct `ans` after the last of those three objects is destructed. However if `a` and/or `b` and/or `c` fail to use `ans` in their constructors and/or if any code anywhere gets the address of `ans` and hands it to some other static object, all bets are off and you have to be very, very careful.

There is a third approach that handles both the static initialization and static deinitialization situations, but it has other non-trivial costs. I'm too lazy (and busy!) to write any more FAQs today so if you're interested in that third approach, you'll have to buy a book that describes that

third approach in detail. The [C++ FAQs book](#) is one of those books, and it also gives the cost/benefit analysis to decide if/when that third approach should be used.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.15] How do I prevent the "static initialization order fiasco" for my static data members?

Just use [the same technique just described](#), but this time use a static member function rather than a global function.

Suppose you have a class `x` that has a static `Fred` object:

```
// File X.hpp

class X {
public:
    ...

private:
    static Fred x_;
};
```

Naturally this static member is initialized separately:

```
// File X.cpp

#include "X.hpp"

Fred X::x_;
```

Naturally also the `Fred` object will be used in one or more of `x`'s methods:

```
void X::someMethod()
{
    x_.goBowling();
}
```

But now the "disaster scenario" is if someone somewhere somehow calls this method before the `Fred` object gets constructed. For example, if someone else creates a static `x` object and invokes its `someMethod()` method during static initialization, then you're at the mercy of the compiler as to whether the compiler will construct `X::x_` before or after the `someMethod()` is called. (Note that the ANSI/ISO C++ committee is working on this problem, but compilers aren't yet generally available that handle these changes; watch this space for an update in the future.)

In any event, it's always portable and safe to change the `X::x_` static data member into a static member function:

```
// File X.hpp

class X {
public:
```



```
...  
private:  
    static Fred& x();  
};
```

Naturally this static member is initialized separately:

```
// File X.cpp  
  
#include "X.hpp"  
  
Fred& X::x()  
{  
    static Fred* ans = new Fred();  
    return *ans;  
}
```

Then you simply change any usages of `x_` to `x()`:

```
void X::someMethod()  
{  
    x().goBowling();  
}
```

If you're super performance sensitive and you're concerned about the overhead of an extra function call on each invocation of `X::someMethod()` you can set up a `static Fred&` instead. As you recall, static local are only initialized once (the first time control flows over their declaration), so this will call `x::x()` only once: the first time `X::someMethod()` is called:

```
void X::someMethod()  
{  
    static Fred& x = X::x();  
    x.goBowling();  
}
```

Note: The static initialization order fiasco can also, [in some cases](#), apply to built-in/intrinsic types.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.16] Do I need to worry about the "static initialization order fiasco" for variables of built-in/intrinsic types?

Yes.

If you initialize your built-in/intrinsic type using a function call, the static initialization order fiasco is able to kill you just as bad as with user-defined/class types. For example, the following code shows the failure:

```
#include <iostream>  
  
int f(); // forward declaration
```

```

int g(); // forward declaration

int x = f();
int y = g();

int f()
{
    std::cout << "using 'y' (which is " << y << ")\n";
    return 3*y + 7;
}

int g()
{
    std::cout << "initializing 'y'\n";
    return 5;
}

```

The output of this little program will show that it uses `y` before initializing it. The solution, as before, is the *Construct On First Use Idiom*:

```

#include <iostream>

int f(); // forward declaration
int g(); // forward declaration

int& x()
{
    static int ans = f();
    return ans;
}

int& y()
{
    static int ans = g();
    return ans;
}

int f()
{
    std::cout << "using 'y' (which is " << y() << ")\n";
    return 3*y() + 7;
}

int g()
{
    std::cout << "initializing 'y'\n";
    return 5;
}

```

Of course you might be able to simplify this by moving the initialization code for `x` and `y` into their respective functions:

```

#include <iostream>

int& y(); // forward declaration

int& x()
{
    static int ans;
}

```

```

static bool firstTime = true;
if (firstTime) {
    firstTime = false;
    std::cout << "using 'y' (which is " << y() << ")\n";
    ans = 3*y() + 7;
}

return ans;
}

int& y()
{
    static int ans;

    static bool firstTime = true;
    if (firstTime) {
        firstTime = false;
        std::cout << "initializing 'y'\n";
        ans = 5;
    }

    return ans;
}

```

And, if you can get rid of the print statements you can further simplify these to something really simple:

```

int& y(); // forward declaration

int& x()
{
    static int ans = 3*y() + 7;
    return ans;
}

int& y()
{
    static int ans = 5;
    return ans;
}

```

Furthermore, since `y` is initialized using a constant expression, it no longer needs its wrapper function — it can be a simple variable again.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.17] How can I handle a constructor that fails?

Throw an exception. See [\[17.2\]](#) for details.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.18] What is the "Named Parameter Idiom"? UPDATED!

[Recently changed friend File; to friend class File; thanks to [Mehri](#) and [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

It's a fairly useful way to exploit [method chaining](#).

The fundamental problem solved by the Named Parameter Idiom is that C++ only supports *positional parameters*. For example, a caller of a function isn't allowed to say, "Here's the value for formal parameter xyz, and this other thing is the value for formal parameter pqr." All you can do in C++ (and C and Java) is say, "Here's the first parameter, here's the second parameter, etc." The alternative, called *named parameters* and implemented in the language Ada, is especially useful if a function takes a large number of mostly default -able parameters.

Over the years people have cooked up lots of workarounds for the lack of named parameters in C and C++. One of these involves burying the parameter values in a string parameter then parsing this string at run-time. This is what's done in the second parameter of `fopen()`, for example. Another workaround is to combine all the boolean parameters in a bit -map, then the caller or's a bunch of bit-shifted constants together to produce the actual parameter. This is what's done in the second parameter of `open()`, for example. These approaches work, but the following technique produces caller -code that's more obvious, easier to write, easier to read, and is generally more elegant.

The idea, called the Named Parameter Idiom, is to change the function's parameters to methods of a newly created class, where all these methods return `*this` by reference. Then you simply rename the main function into a parameterless "do -it" method on that class.

We'll work an example to make the previous paragraph easier to understand.

The example will be for the "open a file" concept. Let's say that concept logically requires a parameter for the file's name, and optionally allows parameters for whether the file should be opened read-only vs. read-write vs. write-only, whether or not the file should be created if it doesn't already exist, whether the writing location should be at the end ("append") or the beginning ("overwrite"), the block-size if the file is to be created, whether the I/O is buffered or non-buffered, the buffer-size, whether it is to be shared vs. exclusive access, and probably a few others. If we implemented this concept using a normal function with positional parameters, the caller code would be very difficult to read: there'd be as many as 8 positional parameters, and the caller would probably make a lot of mistakes. So instead we use the Named Parameter Idiom.

Before we go through the implementation, here's what the caller code might look like, assuming you are willing to accept all the function's default parameters:

```
File f = OpenFile("foo.txt");
```

That's the easy case. Now here's what it might look like if you want to change a bunch of the parameters.

```
File f = OpenFile("foo.txt")
    .readonly()
    .createIfNotExist()
    .appendWhenWriting()
    .blockSize(1024)
```

```
.unbuffered()
.exclusiveAccess();
```

Notice how the "parameters", if it's fair to call them that, are in random order (they're not positional) and they all have names. So the programmer doesn't have to remember the order of the parameters, and the names are (hopefully) obvious.

So here's how to implement it: first we create a class (`OpenFile`) that houses all the parameter values as private data members. The required parameters (in this case, the only required parameter is the file's name) is implemented as a normal, positional parameter on `OpenFile`'s constructor, but that constructor doesn't actually open the file. Then all the optional parameters (readonly vs. readwrite, etc.) become methods. These methods (e.g., `readonly()`, `blockSize(unsigned)`, etc.) return a reference to their `this` object so the method calls can be [chained](#).

```
class File;

class OpenFile {
public:
    OpenFile(const std::string& filename);
        // sets all the default values for each data member
    OpenFile& readonly(); // changes readonly_ to true
    OpenFile& readwrite(); // changes readonly_ to false
    OpenFile& createIfNotExist();
    OpenFile& blockSize(unsigned nbytes);
    ...
private:
    friend class File;
    std::string filename_;
    bool readonly_; // defaults to false [for example]
    bool createIfNotExist_; // defaults to false [for example]
    ...
    unsigned blockSize_; // defaults to 4096 [for example]
    ...
};

inline OpenFile::OpenFile(const std::string& filename)
: filename_ (filename)
, readonly_ (false)
, createIfNotExist_ (false)
, blockSize_ (4096u)
{ }

inline OpenFile& OpenFile::readonly()
{ readonly_ = true; return *this; }

inline OpenFile& OpenFile::readwrite()
{ readonly_ = false; return *this; }

inline OpenFile& OpenFile::createIfNotExist()
{ createIfNotExist_ = true; return *this; }

inline OpenFile& OpenFile::blockSize(unsigned nbytes)
{ blockSize_ = nbytes; return *this; }
```

The only other thing to do is make the constructor for class `File` to take an `OpenFile` object:

```
class File {
public:
    File(const OpenFile& params);
    ...
};
```

This constructor gets the actual parameters from the OpenFile object, then actually opens the file:

```
File::File(const OpenFile& params)
{
    ...
}
```

Note that openFile declares File as its [friend](#), that way [openFile doesn't need a bunch of \(otherwise useless\) public: get methods](#).

Since each member function in the chain returns a reference, there is no copying of objects and the chain is highly efficient. Furthermore, if the various member functions are `inline`, the generated object code will probably be on par with C-style code that sets various members of a struct. Of course if the member functions are not `inline`, there may be a slight increase in code size and a slight decrease in performance (but only if the construction occurs on the critical path of a CPU-bound program; this is a can of worms I'll try to avoid opening; read [the C++ FAQs book](#) for a rather thorough discussion of the issues), so it may, in this case, be a tradeoff for making the code more reliable.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[10.19] Why am I getting an error after declaring a Foo object via `Foo x(Bar())`?

NEW!

[Recently created thanks to [James Kanze](#) (in 12/04) and added the alternate syntax `Foo x = Bar()`; thanks to a suggestion from [Mike Dunn](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Because that doesn't create a Foo object - it declares a non-member function that *returns* a Foo object.

This is really going to hurt; you might want to sit down.

First, here's a better explanation of the problem. Suppose there is a class called Bar that has a default ctor. This might even be a library class such as `std::string`, but for now we'll just call it Bar:

```
class Bar {
public:
    Bar();
    ...
};
```

Now suppose there's another class called Foo that has a ctor that takes a Bar. As before, this might be defined by someone other than you.

```
class Foo {
public:
    Foo(const Bar& b);    // or perhaps Foo(Bar b)
    ...
    void blah();
    ...
};
```

Now you want to create a `Foo` object using a temporary `Bar`. In other words, you want to create an object via `Bar()`, and pass that to the `Foo` ctor to create a local `Foo` object called `x`:

```
void yourCode()
{
    Foo x(Bar());    ← you think this creates a FOO object called x...
    x.blah();        ← ...but it doesn't, so this line gives you a bizarre error message
    ...
}
```

It's a long story, but the solution (hope you're sitting down!) is to use `=` in your declaration:

```
void yourCode()
{
    Foo x = Foo(Bar());    ← Yes, Virginia, that thar syntax really works
    x.blah();              ← Ahhhh, this works now — no more error messages
    ...
}
```

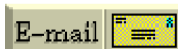
Here's *why* that happens (this part is optional; only read it if you think your future as a programmer is worth two minutes of your precious time today): When the compiler sees `Foo x(Bar())`, it thinks that the `Bar()` part is declaring a non-member function that returns a `Bar` object, so it thinks you are declaring the existence of a *function* called `x` that returns a `Foo` and that takes as a single parameter of type "non-member function that takes nothing and returns a `Bar`."

Now here's the sad part. In fact it's pathetic. Some mindless drone out there is going to skip that last paragraph, then they're going to impose a bizarre, incorrect, irrelevant, and just plain *stupid* coding standard that says something like, "Never create temporaries using a default constructor" or "Always use `=` in all initializations" or something else equally inane. If that's you, please fire yourself before you do any more damage. Those who don't *understand the problem* shouldn't tell others how to solve it. Harumph.

(Okay, that was mostly tongue in cheek. But there's a grain of truth in it. The real problem is that people tend to worship consistency, and they tend to extrapolate from the obscure to the common. That's not wise.)

Follow-up: if your `Foo::Foo(const Bar&)` constructor is not `explicit`, and if `Foo`'s copy constructor is accessible, you can use this syntax instead: `Foo x = Bar();`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[11] Destructors

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [11]:

- [\[11.1\] What's the deal with destructors?](#)
 - [\[11.2\] What's the order that local objects are destructed?](#)
 - [\[11.3\] What's the order that objects in an array are destructed?](#)
 - [\[11.4\] Can I overload the destructor for my class?](#)
 - [\[11.5\] Should I explicitly call a destructor on a local variable?](#)
 - [\[11.6\] What if I want a local to "die" before the close `}` of the scope in which it was created? Can I call a destructor on a local if I *really* want to?](#)
 - [\[11.7\] OK, OK already; I won't explicitly call the destructor of a local; but how do I handle the above situation?](#)
 - [\[11.8\] What if I can't wrap the local in an artificial block?](#)
 - [\[11.9\] But can I explicitly call a destructor if I've allocated my object with `new`?](#)
 - [\[11.10\] What is "placement `new`" and why would I use it?](#)
 - [\[11.11\] When I write a destructor, do I need to explicitly call the destructors for my member objects?](#)
 - [\[11.12\] When I write a derived class's destructor, do I need to explicitly call the destructor for my base class?](#)
 - [\[11.13\] Should my destructor throw an exception when it detects a problem?](#)
 - [\[11.14\] Is there a way to force `new` to allocate memory from a specific memory area?](#)
-

[11.1] What's the deal with destructors?

A destructor gives an object its last rites.

Destructors are used to release any resources allocated by the object. E.g., `class Lock` might lock a semaphore, and the destructor will release that semaphore. The most common example is when the constructor uses `new`, and the destructor uses `delete`.

Destructors are a "prepare to die" member function. They are often abbreviated "dtor".

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[11.2] What's the order that local objects are destructed?

In reverse order of construction: First constructed, last destructed.

In the following example, b's destructor will be executed first, then a's destructor:

```
void userCode()
{
    Fred a;
    Fred b;
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.3] What's the order that objects in an array are destructed?

In reverse order of construction: First constructed, last destructed.

In the following example, the order for destructors will be a[9], a[8], ..., a[1], a[0]:

```
void userCode()
{
    Fred a[10];
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.4] Can I overload the destructor for my class?

No.

You can have only one destructor for a class `Fred`. It's always called `Fred::~~Fred()`. It never takes any parameters, and it never returns anything.

You can't pass parameters to the destructor anyway, since you [never explicitly call a destructor](#) (well, [almost never](#)).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.5] Should I explicitly call a destructor on a local variable?

No!

The destructor will get called *again* at the close `}` of the block in which the local was created. This is a guarantee of the language; it happens automatically; there's no way to stop it from happening. But you can get *really* bad results from calling a destructor on the same object a second time! Bang! You're dead!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.6] What if I want a local to "die" before the close `}` of the scope in which it was created? Can I call a destructor on a local if I *really* want to?

No! [For context, please read [the previous FAQ](#)].

Suppose the (desirable) side effect of destructing a local `File` object is to close the `File`. Now suppose you have an object `f` of a class `File` and you want `File f` to be closed before the end of the scope (i.e., the `}`) of the scope of object `f`:

```
void someCode()
{
    File f;

    ...insert code that should execute when f is still open...

    ← We want the side-effect of f's destructor here!

    ...insert code that should execute after f is closed...
}
```

There is a [simple solution to this problem](#). But in the mean time, remember: [Do not explicitly call the destructor!](#)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.7] OK, OK already; I won't explicitly call the destructor of a local; but how do I handle the above situation?

[For context, please read [the previous FAQ](#)].

Simply wrap the extent of the lifetime of the local in an artificial block `{...}`:

```
void someCode()
{
    {
        File f;
        ...insert code that should execute when f is still open...
    } ← f's destructor will automagically be called here!

    ...insert code here that should execute after f is closed...
}
```

[11.8] What if I can't wrap the local in an artificial block?

Most of the time, you can limit the lifetime of a local by [wrapping the local in an artificial block](#) (`{...}`). But if for some reason you can't do that, add a member function that has a similar effect as the destructor. But *do not call the destructor itself!*

For example, in the case of `class File`, you might add a `close()` method. Typically the destructor will simply call this `close()` method. Note that the `close()` method will need to mark the `File` object so a subsequent call won't re-close an already-closed `File`. E.g., it might set the `fileHandle_` data member to some nonsensical value such as `-1`, and it might check at the beginning to see if the `fileHandle_` is already equal to `-1`:

```
class File {
public:
    void close();
    ~File();
    ...
private:
    int fileHandle_;    // fileHandle_ >= 0 if/only-if it's open
};

File::~~File()
{
    close();
}

void File::close()
{
    if (fileHandle_ >= 0) {
        ...insert code to call the OS to close the file...
        fileHandle_ = -1;
    }
}
```

Note that the other `File` methods may also need to check if the `fileHandle_` is `-1` (i.e., check if the `File` is closed).

Note also that any constructors that don't actually open a file should set `fileHandle_` to `-1`.

[11.9] But can I explicitly call a destructor if I've allocated my object with `new`?

Probably not.

Unless you used [placement new](#), you should simply delete the object rather than explicitly calling the destructor. For example, suppose you allocated the object via a typical `new` expression:

```
Fred* p = new Fred();
```

Then the destructor `Fred::~Fred()` will automatically get called when you delete it via:

```
delete p; // Automatically calls p->~Fred()
```

You should *not* explicitly call the destructor, since doing so won't release the memory that was allocated for the `Fred` object itself. Remember: [delete p does two things](#): it calls the destructor and it deallocates the memory.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.10] What is "placement new" and why would I use it?

There are many uses of placement `new`. The simplest use is to place an object at a particular location in memory. This is done by supplying the place as a pointer parameter to the `new` part of a `new` expression:

```
#include <new>           // Must #include this to use "placement new"
#include "Fred.h"        // Declaration of class Fred

void someCode()
{
    char memory[sizeof(Fred)]; // Line #1
    void* place = memory;      // Line #2

    Fred* f = new(place) Fred(); // Line #3 (see "DANGER" below)
    // The pointers f and place will be equal

    ...
}
```

Line #1 creates an array of `sizeof(Fred)` bytes of memory, which is big enough to hold a `Fred` object. Line #2 creates a pointer `place` that points to the first byte of this memory (experienced C programmers will note that this step was unnecessary; it's there only to make the code more obvious). Line #3 essentially just calls the constructor `Fred::Fred()`. The `this` pointer in the `Fred` constructor will be equal to `place`. The returned pointer `f` will therefore be equal to `place`.

ADVICE: Don't use this "placement new" syntax unless you have to. Use it only when you really care that an object is placed at a particular location in memory. For example, when your hardware has a memory-mapped I/O timer device, and you want to place a `clock` object at that memory location.

DANGER: You are taking sole responsibility that the pointer you pass to the "placement new" operator points to a region of memory that is big enough and is properly aligned for the object type that you're creating. Neither the compiler nor the run-time system make any attempt to check whether you did this right. If your `Fred` class needs to be aligned on a 4 byte boundary but you supplied a location that isn't properly aligned, you can have a serious disaster on your hands (if you don't know what "alignment" means, please don't use the placement new syntax). You have been warned.

You are also solely responsible for destructing the placed object. This is done by explicitly calling the destructor:

```
void someCode()
{
    char memory[sizeof(Fred)];
    void* p = memory;
    Fred* f = new(p) Fred();
    ...
    f->~Fred();    // Explicitly call the destructor for the placed object
}
```

This is about the only time you ever explicitly call a destructor.

Note: there is [a much cleaner but more sophisticated](#) way of handling the destruction / deletion situation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.11] When I write a destructor, do I need to explicitly call the destructors for my member objects?

No. You never need to explicitly call a destructor (except with [placement new](#)).

A class's destructor (whether or not you explicitly define one) *automagically* invokes the destructors for member objects. They are destroyed in the reverse order they appear within the declaration for the class.

```
class Member {
public:
    ~Member();
    ...
};

class Fred {
public:
    ~Fred();
    ...
private:
    Member x_;
    Member y_;
    Member z_;
};

Fred::~~Fred()
{
    // Compiler automagically calls z_. ~Member()
    // Compiler automagically calls y_. ~Member()
    // Compiler automagically calls x_. ~Member()
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.12] When I write a derived class's destructor, do I need to explicitly call the destructor for my base class?

No. You never need to explicitly call a destructor (except with [placement new](#)).

A derived class's destructor (whether or not you explicitly define one) *automagically* invokes the destructors for base class subobjects. Base classes are destructed after member objects. In the event of multiple inheritance, direct base classes are destructed in the reverse order of their appearance in the inheritance list.

```
class Member {
public:
    ~Member();
    ...
};

class Base {
public:
    virtual ~Base();    // A virtual destructor
    ...
};

class Derived : public Base {
public:
    ~Derived();
    ...
private:
    Member x_;
};

Derived::~Derived()
{
    // Compiler automagically calls x_.~Member()
    // Compiler automagically calls Base::~~Base()
}
```

Note: Order dependencies with virtual inheritance are trickier. If you are relying on order dependencies in a virtual inheritance hierarchy, you'll need a lot more information than is in this FAQ.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.13] Should my destructor throw an exception when it detects a problem?

Beware!!! See [this FAQ](#) for details.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[11.14] Is there a way to force `new` to allocate memory from a specific memory area?

Yes. The good news is that these "memory pools" are useful in a number of situations. The bad news is that I'll have to drag you through the mire of how it works before we discuss all the uses. But if you don't know about memory pools, it might be worthwhile to slog through this FAQ — you might learn something useful!

First of all, recall that a memory allocator is simply supposed to return uninitialized bits of memory; it is not supposed to produce "objects." In particular, the memory allocator is not supposed to set the virtual-pointer or any other part of the object, as that is the job of the constructor which runs after the memory allocator. Starting with a simple memory allocator function, `allocate()`, you would use [placement new](#) to construct an object in that memory. In other words, the following is morally equivalent to `new Foo()`:

```
void* raw = allocate(sizeof(Foo)); // line 1
Foo* p = new(raw) Foo();           // line 2
```

Okay, assuming you've used [placement new](#) and have survived the above two lines of code, the next step is to turn your memory allocator into an object. This kind of object is called a "memory pool" or a "memory arena." This lets your users have more than one "pool" or "arena" from which memory will be allocated. Each of these memory pool objects will allocate a big chunk of memory using some specific system call (e.g., shared memory, persistent memory, stack memory, etc.; see below), and will dole it out in little chunks as needed. Your memory -pool class might look something like this:

```
class Pool {
public:
    void* alloc(size_t nbytes);
    void dealloc(void* p);
private:
    ...data members used in your pool object...
};

void* Pool::alloc(size_t nbytes)
{
    ...your algorithm goes here...
}

void Pool::dealloc(void* p)
{
    ...your algorithm goes here...
}
```

Now one of your users might have a `Pool` called `pool`, from which they could allocate objects like this:

```
Pool pool;
...
void* raw = pool.alloc(sizeof(Foo));
Foo* p = new(raw) Foo();
```

Or simply:

```
Foo* p = new(pool.alloc(sizeof(Foo))) Foo();
```

The reason it's good to turn `Pool` into a class is because it lets users create N different pools of memory rather than having one massive pool shared by all users. That allows users to do lots of funky things. For example, if they have a chunk of the system that allocates memory like crazy then goes away, they could allocate all their memory from a `Pool`, then not even bother doing any `deletes` on the little pieces: just deallocate the entire pool at once. Or they could set up a "shared memory" area (where the operating system specifically provides memory that is shared between multiple processes) and have the pool dole out chunks of shared memory rather than process-local memory. Another angle: many systems support a non-standard function often called `alloca()` which allocates a block of memory from the stack rather than the heap. Naturally this block of memory automatically goes away when the function returns, eliminating the need for explicit `deletes`. Someone could use `alloca()` to give the `Pool` its big chunk of memory, then all the little pieces allocated from that `Pool` act like they're local: they automatically vanish when the function returns. Of course the destructors don't get called in some of these cases, and if the destructors do something nontrivial you won't be able to use these techniques, but in cases where the destructor merely deallocates memory, these sorts of techniques can be useful.

Okay, assuming you survived the 6 or 8 lines of code needed to wrap your allocate function as a method of a `Pool` class, the next step is to change the syntax for allocating objects. The goal is to change from the rather clunky syntax `new(pool.alloc(sizeof(Foo))) Foo()` to the [simpler syntax](#) `new(pool) Foo()`. To make this happen, you need to add the following two lines of code just below the definition of your `Pool` class:

```
inline void* operator new(size_t nbytes, Pool& pool)
{
    return pool.alloc(nbytes);
}
```

Now when the compiler sees `new(pool) Foo()`, it calls the above `operator new` and passes `sizeof(Foo)` and `pool` as parameters, and the only function that ends up using the funky `pool.alloc(nbytes)` method is your own `operator new`.

Now to the issue of how to destruct/deallocate the `Foo` objects. Recall that [the brute force approach sometimes used with placement new](#) is to explicitly call the destructor then explicitly deallocate the memory:

```
void sample(Pool& pool)
{
    Foo* p = new(pool) Foo();
    ...
    p->~Foo();           // explicitly call dtor
    pool.dealloc(p);      // explicitly release the memory
}
```

This has several problems, all of which are fixable:

1. The memory will leak if `Foo::~Foo()` throws an exception.
2. The destruction/deallocation syntax is different from what most programmers are used to, so they'll probably screw it up.
3. Users must somehow remember which pool goes with which object. Since the code that allocates is often in a different function from the code that deallocates, programmers will

have to pass around two pointers (a `Foo*` and a `Pool*`), which gets ugly fast (example, what if they had an array of `Foo`s each of which potentially came from a different `Pool`; ugh).

We will fix them in the above order.

Problem #1: plugging the memory leak. When you use the "normal" `new` operator, e.g., `Foo* p = new Foo()`, the compiler generates some special code to handle the case when the constructor throws an exception. The actual code generated by the compiler is functionally similar to this:

```
// This is functionally what happens with Foo* p = new Foo()

Foo* p;

// don't catch exceptions thrown by the allocator itself
void* raw = operator new(sizeof(Foo));

// catch any exceptions thrown by the ctor
try {
    p = new(raw) Foo(); // call the ctor with raw as this
}
catch (...) {
    // oops, ctor threw an exception
    operator delete(raw);
    throw; // rethrow the ctor's exception
}
```

The point is that the compiler deallocates the memory if the ctor throws an exception. But in the case of the "new with parameter" syntax (commonly called "placement new"), the compiler won't know what to do if the exception occurs so by default it does nothing:

```
// This is functionally what happens with Foo* p = new(pool) Foo():

void* raw = operator new(sizeof(Foo), pool);
// the above function simply returns "pool.alloc(sizeof(Foo))"

Foo* p = new(raw) Foo();
// if the above line "throws", pool.dealloc(raw) is NOT called
```

So the goal is to force the compiler to do something similar to what it does with the global `new` operator. Fortunately it's simple: when the compiler sees `new(pool) Foo()`, it looks for a corresponding `operator delete`. If it finds one, it does the equivalent of wrapping the ctor call in a `try` block as shown above. So we would simply provide an `operator delete` with the following signature (be careful to get this right; if the second parameter has a different type from the second parameter of the `operator new(size_t, Pool&)`, the compiler doesn't complain; it simply bypasses the `try` block when your users say `new(pool) Foo()`):

```
void operator delete(void* p, Pool& pool)
{
    pool.dealloc(p);
}
```

After this, the compiler will automatically wrap the ctor calls of your `new` expressions in a `try` block:

```
// This is functionally what happens with Foo* p = new(pool) Foo()

Foo* p;

// don't catch exceptions thrown by the allocator itself
void* raw = operator new(sizeof(Foo), pool);
// the above simply returns "pool.alloc(sizeof(Foo))"

// catch any exceptions thrown by the ctor
try {
    p = new(raw) Foo(); // call the ctor with raw as this
}
catch (...) {
    // oops, ctor threw an exception
    operator delete(raw, pool); // that's the magical line!!
    throw; // rethrow the ctor's exception
}
```

In other words, the one-liner function `operator delete(void* p, Pool& pool)` causes the compiler to automatically plug the memory leak. Of course that function can be, but doesn't have to be, inline.

Problems #2 ("ugly therefore error prone") and #3 ("users must manually associate pool-pointers with the object that allocated them, which is error prone") are solved simultaneously with an additional 10-20 lines of code in one place. In other words, we add 10 -20 lines of code in *one* place (your `Pool` header file) and simplify an arbitrarily large number of other places (every piece of code that *uses* your `Pool` class).

The idea is to implicitly associate a `Pool*` with every allocation. The `Pool*` associated with the global allocator would be `NULL`, but at least conceptually you could say every allocation has an associated `Pool*`. Then you replace the global `operator delete` so it looks up the associated `Pool*`, and if non-`NULL`, calls that `Pool`'s deallocate function. For example, [if\(!\)](#) the normal deallocator used `free()`, the replacement for the global `operator delete` would look something like this:

```
void operator delete(void* p)
{
    if (p != NULL) {
        Pool* pool = /* somehow get the associated 'Pool*' */;
        if (pool == null)
            free(p);
        else
            pool->dealloc(p);
    }
}
```

If you're [not sure if the normal deallocator was free\(\)](#), the easiest approach is also replace the global `operator new` with something that uses `malloc()`. The replacement for the global `operator new` would look something like this (note: this definition ignores a few details such as the `new_handler` loop and the `throw std::bad_alloc()` that happens if we run out of memory):

```
void* operator new(size_t nbytes)
{
    if (nbytes == 0)
        nbytes = 1; // so all alloc's get a distinct address
```

```

void* raw = malloc(nbytes);
...somehow associate the NULL 'Pool*' with 'raw'...
return raw;
}

```

The only remaining problem is to associate a `Pool*` with an allocation. One approach, used in at least one commercial product, is to use a `std::map<void*, Pool*>`. In other words, build a look-up table whose keys are the allocation-pointer and whose values are the associated `Pool*`. For reasons I'll describe in a moment, it is essential that you insert a key/value pair into the map *only* in operator `new(size_t, Pool&)`. In particular, you must not insert a key/value pair from the global operator `new` (e.g., you must not say, `poolMap[p] = NULL` in the global operator `new`). Reason: doing that would create a nasty chicken-and-egg problem — since `std::map` probably uses the global operator `new`, it ends up inserting a new entry every time inserts a new entry, leading to infinite recursion — bang you're dead.

Even though this technique requires a `std::map` look-up for each deallocation, it seems to have acceptable performance, at least in many cases.

Another approach that is faster but might use more memory and is a little trickier is to prepend a `Pool*` just before *all* allocations. For example, if `nbytes` was 24, meaning the caller was asking to allocate 24 bytes, we would allocate 28 (or 32 if you think the machine requires 8-byte alignment for things like `doubles` and/or `long longs`), stuff the `Pool*` into the first 4 bytes, and return the pointer 4 (or 8) bytes from the beginning of what you allocated. Then your global operator `delete` backs off the 4 (or 8) bytes, finds the `Pool*`, and if `NULL`, uses `free()` otherwise calls `pool->dealloc()`. The parameter passed to `free()` and `pool->dealloc()` would be the pointer 4 (or 8) bytes to the left of the original parameter, `p`. If(!) you decide on 4 byte alignment, your code would look something like this (although as before, the following operator `new` code elides the usual out-of-memory handlers):

```

void* operator new(size_t nbytes)
{
    if (nbytes == 0)
        nbytes = 1;
    void* ans = malloc(nbytes + 4); // so all alloc's get a distinct address
    *(Pool**)ans = NULL;           // overallocate by 4 bytes
    return (char*)ans + 4;         // use NULL in the global new
}                                 // don't let users see the Pool*

void* operator new(size_t nbytes, Pool& pool)
{
    if (nbytes == 0)
        nbytes = 1;
    void* ans = pool.alloc(nbytes + 4); // so all alloc's get a distinct address
    *(Pool**)ans = &pool;              // overallocate by 4 bytes
    return (char*)ans + 4;             // put the Pool* here
}                                    // don't let users see the Pool*

void operator delete(void* p)
{
    if (p != NULL) {
        p = (char*)p - 4;              // back off to the Pool*
        Pool* pool = *(Pool**)p;
        if (pool == null)
            free(p);                   // note: 4 bytes left of the original p
    }
}

```

```
    else
        pool->dealloc(p);           // note: 4 bytes left of the original p
    }
}
```

Naturally the last few paragraphs of this FAQ are viable only when you are allowed to change the global operator new and operator delete. If you are not allowed to change these global functions, the first three quarters of this FAQ is still applicable.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[12] Assignment operators

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [12]:

- [\[12.1\] What is "self assignment"?](#)
 - [\[12.2\] Why should I worry about "self assignment"?](#)
 - [\[12.3\] OK, OK, already; I'll handle self-assignment. How do I do it?](#)
-

[12.1] What is "self assignment"?

Self assignment is when someone assigns an object to itself. For example,

```
#include "Fred.hpp"    // Defines class Fred

void userCode(Fred& x)
{
    x = x;    // Self-assignment
}
```

Obviously no one ever explicitly does a self assignment like the above, but since more than one pointer or reference can point to the same object (aliasing), it is possible to have self assignment without knowing it:

```
#include "Fred.hpp"    // Defines class Fred

void userCode(Fred& x, Fred& y)
{
```

```

    x = y;    // Could be self-assignment if &x == &y
}

int main()
{
    Fred z;
    userCode(z, z);
    ...
}

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[12.2] Why should I worry about "self assignment"?

If you don't worry about [self assignment](#), you'll expose your users to some very subtle bugs that have very subtle and often disastrous symptoms. For example, the following class will cause a complete disaster in the case of self-assignment:

```

class Wilma { };

class Fred {
public:
    Fred()           : p_(new Wilma())           { }
    Fred(const Fred& f) : p_(new Wilma(*f.p_)) { }
    ~Fred()          { delete p_; }
    Fred& operator= (const Fred& f)
    {
        // Bad code: Doesn't handle self-assignment!
        delete p_;           // Line #1
        p_ = new Wilma(*f.p_); // Line #2
        return *this;
    }
private:
    Wilma* p_;
};

```

If someone assigns a Fred object to itself, line #1 deletes both `this->p_` and `f.p_` since `*this` and `f` are the same object. But line #2 uses `*f.p_`, which is no longer a valid object. This will likely cause a major disaster.

The bottom line is that *you* the author of class Fred are responsible to [make sure self-assignment on a Fred object is innocuous](#). Do *not* assume that users won't ever do that to your objects. It is *your* fault if your object crashes when it gets a self-assignment.

Aside: the above `Fred::operator= (const Fred&)` has a second problem: [if an exception is thrown](#) while evaluating `new Wilma(*f.p_)` (e.g., [an out-of-memory exception](#) or [an exception in Wilma's copy constructor](#)), `this->p_` will be a dangling pointer — it will point to memory that is no longer valid. This can be solved by allocating the new objects before deleting the old objects.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[12.3] OK, OK, already; I'll handle self-assignment. How do I do it?

[You should worry about self assignment every time you create a class](#). This does *not* mean that you need to add extra code to all your classes: as long as your objects gracefully handle self assignment, it doesn't matter whether you had to add extra code or not.

If you do need to add extra code to your assignment operator, here's a simple and effective technique:

```
Fred& Fred::operator= (const Fred& f)
{
    if (this == &f) return *this;    // Gracefully handle self assignment

    // Put the normal assignment duties here...

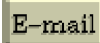
    return *this;
}
```

This explicit test isn't always necessary. For example, if you were to fix [the assignment operator in the previous FAQ](#) to handle [exceptions thrown by new](#) and/or [exceptions thrown by the copy constructor](#) of class Wilma, you might produce the following code. Note that this code has the (pleasant) side effect of automatically handling self assignment as well:

```
Fred& Fred::operator= (const Fred& f)
{
    // This code gracefully (albeit implicitly) handles self assignment
    Wilma* tmp = new Wilma(*f.p_);    // It would be OK if an exception got thrown here
    delete p_;
    p_ = tmp;
    return *this;
}
```

In cases like the previous example (where self assignment is harmless but inefficient), some programmers want to improve the efficiency of self assignment by adding an otherwise unnecessary test, such as "if (this == &f) return *this;". It is generally the wrong tradeoff to make self assignment more efficient by making the non-self assignment case less efficient. For example, adding the above if test to the Fred assignment operator would make the non-self assignment case slightly less efficient (an extra (and unnecessary) conditional branch). If self assignment actually occurred once in a thousand times, the if would waste cycles 99.9% of the time.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[13] Operator overloading

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [13]:

- [\[13.1\] What's the deal with operator overloading?](#)
- [\[13.2\] What are the benefits of operator overloading?](#)
- [\[13.3\] What are some examples of operator overloading?](#)
- [\[13.4\] But operator overloading makes my class look ugly; isn't it supposed to make my code clearer?](#)
- [\[13.5\] What operators can/cannot be overloaded?](#)
- [\[13.6\] Can I overload operator== so it lets me compare two char\[\] using a string comparison?](#)
- [\[13.7\] Can I create a operator** for "to-the-power-of" operations?](#)
- [\[13.8\] Okay, that tells me the operators I *can* override; which operators *should* I override?](#)
- [\[13.9\] What are some guidelines / "rules of thumb" for overloading operators?](#) **NEW!**
- [\[13.10\] How do I create a subscript operator for a Matrix class?](#)
- [\[13.11\] Why shouldn't my Matrix class's interface look like an array-of-array?](#)
- [\[13.12\] Should I design my classes from the outside \(interfaces first\) or from the inside \(data first\)?](#) **UPDATED!**
- [\[13.13\] How can I overload the prefix and postfix forms of operators ++ and --?](#)
- [\[13.14\] Which is more efficient: i++ or ++i?](#)

[13.1] What's the deal with operator overloading?

It allows you to provide an intuitive interface to users of your class, plus makes it possible for [templates](#) to work equally well with classes and built-in/intrinsic types.

Operator overloading allows C/C++ operators to have user-defined meanings on user-defined types (classes). Overloaded operators are syntactic sugar for function calls:

```
class Fred {
public:
    ...
};

#if 0

// Without operator overloading:
Fred add(const Fred& x, const Fred& y);
Fred mul(const Fred& x, const Fred& y);

Fred f(const Fred& a, const Fred& b, const Fred& c)
```

```

{
    return add(add(mul(a,b), mul(b,c)), mul(c,a));    // Yuk...
}

#else

// With operator overloading:
Fred operator+ (const Fred& x, const Fred& y);
Fred operator* (const Fred& x, const Fred& y);

Fred f(const Fred& a, const Fred& b, const Fred& c)
{
    return a*b + b*c + c*a;
}

#endif

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.2] What are the benefits of operator overloading?

By overloading standard operators on a class, you can exploit the intuition of the users of that class. This lets users program in the language of the problem domain rather than in the language of the machine.

The ultimate goal is to reduce both the learning curve and the defect rate.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.3] What are some examples of operator overloading?

Here are a few of the many examples of operator overloading:

- `myString + yourString` might concatenate two `std::string` objects
- `myDate++` might increment a `Date` object
- `a * b` might multiply two `Number` objects
- `a[i]` might access an element of an `Array` object
- `x = *p` might dereference a "smart pointer" that "points" to a disk record — it could seek to the location on disk where `p` "points" and return the appropriate record into `x`

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.4] But operator overloading makes my class look ugly; isn't it supposed to make my code clearer?

Operator overloading [makes life easier for the users of a class](#), not for the developer of the class!

Consider the following example.

```
class Array {
public:
    int& operator[] (unsigned i);      // Some people don't like this syntax
    ...
};

inline
int& Array::operator[] (unsigned i)  // Some people don't like this syntax
{
    ...
}
```

Some people don't like the keyword `operator` or the somewhat funny syntax that goes with it in the body of the class itself. But the operator overloading syntax isn't supposed to make life easier for the *developer* of a class. It's supposed to make life easier for the *users* of the class:

```
int main()
{
    Array a;
    a[3] = 4;    // User code should be obvious and easy to understand...
    ...
}
```

Remember: in a reuse-oriented world, there will usually be many people who use your class, but there is only one person who builds it (yourself); therefore you should do things that favor the many rather than the few.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.5] What operators can/cannot be overloaded?

Most can be overloaded. The only C operators that can't be are `.` and `?:` (and `sizeof`, which is technically an operator). C++ adds a few of its own operators, most of which can be overloaded except `::` and `.*`.

Here's an example of the subscript operator (it returns a reference). First with *out* operator overloading:

```
class Array {
public:
    int& elem(unsigned i)      { if (i > 99) error(); return data[i]; }
private:
    int data[100];
};

int main()
{
    Array a;
```

```
a.elem(10) = 42;
a.elem(12) += a.elem(13);
...
}
```

Now the same logic is presented *with* operator overloading:

```
class Array {
public:
    int& operator[] (unsigned i) { if (i > 99) error(); return data[i]; }
private:
    int data[100];
};

int main()
{
    Array a;
    a[10] = 42;
    a[12] += a[13];
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.6] Can I overload `operator==` so it lets me compare two `char[]` using a string comparison?

No: [at least one operand of any overloaded operator must be of some user-defined type](#) (most of the time that means a `class`).

But even if C++ allowed you to do this, which it doesn't, you wouldn't want to do it anyway since you really should be using a [std::string-like class rather than an array of char in the first place](#) since [arrays are evil](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.7] Can I create a `operator**` for "to-the-power-of" operations?

Nope.

The names of, precedence of, associativity of, and arity of operators is fixed by the language. There is no `operator**` in C++, so you cannot create one for a `class` type.

If you're in doubt, consider that `x ** y` is the same as `x * (*y)` (in other words, the compiler assumes `y` is a pointer). Besides, operator overloading is just syntactic sugar for function calls. Although this particular syntactic sugar can be very sweet, it doesn't add anything fundamental. I suggest you overload `pow(base, exponent)` (a double precision version is in `<cmath>`).

By the way, `operator^` can work for to-the-power-of, except it has the wrong precedence and associativity.

[13.8] Okay, that tells me the operators I *can* override; which operators *should* I override? **NEW!**

[Recently created thanks to [Conrad Weisert](#) for suggesting the topic (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Bottom line: don't confuse your users.

Remember the purpose of operator overloading: to reduce the cost and defect rate in code that uses your class. If you create operators that confuse your users (because they're cool, because they make the code faster, because you need to prove to yourself that you can do it; doesn't really matter why), you've violated the whole reason for using operator overloading in the first place.

[13.9] What are some guidelines / "rules of thumb" for overloading operators? **NEW!**

[Recently created (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Here are a few guidelines / rules of thumb (but be sure to read [the previous FAQ](#) before reading this list):

1. Use common sense. If your overloaded operator makes life easier and safer for your users, do it; otherwise don't. This is the most important guideline. In fact it is, in a very real sense, the only guideline; the rest are just special cases.
2. If you define arithmetic operators, maintain the usual arithmetic identities. For example, if your class defines $x + y$ and $x - y$, then $x + y - y$ ought to return an object that is behaviorally equivalent to x . The term behaviorally equivalent is defined in the bullet on $x == y$ below, but simply put, it means the two objects should ideally act like they have the same state. This should be true even if you decide not to define an $==$ operator for objects of your class.
3. You should provide arithmetic operators only when they make logical sense to users. Subtracting two dates makes sense, logically returning the duration between those dates, so you might want to allow $\text{date1} - \text{date2}$ for objects of your `Date` class (provided you have a reasonable class/type to represent the duration between two `Date` objects). However adding two dates makes no sense: what does it mean to add July 4, 1776 to June 5, 1959? Similarly it makes no sense to multiply or divide dates, so you should not define any of those operators.
4. You should provide mixed-mode arithmetic operators only when they make logical sense to users. For example, it makes sense to add a duration (say 35 days) to a date (say July 4, 1776), so you might define $\text{date} + \text{duration}$ to return a `Date`. Similarly $\text{date} - \text{duration}$ could also return a `Date`. But $\text{duration} - \text{date}$ does not make sense at the

conceptual level (what does it mean to subtract July 4, 1776 from 35 days?) so you should not define that operator.

5. If you provide constructive operators, they should return their result by value. For example, $x + y$ should return its result by value. If it returns by reference, you will probably run into lots of problems figuring out who owns the referent and when the referent will get destructed. Doesn't matter if returning by reference is more efficient; it is *probably wrong*. See the next bullet for more on this point.
6. If you provide constructive operators, they should not change their operands. For example, $x + y$ should not change x . For some crazy reason, programmers often define $x + y$ to be logically the same as $x += y$ because the latter is faster. But remember, your users *expect* $x + y$ to make a copy. In fact they selected the $+$ operator (over, say, the $+=$ operator) precisely because they *wanted* a copy. If they wanted to modify x , they would have used whatever is equivalent to $x += y$ instead. Don't make semantic decisions for your users; it's *their* decision, not yours, whether they want the semantics of $x + y$ vs. $x += y$. Tell them that one is faster if you want, but then step back and let them make the final decision — they know what they're trying to achieve and you do not.
7. If you provide constructive operators, they should allow promotion of the left -hand operand. For example, if your class `Fraction` supports promotion from `int` to `Fraction` (via the non-explicit ctor `Fraction::Fraction(int)`), and if you allow $x - y$ for two `Fraction` objects, you should also allow $42 - y$. In practice that simply means that your operator `-()` should not be a member function of `Fraction`. Typically you will make it a [friend](#), if for no other reason than to [force it into the public: part of the class](#), but even if it is not a friend, it should not be a member.
8. In general, your operator should change its operand(s) if and only if the operands get changed when you apply the same operator to intrinsic types. $x == y$ and $x << y$ should not change either operand; $x *= y$ and $x <= y$ should (but only the left-hand operand).
9. If you define $x++$ and $++x$, maintain the usual identities. For example, $x++$ and $++x$ should have should have the same observable effect on x , and should differ only in what they return. $++x$ should return x by reference; $x++$ should either return a copy (by value) of the original state of x or should have a `void` return-type. You're usually better off returning a copy of the original state of x by value, especially if your class will be used in generic algorithms. The easy way to do that is to implement $x++$ using three lines: make a local copy of `*this`, call $++x$ (i.e., `this->operator++()`), then return the local copy. Similar comments for $x--$ and $--x$.
10. If you define $++x$ and $x += 1$, maintain the usual identities. For example, these expressions should have the same observable behavior, including the same result. Among other things, that means your $+=$ operator should return x by reference. Similar comments for $--x$ and $x -= 1$.
11. If you define `*p` and `p[0]` for pointer-like objects, maintain the usual identities. For example, these two expressions should have the same result and neither should change `p`.
12. If you define `p[i]` and `*(p+i)` for pointer-like objects, maintain the usual identities. For example, these two expressions should have the same result and neither should change `p`. Similar comments for `p[-i]` and `*(p-i)`.
13. Subscript operators generally come in pairs. This is called "const overloading" and is illustrated in several FAQs: [here](#), [here](#), [here](#), [here](#), and [here](#).
14. If you define $x == y$, then $x == y$ should be true if and only if the two objects are behaviorally equivalent. In this bullet, the term "behaviorally equivalent" means the

observable behavior of any operation or sequence of operations applied to `x` will be the same as when applied to `y`. The term "operation" means methods, friends, operators, or just about anything else you can do with these objects (except, of course, the address-of operator). You won't always be able to achieve that goal, but you ought to get close, and you ought to document any variances (other than the address-of operator).

15. If you define `x == y` and `x = y`, maintain the usual identities. For example, after an assignment, the two objects should be equal. Even if you don't define `x == y`, the two objects should be behaviorally equivalent (see above for the meaning of that phrase) after an assignment.
16. If you define `x == y` and `x != y`, you should maintain the usual identities. For example, these expressions should return something convertible to `bool`, neither should change its operands, and `x == y` should have the same result as `!(x != y)`, and vice versa.
17. If you define inequality operators like `x <= y` and `x < y`, you should maintain the usual identities. For example, if `x < y` and `y < z` are both true, then `x < z` should also be true, etc. Similar comments for `x >= y` and `x > y`.
18. If you define inequality operators like `x < y` and `x >= y`, you should maintain the usual identities. For example, `x < y` should have the result as `!(x >= y)`. You can't always do that, but you should get close and you should document any variances. Similar comments for `x > y` and `!(x <= y)`, etc.
19. Avoid overloading short-circuiting operators: `x || y` or `x && y`. The overloaded versions of these do not short-circuit — they evaluate both operands even if the left-hand operand "determines" the outcome, so that confuses users.
20. Avoid overloading the comma operator: `x, y`. The overloaded comma operator does not have the same ordering properties that it has when it is not overloaded, and that confuses users.
21. Don't overload an operator that is non-intuitive to your users. This is called the Doctrine of Least Surprise. For example, although C++ uses `std::cout << x` for printing, and although printing is technically called inserting, and although inserting sort of sounds like what happens when you push an element onto a stack, don't overload `myStack << x` to push an element onto a stack. It might make sense when you're really tired or otherwise mentally impaired, and a few of your friends might think it's "kewl," but just say No.
22. Use common sense. If you don't see "your" operator listed here, you can figure it out. Just remember the ultimate goals of operator overloading: to make life easier for your users, in particular to make their code cheaper to write and more obvious.

Caveat: the list is not exhaustive. That means there are other entries that you might consider "missing." I know.

Caveat: the list contains guidelines, not hard and fast rules. That means almost all of the entries have exceptions, and most of those exceptions are not explicitly stated. I know.

Caveat: please don't email me about the additions or exceptions. I've already spent way too much time on this particular answer.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.10] How do I create a subscript operator for a Matrix class?

Use `operator()` rather than `operator[]`.

When you have multiple subscripts, the cleanest way to do it is with `operator()` rather than with `operator[]`. The reason is that `operator[]` always takes exactly one parameter, but `operator()` can take any number of parameters (in the case of a rectangular matrix, two parameters are needed).

For example:

```
class Matrix {
public:
    Matrix(unsigned rows, unsigned cols);
    double& operator() (unsigned row, unsigned col);
    double operator() (unsigned row, unsigned col) const;
    ...
    ~Matrix(); // Destructor
    Matrix(const Matrix& m); // Copy constructor
    Matrix& operator= (const Matrix& m); // Assignment operator
    ...
private:
    unsigned rows_, cols_;
    double* data_;
};

inline
Matrix::Matrix(unsigned rows, unsigned cols)
    : rows_ (rows)
    , cols_ (cols)
    //data_ <--initialized below (after the 'if/throw' statement)
{
    if (rows == 0 || cols == 0)
        throw BadIndex("Matrix constructor has 0 size");
    data_ = new double[rows * cols];
}

inline
Matrix::~Matrix()
{
    delete[] data_;
}

inline
double& Matrix::operator() (unsigned row, unsigned col)
{
    if (row >= rows_ || col >= cols_)
        throw BadIndex("Matrix subscript out of bounds");
    return data_[cols_*row + col];
}

inline
double Matrix::operator() (unsigned row, unsigned col) const
{
    if (row >= rows_ || col >= cols_)
        throw BadIndex("const Matrix subscript out of bounds");
    return data_[cols_*row + col];
}
```

Then you can access an element of Matrix `m` using `m(i,j)` rather than `m[i][j]`:

```
int main()
{
    Matrix m(10,10);
    m(5,8) = 106.15;
    std::cout << m(5,8);
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.11] Why shouldn't my `Matrix` class's interface look like an array-of-array?

Here's what this FAQ is really all about: Some people build a `Matrix` class that has an `operator[]` that returns a reference to an `Array` object, and that `Array` object has an `operator[]` that returns an element of the `Matrix` (e.g., a reference to a `double`). Thus they access elements of the matrix using syntax like `m[i][j]` rather than [syntax like `m\(i,j\)`](#).

The array-of-array solution obviously works, but it is less flexible than [the `operator\(\)` approach](#). Specifically, there are easy performance tuning tricks that can be done with the `operator()` approach that are more difficult in the `[][]` approach, and therefore the `[][]` approach is more likely to lead to bad performance, at least in some cases.

For example, the easiest way to implement the `[][]` approach is to use a physical layout of the matrix as a dense matrix that is stored in row-major form (or is it column-major; I can't ever remember). In contrast, [the `operator\(\)` approach](#) totally hides the physical layout of the matrix, and that can lead to better performance in some cases.

Put it this way: the `operator()` approach is never worse than, and sometimes better than, the `[][]` approach.

- The `operator()` approach is never worse because it is easy to implement the dense, row-major physical layout using the `operator()` approach, so when that configuration happens to be the optimal layout from a performance standpoint, the `operator()` approach is just as easy as the `[][]` approach (perhaps the `operator()` approach is a tiny bit easier, but I won't quibble over minor nits).
- The `operator()` approach is sometimes better because whenever the optimal layout for a given application happens to be something other than dense, row-major, the implementation is often significantly easier using the `operator()` approach compared to the `[][]` approach.

As an example of when a physical layout makes a significant difference, a recent project happened to access the matrix elements in columns (that is, the algorithm accesses all the elements in one column, then the elements in another, etc.), and if the physical layout is row-major, the accesses can "stride the cache". For example, if the rows happen to be almost as big as the processor's cache size, the machine can end up with a "cache miss" for almost every element access. In this particular project, we got a 20% improvement in performance by changing the mapping from the logical layout (row,column) to the physical layout (column, row).

Of course there are many examples of this sort of thing from numerical methods, and sparse matrices are a whole other dimension on this issue. Since it is, in general, easier to implement a sparse matrix or swap row/column ordering using the `operator()` approach, the `operator()` approach loses nothing and may gain something — it has no down-side and a potential up-side.

Use [the operator\(\) approach](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.12] Should I design my classes from the outside (interfaces first) or from the inside (data first)? UPDATED!

[Recently changed friend `LinkedList`; to friend class `LinkedList`; , and similarly with `LinkedListIterator`, thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

From the outside!

A good interface provides a [simplified view that is expressed in the vocabulary of a user](#). In the case of OO software, the interface is normally the set of public methods of either a single class or a [tight group of classes](#).

First think about what the object logically represents, not how you intend to physically build it. For example, suppose you have a `Stack` class that will be built by containing a `LinkedList`:

```
class Stack {
public:
    ...
private:
    LinkedList list_;
};
```

Should the `Stack` have a `get()` method that returns the `LinkedList`? Or a `set()` method that takes a `LinkedList`? Or a constructor that takes a `LinkedList`? Obviously the answer is *No*, since you should design your interfaces from the outside-in. I.e., users of `Stack` objects don't care about `LinkedList`s; they care about pushing and popping.

Now for another example that is a bit more subtle. Suppose class `LinkedList` is built using a linked list of `Node` objects, where each `Node` object has a pointer to the next `Node`:

```
class Node { /*...*/ };

class LinkedList {
public:
    ...
private:
    Node* first_;
};
```

Should the `LinkedList` class have a `get()` method that will let users access the first `Node`? Should the `Node` object have a `get()` method that will let users follow that `Node` to the next `Node` in

the chain? In other words, what should a `LinkedList` look like from the outside? Is a `LinkedList` really a chain of `Node` objects? Or is that just an implementation detail? And if it is just an implementation detail, how will the `LinkedList` let users access each of the elements in the `LinkedList` one at a time?

The key insight is the realization that a `LinkedList` is *not* a chain of `Nodes`. That may be *how* it is built, but that is not *what* it is. What it is is a sequence of elements. Therefore the `LinkedList` abstraction should provide a `LinkedListIterator` class as well, and that `LinkedListIterator` might have an `operator++` to go to the next element, and it might have a `get()/set()` pair to access its *value* stored in the `Node` (the value in the `Node` element is solely the responsibility of the `LinkedList` user, which is why there is a `get()/set()` pair that allows the user to freely manipulate that value).

Starting from the user's perspective, we might want our `LinkedList` class to support operations that look similar to accessing an array using pointer arithmetic:

```
void userCode(LinkedList& a)
{
    for (LinkedListIterator p = a.begin(); p != a.end(); ++p)
        std::cout << *p << '\n';
}
```

To implement this interface, `LinkedList` will need a `begin()` method and an `end()` method. These return a `LinkedListIterator` object. The `LinkedListIterator` will need a method to go forward, `++p`; a method to access the current element, `*p`; and a comparison operator, `p != a.end()`.

The code follows. The important thing to notice is that `LinkedList` does *not* have any methods that let users access `Nodes`. `Nodes` are an implementation technique that is *completely* buried. This makes the `LinkedList` class safer (no chance a user will mess up the invariants and linkages between the various nodes), easier to use (users don't need to expend extra effort keeping the node-count equal to the actual number of nodes, or any other infrastructure stuff), and more flexible (by changing a single `typedef`, users could change their code from using `LinkedList` to some other list-like class and the bulk of their code would compile cleanly and hopefully with improved performance characteristics).

```
#include <cassert>      // Poor man's exception handling

class LinkedListIterator;
class LinkedList;

class Node {
    // No public members; this is a "private class"
    friend class LinkedListIterator;    // A friend class
    friend class LinkedList;
    Node* next_;
    int elem_;
};

class LinkedListIterator {
public:
    bool operator== (LinkedListIterator i) const;
    bool operator!= (LinkedListIterator i) const;
    void operator++ ();    // Go to the next element
```

```

    int& operator*  ();    // Access the current element
private:
    LinkedListIterator(Node* p);
    Node* p_;
    friend class LinkedList;    // so LinkedList can construct a LinkedListIterator
};

class LinkedList {
public:
    void append(int elem);    // Adds elem after the end
    void prepend(int elem);    // Adds elem before the beginning
    ...
    LinkedListIterator begin();
    LinkedListIterator end();
    ...
private:
    Node* first_;
};

```

Here are the methods that are obviously inlinable (probably in the same header file):

```

inline bool LinkedListIterator::operator==(LinkedListIterator i) const
{
    return p_ == i.p_;
}

inline bool LinkedListIterator::operator!=(LinkedListIterator i) const
{
    return p_ != i.p_;
}

inline void LinkedListIterator::operator++()
{
    assert(p_ != NULL);    //or if (p_==NULL) throw ...
    p_ = p_->next_;
}

inline int& LinkedListIterator::operator*()
{
    assert(p_ != NULL);    //or if (p_==NULL) throw ...
    return p_->elem_;
}

inline LinkedListIterator::LinkedListIterator(Node* p)
: p_(p)
{ }

inline LinkedListIterator LinkedList::begin()
{
    return first_;
}

inline LinkedListIterator LinkedList::end()
{
    return NULL;
}

```

Conclusion: The linked list had two different kinds of data. The values of the elements stored in the linked list are the responsibility of the user of the linked list (and *only* the user; the linked list itself makes no attempt to prohibit users from changing the third element to 5), and the linked

list's infrastructure data (*next* pointers, etc.), whose values are the responsibility of the linked list (and *only* the linked list; e.g., the linked list does not let users change (or even look at!) the various *next* pointers).

Thus the only `get()/set()` methods were to get and set the *elements* of the linked list, but not the infrastructure of the linked list. Since the linked list hides the infrastructure pointers/etc., it is able to make very strong promises regarding that infrastructure (e.g., if it were a doubly linked list, it might guarantee that every forward pointer was matched by a backwards pointer from the next node).

So, we see here an example of where the values of *some* of a class's data is the responsibility of *users* (in which case the class needs to have `get()/set()` methods for that data) but the data that the class wants to control does not necessarily have `get()/set()` methods.

Note: the purpose of this example is *not* to show you how to write a linked-list class. In fact you should *not* "roll your own" linked-list class since you should use one of the "container classes" provided with your compiler. Ideally you'll use one of the [standard container classes](#) such as the `std::list<T>` template.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.13] How can I overload the prefix and postfix forms of operators ++ and --?

Via a dummy parameter.

Since the prefix and postfix ++ operators can have two definitions, the C++ language gives us two different signatures. Both are called `operator++()`, but the prefix version takes no parameters and the postfix version takes a dummy `int`. (Although this discussion revolves around the ++ operator, the -- operator is completely symmetric, and all the rules and guidelines that apply to one also apply to the other.)

```
class Number {
public:
    Number& operator++ ();    // prefix ++
    Number operator++ (int); // postfix ++
};
```

Note the different return types: the prefix version returns by reference, the postfix version by value. If that's not immediately obvious to you, it should be after you see the definitions (and after you remember that `y = x++` and `y = ++x` set `y` to different things).

```
Number& Number::operator++ ()
{
    ...
    return *this;
}

Number Number::operator++ (int)
{
    Number ans = *this;
    ++(*this); // or just call operator++()
```

```
    return ans;
}
```

The other option for the postfix version is to return nothing:

```
class Number {
public:
    Number& operator++ ();
    void    operator++ (int);
};

Number& Number::operator++ ()
{
    ...
    return *this;
}

void Number::operator++ (int)
{
    ++(*this);  // or just call operator++()
}
```

However you must *not* make the postfix version return the 'this' object by reference; you have been warned.

Here's how you use these operators:

```
Number x = /* ... */;
++x;    // calls Number::operator++(), i.e., calls x.operator++()
x++;    // calls Number::operator++(int), i.e., calls x.operator++(0)
```

Assuming the return types are not 'void', you can use them in larger expressions:

```
Number x = /* ... */;
Number y = ++x;  // y will be the new value of x
Number z = x++;  // z will be the old value of x
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[13.14] Which is more efficient: `i++` or `++i`?

`++i` is sometimes faster than, and is never slower than, `i++`.

For intrinsic types like `int`, it doesn't matter: `++i` and `i++` are the same speed. For class types like iterators or the previous FAQ's `Number` class, `++i` very well might be faster than `i++` since the latter might make a copy of the `this` object.

The overhead of `i++`, if it is there at all, won't probably make any practical difference unless your app is CPU bound. For example, if your app spends most of its time waiting for someone to click a mouse, doing disk I/O, network I/O, or database queries, then it won't hurt your performance to waste a few CPU cycles. *However* it's just as easy to type `++i` as `i++`, so why not use the former unless you actually need the old value of `i`.

So if you're writing `i++` as a statement rather than as part of a larger expression, why not just write `++i` instead? You never lose anything, and you sometimes gain something. Old line C programmers are used to writing `i++` instead of `++i`. E.g., they'll say, `for (i = 0; i < 10; i++)` Since this uses `i++` as a statement, not as a part of a larger expression, then you might want to use `++i` instead. For symmetry, I personally advocate that style even when it doesn't improve speed, e.g., for intrinsic types and for class types with postfix operators that return `void`.

Obviously when `i++` appears as a part of a larger expression, that's different: it's being used because it's the only logically correct solution, not because it's an old habit you picked up while programming in C.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
[Download your own copy](#)]

Revised Feb 15, 2005

[14] Friends

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),
[cline@parashift.com](#))

FAQs in section [14]:

- [\[14.1\] What is a friend?](#)
- [\[14.2\] Do friends violate encapsulation?](#)
- [\[14.3\] What are some advantages/disadvantages of using friend functions?](#)
- [\[14.4\] What does it mean that "friendship isn't inherited, transitive, or reciprocal"?](#)
- [\[14.5\] Should my class declare a member function or a friend function?](#)

[14.1] What is a friend?

Something to allow your class to grant access to another class or function.

Friends can be either functions or other classes. A class grants access privileges to its friends. Normally a developer has political and technical control over both the `friend` and member functions of a class (else you may need to get permission from the owner of the other pieces when you want to update your own class).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[14.2] Do friends violate encapsulation?

No! If they're used properly, they *enhance* encapsulation.

You often need to split a class in half when the two halves will have different numbers of instances or different lifetimes. In these cases, the two halves usually need direct access to each other (the two halves *used* to be in the same class, so you haven't *increased* the amount of code that needs direct access to a data structure; you've simply *reshuffled* the code into two classes instead of one). The safest way to implement this is to make the two halves friends of each other.

If you use friends like just described, you'll keep `private` things `private`. People who don't understand this often make naive efforts to avoid using friendship in situations like the above, and often they actually destroy encapsulation. They either use `public` data (grotesque!), or they make the data accessible between the halves via `public` `get()` and `set()` member functions. Having a `public` `get()` and `set()` member function for a `private` datum is OK only when the `private` datum "makes sense" from outside the class (from a user's perspective). In many cases, these `get()/set()` member functions are almost as bad as `public` data: they hide (only) the *name* of the `private` datum, but they don't hide the existence of the `private` datum.

Similarly, if you use friend functions as a syntactic variant of a class's `public` access functions, they don't violate encapsulation any more than a member function violates encapsulation. In other words, a class's friends don't violate the encapsulation barrier: along with the class's member functions, they *are* the encapsulation barrier.

(Many people think of a friend function as something outside the class. Instead, try thinking of a friend function as part of the class's public interface. A friend function in the class declaration doesn't violate encapsulation any more than a public member function violates encapsulation: both have exactly the same authority with respect to accessing the class's non-public parts.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[14.3] What are some advantages/disadvantages of using `friend` functions?

They provide a degree of freedom in the interface design options.

Member functions and `friend` functions are equally privileged (100% vested). The major difference is that a `friend` function is called like `f(x)`, while a member function is called like `x.f()`. Thus the ability to choose between member functions (`x.f()`) and `friend` functions (`f(x)`) allows a designer to select the syntax that is deemed most readable, which lowers maintenance costs.

The major disadvantage of `friend` functions is that they require an extra line of code when you want dynamic binding. To get the effect of a `virtual` friend, the `friend` function should call a hidden (usually protected) `virtual` member function. This is called the [Virtual Friend Function Idiom](#). For example:

```
class Base {  
public:
```

```

    friend void f(Base& b);
    ...
protected:
    virtual void do_f();
    ...
};

inline void f(Base& b)
{
    b.do_f();
}

class Derived : public Base {
public:
    ...
protected:
    virtual void do_f();    // "Override" the behavior of f(Base& b)
    ...
};

void userCode(Base& b)
{
    f(b);
}

```

The statement `f(b)` in `userCode(Base&)` will invoke `b.do_f()`, which is [virtual](#). This means that `Derived::do_f()` will get control if `b` is actually a object of class `Derived`. Note that `Derived` overrides the behavior of the protected [virtual](#) member function `do_f()`; it does *not* have its own variation of the friend function, `f(Base&)`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[14.4] What does it mean that "friendship isn't inherited, transitive, or reciprocal"?

Just because I grant you friendship access to me doesn't automatically grant your kids access to me, doesn't automatically grant your friends access to me, and doesn't automatically grant me access to you.

- I don't necessarily trust the kids of my friends. The privileges of friendship aren't inherited. Derived classes of a friend aren't necessarily friends. If class `Fred` declares that class `Base` is a friend, classes derived from `Base` don't have any automatic special access rights to `Fred` objects.
- I don't necessarily trust the friends of my friends. The privileges of friendship aren't transitive. A friend of a friend isn't necessarily a friend. If class `Fred` declares class `Wilma` as a friend, and class `Wilma` declares class `Betty` as a friend, class `Betty` doesn't necessarily have any special access rights to `Fred` objects.
- You don't necessarily trust me simply because I declare you my friend. The privileges of friendship aren't reciprocal. If class `Fred` declares that class `Wilma` is a friend, `Wilma` objects have special access to `Fred` objects but `Fred` objects do not automatically have special access to `Wilma` objects.

[14.5] Should my class declare a member function or a friend function?

Use a member when you can, and a friend when you have to.

Sometimes friends are syntactically better (e.g., in `class Fred`, friend functions allow the `Fred` parameter to be second, while members require it to be first). Another good use of friend functions are the binary infix arithmetic operators. E.g., `aComplex + aComplex` should be defined as a friend rather than a member if you want to allow `aFloat + aComplex` as well (member functions don't allow promotion of the left hand argument, since that would change the class of the object that is the recipient of the member function invocation).

In other cases, choose a member function over a friend function.



[E-mail the author](#)

[15] Input/output via `<iostream>` and `<cstdio>`

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [15]:

- [\[15.1\] Why should I use `<iostream>` instead of the traditional `<cstdio>`?](#)
- [\[15.2\] Why does my program go into an infinite loop when someone enters an invalid input character?](#)
- [\[15.3\] How can I get `std::cin` to skip invalid input characters?](#)
- [\[15.4\] How does that funky `while \(std::cin >> foo\)` syntax work?](#)
- [\[15.5\] Why does my input seem to process past the end of file?](#)
- [\[15.6\] Why is my program ignoring my input request after the first iteration?](#)
- [\[15.7\] Should I end my output lines with `std::endl` or `'\n'`?](#)
- [\[15.8\] How can I provide printing for my `class Fred`?](#)
- [\[15.9\] But shouldn't I always use a `printon\(\)` method rather than a friend function?](#)
- [\[15.10\] How can I provide input for my `class Fred`?](#)
- [\[15.11\] How can I provide printing for an entire hierarchy of classes?](#)



- [\[15.12\] How can I open a stream in binary mode?](#)
- [\[15.13\] How can I "reopen" std::cin and std::cout in binary mode?](#)
- [\[15.14\] How can I write/read objects of my class to/from a data file?](#)
- [\[15.15\] How can I send objects of my class to another computer \(e.g., via a socket, TCP/IP, FTP, email, a wireless link, etc.\)?](#)
- [\[15.16\] Why can't I open a file in a different directory such as "..\test.dat"?](#)
- [\[15.17\] How can I tell {if a key, which key} was pressed before the user presses the ENTER key?](#)
- [\[15.18\] How can I make it so keys pressed by users are not echoed on the screen?](#)
- [\[15.19\] How can I move the cursor around on the screen?](#)
- [\[15.20\] How can I clear the screen? Is there something like clrscr\(\)?](#)
- [\[15.21\] How can I change the colors on the screen?](#)

[15.1] Why should I use <iostream> instead of the traditional <stdio>?

Increase type safety, reduce errors, allow extensibility, and provide inheritability.

printf() is arguably not broken, and scanf() is perhaps livable despite being error prone, however both are limited with respect to what C++ I/O can do. C++ I/O (using << and >>) is, relative to C (using printf() and scanf()):

- *More type-safe:* With <iostream>, the type of object being I/O'd is known statically by the compiler. In contrast, <stdio> uses "%" fields to figure out the types dynamically.
- *Less error prone:* With <iostream>, there are no redundant "%" tokens that have to be consistent with the actual objects being I/O'd. Removing redundancy removes a class of errors.
- *Extensible:* The C++ <iostream> mechanism allows new user-defined types to be I/O'd without breaking existing code. Imagine the chaos if everyone was simultaneously adding new incompatible "%" fields to printf() and scanf()?!
- *Inheritable:* The C++ <iostream> mechanism is built from real classes such as std::ostream and std::istream. Unlike <stdio>'s FILE*, these are real classes and hence inheritable. This means you can have other user-defined things that look and act like streams, yet that do whatever strange and wonderful things you want. You automatically get to use the zillions of lines of I/O code written by users you don't even know, and they don't need to know about your "extended stream" class.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.2] Why does my program go into an infinite loop when someone enters an invalid input character?

For example, suppose you have the following code that reads integers from std::cin:

```
#include <iostream>
```

```

int main()
{
    std::cout << "Enter numbers separated by whitespace (use -1 to quit): ";
    int i = 0;
    while (i != -1) {
        std::cin >> i;           // BAD FORM — See comments below
        std::cout << "You entered " << i << '\n';
    }
    ...
}

```

The problem with this code is that it lacks any checking to see if someone entered an invalid input character. In particular, if someone enters something that doesn't look like an integer (such as an 'x'), the stream `std::cin` goes into a "failed state," and all subsequent input attempts return immediately without doing anything. In other words, the program enters an infinite loop; if 42 was the last number that was successfully read, the program will print the message `You entered 42` over and over.

An easy way to check for invalid input is to move the input request from the body of the `while` loop into the control-expression of the `while` loop. E.g.,

```

#include <iostream>

int main()
{
    std::cout << "Enter a number, or -1 to quit: ";
    int i = 0;
    while (std::cin >> i) {      // GOOD FORM
        if (i == -1) break;
        std::cout << "You entered " << i << '\n';
    }
    ...
}

```

This will cause the `while` loop to exit either when you hit end -of-file, or when you enter a bad integer, or when you enter `-1`.

(Naturally you can eliminate the `break` by changing the `while` loop expression from `while (std::cin >> i)` to `while ((std::cin >> i) && (i != -1))`, but that's not really the point of this FAQ since this FAQ has to do with iostreams rather than generic structured programming guidelines.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.3] How can I get `std::cin` to skip invalid input characters?

Use `std::cin.clear()` and `std::cin.ignore()`.

```

#include <iostream>
#include <limits>

int main()
{

```

```

int age = 0;

while ((std::cout << "How old are you? ")
      && !(std::cin >> age)) {
    std::cout << "That's not a number; ";
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

std::cout << "You are " << age << " years old\n";
...
}

```

Of course you can also print the error message when the input is out of range. For example, if you wanted the age to be between 1 and 200, you could change the `while` loop to:

```

...
while ((std::cout << "How old are you? ")
      && (!(std::cin >> age) || age < 1 || age > 200)) {
    std::cout << "That's not a number between 1 and 200; ";
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
...

```

Here's a sample run:

```

How old are you? foo
That's not a number between 1 and 200; How old are you? bar
That's not a number between 1 and 200; How old are you? -3
That's not a number between 1 and 200; How old are you? 0
That's not a number between 1 and 200; How old are you? 201
That's not a number between 1 and 200; How old are you? 2
You are 2 years old

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.4] How does that funky `while (std::cin >> foo)` syntax work?

See the [previous FAQ](#) for an example of the "funky `while (std::cin >> foo)` syntax."

The expression `(std::cin >> foo)` calls the appropriate `operator>>` (for example, it calls the `operator>>` that takes an `std::istream` on the left and, if `foo` is of type `int`, an `int&` on the right). The `std::istream` `operator>>` functions return their left argument by convention, which in this case means it will return `std::cin`. Next the compiler notices that the returned `std::istream` is in a boolean context, so it converts that `std::istream` into a boolean.

To convert an `std::istream` into a boolean, the compiler calls a member function called `std::istream::operator void*()`. This returns a `void*` pointer, which is in turn converted to a boolean (`NULL` becomes `false`, any other pointer becomes `true`). So in this case the compiler generates a call to `std::cin.operator void*()`, just as if you had casted it explicitly such as `(void*) std::cin`.

The operator `void*()` cast operator returns some non-NULL pointer if the stream is in a good state, or NULL if it's in a failed state. For example, if you read one too many times (e.g., if you're already at end-of-file), or if the actual info on the input stream isn't valid for the type of `foo` (e.g., if `foo` is an `int` and the data is an 'x' character), the stream will go into a failed state and the cast operator will return NULL.

The reason `operator>>` doesn't simply return a `bool` (or `void*`) indicating whether it succeeded or failed is to support the "cascading" syntax:

```
std::cin >> foo >> bar;
```

The `operator>>` is left-associative, which means the above is parsed as:

```
(std::cin >> foo) >> bar;
```

In other words, if we replace `operator>>` with a normal function name such as `readFrom()`, this becomes the expression:

```
readFrom( readFrom(std::cin, foo), bar);
```

As always, we begin evaluating at the innermost expression. Because of the left-associativity of `operator>>`, this happens to be the left-most expression, `std::cin >> foo`. This expression returns `std::cin` (more precisely, it returns a reference to its left-hand argument) to the next expression. The next expression also returns (a reference to) `std::cin`, but this second reference is ignored since it's the outermost expression in this "expression statement."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.5] Why does my input seem to process past the end of file?

Because the eof state may not get set until after a read is attempted past the end of file. That is, reading the last byte from a file might not set the eof state. E.g., suppose the input stream is mapped to a keyboard — in that case it's not even theoretically possible for the C++ library to predict whether or not the character that the user just typed will be the last character.

For example, the following code might have an off-by-one error with the count `i`:

```
int i = 0;
while (! std::cin.eof()) {    // WRONG! (not reliable)
    std::cin >> x;
    ++i;
    // Work with X ...
}
```

What you really need is:

```
int i = 0;
while (std::cin >> x) {      // RIGHT! (reliable)
    ++i;
```

```
// Work with X ...  
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.6] Why is my program ignoring my input request after the first iteration?

Because the numerical extractor leaves non -digits behind in the input buffer.

If your code looks like this:

```
char name[1000];  
int age;  
  
for (;;) {  
    std::cout << "Name: ";  
    std::cin >> name;  
    std::cout << "Age: ";  
    std::cin >> age;  
}
```

What you really want is:

```
for (;;) {  
    std::cout << "Name: ";  
    std::cin >> name;  
    std::cout << "Age: ";  
    std::cin >> age;  
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
}
```

Of course you might want to change the `for (;;) {` statement to `while (std::cin)`, but don't confuse that with skipping the non-numeric characters at the end of the loop via the line : `std::cin.ignore(...);`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.7] Should I end my output line s with `std::endl` or `'\n'`?

Using `std::endl` flushes the output buffer after sending a `'\n'`, which means `std::endl` is more expensive in performance. Obviously if you need to flush the buffer after sending a `'\n'`, then use `std::endl`; but if you don't need to flush the buffer, the code will run faster if you use `'\n'`.

This code simply outputs a `'\n'`:

```
void f()  
{  
    std::cout << ...stuff... << '\n';  
}
```

This code outputs a '\n', then flushes the output buffer:

```
void g()
{
    std::cout << ...stuff... << std::endl;
}
```

This code simply flushes the output buffer:

```
void h()
{
    std::cout << ...stuff... << std::flush;
}
```

Note: all three of the above examples require `#include <iostream>`

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.8] How can I provide printing for my class Fred?

Use [operator overloading](#) to provide a [friend](#) left-shift operator, `operator<<`.

```
#include <iostream>

class Fred {
public:
    friend std::ostream& operator<< (std::ostream& o, const Fred& fred);
    ...
private:
    int i_;    // Just for illustration
};

std::ostream& operator<< (std::ostream& o, const Fred& fred)
{
    return o << fred.i_;
}

int main()
{
    Fred f;
    std::cout << "My Fred object: " << f << "\n";
    ...
}
```

We use a non-member function (a [friend](#) in this case) since the Fred object is the right-hand operand of the `<<` operator. If the Fred object was supposed to be on the left hand side of the `<<` (that is, `myFred << std::cout` rather than `std::cout << myFred`), we could have used a member function named `operator<<`.

Note that `operator<<` returns the stream. This is so the output operations can be [cascaded](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.9] But shouldn't I always use a `printOn()` method rather than a `friend` function?

No.

The usual reason people want to *always* use a `printOn()` method rather than a `friend` function is because they wrongly believe that friends violate encapsulation and/or that friends are evil. These beliefs are naive and wrong: when used properly, [friends can actually enhance encapsulation](#).

This is not to say that the `printOn()` method approach is *never* useful. For example, it is useful when [providing printing for an entire hierarchy of classes](#). But if you use a `printOn()` method, it should normally be `protected`, not `public`.

For completeness, here is "the `printOn()` method approach." The idea is to have a member function, often called `printOn()`, that does the actual printing, then have `operator<<` call that `printOn()` method. When it is done *wrongly*, the `printOn()` method is `public` so `operator<<` doesn't have to be a `friend` — it can be a simple top-level function that is neither a `friend` nor a member of the class. Here's some sample code:

```
#include <iostream>

class Fred {
public:
    void printOn(std::ostream& o) const;
    ...
};

// operator<< can be declared as a non-friend [NOT recommended!]
std::ostream& operator<< (std::ostream& o, const Fred& fred);

// The actual printing is done inside the printOn() method [NOT recommended!]
void Fred::printOn(std::ostream& o) const
{
    ...
}

// operator<< calls printOn() [NOT recommended!]
std::ostream& operator<< (std::ostream& o, const Fred& fred)
{
    fred.printOn(o);
    return o;
}
```

People wrongly assume that this reduces maintenance cost "since it avoids having a friend function." This is a wrong assumption because:

1. **The member-called-by-top-level-function approach has zero benefit in terms of maintenance cost.** Let's say it takes N lines of code to do the actual printing. In the case of a friend function, those N lines of code will have direct access to the class's private/protected parts, which means whenever someone changes the class's private/protected parts, those N lines of code will need to be scanned and possibly modified, which increases the maintenance cost. However using the `printOn()` method doesn't change this at all: we still have N lines of code that have direct access to the

class's private/protected parts. Thus moving the code from a `friend` function into a member function does *not* reduce the maintenance cost *at all*. Zero reduction. No benefit in maintenance cost. (If anything it's a bit worse with the `printOn()` method since you now have *more* lines of code to maintain since you have an extra function that you didn't have before.)

2. **The member-called-by-top-level-function approach makes the class harder to use, particularly by programmers who are not also class designers.** The approach exposes a public method that programmers are not supposed to call. When a programmer reads the public methods of the class, they'll see two ways to do the same thing. The documentation would need to say something like, "This does exactly the same as that, but don't use this; instead use that." And the average programmer will say, "Huh? Why make the method public if I'm not supposed to use it?" In reality the only reason the `printOn()` method is public is to avoid granting friendship status to `operator<<`, and that is a notion that is somewhere between subtle and incomprehensible to a programmer who simply wants to use the class.

Net: the member-called-by-top-level-function approach has a cost but no benefit. Therefore it is, in general, a bad idea.

Note: if the `printOn()` method is protected or private, the second objection doesn't apply. There are cases when that approach is reasonable, such as when [providing printing for an entire hierarchy of classes](#). Note also that when the `printOn()` method is non-public, `operator<<` needs to be a friend.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.10] How can I provide input for my `class Fred`?

Use [operator overloading](#) to provide a [friend](#) right-shift operator, `operator>>`. This is similar to the [output operator](#), except the parameter doesn't have a [const](#): "Fred&" rather than "const Fred&".

```
#include <iostream>

class Fred {
public:
    friend std::istream& operator>> (std::istream& i, Fred& fred);
    ...
private:
    int i_;    // Just for illustration
};

std::istream& operator>> (std::istream& i, Fred& fred)
{
    return i >> fred.i_;
}

int main()
{
    Fred f;
    std::cout << "Enter a Fred object: ";
    std::cin >> f;
```



```
...  
}
```

Note that `operator>>` returns the stream. This is so the input operations can be [cascaded and/or used in a while loop or if statement](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.11] How can I provide printing for an entire hierarchy of classes?

Provide a [friend operator<<](#) that calls a protected [virtual](#) function:

```
class Base {  
public:  
    friend std::ostream& operator<< (std::ostream& o, const Base& b);  
    ...  
protected:  
    virtual void printOn(std::ostream& o) const;  
};  
  
inline std::ostream& operator<< (std::ostream& o, const Base& b)  
{  
    b.printOn(o);  
    return o;  
}  
  
class Derived : public Base {  
protected:  
    virtual void printOn(std::ostream& o) const;  
};
```

The end result is that `operator<<` *acts* as if it were dynamically bound, even though it's a [friend](#) function. This is called the Virtual Friend Function Idiom.

Note that derived classes override `printOn(std::ostream&) const`. In particular, they do *not* provide their own `operator<<`.

Naturally if Base is an [ABC](#), `Base::printOn(std::ostream&) const` can be declared [pure virtual](#) using the `= 0` syntax.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.12] How can I open a stream in binary mode?

Use `std::ios::binary`.

Some operating systems differentiate between *text* and *binary* modes. In text mode, end-of-line sequences and possibly other things are translated; in binary mode, they are not. For example, in text mode under Windows, `"\r\n"` is translated into `"\n"` on input, and the reverse on output.

To read a file in binary mode, use something like this:

```
#include <string>
#include <iostream>
#include <fstream>

void readBinaryFile(const std::string& filename)
{
    std::ifstream input(filename.c_str(), std::ios::in | std::ios::binary);
    char c;
    while (input.get(c)) {
        ...do something with C here...
    }
}
```

Note: `input >> c` discards leading whitespace, so you won't normally use that when reading binary files.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.13] How can I "reopen" `std::cin` and `std::cout` in binary mode?

This is implementation dependent. Check with your compiler's documentation.

For example, suppose you want to do [binary I/O](#) using `std::cin` and `std::cout`.

Unfortunately there is no standard way to cause `std::cin`, `std::cout`, and/or `std::cerr` to be opened in binary mode. Closing the streams and attempting to reopen them in binary mode might have unexpected or undesirable results.

On systems [where it makes a difference](#), the implementation might provide a way to make them binary streams, but you would have to [check the implementation specifics](#) to find out.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.14] How can I write/read objects of my class to/from a data file?

Read the section on [object serialization](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.15] How can I send objects of my class to another computer (e.g., via a socket, TCP/IP, FTP, email, a wireless link, etc.)?

Read the section on [object serialization](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.16] Why can't I open a file in a different directory such as "..\test.dat"?

Because "\t" is a tab character.

You should use forward slashes in your filenames, even on operating systems that use backslashes (DOS, Windows, OS/2, etc.). For example:

```
#include <iostream>
#include <fstream>

int main()
{
    #if 1
        std::ifstream file("../test.dat"); // RIGHT!
    #else
        std::ifstream file("../\test.dat"); // WRONG!
    #endif

    ...
}
```

Remember, the backslash ("\") is used in string literals to create special characters: "\n" is a newline, "\b" is a backspace, and "\t" is a tab, "\a" is an "alert", "\v" is a vertical-tab, etc. Therefore the file name "\version\next\alpha\beta\test.dat" is interpreted as a bunch of very funny characters. To be safe, use "/version/next/alpha/beta/test.dat" instead, even on systems that use a "\" as the directory separator. This is because the library routines on these operating systems handle "/" and "\" interchangeably.

Of course you *could* use "\\version\\next\\alpha\\beta\\test.dat", but that might hurt you (there's a non-zero chance you'll forget one of the "\", a rather subtle bug since most people don't notice it) and it can't help you (there's no benefit for using "\\" over "/"). Besides "/" is more portable since it works on all flavors of Unix, Plan 9, Inferno, all Windows, OS/2, etc., but "\\" works only on a subset of that list. So "\\" costs you something and gains you nothing: use "/" instead.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.17] How can I tell {if a key, which key} was pressed before the user presses the ENTER key?

This is not a standard C++ feature — C++ doesn't even require your system to *have a keyboard!*. That means every operating system and vendor does it somewhat differently.

Please read the documentation that came with your compiler for details on your particular installation.

(By the way, the process on UNIX typically has two steps: first [set the terminal to single-character mode](#), then use either `select()` or `poll()` to test if a key was pressed. You might be able to adapt [this code](#).)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.18] How can I make it so keys pressed by users are not echoed on the screen?

This is not a standard C++ feature — C++ doesn't even require your system to have a keyboard or a screen. That means every operating system and vendor does it somewhat differently.

Please read the documentation that came with your compiler for details on your particular installation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.19] How can I move the cursor around on the screen?

This is not a standard C++ feature — C++ doesn't even require your system to have a screen. That means every operating system and vendor does it somewhat differently.

Please read the documentation that came with your compiler for details on your particular installation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.20] How can I clear the screen? Is there something like `clrscr()`?

This is not a standard C++ feature — C++ doesn't even require your system to have a screen. That means every operating system and vendor does it somewhat differently.

Please read the documentation that came with your compiler for details on your particular installation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[15.21] How can I change the colors on the screen?

This is not a standard C++ feature — C++ doesn't even require your system to have a screen. That means every operating system and vendor does it somewhat differently.

Please read the documentation that came with your compiler for details on your particular installation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | ©
| [Download your own copy](#)]

Revised Feb 15, 2005

[16] Freestore management

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [16]:

- [\[16.1\] Does delete p delete the pointer p, or the pointed-to-data *p?](#)
- [\[16.2\] Can I free\(\) pointers allocated with new? Can I delete pointers allocated with malloc\(\)?](#)
- [\[16.3\] Why should I use new instead of trustworthy old malloc\(\)?](#)
- [\[16.4\] Can I use realloc\(\) on pointers allocated via new?](#)
- [\[16.5\] Do I need to check for NULL after p = new Fred\(\)?](#)
- [\[16.6\] How can I convince my \(older\) compiler to automatically check new to see if it returns NULL?](#)
- [\[16.7\] Do I need to check for NULL before delete p?](#)
- [\[16.8\] What are the two steps that happen when I say delete p?](#)
- [\[16.9\] In p = new Fred\(\), does the Fred memory "leak" if the Fred constructor throws an exception?](#)
- [\[16.10\] How do I allocate / unallocate an array of things?](#)
- [\[16.11\] What if I forget the \[\] when deleting array allocated via new T\[n\]?](#)
- [\[16.12\] Can I drop the \[\] when deleting array of some built-in type \(char, int, etc\)?](#)
- [\[16.13\] After p = new Fred\[n\], how does the compiler know there are n objects to be destructed during delete\[\] p?](#)
- [\[16.14\] Is it legal \(and moral\) for a member function to say delete this?](#)
- [\[16.15\] How do I allocate multidimensional arrays using new?](#)
- [\[16.16\] But the previous FAQ's code is SOOOO tricky and error prone! Isn't there a simpler way?](#)
- [\[16.17\] But the above Matrix class is specific to Fred! Isn't there a way to make it generic?](#)
- [\[16.18\] What's another way to build a Matrix template?](#) 

- [\[16.19\] Does C++ have arrays whose length can be specified at run-time?](#)
- [\[16.20\] How can I force objects of my class to always be created via `new` rather than as locals or global/static objects?](#)
- [\[16.21\] How do I do simple reference counting?](#) **UPDATED!**
- [\[16.22\] How do I provide reference counting with copy-on-write semantics?](#)
- [\[16.23\] How do I provide reference counting with copy-on-write semantics for a hierarchy of classes?](#) **UPDATED!**
- [\[16.24\] Can you absolutely *prevent* people from subverting the reference counting mechanism, and if so, *should* you?](#)
- [\[16.25\] Can I use a garbage collector in C++?](#)
- [\[16.26\] What are the two kinds of garbage collectors for C++?](#)
- [\[16.27\] Where can I get more info on garbage collectors for C++?](#)

[16.1] Does `delete p` delete the pointer `p`, or the pointed-to-data `*p`?

The pointed-to-data.

The keyword should really be `delete_the_thing_pointed_to_by`. The same abuse of English occurs when freeing the memory pointed to by a pointer in C: `free(p)` really means `free_the_stuff_pointed_to_by(p)`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.2] Can I `free()` pointers allocated with `new`? Can I `delete` pointers allocated with `malloc()`?

No!

It is perfectly legal, moral, and wholesome to use `malloc()` and `delete` in the same program, or to use `new` and `free()` in the same program. *But* it is illegal, immoral, and despicable to call `free()` with a pointer allocated via `new`, or to call `delete` on a pointer allocated via `malloc()`.

Beware! I occasionally get e-mail from people telling me that it works OK for them on machine X and compiler Y. Just because they don't see bad symptoms in a simple test case doesn't mean it won't crash in the field. Even if they *know* it won't crash on their particular compiler doesn't mean it will work safely on another compiler, another platform, or even another version of the same compiler.

Beware! Sometimes people say, "But I'm just working with an array of `char`." Nonetheless do *not* mix `malloc()` and `delete` on the same pointer, or `new` and `free()` on the same pointer! If you allocated via `p = new char[n]`, you *must* use `delete[] p`; you must *not* use `free(p)`. Or if you allocated via `p = malloc(n)`, you *must* use `free(p)`; you must *not* use `delete[] p` or `delete p`! Mixing these up could cause a catastrophic failure at runtime if the code was ported to a new machine, a new compiler, or even a new version of the same compiler.

You have been warned.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.3] Why should I use `new` instead of trustworthy old `malloc()` ?

Constructors/destructors, type safety, overridability.

- Constructors/destructors: unlike `malloc(sizeof(Fred))`, `new Fred()` calls Fred's constructor. Similarly, `delete p` calls `*p`'s destructor.
- Type safety: `malloc()` returns a `void*` which isn't type safe. `new Fred()` returns a pointer of the right type (a `Fred*`).
- Overridability: `new` is an operator that can be overridden by a class, while `malloc()` is not overridable on a per-class basis.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.4] Can I use `realloc()` on pointers allocated via `new`?

No!

When `realloc()` has to copy the allocation, it uses a *bitwise* copy operation, which will tear many C++ objects to shreds. C++ objects should be allowed to copy themselves. They use their own copy constructor or assignment operator.

Besides all that, the heap that `new` uses may *not* be the same as the heap that `malloc()` and `realloc()` use!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.5] Do I need to check for `NULL` after `p = new Fred()` ?

No! (But if you have an old compiler, you may have to [force the new operator to throw an exception if it runs out of memory](#).)

It turns out to be a real pain to always write explicit `NULL` tests after every `new` allocation. Code like the following is very tedious:

```
Fred* p = new Fred();
if (p == NULL)
    throw std::bad_alloc();
```

If your compiler doesn't support (or if you refuse to use) [exceptions](#), your code might be even more tedious:

```
Fred* p = new Fred();
if (p == NULL) {
    std::cerr << "Couldn't allocate memory for a Fred" << std::endl;
    abort();
}
```

Take heart. In C++, if the runtime system cannot allocate `sizeof(Fred)` bytes of memory during `p = new Fred()`, a `std::bad_alloc` exception will be thrown. Unlike `malloc()`, `new` *never* returns `NULL`!

Therefore you should simply write:

```
Fred* p = new Fred();    // No need to check if p is NULL
```

However, if your compiler is old, it may not yet support this. Find out by checking your compiler's documentation under "new". If you have an old compiler, you may have to [force the compiler to have this behavior](#).

Note: If you are using Microsoft Visual C++, to get `new` to throw an exception when it fails [you must #include some standard header in at least one of your .cpp files](#). For example, you could `#include <new>` (or `<iostream>` or `<string>` or ...).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.6] How can I convince my (older) compiler to automatically check `new` to see if it returns `NULL`?

Eventually your compiler will.

If you have an old compiler that doesn't automatically perform [the NULL test](#), you can force the runtime system to do the test by installing a "new handler" function. Your "new handler" function can do anything you want, such as throw an exception, delete some objects and return (in which case operator `new` will retry the allocation), print a message and `abort()` the program, etc.

Here's a sample "new handler" that prints a message and throws an exception. The handler is installed using `std::set_new_handler()`:

```
#include <new>           // To get std::set_new_handler
#include <cstdlib>        // To get abort()
#include <iostream>       // To get std::cerr

class alloc_error : public std::exception {
public:
    alloc_error() : exception() { }
};

void myNewHandler()
{
    // This is your own handler. It can do anything you want.
    throw alloc_error();
}
```



```

}

int main()
{
    std::set_new_handler(myNewHandler);    // Install your "new handler"
    ...
}

```

After the `std::set_new_handler()` line is executed, operator `new` will call your `myNewHandler()` if/when it runs out of memory. This means that `new` will never return `NULL`:

```

Fred* p = new Fred();    // No need to check if p is NULL

```

Note: If your compiler doesn't support [exception handling](#), you can, as a last resort, change the line `throw ...;` to:

```

std::cerr << "Attempt to allocate memory failed!" << std::endl;
abort();

```

Note: If some global/static object's constructor uses `new`, it won't use the `myNewHandler()` function since that constructor will get called before `main()` begins. Unfortunately there's no convenient way to guarantee that the `std::set_new_handler()` will be called before the first use of `new`. For example, even if you put the `std::set_new_handler()` call in the constructor of a global object, you still don't know if the module ("compilation unit") that contains that global object will be elaborated first or last or somewhere inbetween. Therefore you still don't have any guarantee that your call of `std::set_new_handler()` will happen before any other global's constructor gets invoked.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.7] Do I need to check for `NULL` before `delete p`?

No!

The C++ language guarantees that `delete p` will do nothing if `p` is equal to `NULL`. Since you might get the test backwards, and since most testing methodologies force you to explicitly test every branch point, you should *not* put in the redundant `if` test.

Wrong:

```

if (p != NULL)
    delete p;

```

Right:

```

delete p;

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.8] What are the two steps that happen when I say `delete p`?

`delete p` is a two-step process: it calls the destructor, then releases the memory. The code generated for `delete p` is functionally similar to this (assuming `p` is of type `Fred*`):

```
// Original code: delete p;
if (p != NULL) {
    p->~Fred();
    operator delete(p);
}
```

The statement `p->~Fred()` calls the destructor for the `Fred` object pointed to by `p`.

The statement `operator delete(p)` calls the memory deallocation primitive, `void operator delete(void* p)`. This primitive is similar in spirit to `free(void* p)`. (Note, however, that these two are *not* interchangeable; e.g., there is no guarantee that the two memory deallocation primitives even use the same heap!)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.9] In `p = new Fred()`, does the `Fred` memory "leak" if the `Fred` constructor throws an exception?

No.

If an exception occurs during the `Fred` constructor of `p = new Fred()`, the C++ language guarantees that the memory `sizeof(Fred)` bytes that were allocated will automatically be released back to the heap.

Here are the details: `new Fred()` is a two-step process:

1. `sizeof(Fred)` bytes of memory are allocated using the primitive `void* operator new(size_t nbytes)`. This primitive is similar in spirit to `malloc(size_t nbytes)`. (Note, however, that these two are *not* interchangeable; e.g., there is no guarantee that the two memory allocation primitives even use the same heap!).
2. It constructs an object in that memory by calling the `Fred` constructor. The pointer returned from the first step is passed as the `this` parameter to the constructor. This step is wrapped in a `try ... catch` block to handle the case when an exception is thrown during this step.

Thus the actual generated code is functionally similar to:

```
// Original code: Fred* p = new Fred();
Fred* p = (Fred*) operator new(sizeof(Fred));
try {
    new(p) Fred();           // Placement new
} catch (...) {
    operator delete(p);     // Deallocate the memory
    throw;                  // Re-throw the exception
}
```

The statement marked "[Placement new](#)" calls the `Fred` constructor. The pointer `p` becomes the this pointer inside the constructor, `Fred::Fred()`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.10] How do I allocate / unallocate an array of things?

Use `p = new T[n]` and `delete[] p`:

```
Fred* p = new Fred[100];  
...  
delete[] p;
```

Any time you allocate an array of objects via `new` (usually with the `[n]` in the `new` expression), you *must* use `[]` in the `delete` statement. This syntax is necessary because there is no syntactic difference between a pointer to a thing and a pointer to an array of things (something we inherited from C).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.11] What if I forget the `[]` when deleting array allocated via `new T[n]`?

All life comes to a catastrophic end.

It is the programmer's —not the compiler's— responsibility to get the connection between `new T[n]` and `delete[] p` correct. If you get it wrong, neither a compile-time nor a run-time error message will be generated by the compiler. Heap corruption is a likely result. Or worse. Your program will probably die.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.12] Can I drop the `[]` when deleting array of some built-in type (`char`, `int`, etc)?

No!

Sometimes programmers think that the `[]` in the `delete[] p` only exists so the compiler will call the appropriate destructors for all elements in the array. Because of this reasoning, they assume that an array of some built-in type such as `char` or `int` can be deleted without the `[]`. E.g., they assume the following is valid code:

```
void userCode(int n)  
{  
    char* p = new char[n];  
    ...  
}
```

```
delete p;      // ← ERROR! Should be delete[] p!  
}
```

But the above code is wrong, and it can cause a disaster at runtime. In particular, the code that's called for `delete p` is operator `delete(void*)`, but the code that's called for `delete[] p` is operator `delete[](void*)`. The default behavior for the latter is to call the former, but users are allowed to replace the latter with a different behavior (in which case they would normally also replace the corresponding `new` code in operator `new[](size_t)`). If they replaced the `delete[]` code so it wasn't compatible with the `delete` code, and you called the wrong one (i.e., if you said `delete p` rather than `delete[] p`), you could end up with a disaster at runtime.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.13] After `p = new Fred[n]`, how does the compiler know there are `n` objects to be destructed during `delete[] p`?

Short answer: Magic.

Long answer: The run-time system stores the number of objects, `n`, somewhere where it can be retrieved if you only know the pointer, `p`. There are two popular techniques that do this. Both these techniques are in use by commercial-grade compilers, both have tradeoffs, and neither is perfect. These techniques are:

- [Over-allocate the array and put `n` just to the left of the first `Fred` object.](#)
- [Use an associative array with `p` as the key and `n` as the value.](#)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.14] Is it legal (and moral) for a member function to say `delete this`?

As long as you're careful, it's OK for an object to commit suicide (`delete this`).

Here's how I define "careful":

1. You must be absolutely 100% positive sure that `this` object was allocated via `new` (not by `new[]`, nor by [placement new](#), nor a local object on the stack, nor a global, nor a member of another object; but by plain ordinary `new`).
2. You must be absolutely 100% positive sure that your member function will be the last member function invoked on `this` object.
3. You must be absolutely 100% positive sure that the rest of your member function (after the `delete this` line) doesn't touch any piece of `this` object (including calling any other member functions or touching any data members).
4. You must be absolutely 100% positive sure that no one even touches the `this` pointer itself after the `delete this` line. In other words, you must not examine it, compare it with another pointer, compare it with `NULL`, print it, cast it, do anything with it.

Naturally the usual caveats apply in cases where your `this` pointer is a pointer to a base class when you don't have a [virtual destructor](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.15] How do I allocate multidimensional arrays using `new`?

There are many ways to do this, depending on how flexible you want the array sizing to be. On one extreme, if you know all the dimensions at compile-time, you can allocate multidimensional arrays statically (as in C):

```
class Fred { /*...*/ };
void someFunction(Fred& fred);

void manipulateArray()
{
    const unsigned nrows = 10;    // Num rows is a compile-time constant
    const unsigned ncols = 20;    // Num columns is a compile-time constant
    Fred matrix[nrows][ncols];

    for (unsigned i = 0; i < nrows; ++i) {
        for (unsigned j = 0; j < ncols; ++j) {
            // Here's the way you access the (i, j) element:
            someFunction( matrix[i][j] );

            // You can safely "return" without any special delete code:
            if (today == "Tuesday" && moon.isFull())
                return;    // Quit early on Tuesdays when the moon is full
        }
    }

    // No explicit delete code at the end of the function either
}
```

More commonly, the size of the matrix isn't known until run-time but you know that it will be rectangular. In this case you need to use the heap ("freestore"), but at least you are able to allocate all the elements in one freestore chunk.

```
void manipulateArray(unsigned nrows, unsigned ncols)
{
    Fred* matrix = new Fred[nrows * ncols];

    // Since we used a simple pointer above, we need to be VERY
    // careful to avoid skipping over the delete code.
    // That's why we catch all exceptions:
    try {

        // Here's how to access the (i, j) element:
        for (unsigned i = 0; i < nrows; ++i) {
            for (unsigned j = 0; j < ncols; ++j) {
                someFunction( matrix[i*ncols + j] );
            }
        }

        // If you want to quit early on Tuesdays when the moon is full,
```

```

// make sure to do the delete along ALL return paths:
if (today == "Tuesday" && moon.isFull()) {
    delete[] matrix;
    return;
}

...insert code here to fiddle with the matrix...

}
catch (...) {
    // Make sure to do the delete when an exception is thrown:
    delete[] matrix;
    throw;    // Re-throw the current exception
}

// Make sure to do the delete at the end of the function too:
delete[] matrix;
}

```

Finally at the other extreme, you may not even be guaranteed that the matrix is rectangular. For example, if each row could have a different length, you'll need to allocate each row individually. In the following function, `ncols[i]` is the number of columns in row number `i`, where `i` varies between 0 and `nrows-1` inclusive.

```

void manipulateArray(unsigned nrows, unsigned ncols[])
{
    typedef Fred* FredPtr;

    // There will not be a leak if the following throws an exception:
    FredPtr* matrix = new FredPtr[nrows];

    // Set each element to NULL in case there is an exception later.
    // (See comments at the top of the try block for rationale.)
    for (unsigned i = 0; i < nrows; ++i)
        matrix[i] = NULL;

    // Since we used a simple pointer above, we need to be
    // VERY careful to avoid skipping over the delete code.
    // That's why we catch all exceptions:
    try {

        // Next we populate the array. If one of these throws, all
        // the allocated elements will be deleted (see catch below).
        for (unsigned i = 0; i < nrows; ++i)
            matrix[i] = new Fred[ ncols[i] ];

        // Here's how to access the (i, j) element:
        for (unsigned i = 0; i < nrows; ++i) {
            for (unsigned j = 0; j < ncols[i]; ++j) {
                someFunction( matrix[i][j] );
            }
        }

        // If you want to quit early on Tuesdays when the moon is full,
        // make sure to do the delete along ALL return paths:
        if (today == "Tuesday" && moon.isFull()) {
            for (unsigned i = nrows; i > 0; --i)
                delete[] matrix[i-1];
        }
    }
}

```

```

        delete[] matrix;
        return;
    }

    ...insert code here to fiddle with the matrix...

}
catch (...) {
    // Make sure to do the delete when an exception is thrown:
    // Note that some of these matrix[...] pointers might be
    // NULL, but that's okay since it's legal to delete NULL.
    for (unsigned i = nrows; i > 0; --i)
        delete[] matrix[i-1];
    delete[] matrix;
    throw;    // Re-throw the current exception
}

// Make sure to do the delete at the end of the function too.
// Note that deletion is the opposite order of allocation:
for (unsigned i = nrows; i > 0; --i)
    delete[] matrix[i-1];
delete[] matrix;
}

```

Note the funny use of `matrix[i-1]` in the deletion process. This prevents wrap-around of the unsigned value when `i` goes one step below zero.

Finally, note that [pointers and arrays are evil](#). It is normally much better to encapsulate your pointers in a class that has a safe and simple interface. [The following FAQ](#) shows how to do this.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.16] But the previous FAQ's code is SOOOO tricky and error prone! Isn't there a simpler way?

Yep.

The reason the code in [the previous FAQ](#) was so tricky and error prone was that it used pointers, and we know that [pointers and arrays are evil](#). The solution is to encapsulate your pointers in a class that has a safe and simple interface. For example, we can define a `Matrix` class that handles a rectangular matrix so our user code will be vastly simplified when compared to the [the rectangular matrix code from the previous FAQ](#):

```

// The code for class Matrix is shown below...
void someFunction(Fred& fred);

void manipulateArray(unsigned nrows, unsigned ncols)
{
    Matrix matrix(nrows, ncols);    // Construct a Matrix called matrix

    for (unsigned i = 0; i < nrows; ++i) {
        for (unsigned j = 0; j < ncols; ++j) {
            // Here's the way you access the (i, j) element:

```

```

        someFunction( matrix(i,j) );

        // You can safely "return" without any special delete code:
        if (today == "Tuesday" && moon.isFull())
            return;        // Quit early on Tuesdays when the moon is full
    }
}

// No explicit delete code at the end of the function either
}

```

The main thing to notice is the lack of clean -up code. For example, there aren't any `delete` statements in the above code, yet there will be *no* memory leaks, assuming only that the `Matrix` destructor does its job correctly.

Here's the `Matrix` code that makes the above possible:

```

class Matrix {
public:
    Matrix(unsigned nrows, unsigned ncols);
    // Throws a BadSize object if either size is zero
    class BadSize { };

    // Based on the Law Of The Big Three:
    ~Matrix();
    Matrix(const Matrix& m);
    Matrix& operator= (const Matrix& m);

    // Access methods to get the (i, j) element:
    Fred& operator() (unsigned i, unsigned j);
    const Fred& operator() (unsigned i, unsigned j) const;
    // These throw a BoundsViolation object if i or j is too big
    class BoundsViolation { };

private:
    unsigned nrows_, ncols_;
    Fred* data_;
};

inline Fred& Matrix::operator() (unsigned row, unsigned col)
{
    if (row >= nrows_ || col >= ncols_) throw BoundsViolation();
    return data_[row*ncols_ + col];
}

inline const Fred& Matrix::operator() (unsigned row, unsigned col) const
{
    if (row >= nrows_ || col >= ncols_) throw BoundsViolation();
    return data_[row*ncols_ + col];
}

Matrix::Matrix(unsigned nrows, unsigned ncols)
    : nrows_ (nrows)
    , ncols_ (ncols)
    //, data_ <--initialized below (after the 'if/throw' statement)
    {
    if (nrows == 0 || ncols == 0)
        throw BadSize();
    data_ = new Fred[nrows * ncols];
}

```



```

}

Matrix::~~Matrix()
{
    delete[] data_;
}

```

Note that the above `Matrix` class accomplishes two things: it moves some tricky memory management code from the user code (e.g., `main()`) to the class, and it reduces the overall bulk of program. The latter point is important. For example, assuming `Matrix` is even mildly reusable, moving complexity from the users [plural] of `Matrix` into `Matrix` itself [singular] is equivalent to moving complexity from the many to the few. Anyone who's seen *Star Trek 2* knows that the good of the many outweighs the good of the few... or the one.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.17] But the above `Matrix` class is specific to `Fred`! Isn't there a way to make it generic?

Yep; just use [templates](#):

Here's how this can be used:

```

#include "Fred.hpp"          // To get the definition for class Fred

// The code for Matrix<T> is shown below...
void someFunction(Fred& fred);

void manipulateArray(unsigned nrows, unsigned ncols)
{
    Matrix<Fred> matrix(nrows, ncols);    // Construct a Matrix<Fred> called matrix

    for (unsigned i = 0; i < nrows; ++i) {
        for (unsigned j = 0; j < ncols; ++j) {
            // Here's the way you access the (i, j) element:
            someFunction( matrix(i,j) );

            // You can safely "return" without any special delete code:
            if (today == "Tuesday" && moon.isFull())
                return;          // Quit early on Tuesdays when the moon is full
        }
    }

    // No explicit delete code at the end of the function either
}

```

Now it's easy to use `Matrix<T>` for things other than `Fred`. For example, the following uses a `Matrix` of `std::string` (where `std::string` is the standard string class):

```

#include <string>

void someFunction(std::string& s);

```

```

void manipulateArray(unsigned nrows, unsigned ncols)
{
    Matrix<std::string> matrix(nrows, ncols);    // Construct a Matrix<std::string>

    for (unsigned i = 0; i < nrows; ++i) {
        for (unsigned j = 0; j < ncols; ++j) {
            // Here's the way you access the (i, j) element:
            someFunction( matrix(i,j) );

            // You can safely "return" without any special delete code:
            if (today == "Tuesday" && moon.isFull())
                return;    // Quit early on Tuesdays when the moon is full
        }
    }

    // No explicit delete code at the end of the function either
}

```

You can thus get an entire *family* of classes from a [template](#). For example, `Matrix<Fred>`, `Matrix<std::string>`, `Matrix< Matrix<std::string> >`, etc.

Here's one way that the [template](#) can be implemented:

```

template<typename T>    // See section on templates for more
class Matrix {
public:
    Matrix(unsigned nrows, unsigned ncols);
    // Throws a BadSize object if either size is zero
    class BadSize { };

    // Based on the Law Of The Big Three:
    ~Matrix();
    Matrix(const Matrix<T>& m);
    Matrix<T>& operator= (const Matrix<T>& m);

    // Access methods to get the (i, j) element:
    T& operator() (unsigned i, unsigned j);
    const T& operator() (unsigned i, unsigned j) const;
    // These throw a BoundsViolation object if i or j is too big
    class BoundsViolation { };

private:
    unsigned nrows_, ncols_;
    T* data_;
};

template<typename T>
inline T& Matrix<T>::operator() (unsigned row, unsigned col)
{
    if (row >= nrows_ || col >= ncols_) throw BoundsViolation();
    return data_[row*ncols_ + col];
}

template<typename T>
inline const T& Matrix<T>::operator() (unsigned row, unsigned col) const
{
    if (row >= nrows_ || col >= ncols_)
        throw BoundsViolation();
    return data_[row*ncols_ + col];
}

```

```

}

template<typename T>
inline Matrix<T>::Matrix(unsigned nrows, unsigned ncols)
    : nrows_ (nrows)
    , ncols_ (ncols)
    //, data_ <--initialized below (after the 'if/throw' statement)
{
    if (nrows == 0 || ncols == 0)
        throw BadSize();
    data_ = new T[nrows * ncols];
}

template<typename T>
inline Matrix<T>::~~Matrix()
{
    delete[] data_;
}

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.18] What's another way to build a Matrix template? UPDATED!

[Recently added some pros/cons (mostly pros) thanks to a suggestion from [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Use the standard vector template, and make a vector of vector.

The following uses a `std::vector<std::vector<T> >` (note the space between the two > symbols).

```

#include <vector>

template<typename T> // See section on templates for more
class Matrix {
public:
    Matrix(unsigned nrows, unsigned ncols);
    // Throws a BadSize object if either size is zero
    class BadSize { };

    // No need for any of The Big Three!

    // Access methods to get the (i, j) element:
    T& operator() (unsigned i, unsigned j);
    const T& operator() (unsigned i, unsigned j) const;
    // These throw a BoundsViolation object if i or j is too big
    class BoundsViolation { };

    unsigned nrows() const; // #rows in this matrix
    unsigned ncols() const; // #columns in this matrix

private:
    std::vector<std::vector<T> > data_;
};

template<typename T>

```

```

inline unsigned Matrix<T>::nrows() const
{ return data_.size(); }

template<typename T>
inline unsigned Matrix<T>::ncols() const
{ return data_[0].size(); }

template<typename T>
inline T& Matrix<T>::operator() (unsigned row, unsigned col)
{
    if (row >= nrows() || col >= ncols()) throw BoundsViolation();
    return data_[row][col];
}

template<typename T>
inline const T& Matrix<T>::operator() (unsigned row, unsigned col) const
{
    if (row >= nrows() || col >= ncols()) throw BoundsViolation();
    return data_[row][col];
}

template<typename T>
Matrix<T>::Matrix(unsigned nrows, unsigned ncols)
    : data_ (nrows)
{
    if (nrows == 0 || ncols == 0)
        throw BadSize();
    for (unsigned i = 0; i < nrows; ++i)
        data_[i].resize(ncols);
}

```

Note how much simpler this is than [the previous](#): there is no explicit `new` in the constructor, and there is no need for any of [The Big Three](#) (destructor, copy constructor or assignment operator). Simply put, your code is a *lot* less likely to have memory leaks if you use `std::vector` than if you use explicit `new T[n]` and `delete[] p`.

Note also that `std::vector` doesn't force you to allocate numerous chunks of memory. If you prefer to allocate only one chunk of memory for the entire matrix, [as was done in the previous](#), just change the type of `data_` to `std::vector<T>` and add member variables `nrows_` and `ncols_`. You'll figure out the rest: initialize `data_` using `data_(nrows * ncols)`, change `operator()()` to `return data_[row*ncols_ + col];`, etc.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.19] Does C++ have arrays whose length can be specified at run -time?

Yes, in the sense that [the standard library](#) has a `std::vector` template that provides this behavior.

No, in the sense that built-in array types need to have their length specified at compile time.

Yes, in the sense that even built-in array types can specify the first index bounds at run -time. E.g., comparing with the previous FAQ, if you only need the first array dimension to vary then you can just ask `new` for an array of arrays, rather than an array of pointers to arrays:

```
const unsigned ncols = 100;           // ncols = number of columns in the array

class Fred { /*...*/ };

void manipulateArray(unsigned nrows) // nrows = number of rows in the array
{
    Fred (*matrix)[ncols] = new Fred[nrows][ncols];
    ...
    delete[] matrix;
}
```

You can't do this if you need anything other than the first dimension of the array to change at run-time.

But please, don't use arrays unless you have to. [Arrays are evil](#). Use some object of some class if you can. Use arrays only when you have to.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.20] How can I force objects of my class to always be created via `new` rather than as locals or global/static objects?

Use the [Named Constructor Idiom](#).

As usual with the Named Constructor Idiom, the constructors are all `private` or `protected`, and there are one or more public `create()` methods (the so-called "named constructors"), one per constructor. In this case the `create()` methods allocate the objects via `new`. Since the constructors themselves are not `public`, there is no other way to create objects of the class.

```
class Fred {
public:
    // The create() methods are the "named constructors":
    static Fred* create()           { return new Fred();      }
    static Fred* create(int i)      { return new Fred(i);    }
    static Fred* create(const Fred& fred) { return new Fred(fred); }
    ...

private:
    // The constructors themselves are private or protected:
    Fred();
    Fred(int i);
    Fred(const Fred& fred);
    ...
};
```

Now the only way to create `Fred` objects is via `Fred::create()`:

```
int main()
{
    Fred* p = Fred::create(5);
    ...
    delete p;
}
```

```
...  
}
```

Make sure your constructors are in the `protected` section if you expect `Fred` to have derived classes.

Note also that you can make another class `wilma` a [friend](#) of `Fred` if you want to allow a `wilma` to have a member object of class `Fred`, but of course this is a softening of the original goal, namely to force `Fred` objects to be allocated via `new`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.21] How do I do simple reference counting? UPDATED!

[Recently changed `friend FredPtr;` to `friend class FredPtr;` thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

If all you want is the ability to pass around a bunch of pointers to the same object, with the feature that the object will automatically get deleted when the last pointer to it disappears, you can use something like the following "smart pointer" class:

```
// Fred.h  
  
class FredPtr;  
  
class Fred {  
public:  
    Fred() : count_(0) /*...*/ { } // All ctors set count_ to 0!  
    ...  
private:  
    friend class FredPtr; // A friend class  
    unsigned count_;  
    // count_ must be initialized to 0 by all constructors  
    // count_ is the number of FredPtr objects that point at this  
};  
  
class FredPtr {  
public:  
    Fred* operator-> () { return p_; }  
    Fred& operator* () { return *p_; }  
    FredPtr(Fred* p) : p_(p) { ++p->count_; } // p must not be NULL  
    ~FredPtr() { if (--p->count_ == 0) delete p_; }  
    FredPtr(const FredPtr& p) : p_(p.p_) { ++p->count_; }  
    FredPtr& operator= (const FredPtr& p)  
    { // DO NOT CHANGE THE ORDER OF THESE STATEMENTS!  
      // (This order properly handles self-assignment)  
      ++p.p_->count_;  
      if (--p->count_ == 0) delete p_;  
      p_ = p.p_;  
      return *this;  
    }  
private:  
    Fred* p_; // p_ is never NULL  
};
```

Naturally you can use nested classes to rename `FredPtr` to `Fred::Ptr`.

Note that you can soften the "never `NULL`" rule above with a little more checking in the constructor, copy constructor, assignment operator, and destructor. If you do that, you might as well put a `p_ != NULL` check into the "*" and "->" operators (at least as an `assert()`). I would recommend against an operator `Fred*()` method, since that would let people accidentally get at the `Fred*`.

One of the implicit constraints on `FredPtr` is that it must only point to `Fred` objects which have been allocated via `new`. If you want to be really safe, you can enforce this constraint by making all of `Fred`'s constructors `private`, and for each constructor have a `public (static) create()` method which allocates the `Fred` object via `new` and returns a `FredPtr` (not a `Fred*`). That way the *only* way anyone could create a `Fred` object would be to get a `FredPtr` ("`Fred* p = new Fred()`" would be replaced by "`FredPtr p = Fred::create()`"). Thus no one could accidentally subvert the reference counting mechanism.

For example, if `Fred` had a `Fred::Fred()` and a `Fred::Fred(int i, int j)`, the changes to class `Fred` would be:

```
class Fred {
public:
    static FredPtr create();           // Defined below class FredPtr {...};
    static FredPtr create(int i, int j); // Defined below class FredPtr {...};
    ...
private:
    Fred();
    Fred(int i, int j);
    ...
};

class FredPtr { /*...*/ };

inline FredPtr Fred::create()          { return new Fred(); }
inline FredPtr Fred::create(int i, int j) { return new Fred(i,j); }
```

The end result is that you now have a way to use simple reference counting to provide "pointer semantics" for a given object. Users of your `Fred` class explicitly use `FredPtr` objects, which act more or less like `Fred*` pointers. The benefit is that users can make as many copies of their `FredPtr` "smart pointer" objects, and the pointed-to `Fred` object will automatically get deleted when the last such `FredPtr` object vanishes.

If you'd rather give your users "reference semantics" rather than "pointer semantics," you can use [reference counting to provide "copy on write"](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.22] How do I provide reference counting with copy-on-write semantics?

Reference counting can be done with either pointer semantics or reference semantics. The [previous FAQ](#) shows how to do reference counting with pointer semantics. This FAQ shows how to do reference counting with reference semantics.

The basic idea is to allow users to think they're copying your Fred objects, but in reality the underlying implementation doesn't actually do any copying unless and until some user actually tries to modify the underlying Fred object.

Class Fred::Data houses all the data that would normally go into the Fred class. Fred::Data also has an extra data member, count_, to manage the reference counting. Class Fred ends up being a "smart reference" that (internally) points to a Fred::Data.

```
class Fred {
public:

    Fred();                                // A default constructor
    Fred(int i, int j);                    // A normal constructor

    Fred(const Fred& f);
    Fred& operator= (const Fred& f);
    ~Fred();

    void sampleInspectorMethod() const;    // No changes to this object
    void sampleMutatorMethod();            // Change this object

    ...

private:

    class Data {
    public:
        Data();
        Data(int i, int j);
        Data(const Data& d);

        // Since only Fred can access a Fred::Data object,
        // you can make Fred::Data's data public if you want.
        // But if that makes you uncomfortable, make the data private
        // and make Fred a friend class via friend class Fred;
        ...data goes here...

        unsigned count_;
        // count_ is the number of Fred objects that point at this
        // count_ must be initialized to 1 by all constructors
        // (it starts as 1 since it is pointed to by the Fred object that created it)
    };

    Data* data_;
};

Fred::Data::Data()                : count_(1) /*init other data*/ { }
Fred::Data::Data(int i, int j)    : count_(1) /*init other data*/ { }
Fred::Data::Data(const Data& d)    : count_(1) /*init other data*/ { }

Fred::Fred()                     : data_(new Data()) { }
Fred::Fred(int i, int j)         : data_(new Data(i, j)) { }
```



```

Fred::Fred(const Fred& f)
    : data_(f.data_)
{
    ++ data_->count_;
}

Fred& Fred::operator= (const Fred& f)
{
    // DO NOT CHANGE THE ORDER OF THESE STATEMENTS!
    // (This order properly handles self-assignment)
    ++ f.data_->count_;
    if (--data_->count_ == 0) delete data_;
    data_ = f.data_;
    return *this;
}

Fred::~Fred()
{
    if (--data_->count_ == 0) delete data_;
}

void Fred::sampleInspectorMethod() const
{
    // This method promises ("Const") not to change anything in *data_
    // Other than that, any data access would simply use "data_->..."
}

void Fred::sampleMutatorMethod()
{
    // This method might need to change things in *data_
    // Thus it first checks if this is the only pointer to *data_
    if (data_->count_ > 1) {
        Data* d = new Data(*data_);    // Invoke Fred::Data's copy ctor
        -- data_->count_;
        data_ = d;
    }
    assert(data_->count_ == 1);

    // Now the method proceeds to access "data_->..." as normal
}

```

If it is fairly common to call Fred's [default constructor](#), you can avoid all those `new` calls by sharing a common `Fred::Data` object for all Freds that are constructed via `Fred::Fred()`. To avoid static initialization order problems, this shared `Fred::Data` object is created "on first use" inside a function. Here are the changes that would be made to the above code (note that the shared `Fred::Data` object's destructor is never invoked; if that is a problem, either hope you don't have any static initialization order problems, or drop back to the approach described above):

```

class Fred {
public:
    ...
private:
    ...
    static Data* defaultData();
};

Fred::Fred()

```

```

: data_(defaultData())
{
    ++ data_>count_;
}

Fred::Data* Fred::defaultData()
{
    static Data* p = NULL;
    if (p == NULL) {
        p = new Data();
        ++ p->count_;    // Make sure it never goes to zero
    }
    return p;
}

```

Note: You can also provide [reference counting for a hierarchy of classes](#) if your `Fred` class would normally have been a base class.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.23] How do I provide reference counting with copy -on-write semantics for a hierarchy of classes? UPDATED!

[Recently changed `friend Fred;` to `friend class Fred;` thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

The [previous FAQ](#) presented a reference counting scheme that provided users with reference semantics, but did so for a single class rather than for a hierarchy of classes. This FAQ extends the previous technique to allow for a hierarchy of classes. The basic difference is that `Fred::Data` is now the root of a hierarchy of classes, which probably cause it to have some [virtual](#) functions. Note that class `Fred` itself will still not have any `virtual` functions.

The [Virtual Constructor Idiom](#) is used to make copies of the `Fred::Data` objects. To select which derived class to create, the sample code below uses the [Named Constructor Idiom](#), but other techniques are possible (a `switch` statement in the constructor, etc). The sample code assumes two derived classes: `Der1` and `Der2`. Methods in the derived classes are unaware of the reference counting.

```

class Fred {
public:

    static Fred create1(const std::string& s, int i);
    static Fred create2(float x, float y);

    Fred(const Fred& f);
    Fred& operator= (const Fred& f);
    ~Fred();

    void sampleInspectorMethod() const;    // No changes to this object
    void sampleMutatorMethod();           // Change this object

    ...

private:

```

```

class Data {
public:
    Data() : count_(1) { }
    Data(const Data& d) : count_(1) { }           // Do NOT copy the 'count_' member!
    Data& operator= (const Data&) { return *this; } // Do NOT copy the 'count_' member!
    virtual ~Data() { assert(count_ == 0); }       // A virtual destructor
    virtual Data* clone() const = 0;               // A virtual constructor
    virtual void sampleInspectorMethod() const = 0; // A pure virtual function
    virtual void sampleMutatorMethod() = 0;
private:
    unsigned count_;    // count_ doesn't need to be protected
    friend class Fred; // Allow Fred to access count_
};

class Der1 : public Data {
public:
    Der1(const std::string& s, int i);
    virtual void sampleInspectorMethod() const;
    virtual void sampleMutatorMethod();
    virtual Data* clone() const;
    ...
};

class Der2 : public Data {
public:
    Der2(float x, float y);
    virtual void sampleInspectorMethod() const;
    virtual void sampleMutatorMethod();
    virtual Data* clone() const;
    ...
};

Fred(Data* data);
// Creates a Fred smart-reference that owns *data
// It is private to force users to use a createXXX() method
// Requirement: data must not be NULL

Data* data_;    // Invariant: data_ is never NULL
};

Fred::Fred(Data* data) : data_(data) { assert(data != NULL); }

Fred Fred::create1(const std::string& s, int i) { return Fred(new Der1(s, i)); }
Fred Fred::create2(float x, float y)           { return Fred(new Der2(x, y)); }

Fred::Data* Fred::Der1::clone() const { return new Der1(*this); }
Fred::Data* Fred::Der2::clone() const { return new Der2(*this); }

Fred::Fred(const Fred& f)
    : data_(f.data_)
{
    ++ data_>count_;
}

Fred& Fred::operator= (const Fred& f)
{
    // DO NOT CHANGE THE ORDER OF THESE STATEMENTS!
    // (This order properly handles self-assignment)
    ++ f.data_>count_;

```

```

    if (--data_>count_ == 0) delete data_;
    data_ = f.data_;
    return *this;
}

Fred::~Fred()
{
    if (--data_>count_ == 0) delete data_;
}

void Fred::sampleInspectorMethod() const
{
    // This method promises ("Const") not to change anything in *data_
    // Therefore we simply "pass the method through" to *data_:
    data_>sampleInspectorMethod();
}

void Fred::sampleMutatorMethod()
{
    // This method might need to change things in *data_
    // Thus it first checks if this is the only pointer to *data_
    if (data_>count_ > 1) {
        Data* d = data_>clone();    // The Virtual Constructor Idiom
        -- data_>count_;
        data_ = d;
    }
    assert(data_>count_ == 1);

    // Now we "pass the method through" to *data_:
    data_>sampleMutatorMethod();
}

```

Naturally the constructors and sampleXXX methods for Fred::Der1 and Fred::Der2 will need to be implemented in whatever way is appropriate.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.24] Can you absolutely *prevent* people from subverting the reference counting mechanism, and if so, *should* you?

No, and (normally) no.

There are two basic approaches to subverting the reference counting mechanism:

1. The scheme could be subverted if someone got a Fred* (rather than being forced to use a FredPtr). Someone could get a Fred* if class FredPtr has an operator*() that returns a Fred&: FredPtr p = Fred::create(); Fred* p2 = &*p;. Yes it's bizarre and unexpected, but it could happen. This hole could be closed in two ways: overload Fred::operator&() so it returns a FredPtr, or change the return type of FredPtr::operator*() so it returns a FredRef (FredRef would be a class that simulates a reference; it would need to have all the methods that Fred has, and it would need to forward all those method calls to the underlying Fred object; there might be a performance penalty for this second choice depending on how good the compiler is at inlining methods). Another way to fix this is to

eliminate `FredPtr::operator*()` — and lose the corresponding ability to get and use a `Fred&`. But even if you did all this, someone could still generate a `Fred*` by explicitly calling `operator->(): FredPtr p = Fred::create(); Fred* p2 = p.operator->();`.

2. The scheme could be subverted if someone had a leak and/or dangling pointer to a `FredPtr`. Basically what we're saying here is that `Fred` is now safe, but we somehow want to prevent people from doing stupid things with `FredPtr` objects. (And if we could solve that via `FredPtrPtr` objects, we'd have the same problem again with them). One hole here is if someone created a `FredPtr` using `new`, then allowed the `FredPtr` to leak (worst case this is a leak, which is bad but is *usually* a little better than a dangling pointer). This hole could be plugged by declaring `FredPtr::operator new()` as `private`, thus preventing someone from saying `new FredPtr()`. Another hole here is if someone creates a local `FredPtr` object, then takes the address of that `FredPtr` and passed around the `FredPtr*`. If that `FredPtr*` lived longer than the `FredPtr`, you could have a dangling pointer — shudder. This hole could be plugged by preventing people from taking the address of a `FredPtr` (by overloading `FredPtr::operator&()` as `private`), with the corresponding loss of functionality. But even if you did all that, they could still create a `FredPtr&` which is almost as dangerous as a `FredPtr*`, simply by doing this: `FredPtr p; ... FredPtr& q = p;` (or by passing the `FredPtr&` to someone else).

And even if we closed *all* those holes, C++ has those wonderful pieces of syntax called pointer casts. Using a pointer cast or two, a sufficiently motivated programmer can normally create a hole that's big enough to drive a proverbial truck through. (By the way, [pointer casts are evil](#).)

So the lessons here seems to be: (a) you can't prevent espionage no matter how hard you try, and (b) you can easily prevent mistakes.

So I recommend settling for the "low hanging fruit": use the easy -to-build and easy-to-use mechanisms that prevent mistakes, and do n't bother trying to prevent espionage. You won't succeed, and even if you do, it'll (probably) cost you more than it's worth.

So if we can't use the C++ language itself to prevent espionage, are there other ways to do it? Yes. I personally use old fashioned code reviews for that. And since the espionage techniques usually involve some bizarre syntax and/or use of pointer -casts and unions, you can use a tool to point out most of the "hot spots."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.25] Can I use a garbage collector in C++?

Yes.

Compared with the "smart pointer" techniques (see [\[16.21\]](#), the two kinds of garbage collector techniques (see [\[16.26\]](#)) are:

- less portable
- usually more efficient (especially when the average object size is small or in multithreaded environments)

- able to handle "cycles" in the data (reference counting techniques normally "leak" if the data structures can form a cycle)
- sometimes leak other objects (since the garbage collectors are necessarily conservative, they sometimes see a random bit pattern that appears to be a pointer into an allocation, especially if the allocation is large; this can allow the allocation to leak)
- work better with existing libraries (since smart pointers need to be used explicitly, they may be hard to integrate with existing libraries)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.26] What are the two kinds of garbage collectors for C++?

In general, there seem to be two flavors of garbage collectors for C++:

1. *Conservative garbage collectors.* These know little or nothing about the layout of the stack or of C++ objects, and simply look for bit patterns that appear to be pointers. In practice they seem to work with both C and C++ code, particularly when the average object size is small. Here are some examples, in alphabetical order:
 - [Boehm-Demers-Weiser collector](#)
 - [Geodesic Systems collector](#)
2. *Hybrid garbage collectors.* These usually scan the stack conservatively, but require the programmer to supply layout information for heap objects. This requires more work on the programmer's part, but may result in improved performance. Here are some examples, in alphabetical order:
 - [Attardi and Flagella's CMM](#)
 - [Bartlett's mostly copying collector](#)

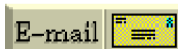
Since garbage collectors for C++ are normally conservative, they can sometimes leak if a bit pattern "looks like" it might be a pointer to an otherwise unused block. Also they sometimes get confused when pointers to a block actually point outside the block's extent (which is illegal, but some programmers simply *must* push the envelope; sigh) and (rarely) when a pointer is hidden by a compiler optimization. In practice these problems are not usually serious, however providing the collector with hints about the layout of the objects can sometimes ameliorate these issues.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[16.27] Where can I get more info on garbage collectors for C++?

For more information, see [the Garbage Collector FAQ](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[17] Exceptions and error handling

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [17]:

- [\[17.1\] What are some ways `try / catch / throw` can improve software quality?](#)
- [\[17.2\] How can I handle a constructor that fails?](#)
- [\[17.3\] How can I handle a destructor that fails?](#)
- [\[17.4\] How should I handle resources if my constructors may throw exceptions?](#)
- [\[17.5\] How do I change the string-length of an array of `char` to prevent memory leaks even if/when someone throws an exception?](#)
- [\[17.6\] What should I throw?](#)
- [\[17.7\] What should I catch?](#) **UPDATED!**
- [\[17.8\] But MFC seems to encourage the use of `catch -by-pointer`; should I do the same?](#)
- [\[17.9\] What does `throw`; \(without an exception object after the `throw` keyword\) mean? Where would I use it?](#)
- [\[17.10\] How do I throw polymorphically?](#)
- [\[17.11\] When I throw this object, how many times will it be copied?](#)
- [\[17.12\] Exception handling seems to make my life more difficult; clearly I'm not the problem, am I??](#) **NEW!**

[17.1] What are some ways `try / catch / throw` can improve software quality?

By eliminating one of the reasons for `if` statements.

The commonly used alternative to `try / catch / throw` is to return a *return code* (sometimes called an *error code*) that the caller explicitly tests via some conditional statement such as `if`. For example, `printf()`, `scanf()` and `malloc()` work this way: the caller is supposed to test the return value to see if the function succeeded.

Although the return code technique is sometimes the most appropriate error handling technique, there are some nasty side effects to adding unnecessary `if` statements:

- **Degrade quality:** It is well known that conditional statements are approximately ten times more likely to contain errors than any other kind of statement. So all other things being equal, if you can eliminate conditionals / conditional statements from your code, you will likely have more robust code.



- **Slow down time-to-market:** Since conditional statements are branch points which are related to the number of test cases that are needed for white-box testing, unnecessary conditional statements increase the amount of time that needs to be devoted to testing. Basically if you don't exercise every branch point, there will be instructions in your code that will *never* have been executed under test conditions until they are seen by your users/customers. That's bad.
- **Increase development cost:** Bug finding, bug fixing, and testing are all increased by unnecessary control flow complexity.

So compared to error reporting via return-codes and `if`, using `try / catch / throw` is likely to result in code that has fewer bugs, is less expensive to develop, and has faster time -to-market. Of course if your organization doesn't have any experiential knowledge of `try / catch / throw`, you might want to use it on a toy project first just to make sure you know what you're doing — you should always get used to a weapon on the firing range before you bring it to the front lines of a shooting war.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.2] How can I handle a constructor that fails?

Throw an exception.

Constructors don't have a return type, so it's not possible to use return codes. The best way to signal constructor failure is therefore to throw an exception. If you don't have the option of using exceptions, the "least bad" work-around is to put the object into a "zombie" state by setting an internal status bit so the object acts sort of like it's dead even though it is technically still alive.

The idea of a "zombie" object has a lot of down-side. You need to add a query ("inspector") member function to check this "zombie" bit so users of your class can find out if their object is truly alive, or if it's a zombie (i.e., a "living dead" object), and just about every place you construct one of your objects (including within a larger object or an array of objects) you need to check that status flag via an `if` statement. You'll also want to add an `if` to your other member functions: if the object is a zombie, do a no-op or perhaps something more obnoxious.

In practice the "zombie" thing gets pretty ugly. Certainly you should prefer exceptions over zombie objects, but if you do not have the option of using exceptions, zombie objects might be the "least bad" alternative.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.3] How can I handle a destructor that fails?

Write a message to a log-file. Or call Aunt Tilda. But do *not* throw an exception!

Here's why (buckle your seat-belts):

The C++ rule is that you must never throw an exception from a destructor that is being called during the "stack unwinding" process of another exception. For example, if someone says `throw Foo()`, the stack will be unwound so all the stack frames between the `throw Foo()` and the `} catch (Foo e) {` will get popped. This is called *stack unwinding*.

During stack unwinding, all the local objects in all those stack frames are destructed. If one of *those* destructors throws an exception (say it throws a `Bar` object), the C++ runtime system is in a no-win situation: should it ignore the `Bar` and end up in the `} catch (Foo e) {` where it was originally headed? Should it ignore the `Foo` and look for a `} catch (Bar e) {` handler? There is no good answer — either choice loses information.

So the C++ language guarantees that it will call `terminate()` at this point, and `terminate()` kills the process. Bang you're dead.

The easy way to prevent this is *never throw an exception from a destructor*. But if you really want to be clever, you can say *never throw an exception from a destructor while processing another exception*. But in this second case, you're in a difficult situation: the destructor itself needs code to handle both throwing an exception and doing "something else", and the caller has no guarantees as to what might happen when the destructor detects an error (it might throw an exception, it might do "something else"). So the whole solution is harder to write. So the easy thing to do is *always* do "something else". That is, *never throw an exception from a destructor*.

Of course the word *never* should be "in quotes" since there is always some situation somewhere where the rule won't hold. But certainly at least 99% of the time this is a good rule of thumb.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.4] How should I handle resources if my constructors may throw exceptions?

Every data member inside your object should clean up its own mess.

If a constructor throws an exception, the object's destructor is *not* run. If your object has already done something that needs to be undone (such as allocating some memory, opening a file, or locking a semaphore), this "stuff that needs to be undone" *must* be remembered by a data member inside the object.

For example, rather than allocating memory into a raw `Fred*` data member, put the allocated memory into a "smart pointer" member object, and the destructor of this smart pointer will delete the `Fred` object when the smart pointer dies. The template `std::auto_ptr` is an example of such as "smart pointer." You can also [write your own reference counting smart pointer](#). You can also [use smart pointers to "point" to disk records or objects on other machines](#).

By the way, if you think your `Fred` class is going to be allocated into a smart pointer, be nice to your users and create a `typedef` within your `Fred` class:

```
#include <memory>

class Fred {
```

```
public:
    typedef std::auto_ptr<Fred> Ptr;
    ...
};
```

That typedef simplifies the syntax of all the code that uses your objects: your users can say `Fred::Ptr` instead of `std::auto_ptr<Fred>`:

```
#include "Fred.h"

void f(std::auto_ptr<Fred> p); // explicit but verbose
void f(Fred::Ptr p); // simpler

void g()
{
    std::auto_ptr<Fred> p1( new Fred() ); // explicit but verbose
    Fred::Ptr p2( new Fred() ); // simpler
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.5] How do I change the string-length of an array of `char` to prevent memory leaks even if/when someone throws an exception?

If what you really want to do is work with strings, don't use an array of `char` in the first place, since [arrays are evil](#). Instead use an object of some string-like class.

For example, suppose you want to get a copy of a string, fiddle with the copy, then append another string to the end of the fiddled copy. The array-of-char approach would look something like this:

```
void userCode(const char* s1, const char* s2)
{
    char* copy = new char[strlen(s1) + 1]; // make a copy
    strcpy(copy, s1); // of s1...

    // use a try block to prevent memory leaks if we get an exception
    // note: we need the try block because we used a "dumb" char* above
    try {

        ...insert code here that fiddles with copy...

        char* copy2 = new char[strlen(copy) + strlen(s2) + 1]; // append s2
        strcpy(copy2, copy); // onto the
        strcpy(copy2 + strlen(copy), s2); // end of
        delete[] copy; // copy...
        copy = copy2;

        ...insert code here that fiddles with copy again...

    } catch (...) {
        delete[] copy; // we got an exception; prevent a memory leak
    }
```

```

    throw;                // re-throw the current exception
}

delete[] copy;           // we did not get an exception; prevent a memory leak
}

```

Using `char*`s like this is tedious and error prone. Why not just use an object of some `string` class? Your compiler probably supplies a `string`-like class, and it's probably just as fast and certainly it's a lot simpler and safer than the `char*` code that you would have to write yourself. For example, if you're using the `std::string` class from the [standardization committee](#), your code might look something like this:

```

#include <string>           // Let the compiler see std::string

void userCode(const std::string& s1, const std::string& s2)
{
    std::string copy = s1;    // make a copy of s1
    ...insert code here that fiddles with copy...
    copy += s2;              // append s2 onto the end of copy
    ...insert code here that fiddles with copy again...
}

```

The `char*` version requires you to write around three times more code than you would have to write with the `std::string` version. Most of the savings came from `std::string`'s automatic memory management: in the `std::string` version, we didn't need to write any code...

- to reallocate memory when we grow the string.
- to `delete[]` anything at the end of the function.
- to catch and re-throw any exceptions.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.6] What should I throw?

C++, unlike just about every other language with exceptions, is very accomodating when it comes to what you can throw. In fact, you can throw anything you like. That begs the question then, what should you throw?

Generally, it's best to throw objects, not built-ins. If possible, you should throw instances of classes that derive (ultimately) from the `std::exception` class. By making your exception class inherit (ultimately) from the standard exception base -class, you are making life easier for your users (they have the option of catching most things via `std::exception`), plus you are probably providing them with more information (such as the fact that your particular exception might be a refinement of `std::runtime_error` or whatever).

The most common practice is to throw a temporary:

```

#include <stdexcept>

class MyException : public std::runtime_error {

```

```
public:
    MyException() : std::runtime_error("MyException") { }
};

void f()
{
    //...
    throw MyException();
}
```

Here, a temporary of type `MyException` is created and thrown. Class `MyException` inherits from class `std::runtime_error` which (ultimately) inherits from class `std::exception`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.7] What should I catch? UPDATED!

[Recently fixed a grammatical error thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

In keeping with the C++ tradition of "there's more than one way to do that" (translation: "give programmers options and tradeoffs so *they* can decide what's best for *them* in *their* situation"), C++ allows you a variety of options for catching.

- You can catch by value.
- You can catch by reference.
- You can catch by pointer.

In fact, you have all the flexibility that you have in declaring function parameters, and the rules for whether a particular exception matches (i.e., will be caught by) a particular catch clause are almost exactly the same as the rules for parameter compatibility when calling a function.

Given all this flexibility, how do you decide what to catch? Simple: unless there's a good reason not to, catch by reference. Avoid catching by value, since that causes a copy to be made and [the copy can have different behavior](#) from what was thrown. Only under very special circumstances should you catch by pointer.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.8] But MFC seems to encourage the use of catch -by-pointer; should I do the same?

Depends. If you're using MFC and catching one of their exceptions, by all means, do it their way. Same goes for any framework: when in Rome, do as the Romans. Don't try to force a framework into your way of thinking, even if "your" way of thinking is "better." If you decide to use a framework, embrace *its* way of thinking — use the idioms that its authors expected you to use.

But if you're creating your own framework and/or a piece of the system that does not directly depend on MFC, then don't catch by pointer just because MFC does it that way. When you're *not* in Rome, you don't *necessarily* do as the Romans. In this case, you should *not*. Libraries like MFC predated the standardization of exception handling in the C++ language, and some of these libraries use a backwards-compatible form of exception handling that requires (or at least encourages) you to catch by pointer.

The problem with catching by pointer is that it's not clear who (if anyone) is responsible for deleting the pointed-to object. For example, consider the following:

```
MyException x;

void f()
{
    MyException y;

    try {
        switch (rand() % 3) {
            case 0: throw new MyException;
            case 1: throw &x;
            case 2: throw &y;
        }
    }
    catch (MyException* p) {
        ...           ← should we delete p here or not???!
    }
}
```

There are three basic problems here:

1. It might be tough to decide whether to `delete p` within the `catch` clause. For example, if object `x` is inaccessible to the scope of the `catch` clause, such as when it's buried in the private part of some class or is `static` within some other compilation unit, it might be tough to figure out what to do.
2. If you solve the first problem by consistently using `new` in the `throw` (and therefore consistently using `delete` in the `catch`), then exceptions always use the heap which can cause problems when the exception was thrown because the system was running low on memory.
3. If you solve the first problem by consistently *not* using `new` in the `throw` (and therefore consistently *not* using `delete` in the `catch`), then you probably won't be able to allocate your exception objects as locals (since then they might get destructed too early), in which case you'll have to worry about thread-safety, locks, semaphores, etc. (`static` objects are not intrinsically thread-safe).

This isn't to say it's not possible to work through these issues. The point is simply this: if you catch by reference rather than by pointer, life is easier. Why make life hard when you don't have to?

The moral: avoid throwing pointer expressions, and avoid catching by pointer, *unless* you're using an existing library that "wants" you to do so.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.9] What does `throw;` (without an exception object after the `throw` keyword) mean? Where would I use it?

You might see code that looks something like this:

```
class MyException {
public:
    ...
    void addInfo(const std::string& info);
    ...
};

void f()
{
    try {
        ...
    }
    catch (MyException& e) {
        e.addInfo("f() failed");
        throw;
    }
}
```

In this example, the statement `throw;` means "re-throw the current exception." Here, a function caught an exception (by non-const reference), modified the exception (by adding information to it), and then re-threw the exception. This idiom can be used to implement a simple form of stack-trace, by adding appropriate catch clauses in the important functions of your program.

Another re-throwing idiom is the "exception dispatcher":

```
void handleException()
{
    try {
        throw;
    }
    catch (MyException& e) {
        ...code to handle MyException...
    }
    catch (YourException& e) {
        ...code to handle YourException...
    }
}

void f()
{
    try {
        ...something that might throw...
    }
    catch (...) {
        handleException();
    }
}
```

This idiom allows a single function (`handleException()`) to be re-used to handle exceptions in a number of other functions.

[17.10] How do I throw polymorphically?

Sometimes people write code like like:

```
class MyExceptionBase { };

class MyExceptionDerived : public MyExceptionBase { };

void f(MyExceptionBase& e)
{
    //...
    throw e;
}

void g()
{
    MyExceptionDerived e;
    try {
        f(e);
    }
    catch (MyExceptionDerived& e) {
        ...code to handle MyExceptionDerived...
    }
    catch (...) {
        ...code to handle other exceptions...
    }
}
```

If you try this, you might be surprised at run-time when your `catch (...)` clause is entered, and not your `catch (MyExceptionDerived&)` clause. This happens because you didn't throw polymorphically. In function `f()`, the statement `throw e;` throws an object with the same type as the *static* type of the expression `e`. In other words, it throws an instance of `MyExceptionBase`. The `throw` statement behaves as-if the thrown object is copied, as opposed to [making a "virtual copy"](#).

Fortunately it's relatively easy to correct:

```
class MyExceptionBase {
public:
    virtual void raise();
};

void MyExceptionBase::raise()
{ throw *this; }

class MyExceptionDerived : public MyExceptionBase {
public:
    virtual void raise();
};

void MyExceptionDerived::raise()
{ throw *this; }

void f(MyExceptionBase& e)
```

```

{
    //...
    e.raise();
}

void g()
{
    MyExceptionDerived e;
    try {
        f(e);
    }
    catch (MyExceptionDerived& e) {
        ...code to handle MyExceptionDerived...
    }
    catch (...) {
        ...code to handle other exceptions...
    }
}

```

Note that the `throw` statement has been moved into a virtual function. The statement `e.raise()` will exhibit polymorphic behavior, since `raise()` is declared `virtual` and `e` was passed by reference. As before, the thrown object will be of the `static` type of the argument in the `throw` statement, but within `MyExceptionDerived::raise()`, that static type is `MyExceptionDerived`, not `MyExceptionBase`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.11] When I throw this object, how many times will it be copied?

Depends. Might be "zero."

Objects that are thrown must have a publicly accessible copy -constructor. The compiler is allowed to generate code that copies the thrown object any number of times, including zero. However even if the compiler never actually copies the thrown object, it must make sure the exception class's copy constructor exists and is accessible.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[17.12] Exception handling seems to make my life more difficult; clearly *I'm* not the problem, am I?? NEW!

[Recently created thanks to an email conversation with [Bob Bell](#) (in 12/04) and rewrote the last bullet thanks to a critique of the old wording from [Jan Ploski](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Absolutely you might be the problem!

The C++ exception handling mechanism can be powerful and useful, but if you use it with the wrong mindset, the result can be a mess. If you're getting bad results, for instance, if your code

seems unnecessarily convoluted or overly cluttered with `try` blocks, you might be suffering from a "wrong mindset." This FAQ gives you a list of some of those wrong mindsets.

Warning: do *not* be simplistic about these "wrong mindsets." They are guidelines and ways of thinking, not hard and fast rules. Sometimes you will do the exact opposite of what they recommend — do *not* write me about some situation that is an exception (no pun intended) to one or more of them — I *guarantee* that there are exceptions. That's not the point.

Here are some "wrong exception-handling mindsets" in no apparent order:

- *The return-codes mindset:* This causes programmers to clutter their code with gobs of `try` blocks. Basically they think of a `throw` as a glorified return code, and a `try/catch` as a glorified "if the return code indicates an error" test, and they put one of these `try` blocks around just about every function that can `throw`.
- *The Java mindset:* In Java, non-memory resources are reclaimed via explicit `try/finally` blocks. When this mindset is used in C++, it results in a large number of unnecessary `try` blocks, which, compared with RAII, clutters the code and makes the logic harder to follow. Essentially the code swaps back and forth between the "good path" and the "bad path" (the latter meaning the path taken during an exception). With RAII, the code is mostly optimistic — it's all the "good path," and the cleanup code is buried in destructors of the resource-owning objects. This also helps reduce the cost of code reviews and unit-testing, since these "resource-owning objects" can be validated in isolation (with explicit `try/catch` blocks, each copy must be unit-tested and inspected individually; they cannot be handled as a group).
- *Organizing the exception classes around the physical thrower rather than the logical reason for the throw:* For example, in a banking app, suppose any of five subsystems might throw an exception when the customer has insufficient funds. The right approach is to throw an exception representing the *reason* for the throw, e.g., an "insufficient funds exception"; the wrong mindset is for each subsystem to throw a subsystem-specific exception. For example, the `Foo` subsystem might throw objects of class `FooException`, the `Bar` subsystem might throw objects of class `BarException`, etc. This often leads to extra `try/catch` blocks, e.g., to catch a `FooException`, repackage it into a `BarException`, then throw the latter. In general, exception classes should represent the problem, not the chunk of code that noticed the problem.
- *Using the bits / data within an exception object to differentiate different categories of errors:* Suppose the `Foo` subsystem in our banking app throws exceptions for bad account numbers, for attempting to liquidate an illiquid asset, and for insufficient funds. When these three logically distinct kinds of errors are represented by the same exception class, the catchers need to say `if` to figure out what the problem really was. If your code wants to handle only bad account numbers, you need to catch the master exception class, then use `if` to determine whether it is one you really want to handle, and if not, to rethrow it. In general, the preferred approach is for the error condition's logical category to get encoded into the *type* of the exception object, not into the *data* of the exception object.
- *Designing exception classes on a subsystem by subsystem basis:* In the bad old days, the specific meaning of any given return-code was local to a given function or API. Just because one function uses the return-code of 3 to mean "success," it was still perfectly acceptable for another function to use 3 to mean something entirely different, e.g., "failed due to out of memory." Consistency has always been *preferred*, but often that didn't happen because it didn't *need* to happen. People coming with that mentality often treat C++ exception-handling the same way: they assume exception classes can be localized

to a subsystem. That causes no end of grief, e.g., lots of extra `try` blocks to catch then throw a repackaged variant of the same exception. In large systems, exception hierarchies *must* be designed with a system-wide mindset. Exception classes cross subsystem boundaries — they are part of the intellectual glue that holds the architecture together.

- *Use of raw (as opposed to smart) pointers:* This is actually just a special case of non-RAII coding, but I'm calling it out because it is so common. The result of using raw pointers is, as above, lots of extra `try/catch` blocks whose only purpose in life is to delete an object then re-throw the exception.
- *Confusing logical errors with runtime situations:* For example, suppose you have a function `f(Foo* p)` that must never be called with the NULL pointer. However you discover that somebody somewhere is sometimes passing a NULL pointer anyway. There are two possibilities: either they are passing NULL because they got bad data from an external user (for example, the user forgot to fill in a field and that ultimately resulted in a NULL pointer) or they just plain made a mistake in their own code. In the former case, you should throw an exception since it is a runtime situation (i.e., something you can't detect by a careful code-review; it is not a bug). In the latter case, you should definitely fix the bug in the caller's code. You can still add some code to write a message in the log-file if it ever happens again, and you can even throw an exception if it ever happens again, but you must not merely change the code within `f(Foo* p)`; you must, *must*, *MUST* fix the code in the caller(s) of `f(Foo* p)`.

There are other "wrong exception-handling mindsets," but hopefully those will help you out. And remember: don't take those as hard and fast rules. They are guidelines, and there are exceptions to each.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[18] Const correctness

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [18]:

- [\[18.1\] What is "const correctness"?](#)
- [\[18.2\] How is "const correctness" related to ordinary type safety?](#)
- [\[18.3\] Should I try to get things const correct "sooner" or "later"?](#)
- [\[18.4\] What does "const Fred* p" mean?](#)



- [\[18.5\] What's the difference between "const Fred* p", "Fred* const p" and "const Fred* const p"?](#)
- [\[18.6\] What does "const Fred& x" mean?](#)
- [\[18.7\] Does "Fred& const x" make any sense?](#)
- [\[18.8\] What does "Fred const& x" mean?](#)
- [\[18.9\] What does "Fred const* x" mean?](#)
- [\[18.10\] What is a "const member function"?](#)
- [\[18.11\] What do I do if I want a const member function to make an "invisible" change to a data member?](#)
- [\[18.12\] Does const_cast mean lost optimization opportunities?](#)
- [\[18.13\] Why does the compiler allow me to change an int after I've pointed at it with a const int*? **UPDATED!**](#)
- [\[18.14\] Does "const Fred* p" mean that *p can't change?](#)
- [\[18.15\] Why am I getting an error converting a Foo** → const Foo**?](#)

[18.1] What is "const correctness"?

A good thing. It means using the keyword `const` to prevent `const` objects from getting mutated.

For example, if you wanted to create a function `f()` that accepted a `std::string`, plus you want to promise callers not to change the caller's `std::string` that gets passed to `f()`, you can have `f()` receive its `std::string` parameter...

- `void f1(const std::string& s);` *// Pass by reference-to-const*
- `void f2(const std::string* sptr);` *// Pass by pointer-to-const*
- `void f3(std::string s);` *// Pass by value*

In the *pass by reference-to-const* and *pass by pointer-to-const* cases, any attempts to change to the caller's `std::string` within the `f()` functions would be flagged by the compiler as an error at compile-time. This check is done entirely at compile-time: there is no run-time space or speed cost for the `const`. In the *pass by value* case (`f3()`), the called function gets a copy of the caller's `std::string`. This means that `f3()` can change its local copy, but the copy is destroyed when `f3()` returns. In particular `f3()` cannot change the caller's `std::string` object.

As an opposite example, if you wanted to create a function `g()` that accepted a `std::string`, but you want to let callers know that `g()` might change the caller's `std::string` object. In this case you can have `g()` receive its `std::string` parameter...

- `void g1(std::string& s);` *// Pass by reference-to-non-const*
- `void g2(std::string* sptr);` *// Pass by pointer-to-non-const*

The lack of `const` in these functions tells the compiler that they are allowed to (but are not required to) change the caller's `std::string` object. Thus they can pass their `std::string` to any of the `f()` functions, but only `f3()` (the one that receives its parameter "by value") can pass its `std::string` to `g1()` or `g2()`. If `f1()` or `f2()` need to call either `g()` function, a local copy of the

`std::string` object must be passed to the `g()` function; the parameter to `f1()` or `f2()` cannot be directly passed to either `g()` function. E.g.,

```
void g1(std::string& s);

void f1(const std::string& s)
{
    g1(s);           // Compile-time Error since s is const

    std::string localCopy = s;
    g1(localCopy);    // OK since localCopy is not const
}
```

Naturally in the above case, any changes that `g1()` makes are made to the `localCopy` object that is local to `f1()`. In particular, no changes will be made to the `const` parameter that was passed by reference to `f1()`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.2] How is "`const` correctness" related to ordinary type safety?

Declaring the `const`-ness of a parameter is just another form of type safety. It is almost as if a `const std::string`, for example, is a different class than an ordinary `std::string`, since the `const` variant is missing the various mutative operations in the non-`const` variant (e.g., you can imagine that a `const std::string` simply doesn't have an assignment operator).

If you find ordinary type safety helps you get systems correct (it does; especially in large systems), you'll find `const` correctness helps also.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.3] Should I try to get things `const` correct "sooner" or "later"?

At the very, very, very beginning.

Back-patching `const` correctness results in a snowball effect: every `const` you add "over here" requires four more to be added "over there."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.4] What does "`const Fred* p`" mean?

It means `p` points to an object of class `Fred`, but `p` can't be used to change that `Fred` object (naturally `p` could also be `NULL`).

For example, if class Fred has a [const member function](#) called `inspect()`, saying `p->inspect()` is OK. But if class Fred has a [non-const member function](#) called `mutate()`, saying `p->mutate()` is an error (the error is caught by the compiler; no run-time tests are done, which means `const` doesn't slow your program down).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.5] What's the difference between "`const Fred* p`", "`Fred* const p`" and "`const Fred* const p`"?

You have to read pointer declarations right-to-left.

- `const Fred* p` means "p points to a Fred that is `const`" — that is, the Fred object can't be changed [via p](#).
- `Fred* const p` means "p is a `const` pointer to a Fred" — that is, you can change the Fred object [via p](#), but you can't change the pointer p itself.
- `const Fred* const p` means "p is a `const` pointer to a `const` Fred" — that is, you can't change the pointer p itself, nor can you change the Fred object [via p](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.6] What does "`const Fred& x`" mean?

It means x aliases a Fred object, but x can't be used to change that Fred object.

For example, if class Fred has a [const member function](#) called `inspect()`, saying `x.inspect()` is OK. But if class Fred has a [non-const member function](#) called `mutate()`, saying `x.mutate()` is an error (the error is caught by the compiler; no run-time tests are done, which means `const` doesn't slow your program down).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.7] Does "`Fred& const x`" make any sense?

No, it is nonsense.

To find out what the above declaration means, [you have to read it right-to-left](#). Thus "`Fred& const x`" means "x is a `const` reference to a Fred". But that is redundant, since references are always `const`. [You can't resear a reference](#). Never. With or without the `const`.

In other words, "`Fred& const x`" is functionally equivalent to "`Fred& x`". Since you're gaining nothing by adding the `const` after the `&`, you shouldn't add it since it will confuse people. I.e., the `const` will make some people think that the Fred is `const`, as if you had said "`const Fred& x`".

[18.8] What does "Fred const& x" mean?

Fred const& x is functionally equivalent to [const Fred& x](#). However, the real question is which *should* be used.

Answer: absolutely *no one* should pretend they can make decisions for *your* organization until they know something about your organization. One size does not fit all; there is no "right" answer for all organizations, so do not allow *anyone* to make a knee-jerk decision in *either* direction. "Think" is not a four-letter word.

For example, some organizations value consistency and have tons of code using `const Fred&`; for those, `Fred const&` would be a bad decision independent of its merits. There are lots of other business scenarios, some of which produce a preference for `Fred const&`, others a preference for `const Fred&`.

Use a style that is appropriate for your organization's *average maintenance programmer*. Not the gurus, not the morons, but the average maintenance programmer. Unless you're willing to fire them and hire new ones, make sure that *they* understand your code. Make a business decision based on your realities, not based on someone else's assumptions.

You'll need to overcome a little inertia to go with `Fred const&`. Most current C++ books use `const Fred&`, most programmers learned C++ with that syntax, and most programmers still use that syntax. That doesn't mean `const Fred&` is necessarily better for your organization, but it does mean you may get some confusion and mistakes during the transition and/or when you integrate new people. Some organizations are convinced the benefits of `Fred const&` outweigh the costs; others, apparently, are not.

Another caveat: if you decide to use `Fred const& x`, do something to make sure your people don't mis-type it as [the nonsensical "Fred& const x"](#).

[18.9] What does "Fred const* x" mean?

Fred const* x is functionally equivalent to [const Fred* x](#). However, the real question is which *should* be used.

Answer: absolutely *no one* should pretend they can make decisions for *your* organization until they know something about your organization. One size does not fit all; there is no "right" answer for all organizations, so do not allow *anyone* to make a knee-jerk decision in *either* direction. "Think" is not a four-letter word.

For example, some organizations value consistency and have tons of code using `const Fred*`; for those, `Fred const*` would be a bad decision independent of its merits. There are lots of other

business scenarios, some of which produce a preference for `Fred const*`, others a preference for `const Fred*`.

Use a style that is appropriate for your organization's *average maintenance programmer*. Not the gurus, not the morons, but the average maintenance programmer. Unless you're willing to fire them and hire new ones, make sure that *they* understand your code. Make a business decision based on your realities, not based on someone else's assumptions.

You'll need to overcome a little inertia to go with `Fred const*`. Most current C++ books use `const Fred*`, most programmers learned C++ with that syntax, and most programmers still use that syntax. That doesn't mean `const Fred*` is necessarily better for your organization, but it does mean you may get some confusion and mistakes during the transition and/or when you integrate new people. Some organizations are convinced the benefits of `Fred const*` outweigh the costs; others, apparently, are not.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.10] What is a "const member function"?

A member function that inspects (rather than mutates) its object.

A const member function is indicated by a `const` suffix just after the member function's parameter list. Member functions with a `const` suffix are called "const member functions" or "inspectors." Member functions without a `const` suffix are called "non-const member functions" or "mutators."

```
class Fred {
public:
    void inspect() const;    // This member promises NOT to change *this
    void mutate();          // This member function might change *this
};

void userCode(Fred& changeable, const Fred& unchangeable)
{
    changeable.inspect();    // OK: doesn't change a changeable object
    changeable.mutate();    // OK: changes a changeable object

    unchangeable.inspect();  // OK: doesn't change an unchangeable object
    unchangeable.mutate();  // ERROR: attempt to change unchangeable object
}
```

The error in `unchangeable.mutate()` is caught at compile time. There is no runtime space or speed penalty for `const`.

The trailing `const` on `inspect()` member function means that the *abstract* (client-visible) state of the object isn't going to change. This is slightly different from promising that the "raw bits" of the object's `struct` aren't going to change. C++ compilers aren't allowed to take the "bitwise" interpretation unless they can solve the aliasing problem, which normally can't be solved (i.e., a non-const alias could exist which could modify the state of the object). Another (important)

insight from this aliasing issue: pointing at an object with a pointer-to-const doesn't guarantee that the object won't change; it promises only that the object won't change *via that pointer*.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.11] What do I do if I want a `const` member function to make an "invisible" change to a data member?

Use `mutable` (or, as a last resort, use `const_cast`).

A small percentage of inspectors need to make innocuous changes to data members (e.g., a `Set` object might want to cache its last lookup in hopes of improving the performance of its next lookup). By saying the changes are "innocuous," I mean that the changes wouldn't be visible from outside the object's interface (otherwise the member function would be a mutator rather than an inspector).

When this happens, the data member which will be modified should be marked as `mutable` (put the `mutable` keyword just before the data member's declaration; i.e., in the same place where you could put `const`). This tells the compiler that the data member is allowed to change during a `const` member function. If your compiler doesn't support the `mutable` keyword, you can cast away the `const`'ness of this via the `const_cast` keyword (but see the **NOTE** below before doing this). E.g., in `Set::lookup() const`, you might say,

```
Set* self = const_cast<Set*>(this);  
// See the NOTE below before doing this!
```

After this line, `self` will have the same bits as `this` (e.g., `self == this`), but `self` is a `Set*` rather than a `const Set*` (technically a `const Set* const`, but the right-most `const` is irrelevant to this discussion). Therefore you can use `self` to modify the object pointed to by `this`.

NOTE: there is an extremely unlikely error that can occur with `const_cast`. It only happens when three very rare things are combined at the same time: a data member that ought to be `mutable` (such as is discussed above), a compiler that doesn't support the `mutable` keyword, and an object that was originally defined to be `const` (as opposed to a normal, non-`const` object that is pointed to by a pointer-to-`const`). Although this combination is so rare that it may never happen to you, if it ever did happen the code may not work (the Standard says the behavior is undefined).

If you ever want to use `const_cast`, use `mutable` instead. In other words, if you ever need to change a member of an object, and that object is pointed to by a pointer-to-`const`, the safest and simplest thing to do is add `mutable` to the member's declaration. You can use `const_cast` if you are *sure* that the actual object isn't `const` (e.g., if you are sure the object is declared something like this: `Set s;`), but if the object itself might be `const` (e.g., if it might be declared like: `const Set s;`), use `mutable` rather than `const_cast`.

Please don't write and tell me that version X of compiler Y on machine Z allows you to change a non-`mutable` member of a `const` object. I don't care — it is illegal according to the language and

your code will probably fail on a different compiler or even a different version (an upgrade) of the same compiler. Just say no. Use `mutable` instead.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.12] Does `const_cast` mean lost optimization opportunities?

In theory, yes; in practice, no.

Even if the language outlawed `const_cast`, the only way to avoid flushing the register cache across a `const` member function call would be to solve the aliasing problem (i.e., to prove that there are no non-`const` pointers that point to the object). This can happen only in rare cases (when the object is constructed in the scope of the `const` member function invocation, and when all the non-`const` member function invocations between the object's construction and the `const` member function invocation are statically bound, and when every one of these invocations is also `inline`, and when the constructor itself is `inline`, and when any member functions the constructor calls are `inline`).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.13] Why does the compiler allow me to change an `int` after I've pointed at it with a `const int*`? **UPDATED!**

[Recently initialized variable `x` in `main()` thanks to [Yecheil M. Kimchi](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Because "`const int* p`" means "`p` promises not to change the `*p`," *not* "`*p` promises not to change."

Causing a `const int*` to point to an `int` doesn't `const`-ify the `int`. The `int` can't be changed via the `const int*`, but if someone else has an `int*` (note: no `const`) that points to ("aliases") the same `int`, then that `int*` can be used to change the `int`. For example:

```
void f(const int* p1, int* p2)
{
    int i = *p1;           // Get the (original) value of *p1
    *p2 = 7;               // If p1 == p2, this will also change *p1
    int j = *p1;           // Get the (possibly new) value of *p1
    if (i != j) {
        std::cout << "*p1 changed, but it didn't change via pointer p1!\n";
        assert(p1 == p2); // This is the only way *p1 could be different
    }
}

int main()
{
    int x = 5;
    f(&x, &x);             // This is perfectly legal (and even moral!)
```

```
...  
}
```

Note that `main()` and `f(const int*,int*)` could be in different compilation units that are compiled on different days of the week. In that case there is no way the compiler can possibly detect the aliasing at compile time. Therefore there is no way we could make a language rule that prohibits this sort of thing. In fact, we wouldn't even want to make such a rule, since in general it's considered a feature that you can have many pointers pointing to the same thing. The fact that one of those pointers promises not to change the underlying "thing" is just a promise made by the *pointer*; it's *not* a promise made by the "thing".

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.14] Does "const Fred* p" mean that *p can't change?

No! (This is related to [the FAQ about aliasing of int pointers](#).)

"const Fred* p" means that the Fred can't be changed via pointer p, but there might be other ways to get at the object without going through a const (such as an aliased non-const pointer such as a Fred*). For example, if you have two pointers "const Fred* p" and "Fred* q" that point to the same Fred object (aliasing), pointer q can be used to change the Fred object but pointer p cannot.

```
class Fred {  
public:  
    void inspect() const;    // A const member function  
    void mutate();          // A non-CONST member function  
};  
  
int main()  
{  
    Fred f;  
    const Fred* p = &f;  
    Fred* q = &f;  
  
    p->inspect();            // OK: No change to *p  
    p->mutate();             // Error: Can't change *p via p  
  
    q->inspect();            // OK: q is allowed to inspect the object  
    q->mutate();            // OK: q is allowed to mutate the object  
  
    f.inspect();            // OK: f is allowed to inspect the object  
    f.mutate();            // OK: f is allowed to mutate the object  
  
    ...  
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[18.15] Why am I getting an error converting a `Foo**` → `const Foo**`?

Because converting `Foo**` → `const Foo**` would be invalid and dangerous.

C++ allows the (safe) conversion `Foo*` → `const Foo*`, but gives an error if you try to implicitly convert `Foo**` → `const Foo**`.

The rationale for why that error is a good thing is given below. But first, here is the most common solution: simply change `const Foo**` to `const Foo* const*`:

```
class Foo { /*...*/ };

void f(const Foo** p);
void g(const Foo* const* p);

int main()
{
    Foo** p = /*...*/;
    ...
    f(p); // ERROR: it's illegal and immoral to convert Foo** to const Foo**
    g(p); // OK: it's legal and moral to convert Foo** to const Foo* const*
    ...
}
```

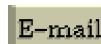
The reason the conversion from `Foo**` → `const Foo**` is dangerous is that it would let you silently and accidentally modify a `const Foo` object without a cast:

```
class Foo {
public:
    void modify(); // make some modify to the this object
};

int main()
{
    const Foo x;
    Foo* p;
    const Foo** q = &p; // q now points to p; this is (fortunately!) an error
    *q = &x;           // p now points to X
    p->modify();        // Ouch: modifies a const Foo!!
    ...
}
```

Reminder: *please* do *not* pointer-cast your way around this. Just Say No!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[19] Inheritance — basics

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [19]:

- [\[19.1\] Is inheritance important to C++?](#)
 - [\[19.2\] When would I use inheritance?](#)
 - [\[19.3\] How do you express inheritance in C++?](#)
 - [\[19.4\] Is it OK to convert a pointer from a derived class to its base class?](#)
 - [\[19.5\] What's the difference between public, private, and protected?](#)
 - [\[19.6\] Why can't my derived class access private things from my base class?](#)
 - [\[19.7\] How can I protect derived classes from breaking when I change the internal parts of the base class?](#)
 - [\[19.8\] I've been told to never use protected data, and instead to always use private data with protected access functions. Is that a good rule?](#)
 - [\[19.9\] Okay, so exactly how should I decide whether to build a "protected interface"?](#)
-

[19.1] Is inheritance important to C++?

Yep.

Inheritance is what separates abstract data type (ADT) programming from OO programming.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[19.2] When would I use inheritance?

As a specification device.

Human beings abstract things on two dimensions: part-of and kind-of. A Ford Taurus is-a-kind-of-a Car, and a Ford Taurus has-a Engine, Tires, etc. The part-of hierarchy has been a part of software since the ADT style became relevant; inheritance adds "the other" major dimension of decomposition.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[19.3] How do you express inheritance in C++?

By the : public syntax:

```
class Car : public Vehicle {
public:
    ...
};
```

We state the above relationship in several ways:

- Car is "a kind of a" Vehicle
- Car is "derived from" Vehicle
- Car is "a specialized" Vehicle
- Car is a "subclass" of Vehicle
- Car is a "derived class" of Vehicle
- Vehicle is the "base class" of Car
- Vehicle is the "superclass" of Car (this not as common in the C++ community)

(Note: this FAQ has to do with public inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[19.4] Is it OK to convert a pointer from a derived class to its base class?

Yes.

An object of a derived class is a kind of the base class. Therefore the conversion from a derived class pointer to a base class pointer is perfectly safe, and happens all the time. For example, if I am pointing at a car, I am in fact pointing at a vehicle, so converting a `Car*` to a `Vehicle*` is perfectly safe and normal:

```
void f(Vehicle* v);
void g(Car* c) { f(c); } // Perfectly safe; no cast
```

(Note: this FAQ has to do with public inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[19.5] What's the difference between public, private, and protected?

- A member (either data member or member function) declared in a private section of a class can only be accessed by member functions and [friends](#) of that class
- A member (either data member or member function) declared in a protected section of a class can only be accessed by member functions and [friends](#) of that class, and by member functions and [friends](#) of derived classes
- A member (either data member or member function) declared in a public section of a class can be accessed by anyone

[19.6] Why can't my derived class access `private` things from my base class?

To protect you from future changes to the base class.

Derived classes do not get access to `private` members of a base class. This effectively "seals off" the derived class from any changes made to the `private` members of the base class.

[19.7] How can I protect derived classes from breaking when I change the internal parts of the base class?

A class has two distinct interfaces for two distinct sets of clients:

- It has a `public` interface that serves unrelated classes
- It has a `protected` interface that serves derived classes

Unless you expect all your derived classes to be built by your own team, you should declare your base class's data members as `private` and use `protected inline` access functions by which derived classes will access the `private` data in the base class. This way the `private` data declarations can change, but the derived class's code won't break (unless you change the `protected` access functions).

[19.8] I've been told to never use protected data, and instead to always use private data with protected access functions. Is that a good rule?

Nope.

Whenever someone says to you, "You should *always* make data private," stop right there — it's an "always" or "never" rule, and those rules are what I call one-size-fits-all rules. The real world isn't that simple.

Here's the way I say it: if I expect derived classes, I should ask this question: who will create them? If the people who will create them will be outside your team, or if there are a *huge* number of derived classes, then and only then is it worth creating a `protected` interface and using `private` data. If I expect the derived classes to be created by my own team and to be reasonable in number, it's just not worth the trouble: use `protected` data. And hold your head up, don't be ashamed: it's the *right thing* to do!

The benefit of protected access functions is that you won't break your derived classes as often as you would if your data was protected. Put it this way: if you believe your users will be outside your team, you should do a lot more than just provide get/set methods for your private data. You should actually create another interface. You have a public interface for one set of users, and a protected interface for another set of users. But they both need an interface that is carefully designed — designed for stability, usability, performance, etc. And at the end of the day, the real benefit of privatizing your data is to avoid breaking your derived classes when you change that data structure.

But if your own team is creating the derived classes, and there are a reasonably small number of them, it's simply not worth the effort: use protected data. Some purists (translation: people who've never stepped foot in the real world, people who've spent their entire lives in an ivory tower, people who don't understand words like "customer" or "schedule" or "deadline" or "ROI") think that *everything* ought to be reusable and *everything* ought to have a clean, easy to use interface. Those kinds of people are dangerous: they often make your project late, since they make everything equally important. They're basically saying, "We have 100 tasks, and I have carefully prioritized them: they are all priority 1." They make the notion of priority meaningless.

You simply will not have enough time to make life easy for *everyone*, so the very best you can do is make life easy for a subset of the world. Prioritize. Select the people that matter most and spend time making stable interfaces for them. You may not like this, but everyone is *not* created equal; some people actually *do* matter more than others. We have a word for those important people. We call them "customers."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[19.9] Okay, so exactly how should I decide whether to build a "protected interface"?

Three keys: ROI, ROI and ROI.

Every interface you build has a cost and a benefit. Every reusable component you build has a cost and a benefit. Every test case, every cleanly structured thing -a-ma-bob, every investment of any sort. You should *never* invest *any* time or *any* money in *any* thing if there is not a positive return on that investment. If it costs your company more than it saves, don't do it!

Not everyone agrees with me on this; they have a right to be wrong. For example, people who live sufficiently far from the real world act like *every* investment is good. After all, they reason, if you wait long enough, it might someday save somebody some time. Maybe. We hope.

That whole line of reasoning is unprofessional and irresponsible. You don't have infinite time, so invest it wisely. Sure, if you live in an ivory tower, you don't have to worry about those pesky things called "schedules" or "customers." But in the real world, you work within a schedule, and you must therefore invest your time only where you'll get good pay-back.

Back to the original question: when should you invest time in building a protected interface? Answer: when you get a good return on that investment. If it's going to cost you an hour, make sure it saves somebody more than an hour, and make sure the savings isn't "someday over the rainbow." If you can save an hour *within* the current project, it's a no-brainer: go for it. If it's going

to save some other project an hour someday maybe we hope, then don't do it. And if it's in between, your answer will depend on exactly how your company trades off the future against the present.

The point is simple: do *not* do something that could damage your schedule. (Or if you do, make sure you never work with me; I'll have your head on a platter.) Investing is good *if* there's a pay-back for that investment. Don't be naive and childish; grow up and realize that some investments are bad because they, in balance, cost more than they return.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

  [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[20] Inheritance — virtual functions

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [20]:

- [\[20.1\] What is a "virtual member function"?](#)
 - [\[20.2\] How can C++ achieve dynamic binding yet also static typing?](#)
 - [\[20.3\] What's the difference between how virtual and non-virtual member functions are called?](#)
 - [\[20.4\] I have a heterogeneous list of objects, and my code needs to do class-specific things to the objects. Seems like this ought to use dynamic binding but can't figure it out. What should I do?](#)
 - [\[20.5\] When should my destructor be virtual?](#)
 - [\[20.6\] What is a "virtual constructor"?](#)
-

[20.1] What is a "virtual member function"?

From an OO perspective, it is the single most important feature of C++: [\[6.8\]](#), [\[6.9\]](#).

A virtual function allows derived classes to replace the implementation provided by the base class. The compiler makes sure the replacement is always called whenever the object in question is actually of the derived class, even if the object is accessed by a base pointer rather than a derived pointer. This allows algorithms in the base class to be replaced in the derived class, even if users don't know about the derived class.

The derived class can either fully replace ("override") the base class member function, or the derived class can partially replace ("augment") the base class member function. The latter is accomplished by having the derived class member function call the base class member function, if desired.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[20.2] How can C++ achieve dynamic binding yet also static typing?

When you have a pointer to an object, the object may actually be of a class that is derived from the class of the pointer (e.g., a `Vehicle*` that is actually pointing to a `Car` object; this is called "polymorphism"). Thus there are two types: the (static) type of the pointer (`Vehicle`, in this case), and the (dynamic) type of the pointed-to object (`Car`, in this case).

Static typing means that the legality of a member function invocation is checked at the earliest possible moment: by the compiler at compile time. The compiler uses the static type of the pointer to determine whether the member function invocation is legal. If the type of the pointer can handle the member function, certainly the pointed-to object can handle it as well. E.g., if `Vehicle` has a certain member function, certainly `Car` also has that member function since `Car` is a kind-of `Vehicle`.

Dynamic binding means that the address of the code in a member function invocation is determined at the last possible moment: based on the dynamic type of the object at run time. It is called "dynamic binding" because the binding to the code that actually gets called is accomplished dynamically (at run time). Dynamic binding is a result of `virtual` functions.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[20.3] What's the difference between how `virtual` and `non-virtual` member functions are called?

`Non-virtual` member functions are resolved statically. That is, the member function is selected statically (at compile-time) based on the type of the pointer (or reference) to the object.

In contrast, `virtual` member functions are resolved dynamically (at run-time). That is, the member function is selected dynamically (at run-time) based on the type of the object, not the type of the pointer/reference to that object. This is called "dynamic binding." Most compilers use some variant of the following technique: if the object has one or more `virtual` functions, the compiler puts a hidden pointer in the object called a "virtual -pointer" or "v-pointer." This v-pointer points to a global table called the "virtual -table" or "v-table."

The compiler creates a v-table for each class that has at least one `virtual` function. For example, if class `Circle` has `virtual` functions for `draw()` and `move()` and `resize()`, there would be exactly one v-table associated with class `Circle`, even if there were a gazillion `Circle` objects, and the v-pointer of each of those `Circle` objects would point to the `Circle` v-table. The v-table itself has pointers to each of the `virtual` functions in the class. For example, the `Circle` v-

table would have three pointers: a pointer to `Circle::draw()`, a pointer to `Circle::move()`, and a pointer to `Circle::resize()`.

During a dispatch of a virtual function, the run-time system follows the object's v-pointer to the class's v-table, then follows the appropriate slot in the v-table to the method code.

The space-cost overhead of the above technique is nominal: an extra pointer per object (but only for objects that will need to do dynamic binding), plus an extra pointer per method (but only for virtual methods). The time-cost overhead is also fairly nominal: compared to a normal function call, a virtual function call requires two extra fetches (one to get the value of the v-pointer, a second to get the address of the method). None of this runtime activity happens with non-virtual functions, since the compiler resolves non-virtual functions exclusively at compile-time based on the type of the pointer.

Note: the above discussion is simplified considerably, since it doesn't account for extra structural things like multiple inheritance, virtual inheritance, RTTI, etc., nor does it account for space/speed issues such as page faults, calling a function via a pointer-to-function, etc. If you want to know about those other things, please ask [comp.lang.c++. PLEASE DO NOT SEND E-MAIL TO ME!](#)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[20.4] I have a heterogeneous list of objects, and my code needs to do class-specific things to the objects. Seems like this ought to use dynamic binding but can't figure it out. What should I do?

It's surprisingly easy.

Suppose there is a base class `Vehicle` with derived classes `Car` and `Truck`. The code traverses a list of `Vehicle` objects and does different things depending on the type of `Vehicle`. For example it might weigh the `Truck` objects (to make sure they're not carrying too heavy of a load) but it might do something different with a `Car` object — check the registration, for example.

The initial solution for this, at least with most people, is to use an `if` statement. E.g., "if the object is a `Truck`, do this, else if it is a `Car`, do that, else do a third thing":

```
typedef std::vector<Vehicle*> VehicleList;

void myCode(VehicleList& v)
{
    for (VehicleList::iterator p = v.begin(); p != v.end(); ++p) {
        Vehicle& v = **p; // just for shorthand

        // generic code that works for any vehicle...
        ...

        // perform the "foo-bar" operation.
        // note: the details of the "foo-bar" operation depend
        // on whether we're working with a car or a truck.
        if (v is a Car) {
```

```

        // car-specific code that does "foo-bar" on car v
        ...
    } else if (v is a Truck) {
        // truck-specific code that does "foo-bar" on truck v
        ...
    } else {
        // semi-generic code that does "foo-bar" on something else
        ...
    }

    // generic code that works for any vehicle...
    ...
}
}

```

The problem with this is what I call "else-if-heimer's disease" (say it fast and you'll understand). The above code gives you else-if-heimer's disease because eventually you'll forget to add an `else if` when you add a new derived class, and you'll probably have a bug that won't be detected until run-time, or worse, when the product is in the field.

The solution is to use dynamic binding rather than dynamic typing. Instead of having (what I call) the live-code dead-data metaphor (where the code is alive and the car/truck objects are relatively dead), we move the code into the data. This is a slight variation of Bertrand Meyer's *Inversion Principle*.

The idea is simple: use the *description* of the code within the `{...}` blocks of each `if` (in this case it is "the foo-bar operation"; obviously your name will be different). Just pick up this descriptive name and use it as the name of a new `virtual` member function in the base class (in this case we'll add a `fooBar()` member function to class `Vehicle`).

```

class Vehicle {
public:
    // performs the "foo-bar" operation
    virtual void fooBar() = 0;
};

```

Then you remove the whole `if...else if...` block and replace it with a simple call to this `virtual` function:

```

typedef std::vector<Vehicle*> VehicleList;

void myCode(VehicleList& v)
{
    for (VehicleList::iterator p = v.begin(); p != v.end(); ++p) {
        Vehicle& v = **p; // just for shorthand

        // generic code that works for any vehicle...
        ...

        // perform the "foo-bar" operation.
        v.fooBar();

        // generic code that works for any vehicle...
        ...
    }
}

```

Finally you *move* the code that used to be in the `{...}` block of each `if` into the `fooBar()` member function of the appropriate derived class:

```
class Car : public Vehicle {
public:
    virtual void fooBar();
};

void Car::fooBar()
{
    // car-specific code that does "foo-bar" on 'this'
    ... ← this is the code that was in {...} of if (v is a Car)
}

class Truck : public Vehicle {
public:
    virtual void fooBar();
};

void Truck::fooBar()
{
    // truck-specific code that does "foo-bar" on 'this'
    ... ← this is the code that was in {...} of if (v is a Truck)
}
```

If you actually have an `else` block in the original `myCode()` function (see above for the "semi-generic code that does the 'foo-bar' operation on something other than a Car or Truck"), change `Vehicle's fooBar()` from pure virtual to plain virtual and move the code into that member function:

```
class Vehicle {
public:
    // performs the "foo-bar" operation
    virtual void fooBar();
};

void Vehicle::fooBar()
{
    // semi-generic code that does "foo-bar" on something else
    ... ← this is the code that was in {...} of the else
    // you can think of this as "default" code...
}
```

That's it!

The point, of course, is that we try to avoid decision logic with decisions based on the kind -of derived class you're dealing with. In other words, you're trying to avoid `if the object is a car do xyz, else if it's a truck do pqr`, etc., because that leads to else-if-heimer's disease.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[20.5] When should my destructor be `virtual`?

When someone will delete a derived-class object via a base-class pointer.

In particular, here's when you need to make your destructor `virtual`:

- *if* someone will derive from your class,
- *and if* someone will say `new Derived`, where `Derived` is derived from your class,
- *and if* someone will say `delete p`, where the actual object's type is `Derived` but the pointer `p`'s type is your class.

Confused? Here's a simplified rule of thumb that usually protects you and usually doesn't cost you anything: make your destructor `virtual` if your class has *any* virtual functions. Rationale:

- that *usually* protects you because most base classes have at least one `virtual` function.
- that *usually* doesn't cost you anything because there is no added per-object space-cost for the second or subsequent `virtual` in your class. In other words, you've already paid all the per-object space-cost that you'll ever pay once you add the first `virtual` function, so the `virtual` destructor doesn't add any additional per-object space cost. (Everything in this bullet is *theoretically* compiler-specific, but in practice it will be valid on almost all compilers.)

Note: if your base class has a `virtual` destructor, then your destructor is automatically `virtual`. You might need an explicit destructor for other reasons, but there's no need to redeclare a destructor simply to make sure it is `virtual`. No matter whether you declare it with the `virtual` keyword, declare it without the `virtual` keyword, or don't declare it at all, it's still `virtual`.

BTW, if you're interested, here are the mechanical details of *why* you need a `virtual` destructor when someone says `delete` using a `Base` pointer that's pointing at a `Derived` object. When you say `delete p`, and the class of `p` has a `virtual` destructor, the destructor that gets invoked is the one associated with the type of the object `*p`, not necessarily the one associated with the type of the pointer. This is A Good Thing. In fact, violating that rule makes your program undefined. The technical term for that is, "Yuck."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[20.6] What is a "virtual constructor"?

An idiom that allows you to do something that C++ doesn't directly support.

You can get the effect of a `virtual` constructor by a `virtual clone()` member function (for copy constructing), or a `virtual create()` member function (for the [default constructor](#)).

```
class Shape {
public:
    virtual ~Shape() { }                // A virtual destructor
    virtual void draw() = 0;            // A pure virtual function
    virtual void move() = 0;
    ...
    virtual Shape* clone() const = 0;    // Uses the copy constructor
    virtual Shape* create() const = 0;    // Uses the default constructor
```

```
};

class Circle : public Shape {
public:
    Circle* clone() const;    // Covariant Return Types; see below
    Circle* create() const;  // Covariant Return Types; see below
    ...
};

Circle* Circle::clone() const { return new Circle(*this); }
Circle* Circle::create() const { return new Circle(); } }
```

In the `clone()` member function, the `new Circle(*this)` code calls `Circle`'s copy constructor to copy the state of `this` into the newly created `Circle` object. (Note: unless `Circle` is known to be [final \(AKA a leaf\)](#), you can reduce the chance of [slicing](#) by making its copy constructor protected.) In the `create()` member function, the `new Circle()` code calls `Circle`'s [default constructor](#).

Users use these as if they were "virtual constructors":

```
void userCode(Shape& s)
{
    Shape* s2 = s.clone();
    Shape* s3 = s.create();
    ...
    delete s2;    // You need a virtual destructor here
    delete s3;
}
```

This function will work correctly regardless of whether the `Shape` is a `Circle`, `Square`, or some other kind-of `Shape` that doesn't even exist yet.

Note: The return type of `Circle`'s `clone()` member function is intentionally different from the return type of `Shape`'s `clone()` member function. This is called *Covariant Return Types*, a feature that was not originally part of the language. If your compiler complains at the declaration of `Circle* clone() const` within class `Circle` (e.g., saying "The return type is different" or "The member function's type differs from the base class virtual function by return type alone"), you have an old compiler and you'll have to change the return type to `Shape*`.

Note: If you are using Microsoft Visual C++ 6.0, you need to change the return types in the derived classes to `Shape*`. This is because MS VC++ 6.0 does not support this feature of the language. Please do *not* write me about this; the above code is correct with respect to the C++ Standard (see 10.3p5); the problem is with MS VC++ 6.0. Fortunately covariant return types are properly supported by MS VC++ 7.0.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
[Download your own copy](#)]

Revised Feb 15, 2005



[21] Inheritance — proper inheritance and substitutability

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [21]:

- [\[21.1\] Should I hide member functions that were public in my base class?](#)
 - [\[21.2\] Converting `Derived\*` → `Base\*` works OK; why doesn't `Derived\*\*` → `Base\*\*` work?](#)
 - [\[21.3\] Is a parking-lot-of-car a kind-of parking-lot-of-vehicle?](#)
 - [\[21.4\] Is an array of `Derived` a kind-of array of `Base`?](#)
 - [\[21.5\] Does array-of-`Derived` is-not-a-kind-of array-of-`Base` mean arrays are bad?](#)
 - [\[21.6\] Is a `Circle` a kind-of an `Ellipse`?](#)
 - [\[21.7\] Are there other options to the "`Circle` is/isnot kind-of `Ellipse`" dilemma?](#)
 - [\[21.8\] But I have a Ph.D. in Mathematics, and I'm sure a `Circle` is a kind of an `Ellipse`! Does this mean Marshall Cline is stupid? Or that C++ is stupid? Or that OO is stupid?](#)
UPDATED!
 - [\[21.9\] Perhaps `Ellipse` should inherit from `Circle` then?](#)
 - [\[21.10\] But my problem doesn't have anything to do with circles and ellipses, so what good is that silly example to me?](#)
 - [\[21.11\] How could "it depend"??!? Aren't terms like "`Circle`" and "`Ellipse`" defined mathematically?](#)
 - [\[21.12\] If `SortedList` has exactly the same public interface as `List`, is `SortedList` a kind-of `List`?](#)
-

[21.1] Should I hide member functions that were public in my base class?

Never, never, never do this. Never. *Never!*

Attempting to hide (eliminate, revoke, privatize) inherited public member functions is an all-too-common design error. It usually stems from muddy thinking.

(Note: this FAQ has to do with public inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.2] Converting `Derived*` → `Base*` works OK; why doesn't `Derived**` → `Base**` work?

Because converting `Derived**` $\rightarrow$ `Base**` would be invalid and dangerous.

C++ allows the conversion `Derived*` $\rightarrow$ `Base*`, since a `Derived` object is a kind of a `Base` object. However trying to convert `Derived**` $\rightarrow$ `Base**` is flagged as an error. Although this error may not be obvious, it is nonetheless a good thing. For example, if you could convert `Car**` $\rightarrow$ `Vehicle**`, and if you could similarly convert `NuclearSubmarine**` $\rightarrow$ `Vehicle**`, you could assign those two pointers and end up making a `Car*` point at a `NuclearSubmarine`:

```
class Vehicle {
public:
    virtual ~Vehicle() { }
    virtual void startEngine() = 0;
};

class Car : public Vehicle {
public:
    virtual void startEngine();
    virtual void openGasCap();
};

class NuclearSubmarine : public Vehicle {
public:
    virtual void startEngine();
    virtual void fireNuclearMissle();
};

int main()
{
    Car    car;
    Car*   carPtr = &car;
    Car**  carPtrPtr = &carPtr;
    Vehicle** vehiclePtrPtr = carPtrPtr; // This is an error in C++
    NuclearSubmarine sub;
    NuclearSubmarine* subPtr = &sub;
    *vehiclePtrPtr = subPtr;
    // This last line would have caused carPtr to point to sub !
    carPtr->openGasCap(); // This might call fireNuclearMissle()!
    ...
}
```

In other words, if it were legal to convert `Derived**` $\rightarrow$ `Base**`, the `Base**` could be dereferenced (yielding a `Base*`), and the `Base*` could be made to point to an object of a *different* derived class, which could cause serious problems for national security (who knows what would happen if you invoked the `openGasCap()` member function on what you thought was a `car`, but in reality it was a `NuclearSubmarine`!! Try the above code out and see what it does — on most compilers it will call `NuclearSubmarine::fireNuclearMissle()`!

(BTW you'll need to use a pointer cast to get it to compile. Suggestion: try to compile it without a pointer cast to see what the compiler does. If you're really quiet when the error message appears on the screen, you should be able to hear the muffled voice of your compiler pleading with you, "Please don't use a pointer cast! Pointer casts prevent me from telling you about errors in your code, but they don't make your errors go away! Pointer casts are [evil](#)!" At least that's what my compiler says.)

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.3] Is a parking-lot-of-car a kind-of parking-lot-of-vehicle?

Nope.

I know it sounds strange, but it's true . You can think of this as a direct consequence of [the previous FAQ](#), or you can reason it this way: if the kind-of relationship were valid, then someone could point a parking-lot-of-vehicle pointer at a parking-lot-of-car, which would allow someone to add *any* kind of vehicle to a parking-lot-of-car (assuming parking-lot-of-vehicle has a member function like `add(Vehicle&)`). In other words, you could park a Bicycle, SpaceShuttle, or even a NuclearSubmarine in a parking-lot-of-car. Certainly it would be surprising if someone accessed what they thought was a car from the parking-lot-of-car, only to find that it is actually a NuclearSubmarine. Gee, I wonder what the `openGasCap()` method would do??

Perhaps this will help: a container of `Thing` is *not* a kind-of container of `Anything` even if a `Thing` is a kind-of an `Anything`. Swallow hard; it's true.

You don't have to like it. But you do have to accept it.

One last example which we use in our OO/C++ training courses: "A Bag-of-Apple is *not* a kind-of Bag-of-Fruit." If a Bag-of-Apple *could* be passed as a Bag-of-Fruit, someone could put a Banana into the Bag, even though it is supposed to only contain Apples!

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.4] Is an array of Derived a kind-of array of Base?

Nope.

This is a corollary of the previous FAQ. Unfortunately this one can get you into a *lot* of hot water. Consider this:

```
class Base {
public:
    virtual void f();           //1
};

class Derived : public Base {
public:
    ...
private:
```

```

    int i_; // 2
};

void userCode(Base* arrayOfBase)
{
    arrayOfBase[1].f(); // 3
}

int main()
{
    Derived arrayOfDerived[10]; // 4
    userCode(arrayOfDerived); // 5
    ...
}

```

The compiler thinks this is perfectly type-safe. Line 5 converts a `Derived*` to a `Base*`. But in reality it is horrendously evil: since `Derived` is larger than `Base`, the pointer arithmetic done on line 3 is incorrect: the compiler uses `sizeof(Base)` when computing the address for `arrayOfBase[1]`, yet the array is an array of `Derived`, which means the address computed on line 3 (and the subsequent invocation of member function `f()`) isn't even at the beginning of any object! It's smack in the middle of a `Derived` object. Assuming your compiler uses the usual approach to [virtual](#) functions, this will reinterpret the `int i_` of the first `Derived` as if it pointed to a virtual table, it will follow that "pointer" (which at this point means we're digging stuff out of a random memory location), and grab one of the first few words of memory at that location and interpret them as if they were the address of a C++ member function, then load that (random memory location) into the instruction pointer and begin grabbing machine instructions from that memory location. The chances of this crashing are very high.

The root problem is that C++ can't distinguish between a pointer -to-a-thing and a pointer-to-an-array-of-things. Naturally C++ "inherited" this feature from C.

NOTE: If we had used an array-like *class* (e.g., `std::vector<Derived>` from [the standard library](#)) instead of using a raw array, this problem would have been properly trapped as an error at compile time rather than a run-time disaster.

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.5] Does array-of-Derived is-not-a-kind-of array-of-Base mean arrays are bad?

Yes, [arrays are evil](#). (only half kidding).

Seriously, arrays are very closely related to pointers, and pointers are notoriously difficult to deal with. But if you have a complete grasp of why the above few FAQs were a problem from a design perspective (e.g., if you really know why a container of `Thing` is not a kind-of container of `Anything`), and if you think everyone else who will be maintaining your code also has a full grasp on these OO design truths, then you should feel free to use arrays. But if you're like most people, you should use a template container class such as `std::vector<T>` from [the standard library](#) rather than raw arrays.

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.6] Is a circle a kind-of an `Ellipse`?

Depends. But not if `Ellipse` guarantees it can change its size asymmetrically.

For example, if `Ellipse` has a `setSize(x,y)` member function that promises the object's `width()` will be `x` and its `height()` will be `y`, `Circle` can't be a kind-of `Ellipse`. Simply put, if `Ellipse` can do something `Circle` can't, then `Circle` can't be a kind of `Ellipse`.

This leaves two valid relationships between `Circle` and `Ellipse`:

- Make `Circle` and `Ellipse` completely unrelated classes
- Derive `Circle` and `Ellipse` from a base class representing "Ellipses that can't *necessarily* perform an `unequal-setSize()` operation"

In the first case, `Ellipse` could be derived from `class AsymmetricShape`, and `setSize(x,y)` could be introduced in `AsymmetricShape`. However `Circle` could be derived from `SymmetricShape` which has a `setSize(size)` member function.

In the second case, `class Oval` could only have `setSize(size)` which sets both the `width()` and the `height()` to `size`. `Ellipse` and `Circle` could both inherit from `Oval`. `Ellipse` —but not `Circle`— could add the `setSize(x,y)` operation (but beware of the [hiding rule](#) if the same member function name `setSize()` is used for both operations).

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

(Note: `setSize(x,y)` isn't sacred. Depending on your goals, it may be okay to prevent users from changing the dimensions of an `Ellipse`, in which case it would be a valid design choice to not have a `setSize(x,y)` method in `Ellipse`. However this series of FAQs discusses what to do when you want to create a derived class of a *pre-existing* base class that has an "unacceptable" method in it. Of course the ideal situation is to discover this problem when the base class doesn't yet exist. But life isn't always ideal...)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.7] Are there other options to the "`Circle` is/isnot kind-of `Ellipse`" dilemma?

If you claim that all `Ellipses` can be squashed asymmetrically, and you claim that `Circle` is a kind-of `Ellipse`, and you claim that `Circle` can't be squashed asymmetrically, clearly you've got to revoke one of your claims. You can get rid of `Ellipse::setSize(x,y)`, get rid of the inheritance relationship between `Circle` and `Ellipse`, or admit that your `Circles` aren't

necessarily circular. You can also get rid of `circle` completely, where circleness is just a temporary state of an `Ellipse` object rather than a permanent quality of the object.

Here are the two most common traps new OO/C++ programmers regularly fall into. They attempt to use coding hacks to cover up a broken design, e.g., they might redefine `Circle::setSize(x,y)` to throw an exception, call `abort()`, choose the average of the two parameters, or to be a no-op. Unfortunately all these hacks will surprise users, since users are expecting `width() == x` and `height() == y`. The one thing you must not do is surprise your users.

If it is important to you to retain the "circle is a kind-of `Ellipse`" inheritance relationship, you can weaken the promise made by `Ellipse`'s `setSize(x,y)`. E.g., you could change the promise to, "This member function *might* set `width()` to `x` and/or it *might* set `height()` to `y`, or it might do *nothing*". Unfortunately this dilutes the contract into dribble, since the user can't rely on any meaningful behavior. The whole hierarchy therefore begins to be worthless (it's hard to convince someone to use an object if you have to shrug your shoulders when asked what the object does for them).

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

(Note: `setSize(x,y)` isn't sacred. Depending on your goals, it may be okay to prevent users from changing the dimensions of an `Ellipse`, in which case it would be a valid design choice to not have a `setSize(x,y)` method in `Ellipse`. However this series of FAQs discusses what to do when you want to create a derived class of a *pre-existing* base class that has an "unacceptable" method in it. Of course the ideal situation is to discover this problem when the base class doesn't yet exist. But life isn't always ideal...)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.8] But I have a Ph.D. in Mathematics, and I'm sure a Circle is a kind of an Ellipse! Does this mean Marshall Cline is stupid? Or that C++ is stupid? Or that OO is stupid? **UPDATED!**

[Recently added the paragraph on constant Ellipses thanks to [Luke Palmer](#) (in 12/04). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Actually, it doesn't mean any of these things. But I'll tell you what it does mean — you may not like what I'm about to say: it means your intuitive notion of "kind of" is leading you to make bad inheritance decisions. Your tummy is lying to you about what good inheritance really means — stop believing those lies.

Look, I have received and answered dozens of passionate e-mail messages about this subject. I have taught it hundreds of times to thousands of software professionals all over the place. I know it goes against your intuition. But trust me; your intuition is wrong, where "wrong" means "will cause you to make bad inheritance decisions in OO design/programming."

Here's how to make good inheritance decisions in OO design/programming: recognize that the derived class objects must be *substitutable* for the base class objects. That means objects of the

derived class must *behave* in a manner consistent with the promises made in the base class' contract. Once you believe this, and I fully recognize that you might not yet but you will if you work at it with an open mind, you'll see that `setSize(x,y)` violates this substitutability.

There are three ways to fix this problem:

1. Soften the promises made by `setSize(x,y)` in base class `Ellipse`, or perhaps remove that method completely, at the risk of breaking existing code that calls `setSize(x,y)`.
2. Strengthen the promises made by `setSize(x,y)` in the derived class `Circle`, which really means allowing a `Circle` to have a different height than width — an asymmetrical circle; hmmm.
3. Drop the inheritance relationship, possibly getting rid of class `Circle` completely (in which case circleness would simply be a temporary state of an `Ellipse` rather than a permanent constraint on the object).

Sorry, but there simply are no other choices.

You must make the base class weaker (weaken `Ellipse` to the point that it no longer guarantees you can set its width and height to different values), make the derived class stronger (empower a `Circle` with the ability to be both symmetric and, ahem, asymmetric), or admit that a `Circle` is not substitutable for `Ellipse`.

Important: there really are no other choices than the above three. In particular:

1. *PLEASE* don't write me and tell me that a fourth option is to derive both `Circle` and `Ellipse` from a third common base class. That's *not* a fourth solution. That's just a repackaging of solution #3: it works precisely because it removes the inheritance relationship between `Circle` and `Ellipse`.
2. *PLEASE* don't write me and tell me that a fourth option is to prevent users from changing the dimensions of an "Ellipse." That is *not* a fourth solution. That's just a repackaging of solution #1: it works precisely because it removes that guarantee that `setSize(x,y)` actually sets the width and height.
3. *PLEASE* don't write me and tell me that you've decided one of these three is "the best" solution. Doing that would show you had missed the whole point of this FAQ, specifically that bad inheritance is subtle but fortunately you have three (not one; not two; but three) possible ways to dig yourself out. So when you run into bad inheritance, please try all three of these techniques and select the best, perhaps "least bad," of the three. Don't throw out two of these tools ahead of time: try them all.

(Note: this FAQ has to do with public inheritance; [private and protected inheritance](#) are different.)

(Note: some people correctly point out that a *constant* `Circle` is substitutable for a *constant* `Ellipse`. That's true, but it's really not a fourth option: it's really just a special case of option #1, since it works precisely because a constant `Ellipse` doesn't have a `setSize(x,y)` method.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.9] Perhaps Ellipse should inherit from Circle then?

If `Circle` is the base class and `Ellipse` is the derived class, then you run into a whole new set of problems. For example, suppose `Circle` has a `radius()` method. Then `Ellipse` will also need to have a `radius()` method, but that doesn't make much sense: what does it even mean for a possibly asymmetric ellipse to have a radius?

If you get over that hurdle, such as by having `Ellipse::radius()` return the average of the major and minor axes or whatever, then there is a problem with the relationship between `radius()` and `area()`. Suppose `Circle` has an `area()` method that promises to return 3.14159 [etc] times the square whatever `radius()` returns. Then either `Ellipse::area()` will not return the true area of the ellipse, or you'll have to stand on your head to get `radius()` to return something that matches the above formula.

Even if you get past that one, such as by having `Ellipse::radius()` return the square root of the ellipse's area divided by π , you'll get stuck by the `circumference()` method. Suppose `Circle` has a `circumference()` method that promises to return two times π times whatever is returned by `radius()`. Now you're stuck: there's no way to make all those constraints work out for `Ellipse`: the `Ellipse` class will have to lie about its area, its circumference, or both.

Bottom line: you can make anything inherit from anything *provided* the methods in the derived class abide by the promises made in the base class. But you ought not to use inheritance just because you feel like it, or just because you want to get code reuse. You should use inheritance (a) only if the derived class's methods can abide by all the promises made in the base class, and (b) only if you don't think you'll confuse your users, and (c) only if there's something to be gained by using the inheritance — some real, measurable improvement in time, money or risk.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.10] But my problem doesn't have anything to do with circles and ellipses, so what good is that silly example to me?

Ahhh, there's the rub. You *think* the `Circle`/`Ellipse` example is just a silly example. But in reality, *your* problem is an isomorphism to that example.

I don't care what your inheritance problem is, but all —yes *all*— bad inheritances boil down to the `Circle-is-not-a-kind-of-Ellipse` example.

Here's why: Bad inheritances always have a base class with an extra capability (often an extra member function or two; sometimes an extra promise made by one or a combination of member functions) that a derived class can't satisfy. You've either got to make the base class weaker, make the derived class stronger, or eliminate the proposed inheritance relationship. I've seen lots and lots and lots of these bad inheritance proposals, and believe me, they all boil down to the `Circle`/`Ellipse` example.

Therefore, if you truly understand the `Circle`/`Ellipse` example, you'll be able to recognize bad inheritance everywhere. If you don't understand what's going on with the `Circle`/`Ellipse`

problem, the chances are high that you'll make some very serious and very expensive inheritance mistakes.

Sad but true.

(Note: this FAQ has to do with `public` inheritance; [private and protected inheritance](#) are different.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.11] How could "it depend"??? Aren't terms like "Circle" and "Ellipse" defined mathematically?

It's irrelevant that those terms are defined mathematically. That irrelevance is why "it depends."

The first step in any rational discussion is to define terms. In this case, the first step is to define the terms `Circle` and `Ellipse`. Believe it or not, most heated disagreements over whether class `Circle` should/shouldn't inherit from class `Ellipse` are caused by incompatible definitions of those terms.

The key insight is to forget mathematics and "the real world," and instead accept as final the only definitions that are relevant for answering the question: the classes themselves. Take `Ellipse`. You created a class with that name, so the one and only final arbiter of what you meant by that term is your class. People who try to mix "the real world" into the discussion get hopelessly confused, and often get into heated (and, sadly, meaningless) arguments.

Since so many people just don't get it, here's an example. Suppose your program says `class Foo : public Bar { ... }`. This *defines* what you mean by the term `Foo`: the one, final, unambiguous, precise definition of `Foo` is given by unioning the public parts of `Foo` with the public parts of its base class, `Bar`. Now suppose you decide to rename `Bar` to `Ellipse` and `Foo` to `Circle`. This means that you (yes you; not "mathematics"; not "history"; not "precedence" nor Euclid nor Euler nor any other famous mathematician; little old you) have *defined* the meaning of the term `Circle` within your program. If you defined it in a way that didn't correspond to people's intuitive notion of circles, then you probably should have chosen a better label for your class, but nonetheless your definition is the one, final, unambiguous, precise definition of the term `Circle` in your program. If somebody else outside your program defines the same term differently, that other definition is irrelevant to questions about your program, even if the "somebody else" is Euclid. *Within your program*, you define the terms, and the term `Circle` is defined by your class named `Circle`.

Simply put, when we are asking questions about words defined in *your* program, we must use *your* definitions of those terms, not Euclid's. That is why the ultimate answer to the question is "it depends." It depends because the answer to whether the thing your program calls `Circle` is properly substitutable for the thing your program calls `Ellipse` depends on exactly how *your program* defines those terms. It's ridiculous and misleading to use Euclid's definition when trying to answer questions about your classes in your program; we *must* use your definitions.

When someone gets heated about this, I always suggest changing the labels to terms that have no predetermined connotations, such as `Foo` and `Bar`. Since those terms do not evoke any mathematical relationships, people naturally go to the class definition to find out exactly what the programmer had in mind. But as soon as we rename the class from `Foo` to `Circle`, some people suddenly think *they* can control the meaning of the term; they're wrong and silly. The definition of the term is still spelled out exclusively by the class itself, not by any outside entity.

Next insight: inheritance means "is substitutable for." It does *not* mean "is a" (since that is ill defined) and it does *not* mean "is a kind of" (also ill defined). Substitutability is well defined: to be substitutable, the derived class is allowed (not required) to add (not remove) public methods, and for each public method inherited from the base class, the derived class is allowed (not required) to weaken preconditions and/or strengthen postconditions (not the other way around). Further the derived class is allowed to have completely different constructors, static methods, and non-public methods.

Back to `Ellipse` and `Circle`: if you define the term `Ellipse` to mean something that can be resized asymmetrically (e.g., its methods let you change the width and height independently and guarantee that the width and height will actually change to the specified values), then that is the final, precise definition of the term `Ellipse`. If you define the thing called `Circle` as something that cannot be resized asymmetrically, then that is also your prerogative, and it is the final, precise definition of the term `Circle`. If you defined those terms in that way, then obviously the thing you called `Circle` is not substitutable for the thing you called `Ellipse`, therefore the inheritance would be improper. QED.

So the answer is *always* "it depends." In particular, it depends on the *behaviors* of the base and derived classes. It does not depend on the *name* of the base and derived classes, since those are arbitrary labels. (I'm not advocating sloppy names; I am, however, saying that you must not use your intuitive connotation of a name to assume you know what a class does. A class does what it does, not what you think it ought to do based on its name.)

It bothers (some) people that the thing you called `Circle` might not be substitutable for the thing you called `Ellipse`, and to those people I have only two things to say: (a) get over it, and (b) change the labels of the classes if that makes you feel more comfortable. For example, rename `Ellipse` to `ThingThatCanBeResizedAsymmetrically` and `Circle` to `ThingThatCannotBeResizedAsymmetrically`.

Unfortunately I honestly believe that people who feel better after renaming the things are missing the point. The point is this: in OO, a thing is defined by how it *behaves*, not by the label used to name it. Obviously it's important to choose good names, but even so, the name chosen does not define the thing. The definition of the thing is specified by the public methods, including the contracts (preconditions and postconditions) of those methods. Inheritance is proper or improper based on the classes' behaviors, not their names.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[21.12] If `sortedList` has *exactly* the same public interface as `List`, is `sortedList` a kind-of `List`?

Probably not.

The most important insight is that the answer depends on the details of the base class's contract. It is not enough to know that the public interfaces / method signatures are compatible; one also needs to know if the contracts / behaviors are compatible.

The important part of the previous sentence are the words "contracts / behaviors." That phrase goes well beyond the public interface = method signatures = method names and parameter types and constness. A method's contract means its advertised behavior = advertised requirements and promises = advertised preconditions and postconditions. So if the base class has a method `void insert(const Foo& x)`, the contract of that method includes the signature (meaning the name `insert` and the parameter `const Foo&`), but goes well beyond that to include the method's advertised preconditions and postconditions.

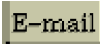
The other important word is *advertised*. The intention here is to differentiate between the code inside the method (assuming the base class's method *has* code; i.e., assuming it's not an unimplemented pure virtual function) and the promises made outside the method. This is where things get tricky. Suppose `List::insert(const Foo& x)` inserts a copy of `x` at the end of *this List*, and the override of that method in `SortedList` inserts `x` in the proper sort-order. Even though the override behaves in a way that is incompatible with the base class's code, the inheritance might still be proper *if* the base class makes a "weak" or "adaptable" promise. For example, if the advertised promise of `List::insert(const Foo& x)` is something vague like, "Promises a copy of `x` will be inserted somewhere within *this List*," then the inheritance is probably okay since the override abides by the advertised behavior even though it is incompatible with the implemented behavior.

The derived class must do what the base class *promises*, not what it actually does.

The key is that we've separated the advertised behavior ("specification") from implemented behavior ("implementation"), and we rely on the specification rather than the implementation. This is very important because in a large percentage of the cases the base class's method is an unimplemented pure virtual — the only thing that can be relied on is the specification — there simply is no implementation on which to rely.

Back to `SortedList` and `List`: it seems likely that `List` has one or more methods that have contracts which guarantee order, and therefore `SortedList` is probably not a kind-of `List`. For example, if `List` has a method that lets you reorder things, prepend things, append things, or change the `i`th element, and if those methods make the typical advertised promise, then `SortedList` would need to violate that advertised behavior and the inheritance would be improper. But it all depends on what the base class advertises — on the base class's contract.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[22] Inheritance — abstract base classes (ABCs)

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [22]:

- [\[22.1\] What's the big deal of separating interface from implementation?](#)
 - [\[22.2\] How do I separate interface from implementation in C++ \(like Modula-2\)?](#)
 - [\[22.3\] What is an ABC?](#)
 - [\[22.4\] What is a "pure virtual" member function?](#)
 - [\[22.5\] How do you define a copy constructor or assignment operator for a class that contains a pointer to a \(abstract\) base class?](#)
-

[22.1] What's the big deal of separating interface from implementation?

Interfaces are a company's most valuable resources. Designing an interface takes longer than whipping together a concrete class which fulfills that interface. Furthermore interfaces require the time of more expensive people.

Since interfaces are so valuable, they should be protected from being tarnished by data structures and other implementation artifacts. Thus you should separate interface from implementation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[22.2] How do I separate interface from implementation in C++ (like Modula -2)?

Use an [ABC](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[22.3] What is an ABC?

An abstract base class.

At the design level, an abstract base class (ABC) corresponds to an abstract concept. If you asked a mechanic if he repaired vehicles, he'd probably wonder what *kind-of* vehicle you had in mind. Chances are he doesn't repair space shuttles, ocean liners, bicycles, or nuclear submarines. The problem is that the term "vehicle" is an abstract concept (e.g., you can't build a

"vehicle" unless you know what kind of vehicle to build). In C++, `class Vehicle` would be an ABC, with `Bicycle`, `SpaceShuttle`, etc, being derived classes (an `OceanLiner` is-a-kind-of-a vehicle). In real-world OO, ABCs show up all over the place.

At the programming language level, an ABC is a `class` that has one or more [pure virtual](#) member functions. You cannot make an object (instance) of an ABC.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[22.4] What is a "pure virtual" member function?

A member function declaration that turns a normal class into an abstract class (i.e., an ABC). You normally only implement it in a derived class.

Some member functions exist in concept; they don't have any reasonable definition. E.g., suppose I asked you to draw a `Shape` at location (x, y) that has size 7. You'd ask me "what kind of shape should I draw?" (circles, squares, hexagons, etc, are drawn differently). In C++, we must indicate the existence of the `draw()` member function (so users can call it when they have a `Shape*` or a `Shape&`), but we recognize it can (logically) be defined only in derived classes:

```
class Shape {
public:
    virtual void draw() const = 0;    // = 0 means it is "pure virtual"
    ...
};
```

This pure virtual function makes `Shape` an ABC. If you want, you can think of the `= 0;` syntax as if the code were at the `NULL` pointer. Thus `Shape` promises a service to its users, yet `Shape` isn't able to provide any code to fulfill that promise. This *forces* any actual object created from a [concrete] class derived from `Shape` to have the indicated member function, even though the base class doesn't have enough information to actually *define* it yet.

Note that it is possible to provide a definition for a pure virtual function, but this usually confuses novices and is best avoided until later.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[22.5] How do you define a copy constructor or assignment operator for a class that contains a pointer to a (abstract) base class?

If the class "owns" the object pointed to by the (abstract) base class pointer, use the [Virtual Constructor Idiom](#) in the (abstract) base class. As usual with this idiom, we declare a [pure virtual](#) `clone()` method in the base class:

```
class Shape {
public:
    ...
    virtual Shape* clone() const = 0;    // The Virtual \(Copy\) Constructor
```

```
...  
};
```

Then we implement this `clone()` method in each derived class. Here is the code for derived class `Circle`:

```
class Circle : public Shape {  
public:  
    ...  
    virtual Circle* clone() const;  
    ...  
};  
  
Circle* Circle::clone() const  
{  
    return new Circle(*this);  
}
```

(Note: the return type in the derived class is [intentionally different](#) from the one in the base class.)

Here is the code for derived class `Square`:

```
class Square : public Shape {  
public:  
    ...  
    virtual Square* clone() const;  
    ...  
};  
  
Square* Square::clone() const  
{  
    return new Square(*this);  
}
```

Now suppose that each `Fred` object "has-a" `Shape` object. Naturally the `Fred` object doesn't know whether the `Shape` is `Circle` or a `Square` or ... `Fred`'s copy constructor and assignment operator will invoke `Shape`'s `clone()` method to copy the object:

```
class Fred {  
public:  
    // p must be a pointer returned by new; it must not be NULL  
    Fred(Shape* p)  
        : p_(p) { assert(p != NULL); }  
    ~Fred()  
        { delete p_; }  
    Fred(const Fred& f)  
        : p_(f.p_->clone()) { }  
    Fred& operator= (const Fred& f)  
        {  
        if (this != &f) {                // Check for self-assignment  
            Shape* p2 = f.p_->clone();    // Create the new one FIRST...  
            delete p_;                    // ...THEN delete the old one  
            p_ = p2;  
        }  
        return *this;  
    }  
}
```

```
...
private:
    Shape* p_;
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[23] Inheritance — what your mother never told you

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [23]:

- [\[23.1\] Is it okay for a non-virtual function of the base class to call a virtual function?](#)
- [\[23.2\] That last FAQ confuses me. Is it a different strategy from the other ways to use virtual functions? What's going on?](#)
- [\[23.3\] When my base class's constructor calls a virtual function on its this object, why doesn't my derived class's override of that virtual function get invoked?](#)
- [\[23.4\] Okay, but is there a way to simulate that behavior as if dynamic binding worked on the this object within my base class's constructor?](#)
- [\[23.5\] Should a derived class redefine \("override"\) a member function that is non-virtual in a base class?](#)
- [\[23.6\] What's the meaning of, Warning: Derived::f\(char\) hides Base::f\(double\) ?](#)
UPDATED!
- [\[23.7\] What does it mean that the "virtual table" is an unresolved external?](#)
- [\[23.8\] How can I set up my class so it won't be inherited from?](#)
- [\[23.9\] How can I set up my member function so it won't be overridden in a derived class?](#)

[23.1] Is it okay for a non-virtual function of the base class to call a virtual function?

Yes. It's sometimes (*not always!*) a great idea. For example, suppose all `Shape` objects have a common algorithm for printing, but this algorithm depends on their area and they all have a potentially different way to compute their area. In this case `Shape's area()` method would necessarily have to be `virtual` (probably `pure virtual`) but `Shape::print()` could, [if we were](#)



[guaranteed no derived class wanted a different algorithm for printing](#), be a non-virtual defined in the base class Shape.

```
#include "Shape.hpp"

void Shape::print() const
{
    float a = this->area(); // area() is pure virtual
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.2] That last FAQ confuses me. Is it a different strategy from the other ways to use virtual functions? What's going on?

Yes, it is a different strategy. Yes, there really are two different basic ways to use virtual functions:

1. Suppose you have the situation described in the previous FAQ: you have a method whose overall structure is the same for each derived class, but has little pieces that are different in each derived class. So the algorithm is the same, but the primitives are different. In this case you'd write the overall algorithm in the base class as a public method (that's sometimes non-virtual), and you'd write the little pieces in the derived classes. The little pieces would be declared in the base class (they're often protected, they're often pure virtual, and they're *certainly* virtual), and they'd ultimately be defined in each derived class. The most critical question in this situation is whether or not the public method containing the overall algorithm should be virtual. The answer is to make it virtual [if you think that some derived class might need to override it](#).
2. Suppose you have the exact opposite situation from the previous FAQ, where you have a method whose overall structure is different in each derived class, yet it has little pieces that are the same in most (if not all) derived classes. In this case you'd put the overall algorithm in a public virtual that's ultimately defined in the derived classes, and the little pieces of common code can be written once (to avoid code duplication) and stashed somewhere (anywhere!). A common place to stash the little pieces is in the protected part of the base class, but that's not necessary and it might not even be best. Just find a place to stash them and you'll be fine. Note that if you do stash them in the base class, you should normally make them protected, since normally they do things that public users don't need/want to do. Assuming they're protected, they probably shouldn't be virtual: if the derived class doesn't like the behavior in one of them, it doesn't have to call that method.

For emphasis, the above list is a both/and situation, not an either/or situation. In other words, you don't have to choose between these two strategies on any given class. It's perfectly normal to have method `f()` correspond to strategy #1 while method `g()` corresponds to strategy #2. In other words, it's perfectly normal to have both strategies working in the same class.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.3] When my base class's constructor calls a virtual function on its this object, why doesn't my derived class's override of that virtual function get invoked?

Because that would be very dangerous, and C++ is protecting you from that danger.

The rest of this FAQ gives a rationale for why C++ needs to protect you from that danger, but before we start that, be advised that you can get the *effect as if* dynamic binding worked on the this object even during a constructor via [The Dynamic Binding During Initialization Idiom](#).

First, here is an example to explain exactly what C++ actually does:

```
#include <iostream>
#include <string>

void println(const std::string& msg)
{ std::cout << msg << '\n'; }

class Base {
public:
    Base() { println("Base::Base()"); virt(); }
    virtual void virt() { println("Base::virt()"); }
};

class Derived : public Base {
public:
    Derived() { println("Derived::Derived()"); virt(); }
    virtual void virt() { println("Derived::virt()"); }
};

int main()
{
    Derived d;
    ...
}
```

The output from the above program will be:

```
Base::Base()
Base::virt() // ← Not Derived::virt()
Derived::Derived()
Derived::virt()
```

The rest of this FAQ describes *why* C++ does the above. If you're happy merely knowing *what* C++ does without knowing *why*, feel free to skip this stuff.

The explanation for this behavior comes from combining two facts:

1. When you create a Derived object, it first calls Base's constructor. That's why it prints Base::Base() before Derived::Derived().
2. While executing Base::Base(), the this object is not yet of type Derived; its type is still merely Base. That's why the call to [virtual](#) function virt() within Base::Base() binds to Base::virt() even though an override exists in Derived.

Now some of you are still curious, saying to yourself, "Hmmm, but I still wonder *why* the *this* object is merely of type `Base` during `Base::Base()`." If that's you, the answer is that C++ is protecting you from serious and subtle bugs. In particular, if the above rule were different, you could easily use objects *before* they were initialized, and that would cause no end of grief and havoc.

Here's how: imagine for the moment that calling `this->virt()` within `Base::Base()` ended up invoking the override `Derived::virt()`. Overrides can (and often do!) access non-static data members declared in the `Derived` class. But since the non-static data members declared in `Derived` are not initialized during the call to `virt()`, any use of them within `Derived::virt()` would be a "use before initialized" error. Bang, you're dead.

So fortunately the C++ language doesn't let this happen: it makes sure any call to `this->virt()` that occurs while control is flowing through `Base`'s constructor will end up invoking `Base::virt()`, not the override `Derived::virt()`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.4] Okay, but is there a way to *simulate* that behavior as *if* dynamic binding worked on the *this* object within my base class's constructor?

Yes.

To clarify, we're talking about this situation:

```
class Base {
public:
    Base();
    ...
    virtual void foo(int n) const; // often pure virtual
    virtual double bar() const;   // often pure virtual
    // if you don't want outsiders calling these, make them protected
};

Base::Base()
{
    ... foo(42) ... bar() ...
    // these will not use dynamic binding
    // goal: simulate dynamic binding in those calls
}

class Derived : public Base {
public:
    ...
    virtual void foo(int n) const;
    virtual double bar() const;
};
```

This FAQ shows some ways to *simulate* dynamic binding as *if* the calls made in `Base`'s constructor dynamically bound to the *this* object's derived class. The ways we'll show have tradeoffs, so choose the one that best fits your needs, or make up another.

The first approach is a two-phase initialization. In Phase I, someone calls the actual constructor; in Phase II, someone calls an "init" method on the object. Dynamic binding on the `this` object works fine during Phase II, and Phase II is *conceptually* part of construction, so we simply move some code from the original `Base::Base()` into `Base::init()`.

```
class Base {
public:
    void init(); // may or may not be virtual
    ...
    virtual void foo(int n) const; // often pure virtual
    virtual double bar() const;    // often pure virtual
};

void Base::init()
{
    ... foo(42) ... bar() ...
    // most of this is copied from the original Base::Base()
}

class Derived : public Base {
public:
    ...
    virtual void foo(int n) const;
    virtual double bar() const;
};
```

The only remaining issues are determining *where* to call Phase I and *where* to call Phase II. There are many variations on where these calls can live; we will consider two.

The first variation is simplest initially, though the code that actually wants to create objects requires a tiny bit of programmer self-discipline, which in practice means you're doomed. Seriously, if there are only one or two places that actually create objects of this hierarchy, the programmer self-discipline is quite localized and shouldn't cause problems.

In this variation, the code that is creating the object explicitly executes both phases. When executing Phase I, the code creating the object either knows the object's exact class (e.g., `new Derived()` or perhaps a local `Derived` object), or doesn't know the object's exact class (e.g., [the virtual constructor idiom](#) or some other factory). The "doesn't know" case is strongly preferred when you want to make it easy to plug-in new derived classes.

Note: Phase I often, but not always, allocates the object from the heap. When it does, you should store the pointer in some sort of [managed pointer](#), such as a `std::auto_ptr`, a [reference counted pointer](#), or some other object whose [destructor deletes the allocation](#). This is the best way to prevent memory leaks when Phase II might [throw exceptions](#). The following example assumes Phase I allocates the object from the heap.

```
#include <memory>

void joe_user()
{
    std::auto_ptr<Base> p(/*...somehow create a Derived object via new...*/);
    p->init();
    ...
}
```

The second variation is to combine the first two lines of the `joe_user` function into some `create` function. That's almost always the right thing to do when there are lots of `joe_user`-like functions. For example, if you're using some kind of factory, such as a registry and [the virtual constructor idiom](#), you could move those two lines into a static method called `Base::create()`:

```
#include <memory>

class Base {
public:
    ...
    typedef std::auto_ptr<Base> Ptr; // typedefs simplify the code
    static Ptr create();
    ...
};

Base::Ptr Base::create()
{
    Ptr p(/*...use a factory to create a Derived object via new...*/);
    p->init();
    return p;
}
```

This simplifies all the `joe_user`-like functions (a little), but more importantly, it reduces the chance that any of them will create a `Derived` object without also calling `init()` on it.

```
void joe_user()
{
    Base::Ptr p = Base::create();
    ...
}
```

If you're sufficiently clever and motivated, you can even *eliminate* the chance that someone could create a `Derived` object without also calling `init()` on it. An important step in achieving that goal is to [make `Derived`'s constructors, including its copy constructor, protected or private](#).

The next approach does not rely on a two-phase initialization, instead using a second hierarchy whose only job is to house methods `foo()` and `bar()`. This approach doesn't always work, and in particular it doesn't work in cases when `foo()` and `bar()` need to access the instance data declared in `Derived`, but it is conceptually quite simple and clean and is commonly used.

Let's call the base class of this second hierarchy `Helper`, and its derived classes `Helper1`, `Helper2`, etc. The first step is to move `foo()` and `bar()` into this second hierarchy:

```
class Helper {
public:
    virtual void foo(int n) const = 0;
    virtual double bar() const = 0;
};

class Helper1 : public Helper {
public:
    virtual void foo(int n) const;
    virtual double bar() const;
};
```

```
class Helper2 : public Helper {
public:
    virtual void foo(int n) const;
    virtual double bar() const;
};
```

Next, remove `init()` from `Base` (since we're no longer using the two-phase approach), remove `foo()` and `bar()` from `Base` and `Derived` (`foo()` and `bar()` are now in the `Helper` hierarchy), and change the signature of `Base`'s constructor so it takes a `Helper` by reference:

```
class Base {
public:
    Base(const Helper& h);
    ...    // remove init() since not using two-phase this time
    ...    // remove foo() and bar() since they're in Helper
};

class Derived : public Base {
public:
    ...    // remove foo() and bar() since they're in Helper
};
```

We then define `Base::Base(const Helper& h)` so it calls `h.foo(42)` and `h.bar()` in exactly those places that `init()` used to call `this->foo(42)` and `this->bar()`:

```
Base::Base(const Helper& h)
{
    ... h.foo(42) ... h.bar() ...
    // almost identical to the original Base::Base()
    // but with h. in calls to h.foo() and h.bar()
}
```

Finally we change `Derived`'s constructor to pass a (perhaps temporary) object of an appropriate `Helper` derived class to `Base`'s constructor (using the [init list syntax](#)). For example, `Derived` would pass an instance of `Helper2` if it happened to contain the behaviors that `Derived` wanted for methods `foo()` and `bar()`:

```
Derived::Derived()
: Base(Helper2())    // ← the magic happens here
{
    ...
}
```

Note that `Derived` can pass values into the `Helper` derived class's constructor, but it *must not* pass any data members that actually live inside the `this` object. While we're at it, let's explicitly say that `Helper::foo()` and `Helper::bar()` must not access data members of the `this` object, particularly data members declared in `Derived`. (Think about when those data members are initialized and you'll see why.)

Of course the choice of which `Helper` derived class could be made out in the `joe_user`-like function, in which case it would be passed into the `Derived` ctor and then up to the `Base` ctor:

```
Derived::Derived(const Helper& h)
: Base(h)
{
```

```
...  
}
```

If the Helper objects don't need to hold any data, that is, if each is *merely* a collection of its methods, then you can simply pass [static member functions](#) instead. This might be simpler since it entirely eliminates the Helper hierarchy.

```
class Base {  
public:  
    typedef void (*FooFn)(int); // typedefs simplify  
    typedef double (*BarFn)(); // the rest of the code  
    Base(FooFn foo, BarFn bar);  
    ...  
};  
  
Base::Base(FooFn foo, BarFn bar)  
{  
    ... foo(42) ... bar() ...  
    // almost identical to the original Base::Base()  
    // except calls are made via function pointers.  
}
```

The Derived class is also easy to implement:

```
class Derived : public Base {  
public:  
    Derived();  
    static void foo(int n); // the static is important!  
    static double bar(); // the static is important!  
    ...  
};  
  
Derived::Derived()  
: Base(foo, bar) // ← pass the function-ptrs into Base's ctor  
{  
    ...  
}
```

As before, the functionality for `foo()` and/or `bar()` can be passed in from the `joe_user`-like functions. In that case, `Derived`'s ctor just accepts them and passes them up into `Base`'s ctor:

```
Derived::Derived(FooFn foo, BarFn bar)  
: Base(foo, bar)  
{  
    ...  
}
```

A final approach is to use templates to "pass" the functionality into the derived classes. This is similar to the case where the `joe_user`-like functions choose the initializer-function or the Helper derived class, but instead of using function pointers or dynamic binding, it wires the code into the classes via templates.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.5] Should a derived class redefine ("override") a member function that is non-virtual in a base class?

It's legal, but it ain't moral.

Experienced C++ programmers will sometimes redefine a non-virtual function for efficiency (e.g., if the derived class implementation can make better use of the derived class's resources) or to get around the [hiding rule](#). However the client-visible effects must be *identical*, since non-virtual functions are dispatched based on the static type of the pointer/reference rather than the dynamic type of the pointed-to/referenced object.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.6] What's the meaning of, warning: Derived::f(char) hides Base::f(double) ?

UPDATED!

[Recently included a new example, changed the parameter types so the behavior seems more bizarre/ominous, and added a note about warnings not being standardized, thanks to [Daniel Kabs](#) (in 12/04). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

It means you're going to die.

Here's the mess you're in: if Base declares a member function `f(double x)`, and Derived declares a member function `f(char c)` (same name but different parameter types and/or constness), then the Base `f(double x)` is "hidden" rather than "overloaded" or "overridden" (even if the Base `f(double x)` is [virtual](#)).

```
class Base {
public:
    void f(double x);  ← doesn't matter whether or not this is virtual
};

class Derived : public Base {
public:
    void f(char c);  ← doesn't matter whether or not this is virtual
};

int main()
{
    Derived* d = new Derived();
    Base* b = d;
    b->f(65.3);  ← okay: passes 65.3 to f(double x)
    d-
>f(65.3);  ← bizarre: converts 65.3 to a char ('A' if ASCII) and passes it to f(char c); does NOT call f(double x)!!
    return 0;
}
```

Here's how you get out of the mess: Derived must have a using declaration of the hidden member function. For example,

```

class Base {
public:
    void f(double x);
};

class Derived : public Base {
public:
    using Base::f;  ← This un-hides Base::f(double x)
    void f(char c);
};

```

If the using syntax isn't supported by your compiler, redefine the hidden Base member function(s), [even if they are non-virtual](#). Normally this re-definition merely calls the hidden Base member function using the :: syntax. E.g.,

```

class Derived : public Base {
public:
    void f(double x) { Base::f(x); }  ← The redefinition merely calls Base::f(double x)
    void f(char c);
};

```

Note: the hiding problem also occurs if class Base declares a method f(char).

Note: warnings are not part of the standard, so your compiler may or may not give the above warning.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.7] What does it mean that the "virtual table" is an unresolved external?

If you get a link error of the form "Error: Unresolved or undefined symbols detected: virtual table for class Fred," you probably have an undefined [virtual](#) member function in class Fred.

The compiler typically creates a magical data structure called the "virtual table" for classes that have virtual functions (this is how it handles [dynamic binding](#)). Normally you don't have to know about it at all. But if you forget to define a virtual function for class Fred, you will sometimes get this linker error.

Here's the nitty gritty: Many compilers put this magical "virtual table" in the compilation unit that defines the first non-inline virtual function in the class. Thus if the first non-inline virtual function in Fred is wilma(), the compiler will put Fred's virtual table in the same compilation unit where it sees Fred::wilma(). Unfortunately if you accidentally forget to define Fred::wilma(), rather than getting a Fred::wilma() is undefined, you may get a "Fred's virtual table is undefined". Sad but true.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.8] How can I set up my class so it won't be inherited from?

This is known as making the class "final" or "a leaf." There are three ways to do it: an easy technical approach, an even easier non-technical approach, and a slightly trickier technical approach.

The (easy) technical approach is to make the class's constructors `private` and to use [the Named Constructor Idiom](#) to create the objects. No one can create objects of a derived class since the base class's constructor will be inaccessible. The "named constructors" themselves could [return by pointer if you want your objects allocated by new](#) or they could [return by value if you want the objects created on the stack](#).

The (even easier) non-technical approach is to put a big fat ugly comment next to the class definition. The comment could say, for example, `// We'll fire you if you inherit from this class` or even just `/*final*/ class Whatever {...};`. Some programmers balk at this because it is enforced by people rather than by technology, but don't knock it on face value: it is quite effective in practice.

A slightly trickier technical approach is to exploit [virtual inheritance](#). Since [the most derived class's ctor needs to directly call the virtual base class's ctor](#), the following guarantees that no concrete class can inherit from class `Fred`:

```
class Fred;

class FredBase {
private:
    friend class Fred;
    FredBase() { }
};

class Fred : private virtual FredBase {
public:
    ...
};
```

Class `Fred` can access `FredBase`'s ctor, since `Fred` is a friend of `FredBase`, but no class derived from `Fred` can access `FredBase`'s ctor, and therefore no one can create a concrete class derived from `Fred`.

If you are in extremely space-constrained environments (such as an embedded system or a handheld with limited memory, etc.), you should be aware that the above technique might add a word of memory to `sizeof(Fred)`. That's because most compilers implement virtual inheritance by adding a pointer in objects of the derived class. This is compiler specific; your mileage may vary.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[23.9] How can I set up my member function so it won't be overridden in a derived class?

This is known as making the method "final" or "a leaf." Here's an easy -to-use solution to this that gives you 90+% of what you want: simply add a comment next to the method and rely on code reviews or random maintenance activities to find violators. The comment could say, for example,

```
// We'll fire you if you override this method or perhaps more likely, /*final*/ void  
theMethod();
```

The advantages to this technique are (a) it is extremely easy/fast/inexpensive to use, and (b) it is quite effective in practice. In other words, you get 90+% of the benefit with almost no cost — lots of bang per buck.

(I'm not aware of a "100% solution" to this problem so this may be the best you can get. If you know of something better, please let me know, cline@parashift.com. But *please* do *not* email me objecting to this solution because it's low-tech or because it doesn't "prevent" people from doing the wrong thing. Who cares whether it's low-tech or high-tech as long as it's effective?!? And nothing in C++ "prevents" people from doing the wrong thing. Using [pointer casts](#) and pointer arithmetic, people can do just about anything they want. C++ makes it easy to do the right thing, but [it doesn't prevent espionage](#). Besides, the original question (see above) asked for something so people *won't* do the wrong thing, not so they *can't* do the wrong thing.)

In any case, this solution should give you most of the potential benefit at almost no cost.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
[Download your own copy](#)]

Revised Feb 15, 2005

[24] Inheritance — private and protected inheritance

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [24]:

- [\[24.1\] How do you express "private inheritance"?](#)
 - [\[24.2\] How are "private inheritance" and "composition" similar?](#)
 - [\[24.3\] Which should I prefer: composition or private inheritance?](#)
 - [\[24.4\] Should I pointer-cast from a private derived class to its base class?](#)
 - [\[24.5\] How is protected inheritance related to private inheritance?](#)
 - [\[24.6\] What are the access rules with private and protected inheritance?](#)
-

[24.1] How do you express "private inheritance"?

When you use : private instead of : public. E.g.,




```
class Foo : private Bar {
public:
    ...
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[24.2] How are "private inheritance" and "composition" similar?

private inheritance is a syntactic variant of composition (AKA aggregation and/or has-a).

E.g., the "Car has-a Engine" relationship can be expressed using *simple composition*:

```
class Engine {
public:
    Engine(int numCylinders);
    void start();           // Starts this Engine
};

class Car {
public:
    Car() : e_(8) { }       // Initializes this Car with 8 cylinders
    void start() { e_.start(); } // Start this Car by starting its Engine
private:
    Engine e_;              // Car has-a Engine
};
```

The "Car has-a Engine" relationship can also be expressed using *private inheritance*:

```
class Car : private Engine { // Car has-a Engine
public:
    Car() : Engine(8) { }    // Initializes this Car with 8 cylinders
    using Engine::start;     // Start this Car by starting its Engine
};
```

There are several similarities between these two variants:

- In both cases there is exactly one Engine member object contained in every car object
- In neither case can users (outsiders) convert a Car* to an Engine*
- In both cases the car class has a start() method that calls the start() method on the contained Engine object.

There are also several distinctions:

- The simple-composition variant is needed if you want to contain several Engines per Car
- The private-inheritance variant can introduce unnecessary multiple inheritance
- The private-inheritance variant allows members of car to convert a Car* to an Engine*
- The private-inheritance variant allows access to the protected members of the base class
- The private-inheritance variant allows car to override Engine's [virtual](#) functions

- The private-inheritance variant makes it *slightly* simpler (20 characters compared to 28 characters) to give car a `start()` method that simply calls through to the `Engine's start()` method

Note that private inheritance is usually used to gain access into the protected members of the base class, but this is usually a short-term solution (translation: a [band-aid](#)).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[24.3] Which should I prefer: composition or private inheritance?

Use composition when you can, private inheritance when you have to.

Normally you don't *want* to have access to the internals of too many other classes, and private inheritance gives you some of this extra power (and responsibility). But private inheritance isn't evil; it's just more expensive to maintain, since it increases the probability that someone will change something that will break your code.

A legitimate, long-term use for private inheritance is when you want to build a class `Fred` that uses code in a class `Wilma`, and the code from class `Wilma` needs to invoke member functions from your new class, `Fred`. In this case, `Fred` calls non-virtuals in `Wilma`, and `Wilma` calls (usually [pure virtuals](#)) in itself, which are overridden by `Fred`. This would be much harder to do with composition.

```
class Wilma {
protected:
    void fredCallsWilma()
    {
        std::cout << "Wilma::fredCallsWilma()\n";
        wilmaCallsFred();
    }
    virtual void wilmaCallsFred() = 0;    // A pure virtual function
};

class Fred : private Wilma {
public:
    void barney()
    {
        std::cout << "Fred::barney()\n";
        Wilma::fredCallsWilma();
    }
protected:
    virtual void wilmaCallsFred()
    {
        std::cout << "Fred::wilmaCallsFred()\n";
    }
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[24.4] Should I pointer-cast from a private derived class to its base class?

Generally, No.

From a member function or [friend](#) of a privately derived class, the relationship to the base class is known, and the upward conversion from `PrivatelyDer*` to `Base*` (or `PrivatelyDer&` to `Base&`) is safe; no cast is needed or recommended.

However users of `PrivatelyDer` should avoid this unsafe conversion, since it is based on a private decision of `PrivatelyDer`, and is subject to change without notice.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[24.5] How is `protected inheritance` related to `private inheritance`?

Similarities: both allow overriding [virtual](#) functions in the `private/protected` base class, neither claims the derived is a kind-of its base.

Dissimilarities: `protected inheritance` allows derived classes of derived classes to know about the inheritance relationship. Thus your grand kids are effectively exposed to your implementation details. This has both benefits (it allows derived classes of the `protected` derived class to exploit the relationship to the `protected` base class) and costs (the `protected` derived class can't change the relationship without potentially breaking further derived classes).

`Protected inheritance` uses the `: protected` syntax:

```
class Car : protected Engine {
public:
    ...
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[24.6] What are the access rules with `private` and `protected inheritance`?

Take these classes as examples:

```
class B { /*...*/ };
class D_priv : private B { /*...*/ };
class D_prot : protected B { /*...*/ };
class D_publ : public B { /*...*/ };
class UserClass { B b; /*...*/ };
```

None of the derived classes can access anything that is `private` in `B`. In `D_priv`, the `public` and `protected` parts of `B` are `private`. In `D_prot`, the `public` and `protected` parts of `B` are `protected`. In `D_publ`, the `public` parts of `B` are `public` and the `protected` parts of `B` are `protected` (`D_publ` is-a-kind-of-a `B`). `class UserClass` can access only the `public` parts of `B`, which "seals off" `UserClass` from `B`.

To make a public member of B so it is public in D_priv or D_prot, state the name of the member with a B:: prefix. E.g., to make member B::f(int, float) public in D_prot, you would say:

```
class D_prot : protected B {
public:
    using B::f; //Note: Not using B::f(int, float)
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[25] Inheritance — multiple and virtual inheritance

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [25]:

- [\[25.1\] How is this section organized?](#)
- [\[25.2\] I've been told that I should never use multiple inheritance. Is that right?](#)
- [\[25.3\] So there are times when multiple inheritance isn't bad?!??](#) UPDATED!
- [\[25.4\] What are some disciplines for using multiple inheritance?](#)
- [\[25.5\] Can you provide an example that demonstrates the above guidelines?](#)
- [\[25.6\] Is there a simple way to visualize all these tradeoffs?](#)
- [\[25.7\] Can you give another example to illustrate the above disciplines?](#)
- [\[25.8\] What is the "dreaded diamond"?](#)
- [\[25.9\] Where in a hierarchy should I use virtual inheritance?](#)
- [\[25.10\] What does it mean to "delegate to a sister class" via virtual inheritance?](#)
- [\[25.11\] What special considerations do I need to know about when I use virtual inheritance?](#)
- [\[25.12\] What special considerations do I need to know about when I inherit from a class that uses virtual inheritance?](#)
- [\[25.13\] What special considerations do I need to know about when I use a class that uses virtual inheritance?](#)
- [\[25.14\] One more time: what is the exact order of constructors in a multiple and/or virtual inheritance situation?](#)

[25.1] How is this section organized?

This section covers a wide spectrum of questions/answers, ranging from the high -level / strategy / design issues, going all the way down to low -level / tactical / programming issues. We cover them in that order.

Please make *sure* you understand the high-level / strategy / design issues. Too many programmers worry about getting "it" to compile without first deciding whether they really want "it" in the first place. So please read the first several FAQs in this section before worrying about the (important) mechanical details in the last several FAQs.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.2] I've been told that I should never use multiple inheritance. Is that right?

Grrrrrrrrr.

It *really* bothers me when people think they know what's best fo r *your* problem even though they've never seen *your* problem!! How can anybody possibly know that multiple inheritance won't help *you* accomplish *your* goals without knowing *your* goals?!?!?!?!!

Next time somebody tells you that you should never use multiple inheritance, look them straight in the eye and say, "One size does not fit all." If they respond with something about *their* bad experience on *their* project, look them in the eye and repeat, slower this time, "One size does not fit all."

People who spout off one-size-fits-all rules presume to make your design decisions without knowing your requirements. They don't know where you're going but know how you should get there.

Don't trust an answer from someone who doesn't know the question.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.3] So there are times when multiple inheritance isn't bad?!??

UPDATED!

[Recently fixed a typo thanks to [Sumit Rajan](#) (in 12/04). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Of course there are!

You won't use it all the time. You might not even use it regularly. But there are some situations where a solution with multiple inheritance is cheaper to build, debug, test, optimize, and maintain than a solution without multiple inheritance. If multiple inheritance cuts your costs, improves your schedule, reduces your risk, and performs well, then please use it.

On the other hand, just because it's there doesn't mean you should use it. Like any tool, use the right tool for the job. If MI (multiple inheritance) helps, use it; if not, don't. And if you have a bad experience with it, don't blame the tool. Take responsibility for your mistakes, and say, "I used the wrong tool for the job; it was my fault." Do *not* say, "Since it didn't help *my* problem, it's bad for all problems in all industries across all time." Good workmen never blame their tools.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.4] What are some disciplines for using multiple inheritance?

M.I. rule of thumb #1: Use inheritance only if doing so will remove `if / switch` statements from the caller code. *Rationale:* this steers people away from "gratuitous" inheritance (either of the single or multiple variety), which is often a good thing. There are a *few* times when you'll use inheritance without dynamic binding, but beware: if you do that a lot, you may have been infected with wrong thinking. In particular, [inheritance is not for code-reuse](#). You sometimes get a little code reuse via inheritance, but the primary purpose for inheritance is dynamic binding, and that is for flexibility. Composition is for code reuse, inheritance is for flexibility. This rule of thumb isn't specific to MI, but is generic to all usages of inheritance.

M.I. rule of thumb #2: Try especially hard to use [ABCs](#) when you use MI. In particular, most classes above the [join class](#) (and often the join class itself) should be ABCs. In this context, "ABC" doesn't simply mean "a class with at least one pure virtual function"; it actually means a *pure* ABC, meaning a class with as little data as possible (often none), and with most (often all) its methods being [pure virtual](#). *Rationale:* this discipline helps you avoid situations where you need to inherit data or code along two paths, plus it encourages you to use inheritance properly. This second goal is subtle but is extremely powerful. In particular, if you're in the habit of using inheritance for code reuse (dubious at best; see above), this rule of thumb will steer you away from MI and perhaps (hopefully!) away from inheritance -for-code-reuse in the first place. In other words, this rule of thumb tends to push people toward inheritance -for-interface-substitutability, which is always safe, and away from inheritance -just-to-help-me-write-less-code-in-my-derived-class, which is often (not always) unsafe.

M.I. rule of thumb #3: Consider [the "bridge" pattern](#) or [nested generalization](#) as possible alternatives to multiple inheritance. This does not imply that there is something "wrong" with MI; it simply implies that there are at least three alternatives, and a wise designer checks out all the alternatives before choosing which is best.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.5] Can you provide an example that demonstrates the above guidelines?

Suppose you have land vehicles, water vehicles, air vehicles, and space vehicles. (Forget the whole concept of amphibious vehicles for this example; pretend they don't exist for this illustration.) Suppose we also have different power sources: gas powered, wind powered, nuclear powered, pedal powered, etc. We *could* use multiple inheritance to tie everything together, but before we do, we should ask a few tough questions:

1. Will the *users* of `LandVehicle` need to have a `Vehicle` that refers to a `LandVehicle` object? In particular, will the *users* call methods on a `Vehicle`-reference and expect the actual implementation of those methods to be specific to `LandVehicle`s?
2. Ditto for `GasPoweredVehicle`s: will the users want a `Vehicle` reference that refers to a `GasPoweredVehicle` object, and in particular will they want to call methods on that `Vehicle` reference and expect the implementations to get overridden by `GasPoweredVehicle`?

If *both* answers are "yes," multiple inheritance is probably the best way to go. But before you close the door on the alternatives, here are a few more "decision criteria." Suppose there are *N* geographies (land, water, air, space, etc.) and *M* power sources (gas, nuclear, wind, pedal, etc.). There are at least three choices for the overall design: the bridge pattern, nested generalization, and multiple inheritance. Each has its pros/cons:

- **With the bridge pattern**, you create two distinct hierarchies: `ABC Vehicle` has derived classes `LandVehicle`, `WaterVehicle`, etc., and `ABC Engine` has derived classes `GasPowered`, `NuclearPowered`, etc. Then the `Vehicle` has an `Engine*` (that is, an `Engine`-pointer), and users mix and match vehicles and engines at run-time. This has the advantage that you only have to write *N+M* derived classes, which means things grow very gracefully: when you add a new geography (incrementing *N*) or engine type (incrementing *M*), you need add only one new derived class. However you have several disadvantages as well: you only have *N+M* derived classes which means you only have at most *N+M* overrides and therefore *N+M* concrete algorithms / data structures. If you ultimately want different algorithms and/or data structures in the *N* M* combinations, you'll have to work hard to make that happen, and you're probably better off with something other than a pure bridge pattern. The other thing the bridge doesn't solve for you is eliminating the nonsensical choices, such as pedal powered space vehicles. You can solve that by adding extra checks when the users combine vehicles and engines at run-time, but it requires a bit of skullduggery, something the bridge pattern doesn't provide for free. The bridge also restricts users since, although there is a common base class above all geographies (meaning a user can pass any kind of vehicle as a `Vehicle`), there is not a common base class above, for example, all gas powered vehicles, and therefore users cannot pass any gas powered vehicle as a `GasPoweredVehicle`. Finally, the bridge has the advantage that it shares code between the group of, for example, water vehicles as well as the group of, for example, gas powered vehicles. In other words, the various gas powered vehicles share the code in derived class `GasPoweredEngine`.
- **With nested generalization**, you pick one of the hierarchies as primary and the other as secondary, and you have a nested hierarchy. For example, if you choose geography as primary, `Vehicle` would have derived classes `LandVehicle`, `WaterVehicle`, etc., and those would each have further derived classes, one per power source type. E.g., `LandVehicle` would have derived classes `GasPoweredLandVehicle`, `PedalPoweredLandVehicle`, `NuclearPoweredLandVehicle`, etc.; `WaterVehicle` would have a similar set of derived classes, etc. This requires you to write roughly *N*M* different derived classes, which means things don't grow gracefully when you increment *N* or *M*, but it gives you the advantage over the bridge that you can have *N*M* different algorithms and data structures. It also gives you fine granular control, since the user cannot select nonsensical combinations, such as pedal powered space vehicles, since the user can select only those combinations that a programmer has decided are reasonable. Unfortunately nested generalization doesn't improve the problem with passing any gas powered vehicle as a common base class, since there is no common base class above the secondary hierarchy, e.g., there is no `GasPoweredVehicle` base class. And finally, it's not obvious

how to share code between all vehicles that use the same power source, e.g., between all gas powered vehicles.

- **With multiple inheritance**, you have two distinct hierarchies, just like the bridge, but you remove the `Engine*` from the bridge and instead create roughly $N \times M$ derived classes below both the hierarchy of geographies and the hierarchy of power sources. It's not as simple as this, since you'll need to change the concept of the `Engine` classes. In particular, you'll want to rename the classes in that hierarchy from, for example, `GasPoweredEngine` to `GasPoweredVehicle`; plus you'll need to make corresponding changes to the methods in the hierarchy. In any case, class `GasPoweredLandVehicle` will multiply inherit from `GasPoweredVehicle` and `LandVehicle`, and similarly with `GasPoweredWaterVehicle`, `NuclearPoweredWaterVehicle`, etc. Like nested generalization, you have to write roughly $N \times M$ classes, which doesn't grow gracefully, but it does give you fine granular control over both which algorithm and data structures to use in the various derived classes as well as which combinations are deemed "reasonable," meaning you simply don't create nonsensical choices like `PedalPoweredSpaceVehicle`. It solves a problem shared by both bridge and nested generalization, namely it allows a user to pass any gas powered vehicle using a common base class. Finally it provides a solution to the code-sharing problem, a solution that is at least as good as that of the bridge solution: it lets all gas powered vehicles share common code when that is desired. We say this is "at least as good as the solution from the bridge" since, unlike the bridge, the derived classes can share common code within gas powered vehicles, but can also, unlike with the bridge, override and replace that code in cases where the shared code is not ideal.

The most important point: there is no universally "best" answer. Perhaps you were hoping I would tell you to always use one or the other of the above choices. I'd be happy to do that except for one minor detail: it'd be a lie. If exactly one of the above was *always* best, then one size would fit all, and we know it does not.

So here's what you have to do: **T H I N K**. You'll have to make a decision. I'll give you some guidelines, but ultimately you will have to decide what is best (or perhaps "least bad") for *your* situation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.6] Is there a simple way to visualize all these tradeoffs?

The following matrix gives an overview of the pros/cons:

	Bridge	Nested generalization	Multiple inheritance
Does it grow gracefully when adding geography or power source?		😞	😞
How much code needs to be written?	($N+M$ chunks)	😞 ($N \times M$ chunks)	😞 ($N \times M$ chunks)
Do you have fine granular control over the	😞		

algorithms and data structures?			
Do you have fine granular control over nonsensical combinations?	😞		
Does it let users treat either base class polymorphically?	😞	😞	
Does it let derived classes share common code from either side?		😞	

Warning: the reader should not be naive in using the above matrix. For example, do not simply

add up the number of and 😞 marks, then decide based on which design has the most good and least bad. The first step in using the above matrix is to find out if there are additional design approaches, that is, additional columns. And don't forget: the bridge and nested generalization columns are really both *pairs* of columns, since in both cases there is an asymmetry that could go in either direction. In other words, one could put an `Engine*` in `Vehicle` or a `Vehicle*` in `Engine` (or both, or some other way to pair them up, such as a small object that contains just a `Vehicle*` and an `Engine*`). Similarly, with nested generalization you could decompose first by geography (land, water, etc.) or first by power source (gas, nuclear, etc.), yielding two distinct designs with distinct tradeoffs.

The second step in using the above matrix is to give a "weight" to each row. For example, in your particular situation, the amount of code that must get written (second row) may be more or less important than the granular control over data structures. The ultimate decision will be made by finding out which approach is best *for your situation*. One size does not fit all — do *not* expect the answer in one project to be the same as the answer in another project.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.7] Can you give another example to illustrate the above disciplines?

This second example is only slightly different from the previous since it is more obviously symmetric. This symmetry tilts the scales slightly toward the multiple inheritance solution, but one of the others still might be best in some situations.

In this example, we have only two categories of vehicles: land vehicles and water vehicles. Then somebody points out that we need amphibious vehicles. Now we get to the good part: the questions.

1. Do we even need a distinct `AmphibiousVehicle` class? Is it also viable to use one of the other classes with a "bit" indicating the vehicle can be both in water and on land? Just because "the real world" has amphibious vehicles doesn't mean we need to mimic that in software.
2. Will the users of `LandVehicle` need to use a `LandVehicle&` that refers to an `AmphibiousVehicle` object? Will they need to call methods on the `LandVehicle&` and expect the actual implementation of those methods to specific to ("overridden in") `AmphibiousVehicle`?

3. Ditto for water vehicles: will the users want a `waterVehicle` that might refer to an `AmphibiousVehicle` object, and in particular to call methods on that reference and expect the implementation will get overridden by `AmphibiousVehicle`?

If we get three "yes" answers, multiple inheritance is probably the right choice. To be sure, you should ask the other questions as well, e.g., the grow-gracefully issue, the granularity of control issues, etc.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.8] What is the "dreaded diamond"?

The "dreaded diamond" refers to a class structure in which a particular class appears more than once in a class's inheritance hierarchy. For example,

```
class Base {
public:
    ...
protected:
    int data_;
};

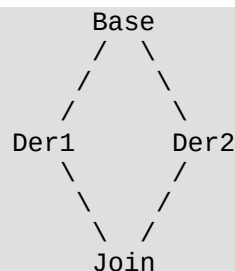
class Der1 : public Base { ... };

class Der2 : public Base { ... };

class Join : public Der1, public Der2 {
public:
    void method()
    {
        data_ = 1;    ← bad: this is ambiguous; see below
    }
};

int main()
{
    Join* j = new Join();
    Base* b = j;    ← bad: this is ambiguous; see below
}
```

Forgive the ASCII-art, but the inheritance hierarchy looks something like this:



Before we explain why the dreaded diamond is dreaded, it is important to note that C++ provides techniques to deal with each of the "dreads." In other words, this structure is often *called* the dreaded diamond, but it really isn't dreaded; it's more ju st something to be aware of.

The key is to realize that Base is inherited twice, which means any data members declared in Base, such as data_ above, will appear twice within a Join object. This can create ambiguities: which data_ did you want to change? For the same reason the conversion from Join* to Base*, or from Join& to Base&, is ambiguous: which Base class subobject did you want?

C++ lets you resolve the ambiguities. For example, instead of saying data_ = 1 you could say Der2::data_ = 1, or you could convert from Join* to a Der1* and then to a Base*. However please, Please, *PLEASE* think before you do that. That is almost always *not* the best solution. The best solution is typically to tell the C++ compiler that only one Base subobject should appear within a Join object, and that is described [next](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.9] Where in a hierarchy should I use virtual inheritance?

Just below the top of the diamond, *not* at the join-class.

To avoid the duplicated base class subobject that occurs with the "dreaded diamond", you should use the `virtual` keyword in the inheritance part of the classes that derive directly from the top of the diamond:

```
class Base {
public:
    ...
protected:
    int data_;
};

class Der1 : public virtual Base {
public:
    ^^^^^^^—this is the key
    ...
};

class Der2 : public virtual Base {
public:
    ^^^^^^^—this is the key
    ...
};

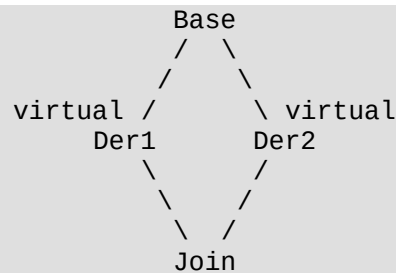
class Join : public Der1, public Der2 {
public:
    void method()
    {
        data_ = 1;  ← good: this is now unambiguous
    }
};

int main()
{
    Join* j = new Join();
}
```

```
Base* b = j;    ← good: this is now unambiguous
}
```

Because of the `virtual` keyword in the base-class portion of `Der1` and `Der2`, an instance of `Join` will have only a single `Base` subobject. This eliminates the ambiguities. This is usually better than using full qualification as described in [the previous FAQ](#).

For emphasis, the `virtual` keyword goes in the hierarchy above `Der1` and `Der2`. It doesn't help to put the `virtual` keyword in the `Join` class itself. In other words, you have to know that a join class will exist when you are creating class `Der1` and `Der2`.



[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.10] What does it mean to "delegate to a sister class" via virtual inheritance?

Consider the following example:

```
class Base {
public:
    virtual void foo() = 0;
    virtual void bar() = 0;
};

class Der1 : public virtual Base {
public:
    virtual void foo();
};

void Der1::foo()
{ bar(); }

class Der2 : public virtual Base {
public:
    virtual void bar();
};

class Join : public Der1, public Der2 {
public:
    ...
};

int main()
{
    Join* p1 = new Join();
    Der1* p2 = p1;
    Base* p3 = p1;
}
```

```
p1->foo();  
p2->foo();  
p3->foo();  
}
```

Believe it or not, when `Der1::foo()` calls `this->bar()`, it ends up calling `Der2::bar()`. Yes, that's right: a class that `Der1` knows nothing about will supply the override of a virtual function invoked by `Der1::foo()`. This "cross delegation" can be a powerful technique for customizing the behavior of polymorphic classes.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.11] What special considerations do I need to know about when I use virtual inheritance?

Generally, virtual base classes are most suitable when the classes that derive from the virtual base, and especially the virtual base itself, are pure abstract classes. This means the classes above the "join class" have very little if any data.

Note: even if the virtual base itself is a pure abstract class with *no* member data, you still probably don't want to remove the virtual inheritance within classes `Der1` and `Der2`. You can use fully qualified names to resolve any ambiguities that arise, and you might even be able to squeeze out a few cycles in some cases, however the object's address is somewhat ambiguous (there are still two base class subobjects in the `Join` object), so simple things like trying to find out if two pointers point at the same instance might be tricky. Just be careful — very careful.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.12] What special considerations do I need to know about when I inherit from a class that uses virtual inheritance?

Initialization list of most-derived-class's ctor directly invokes the virtual base class's ctor.

Because a virtual base class subobject occurs only once in an instance, there are special rules to make sure the virtual base class's constructor and destructor get called exactly once per instance. The C++ rules say that virtual base classes are constructed before all non-virtual base classes. The thing you as a programmer need to know is this: constructors for virtual base classes *anywhere* in your class's inheritance hierarchy are called by the "most derived" class's constructor.

Practically speaking, this means that when you create a concrete class that has a virtual base class, you must be prepared to pass whatever parameters are required to call the virtual base class's constructor. And, of course, if there are several virtual base classes anywhere in your classes ancestry, you must be prepared to call all their constructors. This might mean that the most-derived class's constructor needs more parameters than you might otherwise think.

However, if the author of the virtual base class followed [the guideline in the previous FAQ](#), then the virtual base class's constructor probably takes no parameters since it doesn't have any data to initialize. This means (fortunately!) the authors of the concrete classes that inherit eventually from the virtual base class do not need to worry about taking extra parameters to pass to the virtual base class's ctor.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[25.13] What special considerations do I need to know about when I use a class that uses virtual inheritance?

No C-style downcasts; use `dynamic_cast` instead.

(Rest to be written.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

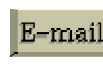
[25.14] One more time: what is the exact order of constructors in a multiple and/or virtual inheritance situation?

The very first constructors to be executed are the virtual base classes anywhere in the hierarchy. They are executed in the order they appear in a depth-first left-to-right traversal of the graph of base classes, where *left to right* refer to the order of appearance of base class names.

After all virtual base class constructors are finished, the construction order is generally from base class to derived class. The details are easiest to understand if you imagine that the very first thing done in the derived class's ctor is to call the base class's ctor (hint: that's the way most compilers actually do it). So if class D inherits multiply from B1 and B2, the constructor for B1 executes first, then the constructor for B2, then the constructor for D. This rule is applied recursively; for example, if B1 inherits from B1a and B1b, and B2 inherits from B2a and B2b, then the final order is B1a, B1b, B1, B2a, B2b, B2, D.

Note that the order B1 and then B2 (or B1a then B1b) is determined by the order that the base classes appear in the declaration of the class, *not* in the order that the initializer appears in the derived class's initialization list.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | ©
| [Download your own copy](#)]

Revised Feb 15, 2005

[26] Built-in / intrinsic / primitive data types

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [26]:

- [\[26.1\] Can `sizeof\(char\)` be 2 on some machines? For example, what about double-byte characters?](#)
 - [\[26.2\] What are the units of `sizeof`?](#)
 - [\[26.3\] Whoa, but what about machines or compilers that support multibyte characters. Are you saying that a "character" and a `char` might be different?!?](#)
 - [\[26.4\] But, but, but what about machines where a `char` has more than 8 bits? Surely you're not saying a C++ byte might have more than 8 bits, are you?!?](#)
 - [\[26.5\] Okay, I could imagine a machine with 9-bit bytes. But surely not 16-bit bytes or 32-bit bytes, right?](#)
 - [\[26.6\] I'm sooooo confused. Would you please go over the rules about bytes, `chars`, and characters one more time?](#)
 - [\[26.7\] What is a "POD type"?](#)
 - [\[26.8\] When initializing non-static data members of built-in / intrinsic / primitive types, should I use the "initialization list" or assignment?](#)
 - [\[26.9\] When initializing static data members of built-in / intrinsic / primitive types, should I worry about the "static initialization order fiasco"?](#)
 - [\[26.10\] Can I define an operator overload that works with built-in / intrinsic / primitive types?](#)
 - [\[26.11\] When I delete an array of some built-in / intrinsic / primitive type, why can't I just say `delete a` instead of `delete\[\] a`?](#)
 - [\[26.12\] How can I tell if an integer is a power of two without looping?](#)
-

[26.1] Can `sizeof(char)` be 2 on some machines? For example, what about double-byte characters?

No, `sizeof(char)` is always 1. Always. It is never 2. Never, never, never.

Even if you think of a "character" as a multi-byte thingy, `char` is not. `sizeof(char)` is always exactly 1. No exceptions, ever.

Look, I know this is going to hurt your head, so please, *please* just read the next few FAQs in sequence and hopefully the pain will go away by sometime next week.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.2] What are the units of `sizeof`?

Bytes.

For example, if `sizeof(Fred)` is 8, the distance between two `Fred` objects in an array of `Freds` will be exactly 8 *bytes*.

As another example, this means `sizeof(char)` is one byte. That's right: one byte. One, one, one, exactly one byte, always one byte. Never two bytes. No exceptions.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.3] Whoa, but what about machines or compilers that support multibyte characters. Are you saying that a "character" and a `char` might be different?!?

Yes that's right: the thing commonly referred to as a "character" might be different from the thing C++ calls a `char`.

I'm really sorry if that hurts, but believe me, it's better to get all the pain over with at once. Take a deep breath and repeat after me: "character and `char` might be different." There, doesn't that feel better? No? Well keep reading — it gets worse.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.4] But, but, but what about machines where a `char` has more than 8 bits? Surely you're not saying a C++ byte might have more than 8 bits, are you?!?

Yep, that's right: a C++ byte might have more than 8 bits.

The C++ language guarantees a byte must always have *at least* 8 bits. But there are implementations of C++ that have more than 8 bits per byte.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.5] Okay, I could imagine a machine with 9-bit bytes. But surely not 16-bit bytes or 32-bit bytes, right?

Wrong.

I have heard of one implementation of C++ that has 64-bit "bytes." You read that right: a byte on that implementation has 64 bits. 64 bits per byte. 64. As in 8 times 8.

And yes, you're right, combining with [the above](#) would mean that a `char` on that implementation would have 64 bits.

[26.6] I'm sooooo confused. Would you please go over the rules about bytes, charS, and characters one more time?

Here are the rules:

- The C++ language gives the programmer the impression that memory is laid out as a sequence of something C++ calls "bytes."
- Each of these things that the C++ language calls a byte has at least 8 bits, but might have more than 8 bits.
- The C++ language guarantees that a `char*` (char pointers) can address individual bytes.
- The C++ language guarantees there are no bits *between* two bytes. This means every bit in memory is part of a byte. If you grind your way through memory via a `char*`, you will be able to see every bit.
- The C++ language guarantees there are no bits that are part of two distinct bytes. This means a change to one byte will never cause a change to a different byte.
- The C++ language gives you a way to find out how many bits are in a byte in your particular implementation: include the header `<climits>`, then the actual number of bits per byte will be given by the `CHAR_BIT` macro.

Let's work an example to illustrate these rules. The PDP -10 has 36-bit words with no hardware facility to address anything within one of those words. That means a pointer can point only at things on a 36-bit boundary: it is not possible for a pointer to point 8 bits to the right of where some other pointer points.

One way to abide by all the above rules is for a PDP -10 C++ compiler to define a "byte" as 36 bits. Another valid approach would be to define a "byte" as 9 bits, and simulate a `char*` by two words of memory: the first could point to the 36-bit word, the second could be a bit-offset within that word. In that case, the C++ compiler would need to add extra instructions when compiling code using `char*` pointers. For example, the code generated for `*p = 'x'` might read the word into a register, then use bit-masks and bit-shifts to change the appropriate 9-bit byte within that word. An `int*` could still be implemented as a single hardware pointer, since C++ allows `sizeof(char*) != sizeof(int*)`.

Using the same logic, it would also be possible to define a PDP -10 C++ "byte" as 12-bits or 18-bits. However the above technique wouldn't allow us to define a PDP-10 C++ "byte" as 8-bits, since 8×4 is 32, meaning every 4th byte we would *skip* 4 bits. A more complicated approach could be used for those 4 bits, e.g., by packing nine bytes (of 8-bits each) into two adjacent 36-bit words. The important point here is that `memcpy()` has to be able to see every bit of memory: there can't be any bits between two adjacent bytes.

Note: one of the popular non-C/C++ approaches on the PDP -10 was to pack 5 bytes (of 7-bits each) into each 36-bit word. However this won't work in C or C++ since $5 \times 7 = 35$, meaning using `char*s` to walk through memory would "skip" a bit every fifth byte (and also because C++ requires bytes to have at least 8 bits).

[26.7] What is a "POD type"?

A type that consists of nothing but **P**lain **O**ld **D**ata.

A POD type is a C++ type that has an equivalent in C, and that uses the same rules as C uses for initialization, copying, layout, and addressing.

As an example, the C declaration `struct Fred x;` does not initialize the members of the `Fred` variable `x`. To make this same behavior happen in C++, `Fred` would need to *not* have any constructors. Similarly to make the C++ version of copying the same as the C version, the C++ `Fred` must not have overloaded the assignment operator. To make sure the other rules match, the C++ version must not have virtual functions, base classes, non-static members that are private or protected, or a destructor. It can, however, have static data members, static member functions, and non-static non-virtual member functions.

The actual definition of a POD type is recursive and gets a little gnarly. Here's a *slightly* simplified definition of POD: a POD type's non-static data members must be public and can be of any of these types: `bool`, any numeric type including the various `char` variants, any enumeration type, any data-pointer type (that is, any type convertible to `void*`), any pointer-to-function type, or any POD type, including arrays of any of these. Note: data-pointers and pointers-to-function are okay, but [pointers-to-member](#) are not. Also note that references are not allowed. In addition, a POD type can't have constructors, virtual functions, base classes, or a non-overloaded assignment operator.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.8] When initializing non-static data members of built-in / intrinsic / primitive types, should I use the "initialization list" or assignment?

For symmetry, it is usually best to initialize all non-static data members in the constructor's "initialization list," even those that are of a built-in / intrinsic / primitive type. The FAQ [shows you why and how](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.9] When initializing static data members of built-in / intrinsic / primitive types, should I worry about the "static initialization order fiasco"?

Yes, if you initialize your built-in / intrinsic / primitive variable by an expression that the compiler doesn't evaluate solely at compile-time. The FAQ [provides several solutions](#) for this (subtle!) problem.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.10] Can I define an operator overload that works with built-in / intrinsic / primitive types?

No, the C++ language requires that your operator overloads take at least one operand of a "class type." The C++ language will not let you define an operator all of whose operands / parameters are of primitive types.

For example, [you can't define an operator== that takes two char*s and uses string comparison](#). That's good news because if `s1` and `s2` are of type `char*`, the expression `s1 == s2` already has a well defined meaning: it compares the two *pointers*, not the two strings pointed to by those pointers. [You shouldn't use pointers anyway. Use `std::string` instead of `char\*`.](#)

If C++ let you redefine the meaning of operators on built-in types, you wouldn't ever know what `1 + 1` is: it would depend on which headers got included and whether one of those headers redefined addition to mean, for example, subtraction.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.11] When I delete an array of some built-in / intrinsic / primitive type, why can't I just say `delete a` instead of `delete[] a`?

Because you can't.

Look, *please* don't write me an email asking me why C++ is what it is. It just is. If you really want a rationale, buy Bjarne Stroustrup's excellent book, "Design and Evolution of C++" (Addison-Wesley publishers). But if your real goal is to write some code, don't waste too much time figuring out *why* C++ has these rules, and instead just abide by its rules.

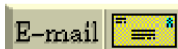
So here's the rule: if `a` points to an array of thingies that was allocated via `new T[n]`, then [you must, must, must](#) delete it via `delete[] a`. Even if the elements in the array are built-in types. Even if they're of type `char` or `int` or `void*`. Even if you don't understand why.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[26.12] How can I tell if an integer is a power of two without looping?

```
inline bool isPowerOf2(int i)
{
    return i > 0 && (i & (i - 1)) == 0;
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[27] Coding standards

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [27]:

- [\[27.1\] What are some good C++ coding standards?](#) UPDATED!
 - [\[27.2\] Are coding standards necessary? Are they sufficient?](#)
 - [\[27.3\] Should our organization determine coding standards from our C experience?](#)
 - [\[27.4\] What's the difference between `<xxx>` and `<xxx.h>` headers?](#)
 - [\[27.5\] Is the `?:` operator evil since it can be used to create unreadable code?](#)
 - [\[27.6\] Should I declare locals in the middle of a function or at the top?](#)
 - [\[27.7\] What source-file-name convention is best? `foo.cpp`? `foo.c`? `foo.cc`?](#)
 - [\[27.8\] What header-file-name convention is best? `foo.H`? `foo.hh`? `foo.hpp`?](#)
 - [\[27.9\] Are there any lint-like guidelines for C++?](#)
 - [\[27.10\] Why do people worry so much about pointer casts and/or reference casts?](#)
 - [\[27.11\] Which is better: identifier names that look like `this` or identifier names that look like `This`?](#)
 - [\[27.12\] Are there any other sources of coding standards?](#) UPDATED!
 - [\[27.13\] Should I use "unusual" syntax?](#)
-

[27.1] What are some good C++ coding standards? UPDATED!

[Recently added last few sentences on the Sutter/Alexandrescu book (in 12/04). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Thank you for reading this answer rather than just trying to set your own coding standards.

But beware that some people on [comp.lang.c++.](#) are very sensitive on this issue. Nearly every software engineer has, at some point, been exploited by someone who used coding standards as a "power play." Furthermore some attempts to set C++ coding standards have been made by those who didn't know what they were talking about, so the standards end up being based on what was the state-of-the-art when the standards setters were writing code. Such impositions generate an attitude of mistrust for coding standards.

Obviously anyone who asks this question wants to be trained so they *don't* run off on their own ignorance, but nonetheless posting a question such as this one to [comp.lang.c++.](#) tends to generate more heat than light.



For an excellent book on the subject, get Sutter and Alexandrescu, *C++ Coding Standards*, 220 pgs, Addison-Wesley, 2005, ISBN 0-321-11358-6. It provides 101 rules, guidelines and best practices. The authors and editors produced some solid material, then did an unusually good job of energizing the peer-review team. All of this improved the book. Buy it.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.2] Are coding standards necessary? Are they sufficient?

Coding standards do not make non-OO programmers into OO programmers; only training and experience do that. If coding standards have merit, it is that they discourage the petty fragmentation that occurs when large organizations coordinate the activities of diverse groups of programmers.

But you really want more than a coding standard. The structure provided by coding standards gives neophytes one less degree of freedom to worry about, which is good. However, pragmatic guidelines should go well beyond pretty-printing standards. Organizations need a consistent *philosophy* of design and implementation. E.g., strong or weak typing? references or pointers in interfaces? stream I/O or stdio? should C++ code call C code? vice versa? how should [ABCs](#) be used? should inheritance be used as an implementation technique or as a specification technique? what testing strategy should be employed? inspection strategy? should interfaces uniformly have a `get()` and/or `set()` member function for each data member? should interfaces be designed from the outside-in or the inside-out? should errors be handled by `try/catch/throw` or by return codes? etc.

What is needed is a "pseudo standard" for detailed *design*. I recommend a three-pronged approach to achieving this standardization: training, [mentoring](#), and libraries. Training provides "intense instruction," mentoring allows OO to be caught rather than just taught, and high quality C++ class libraries provide "long term instruction." There is a thriving commercial market for all three kinds of "training." Advice by organizations who have been through the mill is consistent: *Buy, Don't Build*. Buy libraries, buy training, buy tools, buy consulting. Companies who have attempted to become a self-taught tool-shop as well as an application/system shop have found success difficult.

Few argue that coding standards are "ideal," or even "good," however they are necessary in the kind of organizations/situations described above.

The following FAQs provide some basic guidance in conventions and styles.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.3] Should our organization determine coding standards from our C experience?

No!

No matter how vast your C experience, no matter how advanced your C expertise, being a good C programmer does not make you a good C++ programmer. Converting from C to C++ is more than just learning the syntax and semantics of the ++ part of C++. Organizations who want the promise of OO, but who fail to put the "OO" into "OO programming", are fooling themselves; the balance sheet will show their folly.

C++ coding standards should be tempered by C++ experts. Asking [comp.lang.c++.](#) is a start. Seek out experts who can help guide you away from pitfalls. Get training. Buy libraries and see if "good" libraries pass your coding standards. Do *not* set standards by yourself unless you have considerable experience in C++. Having no standard is better than having a bad standard, since improper "official" positions "harden" bad brain traces. There is a thriving market for both C++ training and libraries from which to pull expertise.

One more thing: whenever something is in demand, the potential for charlatans increases. Look before you leap. Also ask for student-reviews from past companies, since not even expertise makes someone a good communicator. Finally, select a practitioner who can teach, not a full time teacher who has a passing knowledge of the language/paradigm.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.4] What's the difference between `<xxx>` and `<xxx.h>` headers?

The headers in ISO Standard C++ don't have a `.h` suffix. This is something the standards committee changed from former practice. The details are different between headers that existed in C and those that are specific to C++.

The C++ standard library is guaranteed to have 18 standard headers from the C language. These headers come in two standard flavors, `<xxxx>` and `<xxx.h>` (where `xxx` is the basename of the header, such as `stdio`, `stdlib`, etc). These two flavors are identical except the `<xxxx>` versions provide their declarations in the `std` namespace only, and the `<xxx.h>` versions make them available both in `std` namespace and in the global namespace. The committee did it this way so that existing C code could continue to be compiled in C++. However the `<xxx.h>` versions are deprecated, meaning they are standard now but might not be part of the standard in future revisions. (See clause D.5 of the [ISO C++ standard](#).)

The C++ standard library is also guaranteed to have 32 additional standard headers that have no direct counterparts in C, such as `<iostream>`, `<string>`, and `<new>`. You may see things like `#include <iostream.h>` and so on in old code, and some compiler vendors offer `.h` versions for that reason. But be careful: the `.h` versions, if available, may differ from the standard versions. And if you compile some units of a program with, for example, `<iostream>` and others with `<iostream.h>`, the program may not work.

For new projects, use only the `<xxx>` headers, not the `<xxx.h>` headers.

When modifying or extending existing code that uses the old header names, you should probably follow the practice in that code unless there's some important reason to switch to the standard headers (such as a facility available in standard `<iostream>` that was not available in the vendor's `<iostream.h>`). If you need to standardize existing code, make sure to change all

C++ headers in all program units including external libraries that get linked in to the final executable.

All of this affects the standard headers only. You're free to name your own headers anything you like; see [\[27.8\]](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.5] Is the ?: operator evil since it can be used to create unreadable code?

No, but as always, remember that readability is one of the most important things.

Some people feel the ?: ternary operator should be avoided because they find it confusing at times compared to the good old `if` statement. In many cases ?: tends to make your code more difficult to read (and therefore you should replace those usages of ?: with `if` statements), but there are times when the ?: operator is clearer since it can emphasize what's really happening, rather than the fact that there's an `if` in there somewhere.

Let's start with a really simple case. Suppose you need to print the result of a function call. In that case you should put the real goal (printing) at the beginning of the line, and bury the function call within the line since it's relatively incidental (this left-right thing is based on the intuitive notion that most developers think the first thing on a line is the most important thing):

```
// Preferred (emphasizes the major goal — printing):
std::cout << funct();

// Not as good (emphasizes the minor goal — a function call):
functAndPrintOn(std::cout);
```

Now let's extend this idea to the ?: operator. Suppose your real goal is to print something, but you need to do some incidental decision logic to figure out what should be printed. Since the printing is the most important thing conceptually, we prefer to put it first on the line, and we prefer to bury the incidental decision logic. In the example code below, variable `n` represents the number of senders of a message; the message itself is being printed to `std::cout`:

```
int n = /*...*/;    // number of senders

// Preferred (emphasizes the major goal — printing):
std::cout << "Please get back to " << (n==1 ? "me" : "us") << " soon!\n";

// Not as good (emphasizes the minor goal — a decision):
std::cout << "Please get back to ";
if (n == 1)
    std::cout << "me";
else
    std::cout << "us";
std::cout << " soon!\n";
```

All that being said, you can get pretty outrageous and unreadable code ("write only code") using various combinations of `?:`, `&&`, `||`, etc. For example,

```
// Preferred (obvious meaning):  
if (f())  
    g();  
  
// Not as good (harder to understand):  
f() && g();
```

Personally I think the explicit `if` example is clearer since it emphasizes the major thing that's going on (a decision based on the result of calling `f()`) rather than the minor thing (calling `f()`). In other words, the use of `if` here is *good* for precisely the same reason that it was *bad* above: we want to major on the majors and minor on the minors.

In any event, don't forget that readability is the goal (at least it's one of the goals). Your goal should *not* be to avoid certain syntactic constructs such as `?:` or `&&` or `||` or `if` — or even `goto`. If you sink to the level of a "Standards Bigot," you'll ultimately embarrass yourself since there are always counterexamples to any syntax-based rule. If on the other hand you emphasize broad goals and guidelines (e.g., "major on the majors," or "put the most important thing first on the line," or even "make sure your code is obvious and readable"), you're usually much better off.

Code must be written to be read, not by the compiler, but by another human being.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.6] Should I declare locals in the middle of a function or at the top?

Declare near first use.

An object is initialized (constructed) the moment it is declared. If you don't have enough information to initialize an object until half way down the function, you should create it half way down the function when it can be initialized correctly. Don't initialize it to an "empty" value at the top then "assign" it later. The reason for this is runtime performance. Building an object correctly is faster than building it incorrectly and remodeling it later. Simple examples show a factor of 350% speed hit for simple classes like `String`. Your mileage may vary; surely the overall system degradation will be less than 350%, but there *will* be degradation. *Unnecessary* degradation.

A common retort to the above is: "we'll provide `set()` member functions for every datum in our objects so the cost of construction will be spread out." This is worse than the performance overhead, since now you're introducing a maintenance nightmare. Providing a `set()` member function for every datum is tantamount to `public data`: you've exposed your implementation technique to the world. The only thing you've hidden is the physical *names* of your member objects, but the fact that you're using a `List` and a `String` and a `float`, for example, is open for all to see.

Bottom line: Locals should be declared near their first use. Sorry that this isn't familiar to C experts, but new doesn't necessarily mean bad.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.7] What source-file-name convention is best? `foo.cpp`? `foo.c`? `foo.cc`?

If you already have a convention, use it. If not, consult your compiler to see what the compiler expects. Typical answers are: `.cpp`, `.C`, `.cc`, or `.cxx` (naturally the `.c` extension assumes a case-sensitive file system to distinguish `.c` from `.C`).

We've often used `.cpp` for our C++ source files, and we have also used `.c`. In the latter case, when porting to case-insensitive file systems you need to tell the compiler to treat `.c` files as if they were C++ source files (e.g., `-Tdp` for IBM CSet++, `-cpp` for Zortech C++, `-P` for Borland C++, etc.).

The point is that none of these filename extensions are uniformly superior to the others. We generally use whichever technique is preferred by our customer (again, these issues are dominated by business considerations, not by technical considerations).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.8] What header-file-name convention is best? `foo.h`? `foo.hh`? `foo.hpp`?

If you already have a convention, use it. If not, and if you don't need your editor to distinguish between C and C++ files, simply use `.h`. Otherwise use whatever the editor wants, such as `.H`, `.hh`, or `.hpp`.

We've tended to use either `.hpp` or `.h` for our C++ header files.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.9] Are there any lint-like guidelines for C++?

Yes, there are some practices which are generally considered dangerous. However none of these are universally "bad," since situations arise when even the worst of these is needed:

- A class Fred's assignment operator should return `*this` as a `Fred&` (allows chaining of assignments)
- A class with any [virtual](#) functions ought to have a [virtual destructor](#)
- A class with any of {destructor, assignment operator, copy constructor} generally needs all 3
- A class Fred's copy constructor and assignment operator should have `const` in the parameter: respectively `Fred::Fred(const Fred&)` and `Fred& Fred::operator= (const Fred&)`
- When initializing an object's member objects in the constructor, always use initialization lists rather than assignment. The performance difference for user-defined classes can be substantial (3x!)

- Assignment operators should make sure that [self assignment](#) does nothing, [otherwise you may have a disaster](#). In some cases, this may require you to [add an explicit test to your assignment operators](#).
- When you overload operators, abide by [the guidelines](#). For example, in classes that define both += and +, a += b and a = a + b should generally do the same thing; ditto for the other identities of built-in/intrinsic types (e.g., a += 1 and ++a; p[i] and *(p+i); etc). This can be enforced by writing the binary operations using the op= forms. E.g.,

```
Fred operator+ (const Fred& a, const Fred& b)
{
    Fred ans = a;
    ans += b;
    return ans;
}
```

This way the "constructive" binary operators don't even need to be [friends](#). But it is sometimes possible to more efficiently implement common operations (e.g., if class Fred is actually std::string, and += has to reallocate/copy string memory, it may be better to know the eventual length from the beginning).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.10] Why do people worry so much about pointer casts and/or reference casts?

Because they're [evil](#)! (Which means [you should use them sparingly and with great care](#).)

For some reason, programmers are sloppy in their use of pointer casts. They cast this to that all over the place, then they wonder why things don't quite work right. Here's the worst thing: when the compiler gives them an error message, they add a cast to "shut the compiler up," then they "test it" to see if it seems to work. If you have a lot of pointer casts or reference casts, read on.

The compiler will often be silent when you're doing pointer -casts and/or reference casts. Pointer-casts (and reference-casts) tend to shut the compiler up. I think of them as a filter on error messages: the compiler *wants* to complain because it sees you're doing something stupid, but it also sees that it's not allowed to complain due to your pointer-cast, so it drops the error message into the bit-bucket. It's like putting duct tape on the compiler's mouth: it's trying to tell you something important, but you've intentionally shut it up.

A pointer-cast says to the compiler, "Stop thinking and start generating code; I'm smart, you're dumb; I'm big, you're little; I know what I'm doing so just pretend this is assembly language and generate the code." The compiler pretty much blindly generates code when you start casting — you are taking control (and responsibility!) for the outcome. The compiler and the language reduce (and in some cases eliminate!) the guarantees you get as to what will happen. You're on your own.

By way of analogy, even if it's legal to juggle chainsaws, it's stupid. If something goes wrong, don't bother complaining to the chainsaw manufacturer — you did something they didn't guarantee would work. You're on your own.

(To be completely fair, the language does give you *some* guarantees when you cast, at least in a limited subset of casts. For example, it's guaranteed to work as you'd expect if the cast happens to be from an object-pointer (a pointer to a piece of data, as opposed to a pointer-to-function or pointer-to-member) to type `void*` and back to the *same type* of object-pointer. But in a lot of cases you're on your own.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.11] Which is better: identifier names `that_look_like_this` or identifier names `thatLookLikeThis`?

It's a precedent thing. If you have a Pascal or Smalltalk background, `youProbablySquashNamesTogether` like this. If you have an Ada background, `You_Probably_Use_A_Large_Number_Of_Underscores` like this. If you have a Microsoft Windows background, you probably prefer the "Hungarian" style which means `you_jkuidsPrefix` `vndskaIdentifiers` `ncqWith` `ksldjftTheir` `nmdsadType`. And then there are the folks with a Unix C background, who abbrev everything and use very short identifier names. (AND THE FORTRAN PRGMRS LIMIT EVERYTH TO SIX LETTRS.)

So there is no universal standard. If your project team has a particular coding standard for identifier names, use it. But starting another Jihad over this will create a lot more heat than light. From a business perspective, there are only two things that matter: The code should be generally readable, and everyone on the team should use the same style.

Other than that, the differences are minor.

One more thing: don't import a coding style onto platform-specific code where it is foreign. For example, a coding style that seems natural while using a Microsoft library might look bizarre and random while using a UNIX library. Don't do it. Allow different styles for different platforms. (Just in case someone out there isn't reading carefully, don't send me email about the case of common code that is designed to be used/portable to several platforms, since that code wouldn't be platform-specific, so the above "allow different styles" guideline doesn't even apply.)

Okay, one more. Really. Don't fight the coding styles used by automatically generated code (e.g., by tools that generate code). Some people treat coding standards with religious zeal, and they try to get tools to generate code in their local style. Forget it: if a tool generates code in a different style, don't worry about it. Remember money and time?!? This whole coding standard thing was supposed to save money and time; don't turn it into a "money pit."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.12] Are there any other sources of coding standards? **UPDATED!**

*[Recently added a bullet on the Sutter/Alexandrescu book (in 12/04) and hoisted the Sutter/Alexandrescu book out to its proper, primary place; also added (thanks to [Kevlin Henney](#)) links to and insights about *Elemental* and *Industrial Strength C++* (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]*

Yep, there are several.

In my opinion, the best source is [Sutter and Alexandrescu, C++ Coding Standards, 220 pgs, Addison-Wesley, 2005, ISBN 0-321-11358-6](#). I had the privilege of serving as an advisor on that book, and the authors did a great job of energizing the pool of advisors. Everyone collaborated with an intensity and depth that I have not seen previously, and the book is better for it.

Here are a few other sources that you can use as starting points for developing your organization's coding standards (in random order) (some are out of date, some might even be bad; I'm not endorsing any; caveat emptor):

- cdfsga.fnal.gov/computing/coding_guidelines/CodingGuidelines.html
- www.nfra.nl/~seg/cppStdDoc.html
- www.cs.umd.edu/users/cml/resources/cstyle
- www.cs.rice.edu/~dwallach/CPlusPlusStyle.html
- cpptips.hyperformix.com/conventions/cppconventions_1.html
- www.objectmentor.com/resources/articles/naming.htm
- www.arcticlabs.com/codingstandards/
- www.possibility.com/cpp/CppCodingStandard.html
- www.cs.umd.edu/users/cml/cstyle/Wildfire-C++Style.html
- [Industrial Strength C++](#)
- The Ellemtel coding guidelines are available at
 - o membres.lycos.fr/pierret/cpp2.htm
 - o www.cs.umd.edu/users/cml/cstyle/Ellemtel-rules.html
 - o www.doc.ic.ac.uk/lab/cplus/c++.rules/
 - o www.mgl.co.uk/people/kirit/cpprules.html

Notes:

- The Ellemtel guide is dated, but is listed because of its seminal place: it was the first widely distributed and widely adopted set of coding guidelines for C++. It was also the first to castigate the use of protected data.
- Industrial Strength C++ is also dated, but was the first widely published place to mention the use of protected non-virtual destructors in base classes.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[27.13] Should I use "unusual" syntax?

Only when there is a compelling reason to do so. In other words, only when there is no "normal" syntax that will produce the same end-result.

Software decisions should be made based on money. Unless you're in an ivory tower somewhere, when you do something that increases costs, increases risks, increases time, or, in a constrained environment, increases the product's space/speed costs, you've done something "bad." In your mind you should translate all these things into money.

Because of this pragmatic, money-oriented view of software, programmers should avoid non-mainstream syntax whenever there exists a "normal" syntax that would be equivalent. If a programmer does something obscure, other programmers are confused; that costs money. These other programmers will probably introduce bugs (costs money), take longer to maintain the thing (money), have a harder time changing it (missing market windows = money), have a harder time optimizing it (in a constrained environment, somebody will have to spend money for more memory, a faster CPU, and/or a bigger battery), and perhaps have angry customers (money). It's a risk-reward thing: using abnormal syntax carries a risk, but when an equivalent, "normal" syntax would do the same thing, there is no "reward" to ameliorate that risk.

For example, the techniques used in the [Obfuscated C Code Contest](#) are, to be polite, non-normal. Yes many of them are legal, but not everything that is legal is moral. Using strange techniques will confuse other programmers. Some programmers love to "show off" how far they can push the envelope, but that puts their ego above money, and that's unprofessional. Frankly anybody who does that ought to be fired. (And if you think I'm being "mean" or "cruel," I suggest you get an attitude adjustment. Remember this: your company hired you to help it, not to hurt it, and anybody who puts their own personal ego-trips above their company's best interest simply ought to be shown the door.)

As an example of non-mainstream syntax, it's not "normal" to use the `?:` operator as a statement. (Some people don't even like it as an expression, but everyone must admit that there are a *lot* of uses of `?:` out there, so [it is "normal" \(as an expression\) whether people like it or not.](#)) Here is an example of using `?:` as a statement:

```
blah();
blah();
xyz() ? foo() : bar(); // should replace with if/else
blah();
blah();
```

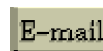
Same goes with using `||` and `&&` as if they are "if-not" and "if" statements, respectively. Yes, those are idioms in Perl, but C++ is not Perl and using these as replacements for `if` statements (as opposed to using them as expressions) is just not "normal" in C++. Example:

```
foo() || bar(); // should replace with if (!foo()) bar();
foo() && bar(); // should replace with if (foo()) bar();
```

Here's another example that seems to work and may even be legal, but it's certainly not normal:

```
void f(const& MyClass x) //use const MyClass& x instead
{
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[28] Learning OO/C++

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [28]:

- [\[28.1\] What is mentoring?](#)
 - [\[28.2\] Should I learn C before I learn OO/C++?](#)
 - [\[28.3\] Should I learn Smalltalk before I learn OO/C++?](#)
 - [\[28.4\] Should I buy one book, or several?](#) UPDATED!
 - [\[28.5\] What are some best-of-breed C++ *morality* guides?](#)
 - [\[28.6\] What are some best-of-breed C++ *legality* guides?](#)
 - [\[28.7\] What are some best-of-breed C++ *programming-by-example* guides?](#)
 - [\[28.8\] Are there other OO books that are relevant to OO/C++?](#)
-

[28.1] What is mentoring?

It's the most important tool in learning OO.

Object-oriented thinking is caught, not just taught. Get cozy with someone who *really* knows what they're talking about, and try to get inside their head and watch them solve problems. Listen. Learn by emulating.

If you're working for a company, get them to bring someone in who can act as a mentor and guide. We've seen gobs and gobs of money wasted by companies who "saved money" by simply buying their employees a book ("Here's a book; read it over the weekend; on Monday you'll be an OO developer").

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[28.2] Should I learn C before I learn OO/C++?

Don't bother.

If your ultimate goal is to learn OO/C++ and you don't already know C, reading books or taking courses in C will not only waste your time, but it will teach you a bunch of things that you'll explicitly have to un-learn when you finally get back on track and learn OO/C++ (e.g., [malloc\(\)](#), [printf\(\)](#), [unnecessary use of switch statements](#), [error-code exception handling](#), [unnecessary use of #define macros](#), etc.).

If you want to learn OO/C++, learn OO/C++. Taking time out to learn C will waste your time and confuse you.

[28.3] Should I learn Smalltalk before I learn OO/C++?

Don't bother.

If your ultimate goal is to learn OO/C++ and you don't already know Smalltalk, reading books or taking courses in Smalltalk will not only waste your time, but it will teach you a bunch of things that you'll explicitly have to un-learn when you finally get back on track and learn OO/C++ (e.g., [dynamic typing](#), [non-subtyping inheritance](#), [error-code exception handling](#), etc.).

Knowing a "pure" OO language doesn't make the transition to OO/C++ any easier. This is not a theory; we have trained and mentored literally thousands of software professionals in OO. In fact, Smalltalk experience can make it *harder* for some people: they need to *unlearn* some rather deep notions about typing and inheritance in addition to needing to learn new syntax and idioms. This unlearning process is especially painful and slow for those who cling to Smalltalk with religious zeal ("C++ is not like Smalltalk, therefore C++ is evil").

If you want to learn OO/C++, learn OO/C++. Taking time out to learn Smalltalk will waste your time and confuse you.

Note: I sit on both the ANSI C++ (X3J16) and ANSI Smalltalk (X3J20) [standardization committees](#). I am [not a language bigot](#). I'm not saying C++ is better or worse than Smalltalk; I'm simply saying that [they are different](#).

[28.4] Should I buy one book, or several? UPDATED!

[Recently added last few sentences recommending purchase of a coding -standards book (in 12/04). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

At least three.

There are three categories of insight and knowledge in OO programming using C++. You should get a great book from each category, not an okay book that tries to do an okay job at everything. The three OO/C++ programming categories are:

- [C++ legality guides — what you can and can't do in C++.](#)
- [C++ morality guides — what you should and shouldn't do in C++.](#)
- [Programming-by-example guides — show lots of examples, normally making liberal use of the C++ standard library.](#)

Legality guides describe all language features with roughly the same level of emphasis; morality guides focus on those language features that you will use most often in typical programming tasks. Legality guides tell you how to get a given feature past the compiler; morality guides tell you whether or not to use that feature in the first place.

Meta comments:

- Don't trade off these categories against each other. You shouldn't argue in favor of one category over the other. They dove-tail.
- The "legality" and "morality" categories are both required. You *must* have a good grasp of both what *can* be done and what *should* be done.

In addition to these (emphasis on "addition"), you should consider at least one book in each of two other categories: at least one book on [OO Design](#) plus at least one book on [coding standards](#). Design books give you ideas and guidelines for thinking at a higher level with objects, and coding standard books establish best practices across your organization, plus help make sure everybody can read each others' code (e.g., so you can move people around if one of the teams falls behind).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[28.5] What are some best-of-breed C++ *morality* guides?

Here's my personal (subjective and selective) short-list of must-read C++ morality guides, alphabetically by author:

- Cline, Lomow, and Girou, *C++ FAQs, Second Edition*, 587 pgs, Addison-Wesley, 1999, ISBN 0-201-30983-1. Covers around 500 topics in a FAQ-like Q&A format.
- Meyers, *Effective C++, Second Edition*, 224 pgs, Addison-Wesley, 1998, ISBN 0-201-92488-9. Covers 50 topics in a short essay format.
- Meyers, *More Effective C++*, 336 pgs, Addison-Wesley, 1996, ISBN 0-201-63371-X. Covers 35 topics in a short essay form at.

Similarities: All three books are extensively illustrated with code examples. All three are excellent, insightful, useful, gold plated books. All three have excellent sales records.

Differences: Cline/Lomow/Girou's examples are complete, working programs rather than code fragments or standalone classes. Meyers contains numerous line-drawings that illustrate the points.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[28.6] What are some best-of-breed C++ *legality* guides?

Here's my personal (subjective and selective) short-list of must-read C++ legality guides, alphabetically by author:

- Lippman and Lajoie, *C++ Primer, Third Edition*, 1237 pgs, Addison-Wesley, 1998, ISBN 0-201-82470-1. Very readable/approachable.
- Stroustrup, *The C++ Programming Language, Third Edition*, 911 pgs, Addison-Wesley, 1998, ISBN 0-201-88954-4. Covers a lot of ground.

Similarities: Both books are excellent overviews of almost every language feature. I reviewed them for back-to-back issues of *C++ Report*, and I said that they are both top notch, gold plated, excellent books. Both have excellent sales records.

Differences: If you don't know C, Lippman's book is better for you. If you know C and you want to cover a lot of ground quickly, Stroustrup's book is better for you.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[28.7] What are some best-of-breed C++ *programming-by-example* guides?

Here's my personal (subjective and selective) short -list of must-read C++ programming-by-example guides:

- Koenig and Moo, *Accelerated C++*, 336 pgs, Addison-Wesley, 2000, ISBN 0-201-70353-X. Lots of examples using the standard C++ library. Truly a programming -by-example book.
- Musser and Saini, *STL Tutorial and Reference Guide, Second Edition* , Addison-Wesley, 2001, ISBN 0-201-037923-6. Lots of examples showing how to use the STL portion of the standard C++ library, plus lots of nitty gritty detail.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[28.8] Are there other OO books that are relevant to OO/C++?

Yes! Tons!

The [morality](#), [legality](#), and [by-example](#) categories listed above were for OO *programming*. The areas of OO analysis and OO design are also relevant, and have their own best -of-breed books.

There are tons and tons of good books in these other areas. The seminal book on OO design patterns is (in my personal, subjective and selective, opinion) a must -read book: Gamma et al., *Design Patterns*, 395 pgs, Addison-Wesley, 1995, ISBN 0-201-63361-2. Describes "patterns" that commonly show up in good OO designs. You *must* read this book if you intend to do OO design work.

[29] Newbie Questions / Answers Updated!

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),

cline@parashift.com)

FAQs in section [29]:

- [\[29.1\] What is this "newbie section" all about?](#)
- [\[29.2\] Where do I start? Why do I feel so confused, so stupid?](#)
- [\[29.3\] Should I use `void main\(\)` or `int main\(\)`?](#)
- [\[29.4\] Should I use `f\(void\)` or `f\(\)`?](#)
- [\[29.5\] What are the criteria for choosing between `short` / `int` / `long` data types?](#) Updated!
- [\[29.6\] What the heck is a `const` variable? Isn't that a contradiction in terms?](#)
- [\[29.7\] Why would I use a `const` variable / `const` identifier as opposed to `#define`?](#)
- [\[29.8\] Are you saying that the preprocessor is evil?](#)
- [\[29.9\] What is the "standard library"? What is included / excluded from it?](#)
- [\[29.10\] How should I lay out my code? When should I use spaces, tabs, and/or newlines in my code?](#)
- [\[29.11\] Is it okay if a lot of numbers appear in my code?](#)
- [\[29.12\] What's the point of the `L`, `U` and `f` suffixes on numeric literals?](#)
- [\[29.13\] I can understand the `and` \(`&&`\) and `or` \(`||`\) operators, but what's the purpose of the `not` \(`!`\) operator?](#)
- [\[29.14\] Is `!\(a < b\)` logically the same as `a >= b`?](#)
- [\[29.15\] What is this NaN thing?](#)
- [\[29.16\] Why is floating point so inaccurate? Why doesn't this print 0.43?](#)
- [\[29.17\] Why doesn't my floating-point comparison work?](#)
- [\[29.18\] What is the type of an enumeration such as `enum Color`? Is it of type `int`?](#) Updated!
- [\[29.19\] If an enumeration type is distinct from any other type, what good is it? What can you do with it?](#)

[29.1] What is this "newbie section" all about?

IT'S A RANDOMLY ORDERED COLLECTION CONTAINING A FEW QUESTIONS NEWBIES MIGHT ASK.

- This section doesn't pretend to be organized. Think of it as random. In truth, think of it as a hurried, initial cut by a busy guy.
- This section doesn't pretend to be complete. Think of it as offering a little help to a few people. It won't help everyone and it might not help you.

HOPEFULLY SOMEDAY I'LL BE ABLE TO IMPROVE THIS SECTION, BUT FOR NOW, IT IS INCOMPLETE AND UNORGANIZED. IF THAT BOTHERS YOU, MY SUGGESTION IS TO CLICK THAT LITTLE X ON THE EXTREME UPPER RIGHT OF YOUR BROWSER WINDOW :-).

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.2] Where do I start? Why do I feel so confused, so stupid?

READ THE FAQ, ESPECIALLY THE [SECTION ON LEARNING C++](#), READ [COMP.LANG.C++](#), READ [BOOKS](#) PLURAL.

BUT IF EVERYTHING STILL SEEMS TOO HARD, IF YOU'RE FEELING BOMBARDED WITH MYSTERIOUS TERMS AND CONCEPTS, IF YOU'RE WONDERING HOW YOU'LL EVER GRASP ANYTHING, DO THIS:

- Type in some C++ code from any of the sources listed above.
- Get it to compile and run.
- Repeat.

THAT'S IT. JUST PRACTICE AND PLAY. HOPEFULLY THAT WILL GIVE YOU A FOOTHOLD.

HERE ARE SOME PLACES YOU CAN GET "SAMPLE PROBLEMS" (IN ALPHABETICAL ORDER):

- The [British Informatics Olympiad](#)
- The [Dictionary of Algorithms and Data Structures](#)
- The [University of Valladolid Programming Contest Site](#)

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.3] Should I use *void main()* or *int main()*?

INT MAIN()

MAIN() MUST RETURN INT. NOT VOID, NOT BOOL, NOT FLOAT. INT. JUST INT, NOTHING BUT INT, ONLY INT.

SOME COMPILERS ACCEPT VOID MAIN(), BUT THAT IS NON-STANDARD AND SHOULDN'T BE USED. INSTEAD USE INT MAIN().

```
void main() // ← BAD; DO NOT DO THIS
{
    ...
}
int main() // ← GOOD
{
    ...
}
int main(int argc, char** argv) // ← GOOD
{
    ...
}
```

AS TO THE SPECIFIC RETURN VALUE, IF YOU DON'T KNOW WHAT ELSE TO RETURN JUST SAY RETURN 0;

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.4] Should I use *f(void)* or *f()*?

F()

C PROGRAMMERS OFTEN USE `F(VOID)` WHEN DECLARING A FUNCTION THAT TAKES NO PARAMETERS, HOWEVER IN C++ THAT IS CONSIDERED BAD STYLE. IN FACT, THE `F(VOID)` STYLE HAS BEEN CALLED [AN "ABOMINATION"](#) BY BJARNE STROUSTRUP, THE CREATOR OF C++, DENNIS RITCHIE, THE CO-CREATOR OF C, AND DOUG MCILROY, HEAD OF THE RESEARCH DEPARTMENT WHERE UNIX WAS BORN.

IF YOU'RE WRITING C++ CODE, YOU SHOULD USE `f()`. THE `F(VOID)` STYLE IS LEGAL IN C++, BUT ONLY TO MAKE IT EASIER TO COMPILE C CODE.

THIS C++ CODE SHOWS THE BEST WAY TO DECLARE A FUNCTION THAT TAKES NO PARAMETERS:

```
void f();    // declares (not defines) a function that takes no parameters
```

THIS C++ CODE BOTH DECLARES AND DEFINES A FUNCTION THAT TAKES NO PARAMETERS:

```
void f()    // declares and defines a function that takes no parameters
{
    ...
}
```

THE FOLLOWING C++ CODE ALSO DECLARES A FUNCTION THAT TAKES NO PARAMETERS, BUT IT USES THE LESS DESIRABLE (SOME WOULD SAY "ABOMINATION") STYLE, `F(VOID)`:

```
void f(void); // undesirable style for C++; use void f() instead
```

ACTUALLY THIS `f()` THING IS ALL YOU NEED TO KNOW ABOUT C++. THAT AND USING THOSE NEW FANGLED `//` COMMENTS. ONCE YOU KNOW THOSE TWO THINGS, YOU CAN CLAIM TO BE A C++ EXPERT. GO FOR IT: TYPE THOSE MAGICAL `"++"` MARKS ON YOUR RESUME. WHO CARES ABOUT ALL THAT OO STUFF — WHY SHOULD YOU BOTHER CHANGING THE WAY YOU THINK? AFTER ALL, THE REALLY IMPORTANT THING ISN'T THINKING; IT'S TYPING IN FUNCTION DECLARATIONS AND COMMENTS. (SIGH; I WISH NOBODY ACTUALLY THOUGHT THAT WAY.)

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.5] What are the criteria for choosing between *short* / *int* / *long* data types? Updated!

[Recently added the disclaimer that `<stdint.h>` is not part of the C++ standard and might not be supplied by your compiler vendor thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

OTHER RELATED QUESTIONS: IF A SHORT INT IS THE SAME SIZE AS AN INT ON MY PARTICULAR IMPLEMENTATION, WHY CHOOSE ONE OR THE OTHER? IF I START TAKING THE ACTUAL SIZE IN BYTES OF THE VARIABLES INTO ACCOUNT, WON'T I BE MAKING MY CODE UNPORTABLE (SINCE THE SIZE IN BYTES MAY DIFFER FROM IMPLEMENTATION TO IMPLEMENTATION)? OR SHOULD I SIMPLY GO WITH SIZES MUCH LARGER THAN I ACTUALLY NEED, AS A SORT OF SAFETY BUFFER?

ANSWER: IT'S USUALLY A GOOD IDEA TO WRITE CODE THAT CAN BE PORTED TO A DIFFERENT OPERATING SYSTEM AND/OR COMPILER. AFTER ALL, IF YOU'RE SUCCESSFUL AT WHAT YOU DO, SOMEONE ELSE MIGHT WANT TO USE IT SOMEWHERE ELSE. THIS CAN BE A LITTLE TRICKY WITH BUILT-IN TYPES LIKE INT AND SHORT, SINCE C++ DOESN'T GIVE GUARANTEED SIZES. HOWEVER C++ GIVES YOU TWO THINGS THAT MIGHT HELP: GUARANTEED MINIMUM SIZES, AND THAT WILL USUALLY BE ALL YOU NEED TO KNOW, AND A STANDARD C HEADER THAT PROVIDES TYPEDEFS FOR SIZED INTEGERS.

C++ GUARANTEES [A CHAR IS EXACTLY ONE BYTE](#) WHICH IS AT LEAST 8 BITS, SHORT IS AT LEAST 16 BITS, INT IS AT LEAST 16 BITS, AND LONG IS AT LEAST 32 BITS. IT ALSO GUARANTEES THE UNSIGNED VERSION OF EACH OF THESE IS THE SAME SIZE AS THE ORIGINAL, FOR EXAMPLE, `sizeof(unsigned short) == sizeof(short)`.

WHEN WRITING PORTABLE CODE, YOU SHOULDN'T MAKE ADDITIONAL ASSUMPTIONS ABOUT THESE SIZES. FOR EXAMPLE, DON'T ASSUME INT HAS 32 BITS. IF YOU HAVE AN INTEGRAL VARIABLE THAT NEEDS AT LEAST 32 BITS, USE A LONG OR UNSIGNED LONG EVEN IF `sizeof(int) == 4` ON YOUR PARTICULAR IMPLEMENTATION. ON THE OTHER HAND, IF YOU HAVE AN INTEGRAL VARIABLE QUANTITY THAT WILL ALWAYS FIT WITHIN 16 BITS AND IF YOU WANT TO MINIMIZE THE USE OF DATA MEMORY, USE A SHORT OR UNSIGNED SHORT EVEN IF YOU KNOW `sizeof(int) == 2` ON YOUR PARTICULAR IMPLEMENTATION.

THE OTHER OPTION IS TO USE THE FOLLOWING STANDARD C HEADER (WHICH MAY OR MAY NOT BE PROVIDED BY YOUR C++ COMPILER VENDOR):

```
#include <stdint.h> /* not part of the C++ standard */
```

THAT HEADER DEFINES TYPEDEFS FOR THINGS LIKE `int32_t` AND `uint16_t`, WHICH ARE A SIGNED 32-BIT INTEGER AND AN UNSIGNED 16-BIT INTEGER, RESPECTIVELY. THERE ARE OTHER GOODIES IN THERE, AS WELL. MY RECOMMENDATION IS THAT YOU USE THESE "SIZED" INTEGRAL TYPES ONLY WHERE THEY ARE ACTUALLY NEEDED. SOME PEOPLE WORSHIP CONSISTENCY, AND THEY ARE SORELY TEMPTED TO USE THESE SIZED INTEGERS EVERYWHERE SIMPLY BECAUSE THEY WERE NEEDED SOMEWHERE. CONSISTENCY IS GOOD, BUT IT IS NOT THE GREATEST GOOD, AND USING THESE TYPEDEFS EVERYWHERE CAN CAUSE SOME HEADACHES AND EVEN POSSIBLE PERFORMANCE ISSUES. BETTER TO USE COMMON SENSE, WHICH OFTEN LEADS YOU TO USE THE NORMAL KEYWORDS, E.G., `int`, `unsigned`, ETC. WHERE YOU CAN, AND USE OF THE EXPLICITLY SIZED INTEGER TYPES, E.G., `int32_t`, ETC. WHERE YOU MUST.

NOTE THAT THERE ARE SOME SUBTLE TRADEOFFS HERE. IN SOME CASES, YOUR COMPUTER MIGHT BE ABLE TO MANIPULATE SMALLER THINGS FASTER THAN BIGGER THINGS, BUT IN OTHER CASES IT IS EXACTLY THE OPPOSITE: INT ARITHMETIC MIGHT BE FASTER THAN SHORT ARITHMETIC ON SOME IMPLEMENTATIONS. ANOTHER TRADEOFF IS DATA-SPACE AGAINST CODE-SPACE: INT ARITHMETIC MIGHT GENERATE LESS BINARY CODE THAN SHORT ARITHMETIC ON SOME IMPLEMENTATIONS. DON'T MAKE SIMPLISTIC ASSUMPTIONS. JUST BECAUSE A PARTICULAR VARIABLE CAN BE DECLARED AS `short` DOESN'T NECESSARILY MEAN IT SHOULD, EVEN IF YOU'RE TRYING TO SAVE SPACE.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.6] What the heck is a *const* variable? Isn't that a contradiction in terms?

IF IT BOTHERS YOU, CALL IT A "CONST IDENTIFIER" INSTEAD.

THE MAIN ISSUE IS TO FIGURE OUT WHAT IT IS; WE CAN FIGURE OUT WHAT TO CALL IT LATER. FOR EXAMPLE, CONSIDER THE SYMBOL `MAX` IN THE FOLLOWING FUNCTION:

```
void f()
{
    const int max = 107;
    ...
    float array[max];
    ...
}
```

IT DOESN'T MATTER WHETHER YOU CALL `MAX` A `CONST` VARIABLE OR A `CONST` IDENTIFIER. WHAT MATTERS IS THAT YOU REALIZE IT IS LIKE A NORMAL VARIABLE IN SOME WAYS (E.G., YOU CAN TAKE ITS ADDRESS OR PASS IT BY `CONST`-REFERENCE), BUT IT IS UNLIKE A NORMAL VARIABLE IN THAT YOU CAN'T CHANGE ITS VALUE.

HERE IS ANOTHER EVEN MORE COMMON EXAMPLE:

```
class Fred {  
public:  
...  
private:  
    static const int max_ = 107;  
...  
};
```

IN THIS EXAMPLE, YOU WOULD NEED TO ADD THE LINE `INT FRED::MAX_;` IN EXACTLY ONE .CPP FILE, TYPICALLY IN `FRED.CPP`.

IT IS GENERALLY CONSIDERED GOOD PROGRAMMING PRACTICE TO [GIVE EACH "MAGIC NUMBER" \(LIKE 107\) A SYMBOLIC NAME](#) AND USE THAT NAME RATHER THAN THE RAW MAGIC NUMBER.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.7] Why would I use a *const* variable / *const* identifier as opposed to `#define`?

`CONST` IDENTIFIERS ARE OFTEN BETTER THAN `#DEFINE` BECAUSE:

- they obey the language's scoping rules
- you can see them in the debugger
- you can take their address if you need to
- you can pass them by *const*-reference if you need to
- they don't create new "keywords" in your program.

IN SHORT, `CONST` IDENTIFIERS ACT LIKE THEY'RE PART OF THE LANGUAGE BECAUSE THEY ARE PART OF THE LANGUAGE. THE PREPROCESSOR CAN BE THOUGHT OF AS A LANGUAGE LAYERED ON TOP OF C++. YOU CAN IMAGINE THAT THE PREPROCESSOR RUNS AS A SEPARATE PASS THROUGH YOUR CODE, WHICH WOULD MEAN YOUR ORIGINAL SOURCE CODE WOULD BE SEEN ONLY BY THE PREPROCESSOR, NOT BY THE C++ COMPILER ITSELF. IN OTHER WORDS, YOU CAN IMAGINE THE PREPROCESSOR SEES YOUR ORIGINAL SOURCE CODE AND REPLACES ALL `#DEFINE` SYMBOLS WITH THEIR VALUES, THEN THE C++ COMPILER PROPER SEES THE MODIFIED SOURCE CODE AFTER THE ORIGINAL SYMBOLS GOT REPLACED BY THE PREPROCESSOR.

THERE ARE CASES WHERE `#DEFINE` IS NEEDED, BUT YOU SHOULD GENERALLY AVOID IT WHEN YOU HAVE THE CHOICE. YOU SHOULD EVALUATE WHETHER TO USE `CONST` VS. `#DEFINE` BASED ON BUSINESS VALUE: TIME, MONEY, RISK. IN OTHER WORDS, ONE SIZE DOES NOT FIT ALL. MOST OF THE TIME YOU'LL USE `CONST` RATHER THAN `#DEFINE` FOR CONSTANTS, BUT SOMETIMES YOU'LL USE `#DEFINE`. BUT PLEASE REMEMBER TO WASH YOUR HANDS AFTERWARDS.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.8] Are you saying that the preprocessor is evil?

YES, THAT'S EXACTLY WHAT I'M SAYING: THE PREPROCESSOR IS [EVIL](#).

EVERY `#DEFINE` MACRO EFFECTIVELY CREATES A NEW KEYWORD IN EVERY SOURCE FILE AND EVERY SCOPE UNTIL THAT SYMBOL IS `#UNDEFD`. THE PREPROCESSOR LETS YOU CREATE A `#DEFINE` SYMBOL THAT IS ALWAYS REPLACED INDEPENDENT OF THE `{...}` SCOPE WHERE THAT SYMBOL APPEARS.

[SOMETIMES WE NEED THE PREPROCESSOR](#), SUCH AS THE `#IFDEF/#DEFINE` WRAPPER WITHIN EACH HEADER FILE, BUT IT SHOULD BE AVOIDED WHEN YOU CAN. "[EVIL](#)" DOESN'T MEAN "NEVER USE." YOU WILL USE EVIL THINGS SOMETIMES, PARTICULARLY WHEN THEY ARE "THE LESSER OF TWO EVILS." BUT THEY'RE STILL EVIL :-)

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.9] What is the "standard library"? What is included / excluded from it?

MOST (NOT ALL) IMPLEMENTATIONS HAVE A "STANDARD INCLUDE" DIRECTORY, SOMETIMES DIRECTORIES PLURAL. IF YOUR IMPLEMENTATION IS LIKE THAT, THE HEADERS IN THE STANDARD LIBRARY ARE PROBABLY A SUBSET OF THE FILES IN THOSE DIRECTORIES. FOR EXAMPLE, `IOSTREAM` AND `STRING` ARE PART OF THE STANDARD LIBRARY, AS IS `CSTRING` AND `CSTDIO`. THERE ARE A BUNCH OF `.H` FILES THAT ARE ALSO PART OF THE STANDARD LIBRARY, BUT NOT EVERY `.H` FILE IN THOSE DIRECTORIES IS PART OF THE STANDARD LIBRARY. FOR EXAMPLE, `STDIO.H` IS BUT `WINDOWS.H` IS NOT.

YOU INCLUDE HEADERS FROM THE STANDARD LIBRARY LIKE THIS:

```
#include <iostream>

int main()
{
    std::cout << "Hello world!\n";
    ...
}
```

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.10] How should I lay out my code? When should I use spaces, tabs, and/or newlines in my code?

THE SHORT ANSWER IS: JUST LIKE THE REST OF YOUR TEAM. IN OTHER WORDS, THE TEAM SHOULD USE A CONSISTENT APPROACH TO WHITESPACE, BUT OTHERWISE PLEASE DON'T WASTE A LOT OF TIME WORRYING ABOUT IT.

HERE ARE A FEW DETAILS:

THERE IS NO UNIVERSALLY ACCEPTED CODING STANDARD WHEN IT COMES TO WHITESPACE. THERE ARE A FEW POPULAR WHITESPACE STANDARDS, SUCH AS THE "ONE TRUE BRACE" STYLE, BUT THERE IS A LOT OF CONTENTION OVER CERTAIN ASPECTS OF ANY GIVEN CODING STANDARD.

MOST WHITESPACE STANDARDS AGREE ON A FEW POINTS, SUCH AS PUTTING A SPACE AROUND INFIX OPERATORS LIKE `X * Y` OR `A - B`. MOST (NOT ALL) WHITESPACE STANDARDS DO NOT PUT SPACES AROUND THE `[ OR ]` IN `A[I]`, AND SIMILAR COMMENTS FOR `( AND )` IN `F(X)`. HOWEVER THERE IS A GREAT DEAL OF CONTENTION OVER VERTICAL WHITESPACE,

PARTICULARLY WHEN IT COMES TO { AND }. FOR EXAMPLE, HERE ARE A FEW OF THE MANY WAYS TO LAY OUT IF (foo()) {
BAR(); BAZ(); }:

```
if (foo()) {  
    bar();  
    baz();  
}  
if (foo())  
{  
    bar();  
    baz();  
}  
if (foo())  
{  
    bar();  
    baz();  
}  
if (foo())  
{  
    bar();  
    baz();  
}  
if (foo()) {  
    bar();  
    baz();  
}
```

...AND OTHERS...

IMPORTANT: Do NOT EMAIL ME WITH REASONS YOUR WHITESPACE APPROACH IS BETTER THAN THE OTHERS. I DON'T CARE. PLUS I WON'T BELIEVE YOU. THERE IS NO OBJECTIVE STANDARD OF "BETTER" WHEN IT COMES TO WHITESPACE SO YOUR OPINION IS JUST THAT: YOUR OPINION. IF YOU WRITE ME AN EMAIL IN SPITE OF THIS PARAGRAPH, I WILL CONSIDER YOU TO BE A HOPELESS GEEK WHO FOCUSES ON NITS. DON'T WASTE YOUR TIME WORRYING ABOUT WHITESPACE: AS LONG AS YOUR TEAM USES A CONSISTENT WHITESPACE STYLE, GET ON WITH YOUR LIFE AND WORRY ABOUT MORE IMPORTANT THINGS.

FOR EXAMPLE, THINGS YOU SHOULD BE WORRIED ABOUT INCLUDE DESIGN ISSUES LIKE WHEN [ABCS](#) SHOULD BE USED, WHETHER INHERITANCE SHOULD BE AN IMPLEMENTATION OR SPECIFICATION TECHNIQUE, WHAT TESTING AND INSPECTION STRATEGIES SHOULD BE USED, WHETHER INTERFACES SHOULD UNIFORMLY HAVE A `GET()` AND/OR `SET()` MEMBER FUNCTION FOR EACH DATA MEMBER, WHETHER INTERFACES SHOULD BE DESIGNED FROM THE OUTSIDE-IN OR THE INSIDE-OUT, WHETHER ERRORS BE HANDLED BY `TRY/CATCH/THROW` OR BY RETURN CODES, ETC. READ THE [FAQ](#) FOR SOME OPINIONS ON THOSE IMPORTANT QUESTIONS, BUT PLEASE DON'T WASTE YOUR TIME ARGUING OVER WHITESPACE. AS LONG AS THE TEAM IS USING A CONSISTENT WHITESPACE STRATEGY, DROP IT.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.11] Is it okay if a lot of numbers appear in my code?

PROBABLY NOT.

IN MANY (NOT ALL) CASES, IT'S BEST TO NAME YOUR NUMBERS SO EACH NUMBER APPEARS ONLY ONCE IN YOUR CODE. THAT WAY, WHEN THE NUMBER CHANGES THERE WILL ONLY BE ONE PLACE IN THE CODE THAT HAS TO CHANGE.

FOR EXAMPLE, SUPPOSE YOUR PROGRAM IS WORKING WITH SHIPPING CRATES. THE WEIGHT OF AN EMPTY CRATE IS 5.7. THE EXPRESSION `5.7 + CONTENTSWEIGHT` PROBABLY MEANS THE WEIGHT OF THE CRATE INCLUDING ITS CONTENTS, MEANING THE NUMBER 5.7 PROBABLY APPEAR MANY TIMES IN THE SOFTWARE. ALL THESE OCCURRENCES OF THE NUMBER 5.7 WILL BE DIFFICULT TO FIND AND CHANGE WHEN (NOT IF) SOMEBODY CHANGES THE STYLE OF CRATES USED IN THIS APPLICATION. THE SOLUTION IS TO MAKE SURE THE VALUE 5.7 APPEARS EXACTLY ONCE, USUALLY AS THE INITIALIZER FOR A `CONST` IDENTIFIER. TYPICALLY THIS WILL BE SOMETHING LIKE `CONST DOUBLE CRATEWEIGHT = 5.7;`. AFTER THAT, `5.7 + CONTENTSWEIGHT` WOULD BE REPLACED BY `CRATEWEIGHT + CONTENTSWEIGHT`.

NOW THAT'S THE GENERAL RULE OF THUMB. BUT UNFORTUNATELY THERE IS SOME FINE PRINT.

SOME PEOPLE BELIEVE ONE SHOULD NEVER HAVE NUMERIC LITERALS SCATTERED IN THE CODE. THEY BELIEVE ALL NUMERIC VALUES SHOULD BE NAMED IN A MANNER SIMILAR TO THAT DESCRIBED ABOVE. THAT RULE, HOWEVER NOBLE IN INTENT, JUST DOESN'T WORK VERY WELL IN PRACTICE. IT IS TOO TEDIOUS FOR PEOPLE TO FOLLOW, AND ULTIMATELY IT COSTS COMPANIES MORE THAN IT SAVES THEM. REMEMBER: THE GOAL OF ALL PROGRAMMING RULES IS TO REDUCE TIME, COST AND RISK. IF A RULE ACTUALLY MAKES THINGS WORSE, IT IS A BAD RULE, PERIOD.

A MORE PRACTICAL RULE IS TO FOCUS ON THOSE VALUES THAT ARE LIKELY TO CHANGE. FOR EXAMPLE, IF A NUMERIC LITERAL IS LIKELY TO CHANGE, IT SHOULD APPEAR ONLY ONCE IN THE SOFTWARE, USUALLY AS THE INITIALIZER OF A `CONST` IDENTIFIER. THIS RULE LETS UNCHANGING VALUES, SUCH AS SOME OCCURRENCES OF 0, 1, -1, ETC., GET CODED DIRECTLY IN THE SOFTWARE SO PROGRAMMERS DON'T HAVE TO SEARCH FOR THE ONE TRUE DEFINITION OF ONE OR ZERO. IN OTHER WORDS, IF A PROGRAMMER WANTS TO LOOP OVER THE INDICES OF A VECTOR, HE CAN SIMPLY WRITE `FOR (INT I = 0; I < V.SIZE(); ++I)`. THE "EXTREMIST" RULE DESCRIBED EARLIER WOULD REQUIRE THE PROGRAMMER TO POKE AROUND ASKING IF ANYBODY ELSE HAS DEFINED A `CONST` IDENTIFIER INITIALIZED TO 0, AND IF NOT, TO DEFINE HIS OWN `CONST INT ZERO = 0;` THEN REPLACE THE LOOP WITH `FOR (INT I = ZERO; I < V.SIZE(); ++I)`. THIS IS ALL A WASTE OF TIME SINCE THE LOOP WILL ALWAYS START WITH 0. IT ADDS COST WITHOUT ADDING ANY VALUE TO COMPENSATE FOR THAT COST.

OBVIOUSLY PEOPLE MIGHT ARGUE OVER EXACTLY WHICH VALUES ARE "LIKELY TO CHANGE," BUT THAT KIND OF JUDGMENT IS WHY YOU GET PAID THE BIG BUCKS: DO YOUR JOB AND MAKE A DECISION. SOME PEOPLE ARE SO AFRAID OF MAKING A WRONG DECISION THAT THEY'LL ADOPT A ONE-SIZE-FITS-ALL RULE SUCH AS "GIVE A NAME TO EVERY NUMBER." BUT IF YOU ADOPT RULES LIKE THAT, YOU'RE GUARANTEED TO HAVE MADE THE WRONG DECISION: THOSE RULES COST YOUR COMPANY MORE THAN THEY SAVE. THEY ARE BAD RULES.

THE CHOICE IS SIMPLE: USE A FLEXIBLE RULE EVEN THOUGH YOU MIGHT MAKE A WRONG DECISION, OR USE A ONE-SIZE-FITS-ALL RULE AND BE GUARANTEED TO MAKE A WRONG DECISION.

THERE IS ONE MORE PIECE OF FINE PRINT: WHERE THE `CONST` IDENTIFIER SHOULD BE DEFINED. THERE ARE THREE TYPICAL CASES:

- If the *const* identifier is used only within a single function, it can be local to that function.
- If the *const* identifier is used throughout a class and nowhere else, it can be *static* within the *private* part of that class.
- If the *const* identifier is used in numerous classes, it can be *static* within the *public* part of the most appropriate class, or perhaps *private* in that class with a *public static* access method.

AS A LAST RESORT, MAKE IT `STATIC` WITHIN A NAMESPACE OR PERHAPS PUT IT IN THE UNNAMED NAMESPACE. TRY VERY HARD TO AVOID USING `#DEFINE` SINCE [THE PREPROCESSOR IS EVIL](#). IF YOU NEED TO USE `#DEFINE` [ANYWAY](#), WASH YOUR HANDS WHEN YOU'RE DONE. AND PLEASE ASK SOME FRIENDS IF THEY KNOW OF A BETTER ALTERNATIVE.

(AS USED THROUGHOUT THE FAQ, "[EVIL](#)" DOESN'T MEAN "NEVER USE IT." THERE ARE TIMES WHEN YOU WILL USE SOMETHING THAT IS "EVIL" SINCE IT WILL BE, IN THOSE PARTICULAR CASES, [THE LESSER OF TWO EVILS](#).)

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.12] What's the point of the *L*, *U* and *f* suffixes on numeric literals?

YOU SHOULD USE THESE SUFFIXES WHEN YOU NEED TO FORCE THE COMPILER TO TREAT THE NUMERIC LITERAL AS IF IT WERE THE SPECIFIED TYPE. FOR EXAMPLE, IF *X* IS OF TYPE *FLOAT*, THE EXPRESSION *X + 5.7* IS OF TYPE *DOUBLE*: IT FIRST PROMOTES THE VALUE OF *X* TO A *DOUBLE*, THEN PERFORMS THE ARITHMETIC USING DOUBLE-PRECISION INSTRUCTIONS. IF THAT IS WHAT YOU WANT, FINE; BUT IF YOU REALLY WANTED IT TO DO THE ARITHMETIC USING SINGLE-PRECISION INSTRUCTIONS, YOU CAN CHANGE THAT CODE TO *X + 5.7f*. NOTE: IT IS [EVEN BETTER TO "NAME" YOUR NUMERIC LITERALS, PARTICULARLY THOSE THAT ARE LIKELY TO CHANGE](#). THAT WOULD REQUIRE YOU TO SAY *X + CRATEWEIGHT* WHERE *CRATEWEIGHT* IS A *CONST FLOAT* THAT IS INITIALIZED TO *5.7f*.

THE *U* SUFFIX IS SIMILAR. IT'S PROBABLY A GOOD IDEA TO USE UNSIGNED INTEGERS FOR VARIABLES THAT ARE ALWAYS ≥ 0 . FOR EXAMPLE, IF A VARIABLE REPRESENTS AN INDEX INTO AN ARRAY, THAT VARIABLE WOULD TYPICALLY BE DECLARED AS AN UNSIGNED. THE MAIN REASON FOR THIS IS IT REQUIRES LESS CODE, AT LEAST IF YOU ARE CAREFUL TO CHECK YOUR RANGES. FOR EXAMPLE, TO CHECK IF A VARIABLE IS BOTH ≥ 0 AND $< \text{MAX}$ REQUIRES TWO TESTS IF EVERYTHING IS SIGNED: IF ($N \geq 0 \ \&\& \ N < \text{MAX}$), BUT CAN BE DONE WITH A SINGLE COMPARISON IF EVERYTHING IS UNSIGNED: IF ($N < \text{MAX}$).

IF YOU END UP USING UNSIGNED VARIABLES, IT IS GENERALLY A GOOD IDEA TO FORCE YOUR NUMERIC LITERALS TO ALSO BE UNSIGNED. THAT MAKES IT EASIER TO SEE THAT THE COMPILER WILL GENERATE "UNSIGNED ARITHMETIC" INSTRUCTIONS. FOR EXAMPLE: IF ($N < 256U$) OR IF ($(N \& 255U) < 32U$). MIXING SIGNED AND UNSIGNED VALUES IN A SINGLE ARITHMETIC EXPRESSION IS OFTEN CONFUSING FOR PROGRAMMERS — THE COMPILER DOESN'T ALWAYS DO WHAT YOU EXPECT IT SHOULD DO.

THE *L* SUFFIX IS NOT AS COMMON, BUT IT IS OCCASIONALLY USED FOR SIMILAR REASONS AS ABOVE: TO MAKE IT OBVIOUS THAT THE COMPILER IS USING LONG ARITHMETIC.

THE BOTTOM LINE IS THIS: IT IS A GOOD DISCIPLINE FOR PROGRAMMERS TO FORCE ALL NUMERIC OPERANDS TO BE OF THE RIGHT TYPE, AS OPPOSED TO RELYING ON THE C++ RULES FOR PROMOTING/DEMOTING NUMERIC EXPRESSIONS. FOR EXAMPLE, IF *X* IS OF TYPE *INT* AND *Y* IS OF TYPE *UNSIGNED*, IT IS A GOOD IDEA TO CHANGE *X + Y* SO THE NEXT PROGRAMMER KNOWS WHETHER YOU INTENDED TO USE UNSIGNED ARITHMETIC, E.G., *UNSIGNED(X) + Y*, OR SIGNED ARITHMETIC: *X + INT(Y)*. THE OTHER POSSIBILITY IS LONG ARITHMETIC: *LONG(X) + LONG(Y)*. BY USING THOSE CASTS, THE CODE IS MORE EXPLICIT AND THAT'S GOOD IN THIS CASE, SINCE A LOT OF PROGRAMMERS DON'T KNOW ALL THE RULES FOR IMPLICIT PROMOTIONS.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.13] I can understand the *and* (*&&*) and *or* (*||*) operators, but what's the purpose of the *not* (*!*) operator?

SOME PEOPLE ARE CONFUSED ABOUT THE *!* OPERATOR. FOR EXAMPLE, THEY THINK THAT *!TRUE* IS THE SAME AS *FALSE*, OR THAT [!\(A < B\) IS THE SAME AS A \$\geq\$ B](#), SO IN BOTH CASES THE *!* OPERATOR DOESN'T SEEM TO ADD ANYTHING.

ANSWER: THE *!* OPERATOR IS USEFUL IN BOOLEAN EXPRESSIONS, SUCH OCCUR IN AN IF OR WHILE STATEMENT. FOR EXAMPLE, LET'S ASSUME *A* AND *B* ARE BOOLEAN EXPRESSIONS, PERHAPS SIMPLE METHOD-CALLS THAT RETURN A *BOOL*. THERE ARE ALL SORTS OF WAYS TO COMBINE THESE TWO EXPRESSIONS:

```
if ( A && B ) ...  
if (!A && B ) ...  
if ( A && !B ) ...
```

```
if (!A && !B) ...  
if (!(A && B)) ...  
if (!(!A && B)) ...  
if (!(A && !B)) ...  
if (!(!A && !B)) ...
```

ALONG WITH A SIMILAR GROUP FORMED USING THE `||` OPERATOR.

NOTE: BOOLEAN ALGEBRA CAN BE USED TO TRANSFORM EACH OF THE `&&`-VERSIONS INTO AN EQUIVALENT `||`-VERSION, SO FROM A TRUTH-TABLE STANDPOINT THERE ARE ONLY 8 LOGICALLY DISTINCT IF STATEMENTS. HOWEVER, SINCE READABILITY IS SO IMPORTANT IN SOFTWARE, PROGRAMMERS SHOULD CONSIDER BOTH THE `&&`-VERSION AND THE LOGICALLY EQUIVALENT `||`-VERSION. FOR EXAMPLE, PROGRAMMERS SHOULD CHOOSE BETWEEN `!A && !B` AND `!(A || B)` BASED ON WHICH ONE IS MORE OBVIOUS TO WHOEVER WILL BE MAINTAINING THE CODE. IN THAT SENSE THERE REALLY ARE 16 DIFFERENT CHOICES.

THE POINT OF ALL THIS IS SIMPLE: THE `!` OPERATOR IS QUITE USEFUL IN BOOLEAN EXPRESSIONS. SOMETIMES IT IS USED FOR READABILITY, AND SOMETIMES IT IS USED BECAUSE EXPRESSIONS LIKE `!(A < B)` ACTUALLY ARE NOT EQUIVALENT TO `A >= B` IN SPITE OF WHAT YOUR GRADE SCHOOL MATH TEACHER TOLD YOU.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.14] Is `!(a < b)` logically the same as `a >= b`?

No!

DESPITE WHAT YOUR GRADE SCHOOL MATH TEACHER TAUGHT YOU, THESE EQUIVALENCES DON'T ALWAYS WORK IN SOFTWARE, ESPECIALLY WITH FLOATING POINT EXPRESSIONS OR USER-DEFINED TYPES.

EXAMPLE: IF `A` IS A FLOATING POINT NAN, THEN BOTH `A < B` AND `A >= B` WILL BE FALSE. THAT MEANS `!(A < B)` WILL BE TRUE AND `A >= B` WILL BE FALSE.

EXAMPLE: IF `A` IS AN OBJECT OF CLASS `FOO` THAT HAS OVERLOADED `OPERATOR<` AND `OPERATOR>=`, THEN IT IS UP TO THE CREATOR OF CLASS `FOO` IF THESE OPERATORS WILL HAVE OPPOSITE SEMANTICS. THEY PROBABLY SHOULD HAVE OPPOSITE SEMANTICS, BUT THAT'S UP TO WHOEVER WROTE CLASS `FOO`.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.15] What is this NaN thing?

NAN MEANS "NOT A NUMBER," AND IS USED FOR FLOATING POINT OPERATIONS.

THERE ARE LOTS OF FLOATING POINT OPERATIONS THAT DON'T MAKE SENSE, SUCH AS DIVIDING BY ZERO, TAKING THE LOG OF ZERO OR A NEGATIVE NUMBER, TAKING THE SQUARE ROOT OF A NEGATIVE NUMBER, ETC. DEPENDING ON YOUR COMPILER, SOME OF THESE OPERATIONS MAY PRODUCE SPECIAL FLOATING POINT VALUES SUCH AS INFINITY (WITH DISTINCT VALUES FOR POSITIVE VS. NEGATIVE INFINITY) AND THE NOT A NUMBER VALUE, NAN.

IF YOUR COMPILER PRODUCES A NAN, IT HAS THE UNUSUAL PROPERTY THAT IT IS NOT EQUAL TO ANY VALUE, INCLUDING ITSELF. FOR EXAMPLE, IF `A` IS NAN, THEN `A == A` IS FALSE. THAT IS THE USUAL WAY TO CHECK IF YOU ARE DEALING WITH A NAN:

```
void funct(double x)
{
    if (x == x) {
        // X is a normal value
        ...
    } else {
        // X is NaN
        ...
    }
}
```

SIMILARLY, IF A IS NAN AND B IS SOME ARBITRARY FLOATING POINT VALUE, A WILL BE NEITHER LESS THAN, EQUAL TO, NOR GREATER THAN B. IN OTHER WORDS, $A < B$, $A \leq B$, $A > B$, $A \geq B$, AND $A == B$ WILL ALL RETURN FALSE.

[\[TOP \]](#) | [\[BOTTOM \]](#) | [\[PREVIOUS SECTION \]](#) | [\[NEXT SECTION \]](#) | [\[SEARCH THE FAQ \]](#)

[29.16] Why is floating point so inaccurate? Why doesn't this print 0.43?

```
#include <iostream>

int main()
{
    float a = 1000.43;
    float b = 1000.0;
    std::cout << a - b << '\n';
    ...
}
```

(ON ONE C++ IMPLEMENTATION, THIS PRINTS 0.429993)

DISCLAIMER: FRUSTRATION WITH ROUNDING/TRUNCATION/APPROXIMATION ISN'T REALLY A C++ ISSUE; IT'S A COMPUTER SCIENCE ISSUE. HOWEVER, PEOPLE KEEP ASKING ABOUT IT ON [COMPLANG.C++](#), SO WHAT FOLLOWS IS A NOMINAL ANSWER.

ANSWER: FLOATING POINT IS AN APPROXIMATION. THE IEEE STANDARD FOR 32 BIT FLOAT SUPPORTS 1 BIT OF SIGN, 8 BITS OF EXPONENT, AND 23 BITS OF MANTISSA. SINCE A NORMALIZED BINARY-POINT MANTISSA ALWAYS HAS THE FORM 1.xxxxx... THE LEADING 1 IS DROPPED AND YOU GET EFFECTIVELY 24 BITS OF MANTISSA. THE NUMBER 1000.43 (AND MANY, MANY OTHERS, INCLUDING SOME REALLY COMMON ONES LIKE 0.1) IS NOT EXACTLY REPRESENTABLE IN FLOAT OR DOUBLE FORMAT. 1000.43 IS ACTUALLY REPRESENTED AS THE FOLLOWING BITPATTERN (THE "S" SHOWS THE POSITION OF THE SIGN BIT, THE "E"S SHOW THE POSITIONS OF THE EXPONENT BITS, AND THE "M"S SHOW THE POSITIONS OF THE MANTISSA BITS):

```
Seeeeeeemmmmmmmmmmmmmmmmmmmmmmmmm
```

THE SHIFTED MANTISSA IS 1111101000.01101110000101 OR $1000 + 7045/16384$. THE FRACTIONAL PART IS 0.429992675781. WITH 24 BITS OF MANTISSA YOU ONLY GET ABOUT 1 PART IN 16M OF PRECISION FOR FLOAT. THE DOUBLE TYPE PROVIDES MORE PRECISION (53 BITS OF MANTISSA).

[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)

[29.17] Why doesn't my floating-point comparison work?

BECAUSE FLOATING POINT ARITHMETIC IS DIFFERENT FROM REAL NUMBER ARITHMETIC.

BOTTOM LINE: NEVER USE `==` TO COMPARE TWO FLOATING POINT NUMBERS.

HERE'S A SIMPLE EXAMPLE:

```
double x = 1.0 / 10.0;
double y = x * 10.0;
if (y != 1.0)
    std::cout << "surprise: " << y << " != 1\n";
```

THE ABOVE "SURPRISE" MESSAGE WILL APPEAR ON SOME (BUT NOT ALL) COMPILERS/MACHINES. BUT EVEN IF YOUR PARTICULAR COMPILER/MACHINE DOESN'T CAUSE THE ABOVE "SURPRISE" MESSAGE (AND IF YOU WRITE ME TELLING ME WHETHER IT DOES, YOU'LL SHOW YOU'VE MISSED THE WHOLE POINT OF THIS FAQ), FLOATING POINT WILL SURPRISE YOU AT SOME POINT. SO READ THIS FAQ AND YOU'LL KNOW WHAT TO DO.

THE REASON FLOATING POINT WILL SURPRISE YOU IS THAT FLOAT AND DOUBLE VALUES ARE NORMALLY REPRESENTED USING A FINITE PRECISION BINARY FORMAT. IN OTHER WORDS, FLOATING POINT NUMBERS ARE NOT REAL NUMBERS. FOR EXAMPLE, IN YOUR MACHINE'S FLOATING POINT FORMAT IT MIGHT BE IMPOSSIBLE TO EXACTLY REPRESENT THE NUMBER 0.1. BY WAY OF ANALOGY, IT'S IMPOSSIBLE TO EXACTLY REPRESENT THE NUMBER ONE THIRD IN DECIMAL FORMAT (UNLESS YOU USE AN INFINITE NUMBER OF DIGITS).

TO DIG A LITTLE DEEPER, LET'S EXAMINE WHAT THE DECIMAL NUMBER 0.625 MEANS. THIS NUMBER HAS A 6 IN THE "TENTHS" PLACE, A 2 IN THE "HUNDRETHS" PLACE, AND A 5 IN THE "THOUSANTHS" PLACE. IN OTHER WORDS, WE HAVE A DIGIT FOR EACH POWER OF 10. BUT IN BINARY, WE MIGHT, DEPENDING ON THE DETAILS OF YOUR MACHINE'S FLOATING POINT FORMAT, HAVE A BIT FOR EACH POWER OF 2. SO THE FRACTIONAL PART MIGHT HAVE A "HALVES" PLACE, A "QUARTERS" PLACE, AN "EIGHTHS" PLACE, "SIXTEENTHS" PLACE, ETC., AND EACH OF THESE PLACES HAS A BIT.

LET'S PRETEND YOUR MACHINE REPRESENTS THE FRACTIONAL PART OF FLOATING POINT NUMBERS USING THE ABOVE SCHEME (IT'S NORMALLY MORE COMPLICATED THAN THAT, BUT IF YOU ALREADY KNOW EXACTLY HOW FLOATING POINT NUMBERS ARE STORED, CHANCES ARE YOU DON'T NEED THIS FAQ TO BEGIN WITH, SO LOOK AT THIS AS A GOOD STARTING POINT). ON THAT PRETEND MACHINE, THE BITS OF THE FRACTIONAL PART OF 0.625 WOULD BE 101: 1 IN THE $\frac{1}{2}$ -PLACE, 0 IN THE $\frac{1}{4}$ -PLACE, AND 1 IN THE $\frac{1}{8}$ -PLACE. IN OTHER WORDS, 0.625 IS $\frac{1}{2} + \frac{1}{8}$.

BUT ON THIS PRETEND MACHINE, 0.1 CANNOT BE REPRESENTED EXACTLY SINCE IT CANNOT BE FORMED AS A SUM OF A FINITE NUMBER OF POWERS OF 2. YOU CAN GET CLOSE BUT YOU CAN'T REPRESENT IT EXACTLY. IN PARTICULAR YOU'D HAVE A 0 IN THE $\frac{1}{2}$ -PLACE, A 0 IN THE $\frac{1}{4}$ -PLACE, A 0 IN THE $\frac{1}{8}$ -PLACE, AND FINALLY A 1 IN THE "SIXTEENTHS" PLACE, LEAVING A REMAINDER OF $\frac{1}{10} - \frac{1}{16} = \frac{3}{80}$. FIGURING OUT THE OTHER BITS IS LEFT AS AN EXERCISE (HINT: LOOK FOR A REPEATING BIT-PATTERN, ANALOGOUS TO TRYING TO REPRESENT $\frac{1}{3}$ OR $\frac{1}{7}$ IN DECIMAL FORMAT).

THE MESSAGE IS THAT SOME FLOATING POINT NUMBERS CANNOT ALWAYS BE REPRESENTED EXACTLY, SO COMPARISONS DON'T ALWAYS DO WHAT YOU'D LIKE THEM TO DO. IN OTHER WORDS, IF THE COMPUTER ACTUALLY MULTIPLIES 10.0 BY 1.0/10.0, IT MIGHT NOT EXACTLY GET 1.0 BACK.

THAT'S THE PROBLEM. NOW HERE'S THE SOLUTION: BE VERY CAREFUL WHEN COMPARING FLOATING POINT NUMBERS FOR EQUALITY (OR WHEN DOING OTHER THINGS WITH FLOATING POINT NUMBERS; E.G., FINDING THE AVERAGE OF TWO FLOATING POINT NUMBERS SEEMS SIMPLE BUT TO DO IT RIGHT REQUIRES AN IF/ELSE WITH AT LEAST THREE CASES).

HERE'S THE WRONG WAY TO DO IT:

```
void dubious(double x, double y)
{
    ...
    if (x == y) // Dubious!
```

```
foo();
...
}
```

IF WHAT YOU REALLY WANT IS TO MAKE SURE THEY'RE "VERY CLOSE" TO EACH OTHER (E.G., IF VARIABLE A CONTAINS THE VALUE 1.0 / 10.0 AND YOU WANT TO SEE IF $(10 * A == 1)$), YOU'LL PROBABLY WANT TO DO SOMETHING FANCIER THAN THE ABOVE:

```
void smarter(double x, double y)
{
    ...
    if (isEqual(x, y)) // Smarter!
        foo();
    ...
}
```

THERE ARE MANY WAYS TO DEFINE THE `isEqual()` FUNCTION, INCLUDING:

```
inline bool isEqual(double x, double y)
{
    const double epsilon = /* some small number such as 1e-5 */;
    return std::abs(x - y) <= epsilon * std::abs(x);
    // see Knuth section 4.2.2 pages 217-218
}
```

NOTE: THE ABOVE SOLUTION IS NOT COMPLETELY SYMMETRIC, MEANING IT IS POSSIBLE FOR `isEqual(x,y) != isEqual(y,x)`. FROM A PRACTICAL STANDPOINT, DOES NOT USUALLY OCCUR WHEN THE MAGNITUDES OF X AND Y ARE SIGNIFICANTLY LARGER THAN EPSILON, BUT YOUR MILEAGE MAY VARY.

FOR OTHER USEFUL FUNCTIONS, CHECK OUT THE FOLLOWING (LISTED ALPHABETICALLY):

- Isaacson, E. and Keller, H., Analysis of Numerical Methods, Dover.
- Kahan, W., <http://cs.berkeley.edu/~wkahan/>.
- Knuth, Donald E., The Art of Computer Programming, Volume II: Seminumerical Algorithms, Addison-Wesley, 1969.
- LAPACK — Linear Algebra Subroutine Library, www.siam.org
- Press et al., Numerical Recipes.
- Ralston and Rabinowitz, A First Course in Numerical Analysis: Second Edition, Dover.
- Stoer, J. and Bulirsch, R., Introduction to Numerical Analysis, Springer Verlag, in German.

DOUBLE-CHECK YOUR ASSUMPTIONS, INCLUDING "OBVIOUS" THINGS LIKE HOW TO COMPUTE AVERAGES, HOW TO SOLVE QUADRATIC EQUATIONS, ETC., ETC. DO NOT ASSUME THE FORMULAS YOU LEARNED IN HIGH SCHOOL WILL WORK WITH FLOATING POINT NUMBERS!

FOR INSIGHTS ON THE UNDERLYING IDEAS AND ISSUES OF FLOATING POINT COMPUTATION, START WITH DAVID GOLDBERG'S PAPER, [WHAT EVERY COMPUTER-SCIENTIST SHOULD KNOW ABOUT FLOATING POINT ARITHMETIC](#) OR [HERE IN PDF FORMAT](#). YOU MIGHT ALSO WANT TO READ [THIS SUPPLEMENT BY DOUG PRIEST](#). THE [COMBINED PAPER + SUPPLEMENT](#) IS ALSO AVAILABLE. YOU MIGHT ALSO WANT TO [GO HERE FOR LINKS TO OTHER FLOATING-POINT TOPICS](#).

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.18] What is the type of an enumeration such as `enum Color`? Is it of type `int`? Updated!

[Recently fixed a typo thanks to [Surendra Phadke](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

AN ENUMERATION SUCH AS `ENUM COLOR { RED, WHITE, BLUE }`; IS ITS OWN TYPE. IT IS NOT OF TYPE `INT`.

WHEN YOU CREATE AN OBJECT OF AN ENUMERATION TYPE, E.G., `COLOR X`;, WE SAY THAT THE OBJECT `X` IS OF TYPE `COLOR`. OBJECT `X` ISN'T OF TYPE "ENUMERATION," AND IT'S NOT OF TYPE `INT`.

AN EXPRESSION OF AN ENUMERATION TYPE CAN BE CONVERTED TO A TEMPORARY `INT`. AN ANALOGY MAY HELP HERE. AN EXPRESSION OF TYPE `FLOAT` CAN BE CONVERTED TO A TEMPORARY `DOUBLE`, BUT THAT DOESN'T MEAN `FLOAT` IS A SUBTYPE OF `DOUBLE`. FOR EXAMPLE, AFTER THE DECLARATION `FLOAT Y`;, WE SAY THAT `Y` IS OF TYPE `FLOAT`, AND THE EXPRESSION `Y` CAN BE CONVERTED TO A TEMPORARY `DOUBLE`. WHEN THAT HAPPENS, A BRAND NEW, TEMPORARY `DOUBLE` IS CREATED BY COPYING SOMETHING OUT OF `Y`. IN THE SAME WAY, A `COLOR` OBJECT SUCH AS `X` CAN BE CONVERTED TO A TEMPORARY `INT`, IN WHICH CASE A BRAND NEW, TEMPORARY `INT` IS CREATED BY COPYING SOMETHING OUT OF `X`. (NOTE: THE ONLY PURPOSE OF THE `FLOAT` / `DOUBLE` ANALOGY IN THIS PARAGRAPH IS TO HELP EXPLAIN HOW EXPRESSIONS OF AN ENUMERATION TYPE CAN BE CONVERTED TO TEMPORARY `INT`S; DO NOT TRY TO USE THAT ANALOGY TO IMPLY ANY OTHER BEHAVIOR!)

THE ABOVE CONVERSION IS VERY DIFFERENT FROM A SUBTYPE RELATIONSHIP, SUCH AS THE RELATIONSHIP BETWEEN DERIVED CLASS `CAR` AND ITS BASE CLASS `VEHICLE`. FOR EXAMPLE, AN OBJECT OF CLASS `CAR`, SUCH AS `CAR Z`;, ACTUALLY IS AN OBJECT OF CLASS `VEHICLE`, THEREFORE YOU CAN BIND A `VEHICLE&` TO THAT OBJECT, E.G., `VEHICLE& V = Z`;. UNLIKE THE PREVIOUS PARAGRAPH, THE OBJECT `Z` IS NOT COPIED TO A TEMPORARY; REFERENCE `V` BINDS TO `Z` ITSELF. SO WE SAY AN OBJECT OF CLASS `CAR` IS A `VEHICLE`, BUT AN OBJECT OF CLASS "COLOR" SIMPLY CAN BE COPIED/CONVERTED INTO A TEMPORARY `INT`. BIG DIFFERENCE.

FINAL NOTE, ESPECIALLY FOR C PROGRAMMERS: [THE C++ COMPILER WILL NOT AUTOMATICALLY CONVERT AN INT EXPRESSION TO A TEMPORARY COLOR](#). SINCE THAT SORT OF CONVERSION IS UNSAFE, IT REQUIRES A CAST, E.G., `COLOR X = COLOR(2)`;. BUT BE SURE YOUR INTEGER IS A VALID ENUMERATION VALUE. IF YOU GO PROVIDE AN ILLEGAL VALUE, YOU MIGHT END UP WITH SOMETHING OTHER THAN WHAT YOU EXPECT. THE COMPILER DOESN'T DO THE CHECK FOR YOU; YOU MUST DO IT YOURSELF.

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]

[29.19] If an enumeration type is distinct from any other type, what good is it? What can you do with it?

LET'S CONSIDER THIS ENUMERATION TYPE: `ENUM COLOR { RED, WHITE, BLUE }`;

THE BEST WAY TO LOOK AT THIS (C PROGRAMMERS: HANG ON TO YOUR SEATS!!) IS THAT THE VALUES OF THIS TYPE ARE `RED`, `WHITE`, AND `BLUE`, AS OPPOSED TO MERELY THINKING OF THOSE NAMES AS CONSTANT `INT` VALUES. THE C++ COMPILER PROVIDES AN AUTOMATIC CONVERSION FROM `COLOR` TO `INT`, AND THE CONVERTED VALUES WILL BE, IN THIS CASE, 0, 1, AND 2 RESPECTIVELY. BUT YOU SHOULDN'T THINK OF `BLUE` AS A FANCY NAME FOR 2. `BLUE` IS OF TYPE `COLOR` AND THERE IS AN AUTOMATIC CONVERSION FROM `BLUE` TO 2, BUT THE INVERSE CONVERSION, FROM `INT` TO `COLOR`, IS NOT PROVIDED AUTOMATICALLY BY THE C++ COMPILER.

HERE IS AN EXAMPLE THAT ILLUSTRATES THE CONVERSION FROM *COLOR* TO *INT*:

```
enum Color { red, white, blue };

void f()
{
    int n;
    n = red;    // change n to 0
    n = white;  // change n to 1
    n = blue;   // change n to 2
}
```

THE FOLLOWING EXAMPLE ALSO DEMONSTRATES THE CONVERSION FROM *COLOR* TO *INT*:

```
void f()
{
    Color x = red;
    Color y = white;
    Color z = blue;

    int n;
    n = x;    // change n to 0
    n = y;    // change n to 1
    n = z;    // change n to 2
}
```

HOWEVER THE INVERSE CONVERSION, FROM *INT* TO *COLOR*, IS NOT AUTOMATICALLY PROVIDED BY THE C++ COMPILER:

```
void f()
{
    Color x;
    x = blue; // change x to blue
    x = 2;    // compile-time error: can't convert int to Color
}
```

THE LAST LINE ABOVE SHOWS THAT ENUMERATION TYPES ARE NOT INTS IN DISGUISE. YOU CAN THINK OF THEM AS *INT* TYPES IF YOU WANT TO, BUT IF YOU DO, YOU MUST REMEMBER THAT THE C++ COMPILER WILL NOT IMPLICITLY CONVERT AN *INT* TO A *COLOR*. IF YOU REALLY WANT THAT, YOU CAN USE A CAST:

```
void f()
{
    Color x;
    x = red;    // change x to red
    x = Color(1); // change x to white
    x = Color(2); // change x to blue
    x = 2;      // compile-time error: can't convert int to Color
}
```

THERE ARE OTHER WAYS THAT ENUMERATION TYPES ARE UNLIKE *INT*. FOR EXAMPLE, ENUMERATION TYPES DON'T HAVE A ++ OPERATOR:


```
void f()
{
    int n = red; // change n to 0
    Color x = red; // change x to red

    n++; // change n to 1
    x++; // compile-time error: can't ++ an enumeration
}
```

[[TOP](#) | [BOTTOM](#) | [PREVIOUS SECTION](#) | [NEXT SECTION](#) | [SEARCH THE FAQ](#)]



[E-MAIL THE AUTHOR](#)

[[C++ FAQ LITE](#) | [TABLE OF CONTENTS](#) | [SUBJECT INDEX](#) | [ABOUT THE AUTHOR](#) | © | [DOWNLOAD YOUR OWN COPY](#)]
 REVISED FEB 15, 2005

[30] Learning C++ if you already know Smalltalk Updated!

(Part of *C++ FAQ Lite*, [Copyright © 1991-2004, Marshall Cline](#), cline@parashift.com)

FAQs in section [30]:

- [\[30.1\] What's the difference between C++ and Smalltalk?](#)
- [\[30.2\] What is "static typing," and how is it similar/dissimilar to Smalltalk?](#)
- [\[30.3\] Which is a better fit for C++: "static typing" or "dynamic typing"?](#)
- [\[30.4\] How do you use inheritance in C++, and is that different from Smalltalk?](#)
- [\[30.5\] What are the practical consequences of differences in Smalltalk/C++ inheritance?](#) Updated!

[30.1] What's the difference between C++ and Smalltalk?

Both fully support the OO paradigm. Neither is categorically and universally ["better" than the other](#). But there are differences. The most important differences are:

- [Static typing vs. dynamic typing](#)
- [Whether inheritance must be used only for subtyping](#)
- [Value vs. reference semantics](#)

Note: Many new C++ programmers come from a Smalltalk background. If that's you, this section will tell you the most important things you need know to make yo ur transition. Please don't get the notion that either language is somehow ["inferior" or "bad"](#), or that this section is promoting one language over the other (I am not a language bigot; I serve on both the ANSI C++ and ANSI Smalltalk [standardization committees](#)). Instead, this section is designed to help you understand (and embrace!) the differences.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[30.2] What is "static typing," and how is it similar/dissimilar to Smalltalk?

Static typing says the compiler checks the type safety of every operation *statically* (at compile-time), rather than to generate code which will check things at run-time. For example, with static typing, the signature matching for function arguments is checked at compile time, not at run-time. An improper match is flagged as an error by the compiler, not by the run-time system.

In OO code, the most common "typing mismatch" is invoking a member function against an object which isn't prepared to handle the operation. E.g., if class Fred has member function `f()` but not `g()`, and fred is an instance of class Fred, then `fred.f()` is legal and `fred.g()` is illegal. C++ (statically typed) catches the error at compile time, and Smalltalk (dynamically typed) catches the error at run-time. (Technically speaking, C++ is like Pascal —*pseudo* statically typed— since pointer casts and unions can be used to violate the typing system; which reminds me: use [pointer casts](#) and unions only as often as you use `gotos`).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[30.3] Which is a better fit for C++: "static typing" or "dynamic typing"?

[For context, please read [the previous FAQ](#)].

If you want to use C++ most effectively, use it as a statically typed language.

C++ is flexible enough that you can (via pointer casts, unions, and `#define` macros) make it "look" like Smalltalk. But don't. Which reminds me: [try to](#) avoid `#define`: it is [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#).

There are places where pointer casts and unions are necessary and even wholesome, but they should be used carefully and sparingly. A pointer cast tells the compiler to believe you. An incorrect pointer cast might corrupt your heap, scribble into memory owned by other objects, call nonexistent member functions, and cause general failures. [It's not a pretty sight](#). If you avoid these and related constructs, you can make your C++ code both safer and faster, since anything that can be checked at compile time is something that doesn't have to be done at run-time.

If you're interested in using a pointer cast, use the new style pointer casts. The most common example of these is to change old-style pointer casts such as `(X*)p` into new-style dynamic casts such as `dynamic_cast<X*>(p)`, where `p` is a pointer and `X` is a type. In addition to `dynamic_cast`, there is `static_cast` and `const_cast`, but `dynamic_cast` is the one that simulates most of the advantages of dynamic typing (the other is the `typeid()` construct; for example, `typeid(*p).name()` will return the name of the type of `*p`).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[30.4] How do you use inheritance in C++, and is that different from Smalltalk?

Some people believe that the purpose of inheritance is code reuse. In C++, this is wrong. Stated plainly, "inheritance is not *for* code reuse."

The purpose of inheritance in C++ is to express interface compliance (subtyping), not to get code reuse. In C++, code reuse usually comes via composition rather than via inheritance. In other words, inheritance is mainly a specification technique rather than an implementation technique.

This is a major difference with Smalltalk, where there is only one form of inheritance (C++ provides private inheritance to mean "share the code but don't conform to the interface", and public inheritance to mean "kind-of"). The Smalltalk language proper (as opposed to coding practice) allows you to have the effect of "hiding" an inherited method by providing an override that calls the "does not understand"

method. Furthermore Smalltalk allows a conceptual "is -a" relationship to exist *apart* from the inheritance hierarchy (subtypes don't have to be derived classes; e.g., you can make something that is -a Stack yet doesn't inherit from class Stack).

In contrast, C++ is more restrictive about inheritance: there's no way to make a "conceptual is -a" relationship without using inheritance (the C++ work -around is to separate interface from implementation via [ABCs](#)). The C++ compiler exploits the added semantic information associated with public inheritance to provide static typing.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[30.5] What are the practical consequences of differences in Smalltalk/C++ inheritance?
Updated!

[Recently fixed the spelling of Maserati thanks to [Alessandro/Gruppo Visione](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

[For context, please read [the previous FAQ](#)].

Smalltalk lets you make a subtype that isn't a derived class, and allows you to make a derived class that isn't a subtype. This allows Smalltalk programmers to be very carefree in putting data (bits , representation, data structure) into a class (e.g., you might put a linked list into class Stack). After all, if someone wants an array-based-Stack, they don't have to inherit from Stack; they could inherit such a class from Array if desired, even though an ArrayBasedStack is *not* a kind-of Array!

In C++, you can't be nearly as carefree. Only mechanism (member function code), but not representation (data bits) can be overridden in derived classes. Therefore you're usually better off *not* putting the data structure in a class. This leads to a stronger reliance on [abstract base classes](#).

I like to think of the difference between an ATV and a [Maserati](#). An ATV (all terrain vehicle) is more fun, since you can "play around" by driving through fields, streams, sidewalks, and the like. A Maserati, on the other hand, gets you there faster, but it forces you to stay on the road. My advice to C++ programmers is simple: stay on the road. Even if you're one of those people who like the "expressive freedom" to drive through the bushes, don't do it in C++; it's not a good fit.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



<mailto:cline@parashift.com>

[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#) | [Download your own copy](#)]

Revised Feb 15, 2005

[31] Reference and value semantics

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [31]:

- [\[31.1\] What is value and/or reference semantics, and which is best in C++?](#)
 - [\[31.2\] What is "virtual data," and how-can / why-would I use it in C++?](#)
 - [\[31.3\] What's the difference between virtual data and dynamic data?](#)
 - [\[31.4\] Should I normally use pointers to freestore allocated objects for my data members, or should I use "composition"?](#)
 - [\[31.5\] What are relative costs of the 3 performance hits associated with allocating member objects from the freestore?](#)
 - [\[31.6\] Are "inline virtual" member functions ever actually "inlined"?](#)
 - [\[31.7\] Sounds like I should never use reference semantics, right?](#)
 - [\[31.8\] Does the poor performance of reference semantics mean I should pass -by-value?](#)
-

[31.1] What is value and/or reference semantics, and which is best in C++?

With reference semantics, assignment is a pointer -copy (i.e., a *reference*). Value (or "copy") semantics mean assignment copies the value, not just the pointer. C++ gives you the choice: use the assignment operator to copy the value (copy/value semantics), or use a pointer -copy to copy a pointer (reference semantics). C++ allows you to override the assignment operator to do anything your heart desires, however the default (and most common) choice is to copy the *value*.

Pros of reference semantics: flexibility and dynamic binding (you get dynamic binding in C++ only when you pass by pointer or pass by reference, not when you pass by value).

Pros of value semantics: speed. "Speed" seems like an odd benefit for a feature that requires an object (vs. a pointer) to be copied, but the fact of the matter is that one usually accesses an object more than one copies the object, so the cost of the occasional copies is (usually) more than offset by the benefit of having an actual object rather than a pointer to an object.

There are three cases when you have an actual object as opposed to a pointer to an object: local objects, global/static objects, and fully contained member objects in a class. The most important of these is the last ("composition").

More info about copy-vs-reference semantics is given in the next FAQs. Please read them all to get a balanced perspective. The first few have intentionally been slanted toward value semantics, so if you only read the first few of the following FAQs, you'll get a warped perspective.

Assignment has other issues (e.g., shallow vs. deep copy) which are not covered here.

[31.2] What is "virtual data," and how-can / why-would I use it in C++?

virtual data allows a derived class to change the exact class of a base class's member object. virtual data isn't strictly "supported" by C++, however it can be simulated in C++. It ain't pretty, but it works.

To simulate virtual data in C++, the base class must have a pointer to the member object, and the derived class must provide a new object to be pointed to by the base class's pointer. The base class would also have one or more normal constructors that provide their own referent (again via new), and the base class's destructor would delete the referent.

For example, class Stack might have an Array member object (using a pointer), and derived class StretchableStack might override the base class member data from Array to StretchableArray. For this to work, StretchableArray would have to inherit from Array, so Stack would have an Array*. Stack's normal constructors would initialize this Array* with a new Array, but Stack would also have a (possibly protected) constructor that would accept an Array* from a derived class. StretchableStack's constructor would provide a new StretchableArray to this special constructor.

Pros:

- Easier implementation of StretchableStack (most of the code is inherited)
- Users can pass a StretchableStack as a kind-of Stack

Cons:

- Adds an extra layer of indirection to access the Array
- Adds some extra freestore allocation overhead (both new and delete)
- Adds some extra dynamic binding overhead (reason given in next FAQ)

In other words, we succeeded at making *our* job easier as the implementer of StretchableStack, but all our users [pay for it](#). Unfortunately the extra overhead was imposed on both users of StretchableStack *and* on users of Stack.

Please read the rest of this section. (You will not get a balanced perspective without the others.)

[31.3] What's the difference between virtual data and dynamic data?

The easiest way to see the distinction is by an analogy with [virtual functions](#): A virtual member function means the declaration (signature) must stay the same in derived classes, but the definition (body) can be overridden. The overriddenness of an inherited member function is a static property of the derived class; it doesn't change dynamically throughout the life of any

particular object, nor is it possible for distinct objects of the derived class to have distinct definitions of the member function.

Now go back and re-read the previous paragraph, but make these substitutions :

- "member function" → "member object"
- "signature" → "type"
- "body" → "exact class"

After this, you'll have a working definition of `virtual data`.

Another way to look at this is to distinguish "per-object" member functions from "dynamic" member functions. A "per-object" member function is a member function that is potentially different in any given instance of an object, and could be implemented by burying a function pointer in the object; this pointer could be `const`, since the pointer will never be changed throughout the object's life. A "dynamic" member function is a member function that will change dynamically over time; this could also be implemented by a function pointer, but the function pointer would not be `const`.

Extending the analogy, this gives us three distinct concepts for data members:

- `virtual data`: the definition (`class`) of the member object is overridable in derived classes provided its declaration ("type") remains the same, and this overriddenness is a static property of the derived class
- `per-object-data`: any given object of a class can instantiate a different conformal (same type) member object upon initialization (usually a "wrapper" object), and the exact class of the member object is a static property of the object that wraps it
- `dynamic-data`: the member object's exact class can change dynamically over time

The reason they all look so much the same is that none of this is "supported" in C++. It's all merely "allowed," and in this case, the mechanism for faking each of these is the same : a pointer to a (probably abstract) base class. In a language that made these "first class" abstraction mechanisms, the difference would be more striking, since they'd each have a different syntactic variant.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[31.4] Should I normally use pointers to freestore allocated objects for my data members, or should I use "composition"?

Composition.

Your member objects should normally be "contained" in the composite object (but not always; "wrapper" objects are a good example of where you want a pointer/reference; also the N-to-1-uses-a relationship needs something like a pointer/reference).

There are three reasons why fully contained member objects ("composition") has better performance than pointers to freestore-allocated member objects:

- Extra layer of indirection every time you need to access the member object
- Extra freestore allocations (new in constructor, delete in destructor)
- Extra dynamic binding (reason given below)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[31.5] What are relative costs of the 3 performance hits associated with allocating member objects from the freestore?

The three performance hits are enumerated in the previous FAQ:

- By itself, an extra layer of indirection is small potatoes
- Freestore allocations can be a performance issue (the performance of the typical implementation of `malloc()` degrades when there are many allocations; OO software can easily become "freestore bound" unless you're careful)
- The extra dynamic binding comes from having a pointer rather than an object. Whenever the C++ compiler can know an object's *exact* class, [virtual](#) function calls can be *statically* bound, which allows inlining. Inlining allows zillions (would you believe half a dozen :-) optimization opportunities such as procedural integration, register lifetime issues, etc. The C++ compiler can know an object's exact class in three circumstances: local variables, global/static variables, and fully-contained member objects

Thus fully-contained member objects allow significant optimizations that wouldn't be possible under the "member objects-by-pointer" approach. This is the main reason that languages which enforce reference-semantics have "inherent" performance challenges.

Note: Please read the next three FAQs to get a balanced perspective!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[31.6] Are "inline virtual" member functions ever actually "inlined"?

Occasionally...

When the object is referenced via a pointer or a reference, a call to a [virtual](#) function cannot be inlined, since the call must be resolved dynamically. Reason: the compiler can't know which actual code to call until run-time (i.e., dynamically), since the code may be from a derived class that was created after the caller was compiled.

Therefore the only time an `inline virtual` call can be inlined is when the compiler knows the "exact class" of the object which is the target of the `virtual` function call. This can happen only when the compiler has an actual object rather than a pointer or reference to an object. I.e., either with a local object, a global/static object, or a fully contained object inside a composite.

Note that the difference between inlining and non-inlining is normally *much* more significant than the difference between a regular function call and a virtual function call. For example, the difference between a regular function call and a virtual function call is often just two extra memory references, but the difference between an inline function and a non-inline function can be as much as an order of magnitude (for zillions of calls to insignificant member functions, loss of inlining virtual functions can result in 25X speed degradation! [Doug Lea, "Customization in C++," proc Userix C++ 1990]).

A practical consequence of this insight: don't get bogged down in the endless debates (or sales tactics!) of compiler/language vendors who compare the cost of a virtual function call on their language/compiler with the same on another language/compiler. Such comparisons are largely meaningless when compared with the ability of the language/compiler to "inline expand" member function calls. I.e., many language implementation vendors make a big stink about how good their dispatch strategy is, but if these implementations don't *inline* member function calls, the overall system performance would be poor, since it is inlining —*not* dispatching— that has the greatest performance impact.

Note: Please read the next two FAQs to see the other side of this coin!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[31.7] Sounds like I should never use reference semantics, right?

Wrong.

Reference semantics are A Good Thing. We can't live without pointers. We just don't want our s/w to be One Gigantic Rats Nest Of Pointers. In C++, you can pick and choose where you want reference semantics (pointers/references) and where you'd like value semantics (where objects physically contain other objects etc). In a large system, there should be a balance. However if you implement absolutely *everything* as a pointer, you'll get enormous speed hits.

Objects near the problem skin are larger than higher level objects. The *identity* of these "problem space" abstractions is usually more important than their "value." Thus reference semantics should be used for problem-space objects.

Note that these problem space objects are normally at a higher level of abstraction than the solution space objects, so the problem space objects normally have a relatively lower frequency of interaction. Therefore C++ gives us an *ideal* situation: we choose reference semantics for objects that need unique identity or that are too large to copy, and we can choose value semantics for the others. Thus the highest frequency objects will end up with value semantics, since we install flexibility where it doesn't hurt us (only), and we install performance where we need it most!

These are some of the many issues that come into play with real OO design. OO/C++ mastery takes time and high quality training. If you want a powerful tool, you've got to invest.

Don't stop now! Read the next FAQ too!!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

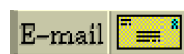
[31.8] Does the poor performance of reference semantics mean I should pass -by-value?

Nope.

The previous FAQ were talking about *member objects*, not parameters. Generally, objects that are part of an inheritance hierarchy should be passed by reference or by pointer, *not* by value, since only then do you get the (desired) dynamic binding (pass -by-value doesn't mix with inheritance, since larger derived class objects get [sliced](#) when passed by value as a base class object).

Unless compelling reasons are given to the contrary, member objects should be by value and parameters should be by reference. The discussion in the previous few FAQs indicates some of the "compelling reasons" for when member objects should be by reference.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[32] How to mix C and C++

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [32]:

- [\[32.1\] What do I need to know when mixing C and C++ code?](#)
- [\[32.2\] How can I include a standard C header file in my C++ code?](#)
- [\[32.3\] How can I include a non-system C header file in my C++ code?](#)
- [\[32.4\] How can I modify my own C header files so it's easier to #include them in C++ code?](#)
- [\[32.5\] How can I call a non-system C function `f\(int, char, float\)` from my C++ code?](#)
- [\[32.6\] How can I create a C++ function `f\(int, char, float\)` that is callable by my C code?](#)
- [\[32.7\] Why is the linker giving errors for C/C++ functions being called from C++/C functions?](#)
- [\[32.8\] How can I pass an object of a C++ class to/from a C function?](#)
- [\[32.9\] Can my C function directly access data in an object of a C++ class?](#)



- [\[32.10\] Why do I feel like I'm "further from the machine" in C++ as opposed to C?](#)

[32.1] What do I need to know when mixing C and C++ code?

There are several caveats:

- You must use your C++ compiler when compiling `main()` (e.g., for static initialization)
- Your C++ compiler should direct the linking process (e.g., so it can get its special libraries)
- Your C and C++ compilers probably need to come from same vendor and have compatible versions (e.g., so they have the same calling conventions)

In addition, you'll need to read the rest of this section to find out how to make your C functions callable by C++ and/or your C++ functions callable by C.

BTW there is another way to handle this whole thing: compile all your code (even your C -style code) using a C++ compiler. That pretty much eliminates the need to mix C and C++, plus it will cause you to be more careful (and possibly —hopefully!— discover some bugs) in your C-style code. The down-side is that you'll need to update your C -style code in certain ways, basically because [the C++ compiler is more careful/picky than your C compiler](#). The point is that the effort required to clean up your C-style code may be less than the effort required to mix C and C++, and as a bonus you get cleaned up C -style code. Obviously you don't have much of a choice if you're not able to alter your C-style code (e.g., if it's from a third-party).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.2] How can I include a standard C header file in my C++ code?

To `#include` a standard header file (such as `<stdio>`), you don't have to do anything unusual. E.g.,

```
// This is C++ code

#include <stdio>                // Nothing unusual in #include line

int main()
{
    std::printf("Hello world\n"); // Nothing unusual in the call either
    ...
}
```

If you think the `std::` part of the `std::printf()` call is unusual, then the best thing to do is "get over it." In other words, it's the standard way to use names in the standard library, so you might as well start getting used to it now.

However if you are compiling C code using your C++ compiler, you don't want to have to tweak all these calls from `printf()` to `std::printf()`. Fortunately in this case the C code will use the

old-style header `<stdio.h>` rather than the new-style header `<cstdio>`, and the magic of namespaces will take care of everything else:

```
/* This is C code that I'm compiling using a C++ compiler */

#include <stdio.h>           /* Nothing unusual in #include line */

int main()
{
    printf("Hello world\n"); /* Nothing unusual in the call either */
    ...
}
```

Final comment: if you have C headers that are *not* part of the standard library, we have somewhat different guidelines for you. There are two cases: either you [can't change the header](#), or you [can change the header](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.3] How can I include a non-system C header file in my C++ code?

If you are including a C header file that isn't provided by the system, you may need to wrap the `#include` line in an `extern "C" { /*...*/ }` construct. This tells the C++ compiler that the functions declared in the header file are C functions.

```
// This is C++ code

extern "C" {
    // Get declaration for f(int i, char c, float x)
    #include "my-C-code.h"
}

int main()
{
    f(7, 'x', 3.14);    // Note: nothing unusual in the call
    ...
}
```

Note: Somewhat different guidelines apply for [C headers provided by the system \(such as `<cstdio>`\)](#) and for [C headers that you can change](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.4] How can I modify my own C header files so it's easier to `#include` them in C++ code?

If you are including a C header file that isn't provided by the system, and if you are able to change the C header, you should strongly consider adding the `extern "C" {...}` logic inside the header to make it easier for C++ users to `#include` it into their C++ code. Since a C compiler

won't understand the `extern "C"` construct, you must wrap the `extern "C" {` and `}` lines in an `#ifdef` so they won't be seen by normal C compilers.

Step #1: Put the following lines at the very top of your C header file (note: the symbol `__cplusplus` is `#defined` if/only-if the compiler is a C++ compiler):

```
#ifdef __cplusplus
extern "C" {
#endif
```

Step #2: Put the following lines at the very bottom of your C header file:

```
#ifdef __cplusplus
}
#endif
```

Now you can `#include` your C header without any `extern "C"` nonsense in your C++ code:

```
// This is C++ code

// Get declaration for f(int i, char c, float x)
#include "my-C-code.h"    // Note: nothing unusual in #include line

int main()
{
    f(7, 'x', 3.14);      // Note: nothing unusual in the call
    ...
}
```

Note: Somewhat different guidelines apply for [C headers provided by the system \(such as `<stdio>`\)](#) and for [C headers that you can't change](#).

Note: `#define` macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#). But they're still [useful sometimes](#). Just wash your hands after using them.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.5] How can I call a non-system C function `f(int, char, float)` from my C++ code?

If you have an individual C function that you want to call, and for some reason you don't have or don't want to `#include` a C header file in which that function is declared, you can declare the individual C function in your C++ code using the `extern "C"` syntax. Naturally you need to use the full function prototype:

```
extern "C" void f(int i, char c, float x);
```

A block of several C functions can be grouped via braces:

```
extern "C" {
    void f(int i, char c, float x);
    int g(char* s, const char* s2);
}
```

```
double sqrtOfSumOfSquares(double a, double b);  
}
```

After this you simply call the function just as if it were a C++ function:

```
int main()  
{  
    f(7, 'x', 3.14);    // Note: nothing unusual in the call  
    ...  
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.6] How can I create a C++ function `f(int, char, float)` that is callable by my C code?

The C++ compiler must know that `f(int, char, float)` is to be called by a C compiler using the [extern "C" construct](#):

```
// This is C++ code  
  
// Declare f(int, char, float) using extern "C":  
extern "C" void f(int i, char c, float x);  
  
...  
  
// Define f(int, char, float) in some C++ module:  
void f(int i, char c, float x)  
{  
    ...  
}
```

The `extern "C"` line tells the compiler that the external information sent to the linker should use C calling conventions and name mangling (e.g., preceded by a single underscore). Since name overloading isn't supported by C, you can't make several overloaded functions simultaneously callable by a C program.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.7] Why is the linker giving errors for C/C++ functions being called from C++/C functions?

If you didn't get your `extern "C"` right, you'll sometimes get linker errors rather than compiler errors. This is due to the fact that C++ compilers usually "mangle" function names (e.g., to support function overloading) differently than C compilers.

See the previous two FAQs on how to use `extern "C"`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.8] How can I pass an object of a C++ class to/from a C function?

Here's an example (for info on `extern "C"`, see the previous two FAQs).

Fred.h:

```
/* This header can be read by both C and C++ compilers */
#ifndef FRED_H
#define FRED_H

#ifdef __cplusplus
class Fred {
public:
    Fred();
    void wilma(int);
private:
    int a_;
};
#else
typedef
    struct Fred
        Fred;
#endif

#ifdef __cplusplus
extern "C" {
#endif

#if defined(__STDC__) || defined(__cplusplus)
    extern void c_function(Fred*);    /* ANSI C prototypes */
    extern Fred* cplusplus_callback_function(Fred*);
#else
    extern void c_function();          /* K&R style */
    extern Fred* cplusplus_callback_function();
#endif

#ifdef __cplusplus
}
#endif

#endif /*FRED_H*/
```

Fred.cpp:

```
// This is C++ code

#include "Fred.h"

Fred::Fred() : a_(0) { }

void Fred::wilma(int a) { }

Fred* cplusplus_callback_function(Fred* fred)
{
    fred->wilma(123);
    return fred;
}
```

main.cpp:

```
// This is C++ code

#include "Fred.h"

int main()
{
    Fred fred;
    c_function(&fred);
    ...
}
```

c-function.c:

```
/* This is C code */

#include "Fred.h"

void c_function(Fred* fred)
{
    cplusplus_callback_function(fred);
}
```

Unlike your C++ code, your C code will not be able to tell that two pointers point at the same object unless the pointers are *exactly* the same type. For example, in C++ it is easy to check if a `Derived*` called `dp` points to the same object as is pointed to by a `Base*` called `bp`: just say `if (dp == bp) ...`. The C++ compiler automatically converts both pointers to the same type, in this case to `Base*`, then compares them. Depending on the C++ compiler's implementation details, this conversion sometimes changes the bits of a pointer's value. However your C compiler will not know how to do that pointer conversion, so the conversion from `Derived*` to `Base*`, for example, must take place in code compiled with a C++ compiler, not in code compiled with a C compiler.

NOTE: you must be especially careful when converting both to `void*` since that conversion will not allow either the C or C++ compiler to do the proper pointer adjustments! For example (continuing from the previous paragraph), if you assigned `dp` and `bp` into two `void*` pointers, say `dpv` and `bpv`, it might be the case that `dpv != bpv` even if `dp == bp`. You have been warned.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.9] Can my C function directly access data in an object of a C++ class?

Sometimes.

(For basic info on passing C++ objects to/from C functions, read the previous FAQ).

You can safely access a C++ object's data from a C function if the C++ class:

- Has no [virtual](#) functions (including inherited virtual functions)
- Has all its data in the same access-level section (private/protected/public)

- Has no fully-contained subobjects with [virtual](#) functions

If the C++ class has any base classes at all (or if any fully contained subobjects have base classes), accessing the data will *technically* be non-portable, since `class` layout under inheritance isn't imposed by the language. However in practice, all C++ compilers do it the same way: the base class object appears first (in left-to-right order in the event of multiple inheritance), and member objects follow.

Furthermore, if the class (or any base class) contains any `virtual` functions, almost all C++ compilers put a `void*` into the object either at the location of the first `virtual` function or at the very beginning of the object. Again, this is not required by the language, but it is the way "everyone" does it.

If the class has any `virtual` base classes, it is even more complicated and less portable. One common implementation technique is for objects to contain an object of the `virtual` base class (`v`) last (regardless of where `v` shows up as a `virtual` base class in the inheritance hierarchy). The rest of the object's parts appear in the normal order. Every derived class that has `v` as a `virtual` base class actually has a *pointer* to the `v` part of the final object.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[32.10] Why do I feel like I'm "further from the machine" in C++ as opposed to C?

Because you are.

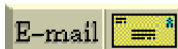
As an OO programming language, C++ allows you to model the problem domain itself, which allows you to program in the language of the problem domain rather than in the language of the solution domain.

One of C's great strengths is the fact that it has "no hidden mechanism": what you see is what you get. You can read a C program and "see" every clock cycle. This is not the case in C++; old line C programmers (such as many of us once were) are often ambivalent (can you say, "hostile"?) about this feature. However after they've made the transition to OO thinking, they often realize that although C++ hides some mechanism from the programmer, it also provides a level of abstraction and economy of expression which lowers maintenance costs without destroying run-time performance.

Naturally you can write bad code in any language; C++ doesn't guarantee any particular level of quality, reusability, abstraction, or any other measure of "goodness."

C++ doesn't try to make it impossible for bad programmers to write bad programs; it enables reasonable developers to create superior software.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[33] Pointers to member functions

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
cline@parashift.com)

FAQs in section [33]:

- [\[33.1\] Is the type of "pointer-to-member-function" different from "pointer-to-function"?](#)
- [\[33.2\] How do I pass a pointer-to-member-function to a signal handler, X event callback, system call that starts a thread/task, etc?](#) **UPDATED!**
- [\[33.3\] Why do I keep getting compile errors \(type mismatch\) when I try to use a member function as an interrupt service routine?](#)
- [\[33.4\] Why am I having trouble taking the address of a C++ function?](#)
- [\[33.5\] How can I avoid syntax errors when calling a member function using a pointer-to-member-function?](#) **UPDATED!**
- [\[33.6\] How do I create and use an array of pointer-to-member-function?](#)
- [\[33.7\] Can I convert a pointer-to-member-function to a void*?](#)
- [\[33.8\] Can I convert a pointer-to-function to a void*?](#)
- [\[33.9\] I need something like function-pointers, but with more flexibility and/or thread-safety; is there another way?](#)
- [\[33.10\] What the heck is a functionoid, and why would I use one?](#) **UPDATED!**
- [\[33.11\] Can you make functionoids faster than normal function calls?](#) **NEW!**

[33.1] Is the type of "pointer-to-member-function" different from "pointer-to-function"?

Yep.

Consider the following function:

```
int f(char a, float b);
```

The type of this function is different depending on whether it is an ordinary function or a non-static member function of some class:

- Its type is "int (*)(char, float)" if an ordinary function
- Its type is "int (Fred::*)(char, float)" if a non-static member function of class Fred



Note: if it's a static member function of class Fred, its type is the same as if it were an ordinary function: "int (*)(char, float)".

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.2] How do I pass a pointer-to-member-function to a signal handler, X event callback, system call that starts a thread/task, etc? UPDATED!

[Recently fixed a grammatical error thanks to [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Don't.

Because a member function is meaningless without an object to invoke it on, you can't do this directly (if The X Window System was rewritten in C++, it would probably pass references to *objects* around, not just pointers to functions; naturally the objects would embody the required function and probably a whole lot more).

As a patch for existing software, use a top-level (non-member) function as a wrapper which takes an object obtained through some other technique. Depending on the routine you're calling, this "other technique" might be trivial or might require a little work on your part. The system call that starts a thread, for example, might require you to pass a function pointer along with a `void*`, so you can pass the object pointer in the `void*`. Many real-time operating systems do something similar for the function that starts a new task. Worst case you could store the object pointer in a global variable; this might be required for Unix signal handlers (but globals are, in general, undesired). In any case, the top-level function would call the desired member function on the object.

Here's an example of the worst case (using a global). Suppose you want to call `Fred::memberFn()` on interrupt:

```
class Fred {
public:
    void memberFn();
    static void staticMemberFn(); // A static member function can usually handle it
    ...
};

// Wrapper function uses a global to remember the object:
Fred* object_which_will_handle_signal;

void Fred_memberFn_wrapper()
{
    object_which_will_handle_signal->memberFn();
}

int main()
{
    /* signal(SIGINT, Fred::memberFn); */ // Can NOT do this
    signal(SIGINT, Fred_memberFn_wrapper); // OK
    signal(SIGINT, Fred::staticMemberFn); // OK usually; see below
}
```

```
...  
}
```

Note: static member functions do not require an actual object to be invoked, so pointers -to-static-member-functions are *usually* type-compatible with regular pointers-to-functions. However, although it probably works on most compilers, it actually would have to be an `extern "C"` non-member function to be correct, since "C linkage" doesn't only cover things like name mangling, but also calling conventions, which might be different between C and C++.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.3] Why do I keep getting compile errors (type mismatch) when I try to use a member function as an interrupt service routine?

This is a special case of the previous two questions, therefore read the previous two answers first.

Non-static member functions have a hidden parameter that corresponds to the `this` pointer. The `this` pointer points to the instance data for the object. The interrupt hardware/firmware in the system is not capable of providing the `this` pointer argument. You must use "normal" functions (non class members) or static member functions as interrupt service routines.

One possible solution is to use a static member as the interrupt service routine and have that function look somewhere to find the instance/member pair that should be called on interrupt. Thus the effect is that a member function is invoked on an interrupt, but for technical reasons you need to call an intermediate function first.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.4] Why am I having trouble taking the address of a C++ function?

Short answer: if you're trying to store it into (or pass it as) a pointer -to-function, then that's the problem — this is a corollary to the previous FAQ.

Long answer: In C++, member functions have an implicit parameter which points to the object (the `this` pointer inside the member function). Normal C functions can be thought of as having a different calling convention from member functions, so the types of their pointers (pointer -to-member-function vs. pointer-to-function) are different and incompatible. C++ introduces a new type of pointer, called a pointer-to-member, which can be invoked only by providing an object.

NOTE: do *not* attempt to "cast" a pointer-to-member-function into a pointer-to-function; the result is undefined and probably disastrous. E.g., a pointer -to-member-function is *not* required to contain the machine address of the appropriate function. As was said in the last example, if you have a pointer to a regular C function, use either a top-level (non-member) function, or a static (class) member function.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.5] How can I avoid syntax errors when calling a member function using a pointer-to-member-function? UPDATED!

[Recently added return value (`int ans = ...`) to make the sample-code more illustrative thanks to [Pete Brunet](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Do *both* a typedef *and* a #define macro.

Step 1: create a typedef:

```
class Fred {
public:
    int f(char x, float y);
    int g(char x, float y);
    int h(char x, float y);
    int i(char x, float y);
    ...
};

// FredMemFn points to a member of Fred that takes (char, float)
typedef int (Fred::*FredMemFn)(char x, float y);
```

Step 2: create a #define macro:

```
#define CALL_MEMBER_FN(object,ptrToMember)  ((object).*(ptrToMember))
```

([Normally I dislike #define macros](#), but [you should use them with pointers to members](#) because they improve the readability and writability of that sort of code.)

Here's how you use these features:

```
void userCode(Fred& fred, FredMemFn memFn)
{
    int ans = CALL_MEMBER_FN(fred,memFn)('x', 3.14);
    // Would normally be: int ans = (fred.*memFn)('x', 3.14);
    ...
}
```

I *strongly* recommend these features. In the real world, member function invocations are a *lot* more complex than the simple example just given, and the difference in readability and writability is significant. [comp.lang.c++.](#) has had to endure hundreds and hundreds of postings from confused programmers who couldn't quite get the syntax right. Almost all these errors would have vanished had they used these features.

Note: #define macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#). But they're [still useful sometimes](#). But you should still feel a vague sense of shame after using them.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.6] How do I create and use an array of pointer -to-member-function?

Use *both* the typedef *and* the #define macro [described earlier](#), and you're 90% done.

Step 1: create a typedef:

```
class Fred {
public:
    int f(char x, float y);
    int g(char x, float y);
    int h(char x, float y);
    int i(char x, float y);
    ...
};

// FredMemFn points to a member of Fred that takes (char, float)
typedef int (Fred::*FredMemFn)(char x, float y);
```

Step 2: create a #define macro:

```
#define CALL_MEMBER_FN(object,ptrToMember)  ((object).*(ptrToMember))
```

Now your array of pointers-to-member-functions is straightforward:

```
FredMemFn a[] = { &Fred::f, &Fred::g, &Fred::h, &Fred::i };
```

And your usage of one of the member function pointers is also straightforward:

```
void userCode(Fred& fred, int memFnNum)
{
    // Assume memFnNum is between 0 and 3 inclusive:
    CALL_MEMBER_FN(fred, a[memFnNum]) ('x', 3.14);
}
```

Note: #define macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#). But they're still useful sometimes. Feel ashamed, feel guilty, but when an evil construct like a macro improves your software, [use it](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.7] Can I convert a pointer-to-member-function to a void*?

No!

```
class Fred {
public:
    int f(char x, float y);
    int g(char x, float y);
    int h(char x, float y);
    int i(char x, float y);
    ...
};
```

```
// FredMemFn points to a member of Fred that takes (char, float)
typedef int (Fred::*FredMemFn)(char x, float y);

#define CALL_MEMBER_FN(object,ptrToMember) ((object).*(ptrToMember))

int callit(Fred& o, FredMemFn p, char x, float y)
{
    return CALL_MEMBER_FN(o,p)(x, y);
}

int main()
{
    FredMemFn p = &Fred::f;
    void* p2 = (void*)p;                // ← illegal!!
    Fred o;
    callit(o, p, 'x', 3.14f);            // okay
    callit(o, FredMemFn(p2), 'x', 3.14f); // might fail!!
    ...
}
```

Please do not email me if the above *seems to work* on your particular version of your particular compiler on your particular operating system. I don't care. It's illegal, period.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.8] Can I convert a pointer-to-function to a void*?

No!

```
int f(char x, float y);
int g(char x, float y);

typedef int(*FuncPtr)(char,float);

int callit(FuncPtr p, char x, float y)
{
    return p(x, y);
}

int main()
{
    FuncPtr p = f;
    void* p2 = (void*)p;                // ← illegal!!
    callit(p, 'x', 3.14f);              // okay
    callit(FuncPtr(p2), 'x', 3.14f);    // might fail!!
    ...
}
```

Please do not email me if the above *seems to work* on your particular version of your particular compiler on your particular operating system. I don't care. It's illegal, period.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.9] I need something like function -pointers, but with more flexibility and/or thread-safety; is there another way?

Use a functionoid.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.10] What the heck is a functionoid, and why would I use one?

UPDATED!

[Recently added a virtual dtor thanks to [Marcus](#); added parameters to `doit()` thanks to [Dirk Bonekaemper](#); made the dtor pure virtual as an optimization (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Functionoids are functions on steroids. Functionoids are strictly more powerful than functions, and that extra power solves some (not all) of the challenges typically faced when you use function-pointers.

Let's work an example showing a traditional use of function -pointers, then we'll translate that example into functionoids. The traditional function -pointer idea is to have a bunch of compatible functions:

```
int funct1(...params...) { ...code... }
int funct2(...params...) { ...code... }
int funct3(...params...) { ...code... }
```

Then you access those by function -pointers:

```
typedef int(*FuncPtr)(...params...);

void myCode(FuncPtr f)
{
    ...
    f(...args-go-here...);
    ...
}
```

Sometimes people create an array of these function -pointers:

```
FuncPtr array[10];
array[0] = funct1;
array[1] = funct1;
array[2] = funct3;
array[3] = funct2;
...
```

In which case they call the function by accessing the array:

```
array[i](...args-go-here...);
```

With functionoids, you first create a base class with a pure -virtual method:

```
class Funct {
public:
```

```

    virtual int doit(int x) = 0;
    virtual ~Func() = 0;
};

inline Func::~~Func() { } // defined even though it's pure virtual; it's faster this way; trust me

```

Then instead of three functions, you create three derived classes:

```

class Func1 : public Func {
public:
    virtual int doit(int x) { ...code from func1... }
};

class Func2 : public Func {
public:
    virtual int doit(int x) { ...code from func2... }
};

class Func3 : public Func {
public:
    virtual int doit(int x) { ...code from func3... }
};

```

Then instead of passing a function -pointer, you pass a `Func*`. I'll create a typedef called `FuncPtr` merely to make the rest of the code similar to the old -fashioned approach:

```

typedef Func* FuncPtr;

void myCode(FuncPtr f)
{
    ...
    f->doit(...args-go-here...);
    ...
}

```

You can create an array of them in almost the same way:

```

FuncPtr array[10];
array[0] = new Func1(...ctor-args...);
array[1] = new Func1(...ctor-args...);
array[2] = new Func3(...ctor-args...);
array[3] = new Func2(...ctor-args...);
...

```

This gives us the first hint about where functionoids are strictly more powerful than function - pointers: the fact that the functionoid approach has arguments you can pass to the ctors (shown above as `...ctor-args...`) whereas the function-pointers version does not. Think of a functionoid object as a freeze-dried function-call (emphasis on the word *call*). Unlike a pointer to a function, a functionoid is (conceptually) a pointer to a *partially called* function. Imagine for the moment a technology that lets you pass some-but-not-all arguments to a function, then lets you freeze -dry that (partially completed) call. Pretend that technology gives you back some sort of magic pointer to that freeze-dried partially-completed function-call. Then later you pass the *remaining* args using that pointer, and the system magically takes your original args (that were freeze -dried), combines them with any local variables that the function calculated prior to being freeze -dried, combines all that with the newly passed args, and continues the function's execution where it left off when it was freeze -dried. That might sound like science fiction, but it's

conceptually what functionoids let you do. *Plus* they let you repeatedly "complete" that freeze-dried function-call with various different "remaining parameters," as often as you like. *Plus* they allow (not require) you to change the freeze-dried state when it gets called, meaning functionoids can remember information from one call to the next.

Okay, let's get our feet back on the ground and we'll work a couple of examples to explain what all that mumbo jumbo really means.

Suppose the original functions (in the old -fashioned function-pointer style) took slightly different parameters.

```
int funct1(int x, float y)
{ ...code... }

int funct2(int x, const std::string& y, int z)
{ ...code... }

int funct3(int x, const std::vector<double>& y)
{ ...code... }
```

When the parameters are different, the old -fashioned function-pointers approach is difficult to use, since the caller doesn't know which parameters to pass (the caller merely has a pointer to the function, not the function's *name* or, when the parameters are different, the number and types of its parameters) (do *not* write me an email about this; yes you can do it, but you have to stand on your head and do messy things; but do *not* write me about it — use functionoids instead).

With functionoids, the situation is, at least sometimes, much better. Since a functionoid can be thought of as a freeze-dried function *call*, just take the un-common args, such as the ones I've called *y* and/or *z*, and make them args to the corresponding ctors. You may also pass the common args (in this case the *int* called *x*) to the ctor, but you don't have to — you have the option of passing it/them to the pure virtual *doit()* method instead. I'll assume you want to pass *x* and into *doit()* and *y* and/or *z* into the ctors:

```
class Funct {
public:
    virtual int doit(int x) = 0;
};
```

Then instead of three functions, you create three derived classes:

```
class Funct1 : public Funct {
public:
    Funct1(float y) : y_(y) { }
    virtual int doit(int x) { ...code from funct1... }
private:
    float y_;
};

class Funct2 : public Funct {
public:
    Funct2(const std::string& y, int z) : y_(y), z_(z) { }
    virtual int doit(int x) { ...code from funct2... }
private:
    std::string y_;
};
```

```

    int z_;
};

class Funct3 : public Funct {
public:
    Funct3(const std::vector<double>& y) : y_(y) { }
    virtual int doit(int x) { ...code from funct3... }
private:
    std::vector<double> y_;
};

```

Now you see that the ctor's parameters get freeze -dried into the functionoid when you create the array of functionoids:

```

FunctPtr array[10];

array[0] = new Funct1(3.14f);

array[1] = new Funct1(2.18f);

std::vector<double> bottlesOfBeerOnTheWall;
bottlesOfBeerOnTheWall.push_back(100);
bottlesOfBeerOnTheWall.push_back(99);
...
bottlesOfBeerOnTheWall.push_back(1);
array[2] = new Funct3(bottlesOfBeerOnTheWall);

array[3] = new Funct2("my string", 42);

...

```

So when the user invokes the `doit()` on one of these functionoids, he supplies the "remaining" args, and the call conceptually combines the original args passed to the ctor with those passed into the `doit()` method:

```
array[i]->doit(12);
```

As I've already hinted, one of the benefits of functionoids is that you can have several instances of, say, `Funct1` in your array, and those instances can have different parameters freeze -dried into them. For example, `array[0]` and `array[1]` are both of type `Funct1`, but the behavior of `array[0]->doit(12)` will be different from the behavior of `array[1]->doit(12)` since the behavior will depend on both the 12 that was passed to `doit()` *and* the args passed to the ctors.

Another benefit of functionoids is apparent if we change the example from an array of functionoids to a local functionoid. To set the stage, let's go back to the old -fashioned function-pointer approach, and imagine that you're trying to pass a comparison -function to a `sort()` or `binarySearch()` routine. The `sort()` or `binarySearch()` routine is called `childRoutine()` and the comparison function-pointer type is called `FunctPtr`:

```

void childRoutine(FunctPtr f)
{
    ...
    f(...args...);
    ...
}

```

Then different callers would pass different function -pointers depending on what they thought was best:

```
void myCaller()
{
    ...
    childRoutine(funcnt1);
    ...
}

void yourCaller()
{
    ...
    childRoutine(funcnt3);
    ...
}
```

We can easily translate this example into one using functionoids:

```
void childRoutine(Funcnt& f)
{
    ...
    f.doit(...args...);
    ...
}

void myCaller()
{
    ...
    Funcnt1 funcnt(...ctor-args...);
    childRoutine(funcnt);
    ...
}

void yourCaller()
{
    ...
    Funcnt3 funcnt(...ctor-args...);
    childRoutine(funcnt);
    ...
}
```

Given this example as a backdrop, we can see two benefits of functionoids over function -pointers. The "ctor args" benefit described above, plus the fact that functionoids can maintain state between calls *in a thread-safe manner*. With plain function-pointers, people normally maintain state between calls via static data. However static data is not intrinsically thread -safe — static data is shared between all threads. The functionoid approach provides you with something that is *intrinsically* thread-safe since the code ends up with thread -local data. The implementation is trivial: change the old -fashioned static datum to an instance data member inside the functionoid's `this` object, and poof, the data is not only thread -local, but it is even safe with recursive calls: each call to `yourCaller()` will have its own distinct `Funcnt3` object with its own distinct instance data.

Note that we've gained something without losing anything. If you *want* thread-global data, functionoids can give you that too: just change it from an instance data member inside the functionoid's `this` object to a static data member within the functionoid's class, or even to a

local-scope static data. You'd be no better off than with function -pointers, but you wouldn't be worse off either.

The functionoid approach gives you a third option which is not available with the old -fashioned approach: the functionoid lets callers decide whether *they* want thread-local or thread-global data. They'd be responsible to use locks in cases where they wanted thread -global data, but at least they'd have the choice. It's easy:

```
void callerWithThreadLocalData()
{
    ...
    Funct1 funct(...ctor-args...);
    childRoutine(funct);
    ...
}

void callerWithThreadGlobalData()
{
    ...
    static Funct1 funct(...ctor-args...); ← the static is the only difference
    childRoutine(funct);
    ...
}
```

Functionoids don't solve every problem encountered when making flexible software, but they are strictly more powerful than function-pointers and they are worth at least evaluating. In fact you can easily prove that functionoids don't lose any power over function -pointers, since you can imagine that the old-fashioned approach of function-pointers is equivalent to having a global(!) functionoid object. Since you can always make a global functionoid object, you haven't lost any ground. QED.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[33.11] Can you make functionoids faster than normal function calls? NEW!

[Recently created thanks to a suggestion from [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Yes.

If you have a small functionoid, and in the real world that's rather common, the cost of the function-call can be high compared to the cost of the work done by the functionoid. In [the previous FAQ](#), functionoids were implemented using [virtual functions](#) and will typically cost you a function-call. An alternate approach uses [templates](#).

The following example is similar in spirit to the one in [the previous FAQ](#). I have renamed `doit()` to `operator()()` to improve the caller code's readability and to allow someone to pass a regular function-pointer:

```
class Funct1 {
public:
    Funct1(float y) : y_(y) { }
```

```

    int operator()(int x) { ...code from funct1... }
private:
    float y_;
};

class Funct2 {
public:
    Funct2(const std::string& y, int z) : y_(y), z_(z) { }
    int operator()(int x) { ...code from funct2... }
private:
    std::string y_;
    int z_;
};

class Funct3 {
public:
    Funct3(const std::vector<double>& y) : y_(y) { }
    int operator()(int x) { ...code from funct3... }
private:
    std::vector<double> y_;
};

```

The difference between this approach and the one in [the previous FAQ](#) is that the functionoid gets "bound" to the caller at compile-time rather than at run-time. Think of it as passing in a parameter: if you know at compile-time the kind of functionoid you ultimately want to pass in, then you can use the above technique, and you can, [at least in typical cases](#), get a speed benefit from having the compiler [inline-expand](#) the functionoid code within the caller. Here is an example:

```

template <typename FunctObj>
void myCode(FunctObj f)
{
    ...
    f(...args-go-here...);
    ...
}

```

When the compiler compiles the above, it [might](#) inline-expand the call which [might](#) improve performance.

Here is one way to call the above:

```

void blah()
{
    ...
    Funct2 x("functionoids are powerful", 42);
    myCode(x);
    ...
}

```

Aside: as was hinted at in the first paragraph above, you may also pass in the names of normal functions (though you might incur the cost of the function call when the caller uses these):

```

void myNormalFunction(int x);

void blah()
{

```

```
...  
myCode(myNormalFunction);  
...  
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

E-mail  [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[34] Container classes

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),
cline@parashift.com)

FAQs in section [34]:

- [\[34.1\] Why should I use container classes rather than simple arrays?](#)
- [\[34.2\] How can I make a perl-like associative array in C++?](#)
- [\[34.3\] Is the storage for a `std::vector<T>` guaranteed to be contiguous?](#)
- [\[34.4\] How can I build a <favorite container> of objects of different types?](#)
- [\[34.5\] How can I insert/access/change elements from a linked list/hashtable/etc?](#)

[34.1] Why should I use container classes rather than simple arrays?

Because arrays are [evil](#).

Let's assume the best case scenario: you're an experienced C programmer, which almost by definition means you're pretty good at working with arrays. You know you can handle the complexity; you've done it for years. And you're smart — the smartest on the team — the smartest in the whole company. But even given all that, *please* read this entire FAQ and think very carefully about it before you go into "business as usual" mode.

Fundamentally it boils down to this simple fact: C++ is not C. That means (this might be painful for you!!) you'll need to set aside some of your hard earned wisdom from your vast experience in C. The two languages simply are different. The "best" way to do something in C is not always the same as the "best" way to do it in C++. If you really want to program in C, please do yourself a favor and program in C. But if you want to be really good at C++, then learn the C++ ways of doing things. You may be a C guru, but if you're just learning C++, you're just learning C++ — you're a newbie. (Ouch; I know that had to hurt. Sorry.)

Here's what you need to realize about containers vs. arrays:

1. Container classes make programmers more productive. So if you insist on using arrays while those around are willing to use container classes, you'll probably be less productive than they are (even if you're smarter and more experienced than they are!).
2. Container classes let programmers write more robust code. So if you insist on using arrays while those around are willing to use container classes, your code will probably have more bugs than their code (even if you're smarter and more experienced).
3. And if you're so smart and so experienced that you can use arrays as fast and as safe as they can use container classes, someone else will probably end up maintaining your code and *they'll* probably introduce bugs. Or worse, you'll be the only one who can maintain your code so management will yank you from development and move you into a full-time maintenance role — just what you always wanted!

Here are some specific problems with arrays:

1. Subscripts don't get checked to see if they are out of bounds. (Note that some container classes, such as `std::vector`, have methods to access elements with or without bounds checking on subscripts.)
2. Arrays often require you to allocate memory from the heap (see below for examples), in which case you must manually make sure the allocation is eventually deleted (even when someone throws an exception). When you use container classes, this memory management is handled automatically, but when you use arrays, you have to manually write a bunch of code (and [unfortunately that code is often subtle and tricky](#)) to deal with this. For example, in addition to writing the code that destroys all the objects and deletes the memory, arrays often also force you to write an extra `try` block with a `catch` clause that destroys all the objects, deletes the memory, then re-throws the exception. This is a real pain in the neck, [as shown here](#). When using container classes, [things are much easier](#).
3. You can't insert an element into the middle of the array, or even add one at the end, unless you allocate the array via the heap, and even then you must allocate a new array and copy the elements.
4. Container classes give you the choice of passing them by reference or by value, but arrays do not give you that choice: they are always passed by reference. If you want to simulate pass-by-value with an array, you have to manually write code that explicitly copies the array's elements (possibly allocating from the heap), along with code to clean up the copy when you're done with it. All this is handled automatically for you if you use a container class.
5. If your function has a non-static local array (i.e., an "auto" array), you cannot return that array, whereas the same is not true for objects of container classes.

Here are some things to think about when using containers:

1. Different C++ containers have different strengths and weaknesses, but for any given job there's usually one of them that is better — clearer, safer, easier/cheaper to maintain, and often more efficient — than an array. For instance,
 - o You might consider a `std::map` instead of manually writing code for a lookup table.
 - o A `std::map` might also be used for a sparse array or sparse matrix.
 - o A `std::vector` is the most array-like of the standard container classes, but it also offers various extra features such as bounds checking via the `at()` member function, insertions/removals of elements, automatic memory management even if

someone throws an exception, ability to be passed both by reference and by value, etc.

- o A `std::string` is [almost always better than an array of `char`](#) (you can think of a `std::string` as a "container class" for the sake of this discussion).
2. Container classes aren't best for *everything*, and sometimes you may need to use arrays. But that should be very rare, and if/when it happens:
- o Please design your container class's public interface in such a way that the code that uses the container class is unaware of the fact that there is an array inside.
 - o The goal is to "bury" the array inside a container class. In other words, make sure there is a very small number of lines of code that directly touch the array (just your own methods of your container class) so everyone else (the users of your container class) can write code that doesn't depend on there being an array inside your container class.

To net this out, arrays really are [evil](#). You may not think so if you're new to C++. But after you write a big pile of code that uses arrays (especially if you make your code leak-proof and exception-safe), you'll learn — the hard way. Or you'll learn the easy way by believing those who've already done things like that. The choice is yours.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[34.2] How can I make a perl-like associative array in C++?

Use the standard class template `std::map<Key, Val>`:

```
#include <string>
#include <map>
#include <iostream>

int main()
{
    // age is a map from string to int
    std::map<std::string, int, std::less<std::string> > age;

    age["Fred"] = 42;                // Fred is 42 years old
    age["Barney"] = 37;              // Barney is 37

    if (todayIsFredsBirthday())      // On Fred's birthday,
        ++ age["Fred"];              // increment Fred's age

    std::cout << "Fred is " << age["Fred"] << " years old\n";
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[34.3] Is the storage for a `std::vector<T>` guaranteed to be contiguous?

Yes.

This means you the following technique is safe:

```
#include <vector>
#include "Foo.h" /* get class Foo */

// old-style code that wants an array
void f(Foo* array, unsigned numFoos);

void g()
{
    std::vector<Foo> v;
    ...
    f(&v[0], v.size()); ← safe
}
```

In general, it means you are guaranteed that $\&v[0] + n == \&v[n]$, where v is a `std::vector<T>` and n is an integer in the range $0 \dots v.size() - 1$.

However `v.begin()` is *not* guaranteed to be a τ^* , which means `v.begin()` is not guaranteed to be the same as `&v[0]`:

```
void g()
{
    std::vector<Foo> v;
    ...
    f(v.begin(), v.size()); ← Error!! Not Guaranteed!!
    ^^^^^^^^^^-- cough, choke, gag; not guaranteed to be the same as &v[0]
}
```

Do **NOT** email me and tell me that `v.begin() == &v[0]` on your particular version of your particular compiler on your particular platform. I don't care, plus that would show that you've *totally* missed the point. The point is to help you know the kind of code that is guaranteed to work correctly on *all* standard-conforming implementations, not to study the vagaries of particular implementations.

Caveat: the above guarantee is currently in the technical corrigendum of the standard and has not, as of this date, officially become a part of the standard. However it will be ratified *Real Soon Now*. In the mean time, the practically important thing is that existing implementations make the storage contiguous, so it is safe to assume that $\&v[0] + n == \&v[n]$.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[34.4] How can I build a <favorite container> of objects of different types?

You can't, but you can fake it pretty well. In C/C++ all arrays are homogeneous (i.e., the elements are all the same type). However, with an extra layer of indirection you can give the appearance of a heterogeneous container (a heterogeneous container is a container where the contained objects are of different types).

There are two cases with heterogeneous containers.

The first case occurs when all objects you want to store in a container are publicly derived from a common base class. You can then declare/define your container to hold pointers to the base class. You indirectly store a derived class object in a container by storing the object's address as an element in the container. You can then access objects in the container indirectly through the pointers (enjoying polymorphic behavior). If you need to know the exact type of the object in the container you can use `dynamic_cast<>` or `typeid()`. You'll probably need the [Virtual Constructor Idiom](#) to copy a container of disparate object types. The downside of this approach is that it makes memory management a little more problematic (who "owns" the pointed-to objects? if you delete these pointed-to objects when you destroy the container, how can you guarantee that no one else has a copy of one of these pointers? if you don't delete these pointed-to objects when you destroy the container, how can you be sure that someone else will eventually do the deleting?). It also makes copying the container more complex (may actually break the container's copying functions since you don't want to copy the pointers, at least not when the container "owns" the pointed-to objects).

The second case occurs when the object types are disjoint — they do not share a common base class. The approach here is to use a handle class. The container is a container of handle objects (by value or by pointer, your choice; by value is easier). Each handle object knows how to "hold on to" (i.e., maintain a pointer to) one of the objects you want to put in the container. You can use either a single handle class with several different types of pointers as instance data, or a hierarchy of handle classes that shadow the various types you wish to contain (requires the container be of handle base class pointers). The downside of this approach is that it opens up the handle class(es) to maintenance every time you change the set of types that can be contained. The benefit is that you can use the handle class(es) to encapsulate most of the ugliness of memory management and object lifetime. Thus using handle objects may be beneficial even in the first case.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[34.5] How can I insert/access/change elements from a linked list/hashtable/etc?

The most important thing to remember is this: don't roll your own from scratch unless there is a compelling reason to do so. In other words, instead of creating your own list or hashtable, use one of the standard class templates such as `std::vector<T>` or `std::list<T>` or whatever.

Assuming you have a compelling reason to build your own container, here's how to handle inserting (or accessing, changing, etc.) the elements.

To make the discussion concrete, I'll discuss how to insert an element into a linked list. This example is just complex enough that it generalizes pretty well to things like vectors, hash tables, binary trees, etc.

A linked list makes it easy insert an element before the first or after the last element of the list, but limiting ourselves to these would produce a library that is too weak (a weak library is almost worse than no library). This answer will be a lot to swallow for novice C++ers, so I'll give a couple of options. The first option is easiest; the second and third are better.

1. Empower the `List` with a "current location," and member functions such as `advance()`, `backup()`, `atEnd()`, `atBegin()`, `getCurrElem()`, `setCurrElem(Elem)`, `insertElem(Elem)`, and `removeElem()`. Although this works in small examples, the notion of a current position makes it difficult to access elements at two or more positions within the list (e.g., "for all pairs `x,y` do the following...").
2. Remove the above member functions from `List` itself, and move them to a separate class, `ListPosition`. `ListPosition` would act as a "current position" within a list. This allows multiple positions within the same list. `ListPosition` would be a [friend](#) of class `List`, so `List` can hide its innards from the outside world (else the innards of `List` would have to be publicized via public member functions in `List`). Note: `ListPosition` can use operator overloading for things like `advance()` and `backup()`, since operator overloading is syntactic sugar for normal member functions.
3. Consider the entire iteration as an atomic event, and create a class template that embodies this event. This enhances performance by allowing the public access member functions (which may be [virtual](#) functions) to be avoided during the access, and this access often occurs within an inner loop. Unfortunately the class template will increase the size of your object code, since templates gain speed by duplicating code. For more, see [Koenig, "Templates as interfaces," JOOP, 4, 5 (Sept 91)], and [Stroustrup, "The C++ Programming Language Third Edition," under "Comparator"].

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
[Download your own copy](#)]

Revised Feb 15, 2005

[35] Templates

UPDATED!

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#),
[cline@parashift.com](#))

FAQs in section [35]:

- [\[35.1\] What's the idea behind templates?](#)
- [\[35.2\] What's the syntax / semantics for a "class template"?](#)
- [\[35.3\] What's the syntax / semantics for a "function template"?](#)
- [\[35.4\] How do I explicitly select which version of a function template should get called?](#)
- [\[35.5\] What is a "parameterized type"?](#)
- [\[35.6\] What is "genericity"?](#)
- [\[35.7\] Why can't I separate the definition of my templates class from it's declaration and put it inside a .cpp file?](#)
- [\[35.8\] How can I avoid linker errors with my template functions?](#)
- [\[35.9\] How can I avoid linker errors with my template classes?](#)



- [\[35.10\] Why do I get linker errors when I use template friends?](#)
- [\[35.11\] How can any human hope to understand these overly verbose template -based error messages?](#)
- [\[35.12\] Why am I getting errors when my template -derived-class accesses something it inherited from its template-base-class?](#) **UPDATED!**
- [\[35.13\] Can the previous problem hurt me silently? Is it possible that the compiler will silently generate the wrong code?](#)

[35.1] What's the idea behind templates?

A template is a cookie-cutter that specifies how to cut cookies that all look pretty much the same (although the cookies can be made of various kinds of dough, they'll all have the same basic shape). In the same way, a class template is a cookie cutter for a description of how to build a family of classes that all look basically the same, and a function template describes how to build a family of similar looking functions.

Class templates are often used to build type safe containers (although this only scratches the surface for how they can be used).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.2] What's the syntax / semantics for a "class template"?

Consider a container class `Array` that acts like an array of integers:

```
// This would go into a header file such as "Array.h"
class Array {
public:
    Array(int len=10)                : len_(len), data_(new int[len]) { }
    ~Array()                        { delete[] data_; }
    int len() const                  { return len_; }
    const int& operator[](int i) const { return data_[check(i)]; }
    int& operator[](int i)           { return data_[check(i)]; }
    Array(const Array&);
    Array& operator= (const Array&);
private:
    int len_;
    int* data_;
    int check(int i) const
    { if (i < 0 || i >= len_) throw BoundsViol("Array", i, len_);
      return i; }
};
```

Repeating the above over and over for `Array` of `float`, of `char`, of `std::string`, of `Array-of-std::string`, etc, will become tedious.

```
// This would go into a header file such as "Array.h"
template<typename T>
class Array {
public:
```

```

    Array(int len=10)                : len_(len), data_(new T[len]) { }
    ~Array()                        { delete[] data_; }
    int len() const                  { return len_; }
    const T& operator[](int i) const { return data_[check(i)]; }
    T& operator[](int i)              { return data_[check(i)]; }
    Array(const Array<T>&);
    Array<T>& operator= (const Array<T>&);
private:
    int len_;
    T* data_;
    int check(int i) const
    { if (i < 0 || i >= len_) throw BoundsViol("Array", i, len_);
      return i; }
};

```

Unlike [template functions](#), template classes (instantiations of class templates) need to be explicit about the parameters over which they are instantiating:

```

int main()
{
    Array<int>          ai;
    Array<float>        af;
    Array<char*>        ac;
    Array<std::string>  as;
    Array< Array<int> > aai;
    ...
}

```

Note the space between the two >'s in the last example. Without this space, the compiler would see a >> (right-shift) token instead of two >'s.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.3] What's the syntax / semantics for a "function template"?

Consider this function that swaps its two integer arguments:

```

void swap(int& x, int& y)
{
    int tmp = x;
    x = y;
    y = tmp;
}

```

If we also had to swap floats, longs, Strings, Sets, and FileSystems, we'd get pretty tired of coding lines that look almost identical except for the type. Mindless repetition is an ideal job for a computer, hence a function template:

```

template<typename T>
void swap(T& x, T& y)
{
    T tmp = x;
    x = y;
    y = tmp;
}

```

Every time we used `swap()` with a given pair of types, the compiler will go to the above definition and will create yet another "template function" as an instantiation of the above. E.g.,

```
int main()
{
    int      i, j; /*...*/ swap(i, j); // Instantiates a swap for int
    float    a, b; /*...*/ swap(a, b); // Instantiates a swap for float
    char     c, d; /*...*/ swap(c, d); // Instantiates a swap for char
    std::string s, t; /*...*/ swap(s, t); // Instantiates a swap for std::string
    ...
}
```

Note: A "template function" is the instantiation of a "function template".

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.4] How do I explicitly select which version of a function template should get called?

When you call a function template, the compiler tries to *deduce* the template type. Most of the time it can do that successfully, but every once in a while you may want to help the compiler deduce the right type — either because it cannot deduce the type at all, or perhaps because it would deduce the wrong type.

For example, you might be calling a function template that doesn't have any parameters of its template argument types, or you might want to force the compiler to do certain promotions on the arguments before selecting the correct function template. In these cases you'll need to explicitly tell the compiler which instantiation of the function template should be called.

Here is a sample function template where the template parameter `T` does not appear in the function's parameter list. In this case the compiler *cannot* deduce the template parameter types when the function is called.

```
template<typename T>
void f()
{
    ...
}
```

To call this function with `T` being an `int` or a `std::string`, you could say:

```
#include <string>

void sample()
{
    f<int>();           // type T will be int in this call
    f<std::string>();   // type T will be std::string in this call
}
```

Here is another function whose template parameters appear in the function's list of formal parameters (that is, the compiler *can* deduce the template type from the actual arguments):

```
template<typename T>
void g(T x)
{
    ...
}
```

Now if you want to force the actual arguments to be promoted before the compiler deduces the template type, you can use the above technique. E.g., if you simply called `g(42)` you would get `g<int>(42)`, but if you wanted to pass `42` to `g<long>()`, you could say this: `g<long>(42)`. (Of course you could also promote the parameter explicitly, such as either `g(long(42))` or even `g(42L)`, but that ruins the example.)

Similarly if you said `g("xyz")` you'd end up calling `g<char*>(char*)`, but if you wanted to call the `std::string` version of `g<>()` you could say `g<std::string>("xyz")`. (Again you could also promote the argument, such as `g(std::string("xyz"))`, but that's another story.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.5] What is a "parameterized type"?

Another way to say, "class templates."

A parameterized type is a type that is parameterized over another type or some value. `List<int>` is a type (`List`) parameterized over another type (`int`).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.6] What is "genericity"?

Yet another way to say, "class templates."

Not to be confused with "generality" (which just means avoiding solutions which are overly specific), "genericity" means class templates.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.7] Why can't I separate the definition of my templates class from it's declaration and put it inside a .cpp file?

If all you want to know is *how* to fix this situation, read [the next FAQ](#). But in order to understand *why* things are the way they are, first accept these facts:

1. A template is not a class or a function. A template is a "p attern" that the compiler uses to generate a family of [classes](#) or [functions](#).

2. In order for the compiler to generate the code, it must see both the template definition (not just declaration) and the specific types/whatever used to "fill in" the template. For example, if you're trying to use a `Foo<int>`, the compiler must see both the `Foo` template and the fact that you're trying to make a specific `Foo<int>`.
3. Your compiler probably doesn't remember the details of one `.cpp` file while it is compiling another `.cpp` file. It *could*, but most do not and if you are reading this FAQ, it almost definitely does not. BTW this is called the "separate compilation model."

Now based on those facts, here's an example that shows why things are the way they are. Suppose you have a template `Foo` defined like this:

```
template<typename T>
class Foo {
public:
    Foo();
    void someMethod(T x);
private:
    T x;
};
```

Along with similar definitions for the member functions:

```
template<typename T>
Foo<T>::Foo()
{
    ...
}

template<typename T>
void Foo<T>::someMethod(T x)
{
    ...
}
```

Now suppose you have some code in file `Bar.cpp` that uses `Foo<int>`:

```
// Bar.cpp

void blah_blah_blah()
{
    ...
    Foo<int> f;
    f.someMethod(5);
    ...
}
```

Clearly somebody somewhere is going to have to use the "pattern" for the constructor definition and for the `someMethod()` definition and instantiate those when `T` is actually `int`. But if you had put the definition of the constructor and `someMethod()` into file `Foo.cpp`, the compiler would see the template code when it compiled `Foo.cpp` and it would see `Foo<int>` when it compiled `Bar.cpp`, but there would never be a time when it saw both the template code and `Foo<int>`. So by rule #2 above, it could never generate the code for `Foo<int>::someMethod()`.

A note to the experts: I have obviously made several simplifications above. This was intentional so please don't complain too loudly. If you know the difference between a `.cpp` file and a

compilation unit, the difference between a class template and a template class, and the fact that templates really aren't just glorified macros, then don't complain: this particular question/answer wasn't aimed at you to begin with. I simplified things so newbies would "get it," even if doing so offends some experts.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.8] How can I avoid linker errors with my template functions?

Tell your C++ compiler which instantiations to make while it is compiling your template function's .cpp file.

As an example, consider the header file `foo.h` which contains the following template function declaration:

```
// file "foo.h"
template<typename T>
extern void foo();
```

Now suppose file `foo.cpp` actually defines that template function:

```
// file "foo.cpp"
#include <iostream>
#include "foo.h"

template<typename T>
void foo()
{
    std::cout << "Here I am!\n";
}
```

Suppose file `main.cpp` uses this template function by calling `foo<int>()`:

```
// file "main.cpp"
#include "foo.h"

int main()
{
    foo<int>();
    ...
}
```

If you compile and (try to) link these two .cpp files, most compilers will generate linker errors. There are three solutions for this. The first solution is to physically move the definition of the template function into the .h file, even if it is not an `inline` function. This solution may (or may not!) cause significant code bloat, meaning your executable size may increase dramatically (or, if your compiler is smart enough, may not; try it and see).

The other solution is to leave the definition of the template function in the .cpp file and simply add the line `template void foo<int>();` to that file:

```
// file "foo.cpp"
#include <iostream>
#include "foo.h"

template<typename T> void foo()
{
    std::cout << "Here I am!\n";
}

template void foo<int>();
```

If you can't modify `foo.cpp`, simply create a new `.cpp` file such as `foo-impl.cpp` as follows:

```
// file "foo-impl.cpp"
#include "foo.cpp"

template void foo<int>();
```

Notice that `foo-impl.cpp` `#includes` a `.cpp` file, not a `.h` file. If that's confusing, click your heels twice, think of Kansas, and repeat after me, "I will do it anyway even though it's confusing." You can trust me on this one. But if you don't trust me or are simply curious, [the rationale is given earlier](#).

If you are using [Comeau C++](#), you probably want to check out the `export` keyword.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.9] How can I avoid linker errors with my template classes?

Tell your C++ compiler which instantiations to make while it is compiling your template class's `.cpp` file.

(If you've already read the previous FAQ, this answer is completely symmetric with that one, so you can probably skip this answer.)

As an example, consider the header file `Foo.h` which contains the following template class. Note that method `Foo<T>::f()` is inline and methods `Foo<T>::g()` and `Foo<T>::h()` are not.

```
// file "Foo.h"
template<typename T>
class Foo {
public:
    void f();
    void g();
    void h();
};

template<typename T>
inline
void Foo<T>::f()
{
    ...
}
```

Now suppose file `Foo.cpp` actually defines the non-inline methods `Foo<T>::g()` and `Foo<T>::h()`:

```
// file "Foo.cpp"
#include <iostream>
#include "Foo.h"

template<typename T>
void Foo<T>::g()
{
    std::cout << "Foo<T>::g()\n";
}

template<typename T>
void Foo<T>::h()
{
    std::cout << "Foo<T>::h()\n";
}
```

Suppose file `main.cpp` uses this template class by creating a `Foo<int>` and calling its methods:

```
// file "main.cpp"
#include "Foo.h"

int main()
{
    Foo<int> x;
    x.f();
    x.g();
    x.h();
    ...
}
```

If you compile and (try to) link these two `.cpp` files, most compilers will generate linker errors. There are three solutions for this. The first solution is to physically move the definition of the template functions into the `.h` file, even if they are not `inline` functions. This solution may (or may not!) cause significant code bloat, meaning your executable size may increase dramatically (or, if your compiler is smart enough, may not; try it and see).

The other solution is to leave the definition of the template function in the `.cpp` file and simply add the line `template Foo<int>;` to that file:

```
// file "Foo.cpp"
#include <iostream>
#include "Foo.h"

...definition of Foo<T>::f() is unchanged -- see above...
...definition of Foo<T>::g() is unchanged -- see above...

template Foo<int>;
```

If you can't modify `Foo.cpp`, simply create a new `.cpp` file such as `Foo-impl.cpp` as follows:

```
// file "Foo-impl.cpp"
#include "Foo.cpp"

template Foo<int>;
```

Notice that `Foo-impl.cpp` `#includes` a `.cpp` file, not a `.h` file. If that's confusing, click your heels twice, think of Kansas, and repeat after me, "I will do it anyway even though it's confusing." You can trust me on this one. But if you don't trust me or are simply curious, [the rationale is given earlier](#).

If you are using [Comeau C++](#), you probably want to check out the `export` keyword.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.10] Why do I get linker errors when I use template friends?

Ah, the intricacies of template friends. Here's an example of what people often want to do:

```
#include <iostream>

template<typename T>
class Foo {
public:
    Foo(const T& value = T());
    friend Foo<T> operator+ (const Foo<T>& lhs, const Foo<T>& rhs);
    friend std::ostream& operator<< (std::ostream& o, const Foo<T>& x);
private:
    T value_;
};
```

Naturally the template will need to actually be *used* somewhere:

```
int main()
{
    Foo<int> lhs(1);
    Foo<int> rhs(2);
    Foo<int> result = lhs + rhs;
    std::cout << result;
    ...
}
```

And of course the various member and friend functions will need to be defined somewhere:

```
template<typename T>
Foo<T>::Foo(const T& value = T())
: value_(value)
{ }

template<typename T>
Foo<T> operator+ (const Foo<T>& lhs, const Foo<T>& rhs)
{ return Foo<T>(lhs.value_ + rhs.value_); }

template<typename T>
std::ostream& operator<< (std::ostream& o, const Foo<T>& x)
{ return o << x.value_; }
```

The snag happens when the compiler sees the `friend` lines way up in the class definition proper. At that moment it does not yet know the `friend` functions are themselves templates; it assumes they are non-templates like this:

```

Foo<int> operator+ (const Foo<int>& lhs, const Foo<int>& rhs)
{ ... }

std::ostream& operator<< (std::ostream& o, const Foo<int>& x)
{ ... }

```

When you call the `operator+` or `operator<<` functions, this assumption causes the compiler to generate a call to the *non*-template functions, but the linker will give you an "undefined external" error because you never actually defined those *non*-template functions.

The solution is to convince the compiler *while it is examining the class body proper* that the `operator+` and `operator<<` functions are themselves templates. There are several ways to do this; one simple approach is pre-declare each template friend function *above* the definition of template class `Foo`:

```

template<typename T> class Foo; // pre-declare the template class itself
template<typename T> Foo<T> operator+ (const Foo<T>& lhs, const Foo<T>& rhs);
template<typename T> std::ostream& operator<< (std::ostream& o, const Foo<T>& x);

```

Also you add `<>` in the friend lines, as shown:

```

#include <iostream>

template<typename T>
class Foo {
public:
    Foo(const T& value = T());
    friend Foo<T> operator+ <> (const Foo<T>& lhs, const Foo<T>& rhs);
    friend std::ostream& operator<< <> (std::ostream& o, const Foo<T>& x);
private:
    T value_;
};

```

After the compiler sees that magic stuff, it will be better informed about the friend functions. In particular, it will realize that the friend lines are referring to functions that are themselves templates. That eliminates the confusion.

Another approach is to *define* the friend function within the class body at the same moment you declare it to be a friend. For example:

```

#include <iostream>

template<typename T>
class Foo {
public:
    Foo(const T& value = T());

    friend Foo<T> operator+ (const Foo<T>& lhs, const Foo<T>& rhs)
    {
        ...
    }

    friend std::ostream& operator<< (std::ostream& o, const Foo<T>& x)
    {
        ...
    }
};

```

```
private:
    T value_;
};
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.11] How can any human hope to understand these overly verbose template-based error messages?

Here's a free tool that [transforms error messages into something more understandable](#). At the time of this writing, it works with the following compilers: Comeau C++, Intel C++, CodeWarrior C++, gcc, Borland C++, Microsoft Visual C++, and EDG C++.

Here's an example showing some unfiltered gcc error messages:

```
rtmap.cpp: In function `int main()':
rtmap.cpp:19: invalid conversion from `int' to `
    std::_Rb_tree_node<std::pair<const int, double> >*'
rtmap.cpp:19:   initializing argument 1 of `std::_Rb_tree_iterator<_Val, _Ref,
    _Ptr>::_Rb_tree_iterator(std::_Rb_tree_node<_Val>*) [with _Val =
    std::pair<const int, double>, _Ref = std::pair<const int, double>&, _Ptr =
    std::pair<const int, double>*]'
rtmap.cpp:20: invalid conversion from `int' to `
    std::_Rb_tree_node<std::pair<const int, double> >*'
rtmap.cpp:20:   initializing argument 1 of `std::_Rb_tree_iterator<_Val, _Ref,
    _Ptr>::_Rb_tree_iterator(std::_Rb_tree_node<_Val>*) [with _Val =
    std::pair<const int, double>, _Ref = std::pair<const int, double>&, _Ptr =
    std::pair<const int, double>*]'
E:/GCC3/include/c++/3.2/bits/stl_tree.h: In member function `void
    std::_Rb_tree<_Key, _Val, _KeyOfValue, _Compare, _Alloc>::insert_unique(_II,
    _II) [with _InputIterator = int, _Key = int, _Val = std::pair<const int,
    double>, _KeyOfValue = std::Select1st<std::pair<const int, double> >,
    _Compare = std::less<int>, _Alloc = std::allocator<std::pair<const int,
    double> >]':
E:/GCC3/include/c++/3.2/bits/stl_map.h:272:   instantiated from `void std::map<
    Key, _Tp, _Compare, _Alloc>::insert(_InputIterator, _InputIterator) [with _Input
    Iterator = int, _Key = int, _Tp = double, _Compare = std::less<int>, _Alloc = st
    d::allocator<std::pair<const int, double> >]'
rtmap.cpp:21:   instantiated from here
E:/GCC3/include/c++/3.2/bits/stl_tree.h:1161: invalid type argument of `unary *
```

Here's what the filtered error messages look like (note: you can configure the tool so it shows more information; this output was generated with settings to strip things down to a minimum):

```
rtmap.cpp: In function `int main()':
rtmap.cpp:19: invalid conversion from `int' to `iter'
rtmap.cpp:19:   initializing argument 1 of `iter(iter)'
rtmap.cpp:20: invalid conversion from `int' to `iter'
rtmap.cpp:20:   initializing argument 1 of `iter(iter)'
stl_tree.h: In member function `void map<int,double>::insert_unique(_II, _II)':
    [STL Decryptor: Suppressed 1 more STL standard header message]
rtmap.cpp:21:   instantiated from here
stl_tree.h:1161: invalid type argument of `unary *
```

Here is the source code to generate the above example:

```

#include <map>
#include <algorithm>
#include <cmath>

const int values[] = { 1,2,3,4,5 };
const int NVALS = sizeof values / sizeof (int);

int main()
{
    using namespace std;

    typedef map<int, double> valmap;

    valmap m;

    for (int i = 0; i < NVALS; i++)
        m.insert(make_pair(values[i], pow(values[i], .5)));

    valmap::iterator it = 100;           // error
    valmap::iterator it2(100);           // error
    m.insert(1,2);                       // error

    return 0;
}

```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.12] Why am I getting errors when my template-derived-class accesses something it inherited from its template-base-class? UPDATED!

[Recently added `public:` thanks to [Alexander Kamotsky](#) and [Leonard "SoulSkorpion" Frankel](#); fixed a grammatical error thanks to [Val Kartchner](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Perhaps surprisingly, the following code is not valid C++, even though some compilers accept it:

```

template<typename T>
class B {
public:
    void f() { }
};

template<typename T>
class D : public B<T> {
public:
    void g()
    {
        f();    ← compiler might give an error here
    }
};

```

This might hurt your head; better if you sit down.

A nondependent name (one that is not dependent on template parameters in some appropriate way) is not looked up in dependent base classes (i.e., base classes that are specified in terms of

one or more template parameters. In the example above `f` in the call `f()` is nondependent while the base `B<T>` is dependent.

This doesn't mean that inheritance doesn't work. A `D<int>` is still derived from a `B<int>` and the compiler still lets you implicitly convert them via the `is-a` relationship. However there is an issue about how names are looked up.

There are various possible workarounds for this. The simplest is to change the call from `f()` to `this->f()`. Since `this` is always implicitly dependent in a template, `this->f` is not looked up until the template is actually instantiated at which point all base classes are considered.

In some cases that isn't feasible, in which case you can insert `using B<T>::f;` just prior to calling `f()`.

A third possibility is to change the call from `f()` to `B<T>::f()`. However that might not give you what you want in cases where `f()` is virtual: it inhibits the virtual dispatch mechanism.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[35.13] Can the previous problem hurt me silently? Is it possible that the compiler will silently generate the wrong code?

Yes (unfortunately).

Since the non-dependent name `f` is not looked up in the dependent base-class `B<T>`, the compiler will search the enclosing scope (such as the enclosing namespace) for the name `f`. This can cause it to silently(!) do the wrong thing.

For example:

```
void f() { }    // a global ("namespace scope") function

template<typename T>
class B {
public:
    void f() { }
};

template<typename T>
class D : public B<T> {
public:
    void g()
    {
        f();
    }
};
```

Here the call within `D<T>::g()` will silently(!) call `::f()` rather than `B<T>::f()`.

You have been warned.



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[36] Serialization and Unserialization

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),
[cline@parashift.com](#))

FAQs in section [36]:

- [\[36.1\] What's this "serialization" thing all about?](#)
 - [\[36.2\] How do I select the best serialization technique?](#)
 - [\[36.3\] How do I decide whether to serialize to human -readable \("text"\) or non-human-readable \("binary"\) format?](#)
 - [\[36.4\] How serialize/unserialize simple types like numbers, characters, strings, etc.?](#)
 - [\[36.5\] How exactly do I read/write simple types in human -readable \("text"\) format?](#)
 - [\[36.6\] How exactly do I read/write simple types in non -human-readable \("binary"\) format?](#)
 - [\[36.7\] How do I serialize objects that aren't part of an inheritance hierarchy and that don't contain pointers to other objects?](#)
 - [\[36.8\] How do I serialize objects that are part of an inheritance hierarchy and that don't contain pointers to other objects?](#)
 - [\[36.9\] How do I serialize objects that contain pointers to other objects, but those pointers form a tree with no cycles and no joins?](#)
 - [\[36.10\] How do I serialize objects that contain pointers to other objects, but those pointers form a tree with no cycles and only "trivial" joins?](#)
 - [\[36.11\] How do I serialize objects that contain pointers to other objects, and those pointers form a graph that might have cycles or non -trivial joins?](#)
 - [\[36.12\] Are there any caveats when serializing / unserializing objects?](#)
 - [\[36.13\] What's all this about graphs, trees, nodes, cycles, join s, and joins at the leaves vs. internal nodes?](#)
-

[36.1] What's this "serialization" thing all about?

It lets you take an object or group of objects, put them on a disk or send them through a wire or wireless transport mechanism, then later, perhaps on an other computer, reverse the process: resurrect the original object(s). The basic mechanisms are to flatten object(s) into a one - dimensional stream of bits, and to turn that stream of bits back into the original object(s).



Like the Transporter on Star Trek , it's all about taking something complicated and turning it into a flat sequence of 1s and 0s, then taking that sequence of 1s and 0s (possibly at another place, possibly at another time) and reconstructing the original complicated "something."

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.2] How do I select the best serialization technique?

There are lots and lots (and lots) of if's, and's and but's, and in reality there are a whole continuum of techniques with lots of dimensions. Because I have a finite amount of time (translation: I don't get paid for any of this), I've simplified it to a [decision between using human-readable \("text"\) or non-human-readable \("binary"\) format](#), followed by a list of five techniques arranged more-or-less in increasing order of sophistication.

You are, of course, not limited to those five techniques. You will probably end up mixing ideas from several techniques. And certainly you can always use a more sophisticated (higher numbered) technique than is actually needed. In fact it might be wise to use a more sophisticated technique than is minimally needed if you believe future changes will require the greater sophistication. So think of this list merely as a good starting point.

There's a lot here, so get ready!

1. [Decide between human-readable \("text"\) and non-human-readable \("binary"\) formats](#). The tradeoffs are non-trivial. Later FAQs show [how to write simple types in text format](#) and [how to write simple types in binary format](#).
2. [Use the least sophisticated solution](#) when the objects to be serialized aren't part of an inheritance hierarchy (that is, when they're all of the same class) and when they don't contain pointers to other objects.
3. [Use the second level of sophistication](#) when the objects to be serialized are part of an inheritance hierarchy, but when they don't contain pointers to other objects.
4. [Use the third level of sophistication](#) when the objects to be serialized contain pointers to other objects, but when those pointers form a [tree](#) with no [cycles](#) and no [joins](#).
5. [Use the fourth level of sophistication](#) when the objects to be serialized contain pointers to other objects, and when those pointers form a [graph](#) that with no [cycles](#), and with [joins at the leaves only](#).
6. [Use the most sophisticated solution](#) when the objects to be serialized contain pointers to other objects, and when those pointers form a [graph](#) that might have [cycles](#) or [joins](#).

Here's that same information arranged like an algorithm:

1. The first step is to [make an eyes-open decision between text- and binary-formats](#).
2. If your objects aren't part of an inheritance hierarchy and don't contain pointers, [use solution #1](#).
3. Else if your objects don't contain pointers to other objects, [use solution #2](#).
4. Else if the [graph](#) of pointers within your objects contain neither [cycles](#) nor [joins](#), [use solution #3](#).

5. Else if the [graph](#) of pointers within your objects don't contain [cycles](#) and if the only [joins are to terminal \(leaf\) nodes](#), [use solution #4](#).
6. Else [use solution #5](#).

Remember: feel free to mix and match, to add to the above list, and, *if* you can justify the added expense, to use a more sophisticated technique than is minimally required.

One more thing: the issues of inheritance and of pointers within the objects are logically unrelated, so there's no theoretical reason for #2 to be any less sophisticated than #3 -5. However in practice it often (not always) works out that way. So please do not think of these categories as somehow sacred — they're somewhat arbitrary, and you are expected to mix and match the solutions to fit your situation. This whole area of serialization has *far* more variants and shades of gray than can be covered in a few questions/answers.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.3] How do I decide whether to serialize to human -readable ("text") or non-human-readable ("binary") format?

Carefully.

There is no "right" answer to this question; it really depends on your goals. Here are a few of the pros/cons of human-readable ("text") format vs. non-human-readable ("binary") format:

- **Text format** is easier to "desk check." That means you won't have to write extra tools to debug the input and output; you can open the serialized output with a text editor to see if it looks right.
- **Binary format** typically uses fewer CPU cycles. However that is relevant only if your application is CPU bound and you intend to do serialization and/or unserialization on an inner loop/bottleneck. Remember: 90% of the CPU time is spent in 10% of the code, which means there won't be any practical performance benefit unless your "CPU meter" is pegged at 100%, and your serialization and/or unserialization code is consuming a healthy portion of that 100%.
- **Text format** lets you ignore programming issues like `sizeof` and little-endian vs. big-endian.
- **Binary format** lets you ignore separations between adjacent values, since many values have fixed lengths.
- **Text format** can produce smaller results when most numbers are small and when you need to textually encode binary results, e.g., uuencode or Base64.
- **Binary format** can produce smaller results when most numbers are large or when you don't need to textually encode binary results.

You might think of others to add as well... The important thing to remember is that one size does not fit all — make a careful decision here.

One more thing: no matter which you choose, you might want to start each file / stream with a "magic" tag and a version number. The version number would indicate the format rules. That

way if you decide to make a radical change in the format, you hopefully will still be able to read the output produced by the old software.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.4] How serialize/unserialize simple types like numbers, characters, strings, etc.?

The answer depends on [your decision regarding human-readable \("text"\) format vs. non-human-readable \("binary"\) format](#):

- Here is [how to serialize/unserialize simple types in human-readable \("text"\) format](#).
- Here is [how to serialize/unserialize simple types in non-human-readable \("binary"\) format](#).

The primitives discussed in those FAQs will be needed for most of the other FAQs in this section.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.5] How exactly do I read/write simple types in human-readable ("text") format?

Before you read this, make sure to [evaluate all the tradeoffs between human-readable and non-human-readable formats](#). The tradeoffs are non-trivial, so you should resist a knee-jerk reaction to do it the way you did it on the last project — one size does not fit all.

After you have made an eyes-open decision to use human-readable ("text") format, you should remember these keys:

- You probably want to use `iostream`'s `>>` and `<<` operators rather than its `read()` and `write()` methods. The `>>` and `<<` operators are better for text mode, whereas `read()` and `write()` are better for binary mode.
- When storing numbers, you'll probably want to add a separator to prevent items from running together. One simple approach is to always add a space (' ') before each number, that way the number 1 followed by the number 2 won't run together and look like a 12. Since the leading space will automatically get soaked up by the `>>` operator, you won't have to do anything explicit to extract the leading space in the code that reads things back in.
- String data is tricky because you have to unambiguously know when the string's body stops. You can't unambiguously terminate all strings with a `'\\n'` or `'''` or even `'\\0'` if some string might contain those characters. You might want to use C++ source-code escape-sequences, e.g., writing `'\\'` followed by `'n'` when you see a newline, etc. After this transformation, you can either make strings go until end-of-line (meaning they are delimited by `'\\n'`) or you can delimit them with `'''`.

- If you use C++-like escape-sequences for your string data, be sure to always use the same number of hex digits after `'\x'` and `'\u'`. I typically use 2 and 4 digits respectively. Reason: if you write a smaller number of hex digits, e.g., if you simply use `stream << "\\x" << hex << unsigned(theChar)`, you'll get errors when the next character in the string happens to be a hex digit. E.g., if the string contains `'\xF'` followed by `'A'`, you should write `"\x0FA"`, not `"\xFA"`.
- If you don't use some sort of escape sequence for characters like `'\n'`, be careful that the operating system doesn't mess up your string data. In particular, if you open a `std::fstream` without `std::ios::binary`, some operating systems translate end-of-line characters.
- Another approach for string data is to prefix the string's data with an integer length, e.g., to write "now is the time" as `15:now is the time`. Note that this can make it hard for people to read/write the file, since the value just after that might not have a visible separator, but you still might find it useful.

Please remember that these are primitives that you will need to use in the other FAQs in this section.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.6] How exactly do I read/write simple types in non-human-readable ("binary") format?

Before you read this, make sure to [evaluate all the tradeoffs between human-readable and non-human-readable formats](#). The tradeoffs are non-trivial, so you should resist a knee-jerk reaction to do it the way you did it on the last project — one size does not fit all.

After you have made an eyes-open decision to use non-human-readable ("binary") format, you should remember these keys:

- Make sure you open the input and output streams using `std::ios::binary`. Do this even if you are on a Unix system since it's easy to do, it documents your intent, and it's one less non-portability to locate and change down the road.
- You probably want to use `istream`'s `read()` and `write()` methods instead of its `>>` and `<<` operators. `read()` and `write()` are better for binary mode; `>>` and `<<` are better for text mode.
- If the binary data might get read by a different computer than the one that wrote it, be very careful about endian issues (little-endian vs. big-endian) and `sizeof` issues. The easiest way to handle this is to anoint one of those two formats as the official "network" format, and to create a header file that contains machine dependencies (I usually call it `machine.h`). That header should define `inline` functions like `readNetworkInt(std::istream& istr)` to read a "network int," and so forth for reading and writing all the primitive types. You can define the format for these pretty much anyway you want. E.g., you might define a "network int" as exactly 32 bits in little endian format. In any case, the functions in `machine.h` will do any necessary endian conversions, `sizeof` conversions, etc. You'll either end up with a different `machine.h` on each machine architecture, or you'll end up with a lot of `#ifdefs` in your `machine.h`, but either way, all

this ugliness will be buried in a single header, and all the rest of your code will be clean(er). Note: the floating point differences are the most subtle and tricky to handle. It can be done, but you'll have to be careful with things like [NaN](#), over- and under-flow, #bits in the mantissa or exponent, etc.

- When space-cost is an issue, such as when you are storing the serialized form in a small memory device or sending it over a slow link, you can compress the stream and/or you can do some manual tricks. The simplest is to store small numbers in a smaller number of bytes. For example, to store an unsigned integer in a stream that has 8-bit bytes, you can hijack the 8th bit of each byte to indicate whether or not there is another byte. That means you get 7 meaningful bits/byte, so 0...127 fit in 1 byte, 128...16384 fit in 2 bytes, etc. If the average number is smaller than around half a billion, this will use less space than storing every four-byte unsigned number in four 8-bit bytes. There are lots of other variations on this theme, e.g., a sorted array of numbers can store the difference between each number, storing extremely small values in unary format, etc.
- String data is tricky because you have to unambiguously know when the string's body stops. You can't unambiguously terminate all strings with a `'\0'` if some string might contain that character; recall that `std::string` can store `'\0'`. The easiest solution is to write the integer length just before the string data. Make sure the integer length is written in "network format" to avoid `sizeof` and endian problems (see the solutions in earlier bullets).

Please remember that these are primitives that you will need to use in the other FAQs in this section.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.7] How do I serialize objects that aren't part of an inheritance hierarchy and that don't contain pointers to other objects?

This is the least sophisticated problem, and not surprisingly, it is also the least sophisticated solution:

- Every class should handle its own serialization and unserialization. You will typically create a member function that serializes the object to some sink (such as a `std::ostream`), and another that allocates a new object, or perhaps changes an existing object, setting the member data based on what it reads from some source (such as a `std::istream`).
- If your object physically contains another object, e.g., a `car` object might have a member variable of type `Engine`, the outer object's `serialize()` member function should simply call the appropriate function associated with the member object.
- Use the primitives described earlier to read/write the simple types in [text](#) or [binary](#) format.
- If a class's data structure might change someday, the class should write out a version number at the beginning of the object's serialized output. The version number simply represents the *serialized format*; it should *not* get incremented simply when the class's behavior changes. This means the version numbers don't need to be fancy — they usually don't need a major and minor number.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.8] How do I serialize objects that are part of an inheritance hierarchy and that don't contain pointers to other objects?

Suppose you want to serialize a "shape" object, where `Shape` is an abstract class with derived classes `Rectangle`, `Ellipse`, `Line`, `Text`, etc. You would declare a [pure virtual](#) function `serialize(std::ostream&) const` within class `Shape`, and make sure the first thing done by each override is to write out the class's identity. For example, `Ellipse::serialize(std::ostream&) const` would write out the identifier `Ellipse` (perhaps [as a simple string](#), but there are several alternatives discussed below).

Things get a little trickier when unserializing the object. You typically start with a static member function in the base class such as `Shape::unserialize(std::istream& istr)`. This is declared to return a `Shape*` or perhaps a [smart pointer such as `Shape::Ptr`](#). It [reads the class-name identifier](#), then uses some sort of creational pattern to create the object. For example, you might have a table that maps from the class name to an object of the class, then use the [Virtual Constructor Idiom](#) to create the object.

Here's a concrete example: Add a [pure virtual](#) method `create(std::istream&) const` within base class `Shape`, and define each override to a one-liner that uses `new` to allocate an object of the appropriate derived class. E.g., `Ellipse::create(std::istream& istr) const` would be `{ return new Ellipse(istr); }`. Add a static `std::map<std::string, Shape*>` object that maps from the class name to a representative (AKA *prototype*) object of the appropriate class; e.g., "Ellipse" would map to a new `Ellipse()`. Function `Shape::unserialize(std::istream& istr)` would [read the class-name](#), throw an exception if it's not in the map (if `(theMap.count(className) == 0)` throw *...something...*), then look up the associated `Shape*` and call its `create()` method: `return theMap[className]->create(istr)`.

The map is typically populated during static initialization. For example, if file `Ellipse.cpp` contains the code for derived class `Ellipse`, it would also contain a static object whose ctor adds that class to the map: `theMap["Ellipse"] = new Ellipse()`.

Notes and caveats:

- It adds a little flexibility if `Shape::unserialize()` passes the class name to the `create()` method. In particular, that would let a derived class be used with two or more names, each with its own "network format." For example, derived class `Ellipse` could be used for both "Ellipse" and "Circle", which might be useful to save space in the output stream or perhaps other reasons.
- It's usually easiest to handle errors during unserialization by throwing an exception. You can return `NULL` if you want, but you will need to move the code that reads the input stream out of the derived class' ctors into the corresponding `create()` methods, and ultimately the result is often that your code is more complicated.
- You must be careful to [avoid the static initialization order fiasco](#) with the map used by `Shape::unserialize()`. This normally means using the [Construct On First Use Idiom](#) for the map itself.
- For the map used by `Shape::unserialize()`, I personally prefer the [Named Constructor Idiom](#) over the [Virtual Constructor Idiom](#) — it simplifies a few steps. Details: I usually

define a typedef within Shape such as `typedef Shape* (*Factory)(std::istream&)`. This means `Shape::Factory` is a "pointer to a function that takes a `std::istream&` and returns a `Shape*`." I then define the map as `std::map<std::string, Factory>`. Finally I populate that map using lines like `theMap["Ellipse"] = Ellipse::create` (where `Ellipse::create(std::istream&)` is now a [static member function](#) of class `Ellipse`, that is, the [Named Constructor Idiom](#)). You'd change the return value in function `Shape::unserialize(std::istream& istr)` from `theMap[className]->create(istr)` to `theMap[className](istr)`.

- If you might need to serialize a `NULL` pointer, it's usually easy since you already write out a class identifier so you can just as easily write out a *pseudo* class identifier like `"NULL"`. You might need an extra `if` statement in `Shape::unserialize()`, but if you chose my preference from the previous bullet, you can eliminate that special case (and generally keep your code clean) by defining static member function `Shape::nullFactory(istream&)` { return `NULL`; }. You add that function to the map as any other: `theMap["NULL"] = Shape::nullFactory`;
- You can make the serialized form smaller and a little faster if you tokenize the class name identifiers. For example, write a class name only the first time it is seen, and for subsequent uses write only a corresponding integer index. A mapping such as `std::map<std::string, unsigned> unique` makes this easy: if a class name is already in the map, write `unique[className]`; otherwise set a variable `unsigned n = unique.size()`, write `n`, write the class name, and set `unique[className] = n`. (Note: be *sure* to copy it into a separate variable. Do *not* say `unique[className] = unique.size()`! You have been warned!) When unserializing, use `std::vector<std::string> unique`, read the number `n`, and if `n == unique.size()`, read a name and add it to the vector. Either way the name will be `unique[n]`. You can also pre-populate the first *N* slots in these tables with the *N* most common names, that way streams won't need to contain any of those strings.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.9] How do I serialize objects that contain pointers to other objects, but those pointers form a tree with no cycles and no joins?

Before we even start, you must understand that the word "tree" does *not* mean that the objects are stored in some sort of tree-like data structure. It simply means that your objects point to each other, and the "with no cycles" part means if you keep following pointers from one object to the next, you never return to an earlier object. Your objects aren't "inside" a tree; they *are* a tree. If you don't understand that, you *really* should read [the lingo FAQ](#) before continuing with this one.

Second, don't use this technique if the [graph](#) might someday contain [cycles](#) or [joins](#).

Graphs with neither cycles nor joins are very common, even with "recursive composition" [design patterns like Composite or Decorator](#). For example, the objects representing an XML document or an HTML document can be represented as a graph without joins or cycles.

The key to serializing these graphs is to ignore a node's *identity* and instead to focus only on its *contents*. A (typically recursive) algorithm dives through the tree and writes the contents as it goes. For example, if the current node happens to have an integer `a`, a pointer `b`, a float `c`, and another pointer `d`, then you first [write the integer](#) `a`, then recursively dive into the child pointed to

by b, then [write the float](#) c, and finally recursively dive into the child pointed to by d. (You don't have to write/read them in the declaration order; the only essential rule is that the reader's order is consistent with the writer's order.)

When unserializing, you need a constructor that takes a `std::istream&`. The constructor for the above object would [read an integer](#) and store the result in a, then would allocate an object to be stored in pointer b (and will pass the `std::istream` to the constructor so it too can read the stream's contents), [read a float](#) into c, and finally will allocate an object to be stored in pointer d. Be sure to use [smart pointers](#) within your objects, [otherwise you will get a leak](#) if an exception is thrown while reading anything but the first pointed-to object.

It is often convenient to use the [Named Constructor Idiom](#) when allocating these objects. This has the advantage that you can [enforce the use of smart pointers](#). To do this in a class `Foo`, write a static method such as `FooPtr Foo::create(std::istream& istr) { return new Foo(istr); }` (where `FooPtr` is a smart pointer to a `Foo`). The alert reader will note how consistent this is with the technique discussed in [the previous FAQ](#) — the two techniques are completely compatible.

If an object can contain a variable number of children, e.g., a `std::vector` of pointers, then the usual approach is to [write the number of children](#) just before recursively diving into the first child. When unserializing, just [read the number of children](#), then use a loop to allocate the appropriate number of child objects.

If a child-pointer might be `NULL`, be sure to handle that in both the writing and reading. This shouldn't be a problem [if your objects use inheritance](#); see that solution for details. Otherwise, if the first serialized character in an object has a known range, use something outside that range. E.g., if the first character of a serialized object is always a digit, use a non-digit like 'N' to mean a `NULL` pointer. Unserialization can use `std::istream::peek()` to check for the 'N' tag. If the first character doesn't have a known range, force one; e.g., write 'x' before each object, then use something else like 'y' to mean `NULL`.

If an object physically contains another object inside itself, as opposed to containing a pointer to the other object, then nothing changes: you still recursively dive into the child -node as if it were via a pointer.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.10] How do I serialize objects that contain pointers to other objects, but those pointers form a tree with no cycles and only "trivial" joins?

As before, the word "tree" does *not* mean that the objects are stored in some sort of tree-like data structure. It simply means your objects have pointers to each other, and the "no cycles" part means you can follow the pointers from one object to the next and never return to an earlier object. The objects are not "inside" a tree; they *are* a tree. If that's doesn't make sense, you *really* should read [the lingo FAQ](#) before continuing with this one.

Use this solution if the [graph](#) contains [joins at the leaf nodes](#), but those joins can be easily reconstructed via a simple look-up table. For example, the parse-tree of an arithmetic expression like $(3*(a+b) - 1/a)$ *might* have joins since a variable-name (like a) can show up more than

once. If you want the graph to use the same exact node -object to represent both occurrences of that variable, then you could use this solution.

Although the above constraints don't fit with those of [the solution without any joins](#), it's so close that you can squeeze things into that solution. Here are the differences:

- During serialization, ignore the join completely.
- During unserializing, create a look-up table, like `std::map<std::string, Node*>`, that maps from the variable name to the associated node.

Caveat: this assumes that *all* occurrences of variable `a` should map to the same node object; if it's more complicated than this, that is, if some occurrences of `a` should map to one object and some to another, you might need to use [a more sophisticated solution](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.11] How do I serialize objects that contain pointers to other objects, and those pointers form a graph that might have cycles or non-trivial joins?

Caveat: the word "graph" does *not* mean that the objects are stored in some sort of data structure. Your objects form a graph because they point to each other. They're not "inside" a graph; they *are* a graph. If that doesn't make sense, you *really* should read [the lingo FAQ](#) before continuing with this one.

Use this solution if the [graph](#) can contain [cycles](#), or if it can contain more complicated kinds of [joins](#) than are allowed by [the solution for trivial joins](#). This solution handles two core issues: it avoids infinite recursion and it writes/reads each node's *identity* in addition its *contents*.

A node's identity doesn't normally have to be consistent between different output streams. For example, if a particular file uses the number 3 to represent node `x`, a different file can use the number 3 to represent a different node, `y`.

There are some clever ways to serialize the graph, but the simplest to describe is a two-pass algorithm that uses an *object-ID map*, e.g., `std::map<Node*, unsigned> oidMap`. The first pass populates our `oidMap`, that is, it builds a mapping from object pointer to the integer that represents the object's identity. It does this by recursively diving through the graph, at each node checking if the node is already in `oidMap`, and if not, adding the node and a unique integer to `oidMap` and recursively diving into the new node's children. The unique integer is often just the initial `oidMap.size()`, e.g., `unsigned n = oidMap.size(); oidMap[nodePtr] = n`. (Yes, we did that in two statements. You must also. Do *not* shorten it to a single statement. You have been warned.)

The second pass iterates through the nodes of `oidMap`, and at each, writes the node's identity (the associated integer) followed by its contents. When writing the contents of a node that contains pointers to other nodes, instead of diving into those "child" objects, it simply writes the identity (the associated integer) of the pointer to those nodes. For example, when your node contains `Node* child`, simply [write the integer](#) `oidMap[child]`. After the second pass, the `oidMap`

can be discarded. In other words, the mapping from `Node*` to unsigned should *not* normally survive beyond the end of the serialization of any given graph.

There are also some clever ways to unserialize the graph, but here again the simplest to describe is a two-pass algorithm. The first pass populates a `std::vector<Node*> v` with objects of the right class, but any child pointers within those objects are all `NULL`. This means `v[3]` will point to the object whose oid is 3, but any child pointers inside that object will be `NULL`. The second pass populates the child pointers inside the objects, e.g., if `v[3]` has a child pointer called `child` that is supposed to point to the object whose oid is 5, the second pass changes `v[3].child` from `NULL` to `v[5]` (obviously encapsulation might prevent it from directly accessing `v[3].child`, but ultimately `v[3].child` gets changed to `v[5]`). After unserializing a given stream, the vector `v` can normally be discarded. In other words, the oids (3, 5, etc.) mean *nothing* when serializing or unserializing a different stream — those numbers are only meaningful within a given stream.

Note: if your objects contain *polymorphic pointers*, that is, base class pointers that might point at derived class objects, then [use the technique described earlier](#). You'll also want to read some of the earlier techniques for handling `NULL`, writing version numbers, etc.

Note: you should seriously consider the [Visitor pattern](#) when recursively diving through the graph, since serialization is probably just one of many different reasons to make that recursive dive, and they'll all need to avoid infinite recursion.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.12] Are there any caveats when serializing / unserializing objects?

One of the things you *don't* want to do, except in unusual circumstances, is to make any changes to the node's data during the traversal. For example, some people feel they can map from `Node*` to integer by simply adding an integer as a data member in the `Node` class. They also sometimes add a `bool haveVisited` flag as another data member within `Node` objects.

But this causes lots of multi-threading and/or performance problems. Your `serialize()` method can no longer be `const`, so even though it is logically just a reader of the node data, and even though it logically makes sense to have multiple threads reading the nodes at the same time, the actual algorithm writes into the nodes. If you fully understand threading and reader/writer conflicts, the best you can hope for is to make your code more complicated and slower (you'll have to block all reader threads whenever *any* thread is doing *anything* with the graph). If you (or those who will maintain the code after you!) don't fully understand reader/writer conflicts, this technique can create very serious and very subtle errors. Reader/writer and writer/writer conflicts are so subtle they often slip through test into the field. Bad news. Trust me. If you don't trust me, talk to someone you do trust. But don't cut corners.

There's lots more I could say, such as several simplifications and special cases, but I've already spent too much time on this. If you want more info, spend some money.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[36.13] What's all this about graphs, trees, nodes, cycles, joins, and joins at the leaves vs. internal nodes?

When your objects contain pointers to other objects, you end up with something computer scientists call a *graph*. Not that your objects are stored inside a tree-like data structure; they *are* a tree-like structure.

Your objects correspond to the graph's *nodes* AKA *vertices*, and the pointers within your objects correspond to the graph's *edges*. The graph is of a special variety called a *rooted, directed* graph. The root object to be serialized corresponds to the graph's *root node*, and the pointers correspond to *directed edges*.

If object *x* has a pointer to object *y*, we say that *x* is a *parent* of *y* and/or that *y* is a *child* of *x*.

A *path* through a graph corresponds to starting with an object, following a pointer to another object, etc., etc. to an arbitrary depth. If there is a path from *x* to *z* we say that *x* is an *ancestor* of *z* and/or that *z* is a *descendent* of *x*.

A *join* in a graph means there are two or more distinct paths to the same object. For example, if *z* is a child of both *x* and *y*, then the graph has a join, and *z* is a *join node*.

A *cycle* in a graph means there is a path from an object back to itself: if *x* has a pointer to itself, or to *y* which points to *x*, or to *y* which points to *z* which points to *x*, etc. A graph is *cyclic* if it has one or more cycles; otherwise it is *acyclic*.

An *internal node* is a node with children. A *leaf node* is a node without children.

As used in this section, the word *tree* means a rooted, directed, acyclic graph. Note that each node within a tree is also a tree.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

E-mail  [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[37] Class libraries

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),
cline@parashift.com)

FAQs in section [37]:

- [\[37.1\] What is the "STL"?](#)

- [\[37.2\] Where can I get a copy of "STL"?](#)
 - [\[37.3\] How can I find a Fred object in an STL container of Fred* such as std::vector<Fred*>?](#)
 - [\[37.4\] Where can I get help on how to use STL?](#)
 - [\[37.5\] How can you tell if you have a dynamically typed C++ class library?](#)
 - [\[37.6\] What is the NIHCL? Where can I get it?](#)
 - [\[37.7\] Where can I ftp the code that accompanies "Numerical Recipes"?](#)
 - [\[37.8\] Why is my executable so large?](#)
 - [\[37.9\] Where can I get tons and tons of more information on C++ class libraries?](#)
-

[37.1] What is the "STL"?

STL ("Standard Templates Library") is a library that consists mainly of (very efficient) container classes, along with some iterators and algorithms to work with the contents of these containers.

Technically speaking the term "STL" is no longer meaningful since the classes provided by the STL have been fully integrated into the standard library, along with other standard classes like `std::ostream`, etc. Nonetheless many people still refer to the STL as if it were a separate thing, so you might as well get used to hearing that term.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.2] Where can I get a copy of "STL"?

Since the classes that were part of the [STL](#) have become part of the standard library, your compiler should provide these classes. If your compiler doesn't include these standard classes, either get an updated version of your compiler or download a copy of the STL classes from one of the following:

- An STL site: <ftp.cs.rpi.edu/pub/stl>
- STL HP official site: butler.hpl.hp.com/stl/
- Mirror site in Europe: www.maths.warwick.ac.uk/ftp/mirrors/c++/stl/
- STL code alternate: <ftp.cs.rpi.edu/stl>
- The SGI implementation: www.sgi.com/tech/stl/
- STLport: www.stlport.org

STL hacks for GCC-2.6.3 are part of the GNU libg++ package 2.6.2.1 or later (and they may be in an earlier version as well). Thanks to Mike Lindner.

Also you may as well get used to some people using "STL" to include the standard string header, "<string>", and others objecting to that usage.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.3] How can I find a `Fred` object in an STL container of `Fred*` such as `std::vector<Fred*>`?

STL functions such as `std::find_if()` help you find a `T` element in a container of `T`'s. But if you have a container of pointers such as `std::vector<Fred*>`, these functions will enable you to find an element that matches a given `Fred*` pointer, but they don't let you find an element that matches a given `Fred` object.

The solution is to use an optional parameter that specifies the "match" function. The following class template lets you compare the objects on the other end of the dereferenced pointers.

```
template<typename T>
class DereferencedEqual {
public:
    DereferencedEqual(const T* p) : p_(p) { }
    bool operator() (const T* p2) const { return *p_ == *p2; }
private:
    const T* p_;
};
```

Now you can use this template to find an appropriate `Fred` object:

```
void userCode(std::vector<Fred*> v, const Fred& match)
{
    std::find_if(v.begin(), v.end(), DereferencedEqual<Fred>(&match));
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.4] Where can I get help on how to use STL?

Here are some resources (in random order):

Rogue Wave's STL Guide: www.ccd.bnl.gov/bcf/cluster/pgi/pgC++_lib/stdlibug/ug1.htm

The STL FAQ: butler.hpl.hp.com/stl/stl.faq

Kenny Zalewski's STL guide: www.cs.rpi.edu/projects/STL/htdocs/stl.html

Mumit's STL Newbie's guide: www.xraylith.wisc.edu/~khan/software/stl/STL.newbie.html

SGL's STL Programmer's guide: www.sgi.com/tech/stl/

There are also some [books that will help](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.5] How can you tell if you have a dynamically typed C++ class library?

- Hint #1: when everything is derived from a single root class, usually `object`.
- Hint #2: when the container classes (`List`, `Stack`, `Set`, etc) are non-templates.
- Hint #3: when the container classes (`List`, `Stack`, `Set`, etc) insert/extract elements as pointers to `object`. This lets you put an `Apple` into such a container, but when you get it out, the compiler knows only that it is derived from `object`, so you have to use a pointer cast to convert it back to an `Apple*`; and you'd better pray a lot that it really *is* an `Apple`, cause [your blood is on your own head](#)).

You can make the pointer cast "safe" by using `dynamic_cast`, but this dynamic testing is just that: dynamic. This coding style is the essence of dynamic typing in C++. You call a function that says "convert this `object` into an `Apple` or give me `NULL` if its not an `Apple`," and you've got dynamic typing: you don't know what will happen until run-time.

When you use templates to implement your containers, the C++ compiler can statically validate 90+% of an application's typing information (the figure "90+" is apocryphal; some claim they always get 100%, those who need [persistence](#) get something less than 100% static type checking). The point is: C++ gets genericity from templates, not from inheritance.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.6] What is the NIHCL? Where can I get it?

NIHCL stands for "National-Institute-of-Health's-class-library." It can be acquired via 128.231.128.7/pub/NIHCL/nihcl-3.0.tar.Z

NIHCL (some people pronounce it "N-I-H-C-L," others pronounce it like "nickel") is a [C++ translation of the Smalltalk class library](#). There are some ways where NIHCL's use of dynamic typing helps (e.g., [persistent](#) objects). There are also places where its use of dynamic typing creates [tension](#) with the static typing of the C++ language.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.7] Where can I ftp the code that accompanies "Numerical Recipes"?

This software is sold and therefore it would be illegal to provide it on the net. However, it's only about \$30.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[37.8] Why is my executable so large?

Many people are surprised by how big executables are, especially if the source code is trivial. For example, a simple "hello world" program can generate an executable that is larger than most people expect (40+K bytes).

One reason executables can be large is that portions of the C++ runtime library might get statically linked with your program. How much gets linked in depends on compiler options regarding whether to statically or dynamically link the standard libraries, on how much of it you are using, and on how the implementer split up the library into pieces. For example, the `<iostream>` library is quite large, and consists of numerous classes and [virtual](#) functions. Using any part of it *might* pull in nearly all of the `<iostream>` code as a result of the interdependencies (however there might be a compiler option to dynamically link these classes, in which case your program might be small).

Another reason executables can be large is if you have turned on debugging (again via a compiler option). In at least one well known compiler, this option can increase the executable size by up to a factor of 10.

You have to consult your compiler manuals or the vendor's technical support for a more detailed answer.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

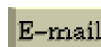
[37.9] Where can I get tons and tons of more information on C++ class libraries?

Three places you should check (not necessarily in this order):

- The *C++ Libraries* FAQ is maintained by [Nikki Locke](#) and is available [with frames](#) and [without frames](#).
- Also you should check out www.mathtools.net/C++/. They have a good pile of stuff organized into (at present) sixty-some categories.
- Also you should check out www.boost.org/. They have some wonderful stuff, some of which will get proposed for standardization the next time around.

Important: none of these lists are exhaustive. If you are looking for some particular functionality that you don't find above, try a Web search such as [Google](#). Also, don't forget to help out the next person via [the Submission Form in the C++ Libraries FAQ](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[38] Compiler dependencies

(Part of [C++ FAQ Lite](#), Copyright © 1991-2004, [Marshall Cline](#), cline@parashift.com)

FAQs in section [38]:

- [\[38.1\] Where can I download a free C++ compiler?](#)
 - [\[38.2\] Where can I get more information on using MFC and Visual C++?](#)
 - [\[38.3\] How do I display text in the status bar using MFC?](#)
 - [\[38.4\] How can I decompile an executable program back into C++ source code?](#)
 - [\[38.5\] Where can I get information about the C++ compiler from {Borland, IBM, Microsoft, Sun, etc.}?](#)
 - [\[38.6\] What's the difference between C++ and Visual C++?](#)
 - [\[38.7\] How do compilers use "over-allocation" to remember the number of elements in an allocated array?](#)
 - [\[38.8\] How do compilers use an "associative array" to remember the number of elements in an allocated array?](#)
 - [\[38.9\] If name mangling was standardized, could I link code compiled with compilers from different compiler vendors?](#)
 - [\[38.10\] GNU C++ \(g++\) produces big executables for tiny programs; Why?](#)
 - [\[38.11\] Is there a yacc-able C++ grammar?](#)
 - [\[38.12\] What is C++ 1.2? 2.0? 2.1? 3.0?](#)
 - [\[38.13\] Is it possible to convert C++ to C?](#)
-

[38.1] Where can I download a free C++ compiler?

Check these out (alphabetically):

- [Borland gives away a command line compiler.](#)
- [Digital Mars.](#)
- [DJGPP is a port of GCC 3.2 to DOS, and includes RHIDE, a DOS -based IDE.](#)
- [MinGW has a port of GCC 3.2 that runs on MS Windows.](#)

Also check out www.idiom.com/free-compilers/LANG/C++-1.html.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.2] Where can I get more information on using MFC and Visual C++?

Here are some resources (in no particular order):

- www.visionx.com/mfcpro/
- www.mvps.org/vcfaq
- www.flounder.com/mvp_tips.htm
- msdn.microsoft.com/archive/en-us/dnarvc/html/msdn_mfcfaq50.asp

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.3] How do I display text in the status bar using MFC?

Use the following code snippet:

```
CString s = "Text";
CStatusBar* p =
(CStatusBar*)AfxGetApp() ->m_pMainWnd->GetDescendantWindow(AFX_IDW_STATUS_BAR);
p->SetPaneText(1, s);
```

This works with MFC v.1.00 which hopefully means it will work with other versions as well.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.4] How can I decompile an executable program back into C++ source code?

You gotta be kidding, right?

Here are a few of the *many* reasons this is not even remotely feasible:

- What makes you think the program was written in C++ to begin with?
- Even if you are sure it was originally written (at least partially) in C++, which one of the gazillion C++ compilers produced it?
- Even if you know the compiler, which particular version of the compiler was used?
- Even if you know the compiler's manufacturer and version number, what compile-time options were used?
- Even if you know the compiler's manufacturer and version number and compile-time options, what third party libraries were linked-in, and what was their version?
- Even if you know all that stuff, most executables have had their debugging information stripped out, so the resulting decompiled code will be totally unreadable.
- Even if you know everything about the compiler, manufacturer, version number, compile-time options, third party libraries, and debugging information, the cost of writing a decompiler that works with even one particular compiler and has even a modest success rate at generating code would be significant — on the par with writing the compiler itself from scratch.

But the biggest question is not *how* you can decompile someone's code, but *why* do you want to do this? If you're trying to reverse-engineer someone else's code, shame on you; go find honest work. If you're trying to recover from losing your own source, the best suggestion I have is to make better backups next time.

(Don't bother writing me email saying there are legitimate reasons for decompiling; I didn't say there weren't.)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.5] Where can I get information about the C++ compiler from {Borland, IBM, Microsoft, Sun, etc.}?

In alphabetical order by vendor name:

- Borland C++ 5.0 FAQs: www.borland.com/cbuilder/index.html
- Comeau C++: www.comeaucomputing.com/
- Digital Mars (free) C++: www.digitalmars.com/
- DJ C++ ("DJGPP"): www.delorie.com/
- Edison Design Group C++: www.edg.com/cpp.html
- GNU C++ ("g++" or "GCC"): gcc.gnu.org/. Note: there are two versions precompiled for Win32: [Cygwin](#) and [minGW](#).
- HP C++: www.tru64unix.compaq.com/linux/compaq_cxx/index.html
- IBM VisualAge C++: www.ibm.com/software/ad/vacpp/
- Intel Reference C++: developer.intel.com/software/products/compilers/
- KAI C++: developer.intel.com/software/products/kcc/
- Metrowerks C++: metrowerks.com or www.metrowerks.com
- Microsoft Visual C++: www.microsoft.com/visualc/
- Open Watcom C++ (an open-source follow-up to Watcom C++): www.openwatcom.org/
- Portland Group C++: www.pgroup.com
- Silicon Graphics C++: www.sgi.com/developers/devtools/languages/c++.html
- Watcom C++: www.sybase.com/products/archivedproducts/watcomc

[If anyone has other suggestions that should go into this list, please let me know; thanks; (cline@parashift.com)].

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.6] What's the difference between C++ and Visual C++?

C++ is the language itself, [Visual C++](#) is a compiler that tries to implement the language.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.7] How do compilers use "over-allocation" to remember the number of elements in an allocated array?

Recall that when you `delete[]` an array, the runtime system [magically knows how many destructors to run](#). This FAQ describes a technique used by some C++ compilers to do this (the other common technique is to [use an associative array](#)).

If the compiler uses the "over-allocation" technique, the code for `p = new Fred[n]` looks something like the following. Note that `WORDSIZE` is an imaginary machine-dependent constant that is at least `sizeof(size_t)`, possibly rounded up for any alignment constraints. On many machines, this constant will have a value of 4 or 8. It is not a real C++ identifier that will be defined for your compiler.

```
// Original code: Fred* p = new Fred[n];
char* tmp = (char*) operator new[] (WORDSIZE + n * sizeof(Fred));
Fred* p = (Fred*) (tmp + WORDSIZE);
*(size_t*)tmp = n;
size_t i;
try {
    for (i = 0; i < n; ++i)
        new(p + i) Fred();                // Placement new
} catch (...) {
    while (i-- != 0)
        (p + i)->~Fred();                // Explicit call to the destructor
    operator delete[] ((char*)p - WORDSIZE);
    throw;
}
```

Then the `delete[] p` statement becomes:

```
// Original code: delete[] p;
size_t n = * (size_t*) ((char*)p - WORDSIZE);
while (n-- != 0)
    (p + n)->~Fred();
operator delete[] ((char*)p - WORDSIZE);
```

Note that the address passed to `operator delete[]` is *not* the same as `p`.

Compared to the [associative array technique](#), this technique is faster, but more sensitive to the problem of programmers saying `delete p` rather than `delete[] p`. For example, if you make a programming error by saying `delete p` where you should have said `delete[] p`, the address that is passed to `operator delete(void*)` is *not* the address of any valid heap allocation. This will probably corrupt the heap. Bang! You're dead!

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.8] How do compilers use an "associative array" to remember the number of elements in an allocated array?

Recall that when you `delete[]` an array, the runtime system [magically knows how many destructors to run](#). This FAQ describes a technique used by some C++ compilers to do this (the other common technique is to [over-allocate](#)).

If the compiler uses the associative array technique, the code for `p = new Fred[n]` looks something like this (where `arrayLengthAssociation` is the imaginary name of a hidden, global associative array that maps from `void*` to `"size_t"`):

```
// Original code: Fred* p = new Fred[n];
Fred* p = (Fred*) operator new[] (n * sizeof(Fred));
size_t i;
try {
    for (i = 0; i < n; ++i)
        new(p + i) Fred();           // Placement new
} catch (...) {
    while (i-- != 0)
        (p + i)->~Fred();           // Explicit call to the destructor
    operator delete[] (p);
    throw;
}
arrayLengthAssociation.insert(p, n);
```

Then the `delete[] p` statement becomes:

```
// Original code: delete[] p;
size_t n = arrayLengthAssociation.lookup(p);
while (n-- != 0)
    (p + n)->~Fred();
operator delete[] (p);
```

Cfront uses this technique (it uses an AVL tree to implement the associative array).

Compared to the [over-allocation technique](#), the associative array technique is slower, but less sensitive to the problem of programmers saying `delete p` rather than `delete[] p`. For example, if you make a programming error by saying `delete p` where you should have said `delete[] p`, only the first `Fred` in the array gets destructed, but the heap *may* survive (unless you've replaced `operator delete[]` with something that doesn't simply call `operator delete`, or unless the destructors for the other `Fred` objects were necessary).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.9] If name mangling was standardized, could I link code compiled with compilers from different compiler vendors?

Short answer: Probably not.

In other words, some people would like to see name mangling standards incorporated into the proposed C++ ANSI standards in an attempt to avoiding having to purchase different versions of class libraries for different compiler vendors. However name mangling differences are one of the smallest differences between implementations, even on the same platform.

Here is a partial list of other differences:

- Number and type of hidden arguments to member functions.
 - is this handled specially?

- o where is the return-by-value pointer passed?
- Assuming a [v-table](#) is used:
 - o what is its contents and layout?
 - o where/how is the adjustment to this made for multiple and/or virtual inheritance?
- How are classes laid out, including:
 - o location of base classes?
 - o handling of virtual base classes?
 - o location of [v-pointers](#), if they are used at all?
- Calling convention for functions, including:
 - o where are the actual parameters placed?
 - o in what order are the actual parameters passed?
 - o how are registers saved?
 - o where does the return value go?
 - o does caller or callee pop the stack after the call?
 - o special rules for passing or returning structs or doubles?
 - o special rules for saving registers when calling leaf functions?
- How is the run-time-type-identification laid out?
- How does the runtime exception handling system know which local objects need to be destructed during an exception throw?

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.10] GNU C++ (g++) produces big executables for tiny programs; Why?

libg++ (the library used by g++) was probably compiled with debug info (`-g`). On some machines, recompiling libg++ without debugging can save lots of disk space (approximately 1 MB; the down-side: you'll be unable to trace into libg++ calls). Merely strip-ping the executable doesn't reclaim as much as recompiling without `-g` followed by subsequent strip-ping the resultant `a.out's`.

Use `size a.out` to see how big the program code and data segments really are, rather than `ls -ls a.out` which includes the symbol table.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.11] Is there a yacc-able C++ grammar?

The primary yacc grammar you'll want is from Ed Willink. Ed believes his grammar is fully compliant with [the ISO/ANSI C++ standard](#), however he doesn't warrant it: "the grammar has not," he says, "been used in anger." You can get [the grammar without action routines](#) or [the grammar with dummy action routines](#). You can also get [the corresponding lexer](#). For those who are interested in how he achieves a context-free parser (by pushing all the ambiguities plus a

small number of repairs to be done later after parsing is complete), you might want to read chapter 4 of [his thesis](#).

There is also a very old yacc grammar that doesn't support templates, exceptions, nor namespaces; plus it deviates from the core language in some subtle ways. You can get that grammar [here](#) or [here](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.12] What is C++ 1.2? 2.0? 2.1? 3.0?

These are not versions of the language, but rather versions of Cfront, which was the original C++ translator implemented by AT&T. It has become generally accepted to use these version numbers as if they were versions of the language itself.

Very roughly speaking, these are the major features:

- 2.0 includes multiple/virtual inheritance and [pure virtual](#) functions
- 2.1 includes semi-nested classes and `delete[] pointerToArray`
- 3.0 includes fully-nested classes, templates and `i++` vs. `++i`
- 4.0 will include exceptions

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[38.13] Is it possible to convert C++ to C?

Depends on what you mean. If you mean, *Is it possible to convert C++ to readable and maintainable C-code?* then sorry, the answer is No — C++ features don't directly map to C, plus the generated C code is not intended for humans to follow. If instead you mean, *Are there compilers which convert C++ to C for the purpose of compiling onto a platform that yet doesn't have a C++ compiler?* then you're in luck — keep reading.

A compiler which compiles C++ to C does full syntax and semantic checking on the program, and just happens to use C code as a way of generating object code. Such a compiler is not merely some kind of fancy macro processor. (And please don't email me claiming these are preprocessors — they are not — they are full compilers.) It is possible to implement all of the features of ISO Standard C++ by translation to C, and except for exception handling, it typically results in object code with efficiency comparable to that of the code generated by a conventional C++ compiler.

Here are some products that perform compilation to C (note: if you know of any other products that do this, please let me know (cline@parashift.com)):

- [Comeau Computing](#) offers a compiler based on [Edison Design Group's front end](#) that outputs C code.
- [LLVM](#) is a downloadable compiler that emits C code. See also [here](#).

- [Cfront](#), the original implementation of C++, done by Bjarne Stroustrup and others at AT&T, generates C code. However it has two problems: it's been difficult to obtain a license since the mid 90s when it started going through a maze of ownership changes, and development ceased at that same time and so it doesn't get bug fixes and doesn't support any of the newer language features (e.g., exceptions, namespaces, RTTI, member templates).
- Contrary to popular myth, as of this writing there is no version of g++ that translates C++ to C. Such a thing seems to be doable, but I am not aware that anyone has actually done it (yet).

Note that you typically need to specify the target platform's CPU, OS and C compiler so that the generated C code will be specifically targeted for this platform. This means: (a) you probably can't take the C code generated for platform X and compile it on platform Y; and (b) it'll be difficult to do the translation yourself — it'll probably be a lot cheaper/safer with one of these tools.

One more time: do *not* email me saying these are just preprocessors — they are not — they are compilers.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[39] Miscellaneous technical issues

UPDATED!

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),
cline@parashift.com)

FAQs in section [39]:

- [\[39.1\] How do I convert a value \(a number, for example\) to a std::string?](#)
- [\[39.2\] How do I convert a std::string to a number?](#)
- [\[39.3\] Can I templatize the above functions so they work with other types?](#)
- [\[39.4\] What should be done with macros that contain if?](#) **UPDATED!**
- [\[39.5\] What should be done with macros that have multiple lines?](#)
- [\[39.6\] What should be done with macros that need to paste two tokens together?](#)
- [\[39.7\] Why can't the compiler find my header file in #include "c:\test.hpp" ?](#)
- [\[39.8\] What are the C++ scoping rules for for loops?](#)
- [\[39.9\] Why can't I overload a function by its return type?](#)
- [\[39.10\] What is "persistence"? What is a "persistent object"?](#)

- [\[39.11\] How can I create two classes that both know about each other?](#)
- [\[39.12\] What special considerations are needed when forward declarations are used with member objects?](#)
- [\[39.13\] What special considerations are needed when forward declarations are used with inline functions?](#)
- [\[39.14\] Why can't I put a forward-declared class in a `std::vector<>`?](#)
- [\[39.15\] Why do some people think `x = ++y + y++` is bad?](#)
- [\[39.16\] What's the deal with "sequence points"?](#)

[39.1] How do I convert a value (a number, for example) to a `std::string`?

There are two easy ways to do this: you can use the `<cstdio>` facilities or the `<iostream>` library. In general, [you should prefer the `<iostream>` library](#).

The `<iostream>` library allows you to convert pretty much anything to a `std::string` using the following syntax (the example converts a `double`, but you could substitute pretty much anything that prints using the `<<` operator):

```
// File: convert.h
#include <iostream>
#include <sstream>
#include <string>
#include <stdexcept>

class BadConversion : public std::runtime_error {
public:
    BadConversion(const std::string& s)
        : std::runtime_error(s)
    { }
};

inline std::string stringify(double x)
{
    std::ostringstream o;
    if (!(o << x))
        throw BadConversion("stringify(double)");
    return o.str();
}
```

The `std::ostringstream` object `o` offers formatting facilities just like those for `std::cout`. You can use manipulators and format flags to control the formatting of the result, just as you can for other `std::cout`.

In this example, we *insert* `x` into `o` via the overloaded insertion operator, `<<`. This invokes the `iostream` formatting facilities to convert `x` into a `std::string`. [The `if` test](#) makes sure the conversion works correctly — it should always succeed for built-in/intrinsic types, but the `if` test is good style.

The expression `o.str()` returns the `std::string` that contains whatever has been inserted into stream `o`, in this case the string value of `x`.

Here's how to use the `stringify()` function:

```
#include "convert.h"

void myCode()
{
    double x = ...;
    ...
    std::string s = "the value is " + stringify(x);
    ...
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.2] How do I convert a `std::string` to a number?

There are two easy ways to do this: you can use the `<cstdio>` facilities or the `<iostream>` library. In general, [you should prefer the `<iostream>` library](#).

The `<iostream>` library allows you to convert a `std::string` to pretty much anything using the following syntax (the example converts a `double`, but you could substitute pretty much anything that can be read using the `>>` operator):

```
// File: convert.h
#include <iostream>
#include <sstream>
#include <string>
#include <stdexcept>

class BadConversion : public std::runtime_error {
public:
    BadConversion(const std::string& s)
        : std::runtime_error(s)
    { }
};

inline double convertToDouble(const std::string& s)
{
    std::istringstream i(s);
    double x;
    if (!(i >> x))
        throw BadConversion("convertToDouble(\"" + s + "\")");
    return x;
}
```

The `std::istringstream` object `i` offers formatting facilities just like those for `std::cin`. You can use manipulators and format flags to control the formatting of the result, just as you can for other `std::cin`.

In this example, we initialize the `std::istringstream` `i` passing the `std::string` `s` (for example, `s` might be the string `"123.456"`), then we *extract* `i` into `x` via the overloaded extraction operator, `>>`. This invokes the `iostream` formatting facilities to convert as much of the string as possible/appropriate based on the type of `x`.

[The if test](#) makes sure the conversion works correctly. For example, if the string contains characters that are inappropriate for the type of `x`, the `if` test will fail.

Here's how to use the `convertToDouble()` function:

```
#include "convert.h"

void myCode()
{
    std::string s = ...a string representation of a number...;
    ...
    double x = convertToDouble(s);
    ...
}
```

You probably want to enhance `convertToDouble()` so it optionally checks that there aren't any left-over characters:

```
inline double convertToDouble(const std::string& s,
                              bool failIfLeftoverChars = true)
{
    std::istringstream i(s);
    double x;
    char c;
    if (!(i >> x) || (failIfLeftoverChars && i.get(c)))
        throw BadConversion("convertToDouble(\"" + s + "\")");
    return x;
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.3] Can I templatize the above functions so they work with other types?

Yes — for any types that support `iostream`-style input/output.

For example, suppose you want to convert an object of class `Foo` to a `std::string`, or perhaps the reverse: from a `std::string` to a `Foo`. You could write a whole family of conversion functions based on the ones shown in the previous FAQs, or you could write a template function so the compiler does the grunt work.

For example, to convert an arbitrary type `T` to a `std::string`, provided `T` supports syntax like `std::cout << x`, you can use this:

```
// File: convert.h
#include <iostream>
#include <sstream>
#include <string>
#include <typeinfo>
#include <stdexcept>

class BadConversion : public std::runtime_error {
public:
    BadConversion(const std::string& s)
        : std::runtime_error(s)
}
```

```

    { }
};

template<typename T>
inline std::string stringify(const T& x)
{
    std::ostringstream o;
    if (!(o << x))
        throw BadConversion(std::string("stringify(")
                               + typeid(x).name() + ")");
    return o.str();
}

```

Here's how to use the `stringify()` function:

```

#include "convert.h"

void myCode()
{
    Foo x;
    ...
    std::string s = "this is a Foo: " + stringify(x);
    ...
}

```

You can also convert *from* any type that supports `istream` input by adding this to file `convert.h`:

```

template<typename T>
inline void convert(const std::string& s, T& x,
                   bool failIfLeftoverChars = true)
{
    std::istringstream i(s);
    char c;
    if (!(i >> x) || (failIfLeftoverChars && i.get(c)))
        throw BadConversion(s);
}

```

Here's how to use the `convert()` function:

```

#include "convert.h"

void myCode()
{
    std::string s = ...a string representation of a Foo...;
    ...
    Foo x;
    convert(s, x);
    ...
    ...code that uses x...
}

```

To simplify your code, particularly for light-weight easy-to-copy types, you probably want to add a return-by-value conversion function to file `convert.h`:

```

template<typename T>
inline T convertTo(const std::string& s,
                  bool failIfLeftoverChars = true)
{
    T x;

```

```
convert(s, x, failIfLeftoverChars);
return x;
}
```

This simplifies your "usage" code some. You call it by explicitly specifying the template parameter `T`:

```
#include "convert.h"

void myCode()
{
    std::string a = ...string representation of an int...;
    std::string b = ...string representation of an int...;
    ...
    if (convertTo<int>(a) < convertTo<int>(b))
        ...;
}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.4] What should be done with macros that contain `if`? UPDATED!

[Recently clarified why we use the `(void)0` part thanks to a suggestion from [Paddu/Padmanabhan V. Karthic](#) (in 2/05). [Click here to go to the next FAQ in the "chain" of recent changes.](#)]

Ideally you'll get rid of the macro. Macros are evil in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#), regardless of whether they contain an `if` (but they're especially evil if they contain an `if`).

Nonetheless, even though macros are evil, [sometimes they are the lesser of the other evils](#). When that happens, read this FAQ so you know how to make them "less bad," then hold your nose and do what's practical.

Here's a naive solution:

```
#define MYMACRO(a,b) \
    if (xyzzzy) asdf()           (Bad)
```

This will cause big problems if someone uses that macro in an `if` statement:

```
if (whatever)
    MYMACRO(foo,bar);
else
    baz;
```

The problem is that the `else baz` nests with the wrong `if`: the compiler sees this:

```
if (whatever)
    if (xyzzzy) asdf();
else baz;
```

Obviously that's a bug.

The easy solution is to require `{...}` everywhere, but there's another solution that I prefer even if there's a coding standard that requires `{...}` everywhere (just in case someone somewhere forgets): add a balancing `else` to the macro definition:

```
#define MYMACRO(a,b) \                               (Good)
    if (xyzzzy) asdf(); \
    else (void)0
```

(The `(void)0` causes the compiler to [generate an error message](#) if you forget to put the `;` after the 'call'.)

Your usage of that macro might look like this:

```
if (whatever)
    MYMACRO(foo, bar);
else          ^—this ; closes off the else (void)0 part
    baz;
```

which will get expanded into a balanced set of `ifs` and `elses`:

```
if (whatever)
    if (xyzzzy)
        asdf();
    else
        (void)0;
else    ^^^^^^^^—that's a do-nothing statement
    baz;
```

Like I said, I personally do the above even when the coding standard calls for `{...}` in all the `ifs`. Call me paranoid, but I sleep better at night and my code has fewer bugs.

There is another approach that old-line C programmers will remember:

```
#define MYMACRO(a,b) \                               (Okay)
    do { \
        if (xyzzzy) asdf(); \
    } while (false)
```

Some people prefer the `do {...} while (false)` approach, though if you choose to use that, be aware that it might cause your compiler to generate [less efficient code](#). Both approaches cause the compiler to give you an error message if you forget the `;` after `MYMACRO(foo, bar)`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.5] What should be done with macros that have multiple lines?

Answer: Choke, gag, cough. Macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#). Kill them *all!!* (Just [kidding](#).)

Seriously, sometimes you need to use them [anyway](#), and when you do, read this to learn some safe ways to write a macro that has multiple statements.

Here's a naive solution:

```
#define MYMACRO(a,b) \                                (Bad)
    statement1; \
    statement2; \
    ... \
    statementN;
```

This can cause problems if someone uses the macro in a context that demands a single statement. E.g.,

```
while (whatever)
    MYMACRO(foo, bar);
```

The naive solution is to wrap the statements inside {...}, such as this:

```
#define MYMACRO(a,b) \                                (Bad)
    { \
        statement1; \
        statement2; \
        ... \
        statementN; \
    }
```

But this will cause compile-time errors with things like the following:

```
if (whatever)
    MYMACRO(foo, bar);
else
    baz;
```

since the compiler will see a } ; else which is illegal:

```
if (whatever)
{
    statement1;
    statement2;
    ...
    statementN;
}; //ERROR: can't have }; before else
else
    baz;
```

One solution is to use a do { <statements go here> } while (false) pseudo-loop. This executes the body of the "loop" exactly once. The macro might look like this:

```
#define MYMACRO(a, b) \                                (Okay)
    do { \
        statement1; \
        statement2; \
        ... \
        statementN; \
    } while (false)
```

Note that there is no ; at the end of the macro definition. The ; gets added by the user of the macro, such as the following:

```

if (whatever)
    MYMACRO(foo, bar); // The ; is added here
else
    baz;

```

This will expand to the following (note that the `;` added by the user goes after (and completes) the `} while (false)` part):

```

if (whatever)
    do {
        statement1;
        statement2;
        ...
        statementN;
    } while (false);
else
    baz;

```

This is an acceptable approach, however some C++ compilers refuse to inline -expand any function that contains a loop, and since this *looks* like a loop, those compilers won't inline -expand any function that contains it. Do some timing tests with your compiler to see; it might be a non-issue to you (besides, you might not be using this within an inline function anyway).

A possibly better solution is to use `if (true) { <statements go here> } else (void)0`

```

#define MYMACRO(a, b) \
    if (true) { \
        statement1; \
        statement2; \
        ... \
        statementN; \
    } else \
        (void)0

```

(Best)

The preprocessor will expand the usage into the following (note the balanced set of `ifs` and `elses`):

```

if (whatever)
    if (true) {
        statement1;
        statement2;
        ...
        statementN;
    } else
        (void)0;
else
    ^^^^^^^^—that's a do-nothing statement
    baz;

```

The purpose of the strange `(void)0` at the end is to make sure you remember to add the `;` just after any usage of the macro. For example, if you forgot the `;` like this...

```

foo();
MYMACRO(a, b)    ← bad news: we forgot the ;
bar();
baz();

```


...then the preprocessor would produce this...

```
foo();
if (true) {
    statement1; \
    statement2; \
    ... \
    statementN; \
} else
    (void)0 bar();    ← fortunately(!) you'll get a compile-time error-message here
baz();
```

...which would result in a compile-time error-message. That error-message is *lot* better than the alternative: without the `(void)0`, the compiler would silently generate the wrong code, since the `bar()` call would erroneously be on the `else` branch of the `if`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.6] What should be done with macros that need to paste two tokens together?

Groan. I really *hate* macros. Yes [they're useful sometimes](#), and yes I use them. But I always wash my hands afterwards. Twice. Macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#).

Okay, here we go again, desperately trying to make an inherently evil thing a little [less evil](#).

First, the basic approach is use the ISO/ANSI C and ISO/ANSI C++ "token pasting" feature: `##`. On the surface this would look like the following:

Suppose you have a macro called "MYMACRO", and suppose you're passing a token as the parameter of that macro, and suppose you want to concatenate that token with the token "Tmp" to create a variable name. For example, the use of `MYMACRO(Foo)` would create a variable named `FooTmp` and the use of `MYMACRO(Bar)` would create a variable named `BarTmp`. In this case the naive approach would be to say this:

```
#define MYMACRO(a) \
    /*...*/ a ## Tmp /*...*/
```

However you need a double layer of indirection when you use `##`. Basically you need to create a special macro for "token pasting" such as:

```
#define name2(a,b)      name2_hidden(a,b)
#define name2_hidden(a,b) a ## b
```

Trust me on this — you really need to do this! (And *please* nobody write me saying it sometimes works without the second layer of indirection. Try concatenating a symbol with `__LINE__` and see what happens then.)

Then replace your use of `a ## Tmp` with `name2(a, Tmp)`:

```
#define MYMACRO(a) \
    /*...*/ name2(a,Tmp) /*...*/
```

And if you have a three-way concatenation to do (e.g., to paste three tokens together), you'd create a `name3()` macro like this:

```
#define name3(a,b,c)      name3_hidden(a,b,c)
#define name3_hidden(a,b,c) a ## b ## c
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.7] Why can't the compiler find my header file in `#include "c:\test.hpp"` ?

Because `"\t"` is a tab character.

You should use forward slashes ("/") rather than backslashes ("\") in your `#include` filenames, even on operating systems that use backslashes such as DOS, Windows, OS/2, etc. For example:

```
#if 1
    #include "/version/next/alpha/beta/test.hpp"    // RIGHT!
#else
    #include "\\version\\next\\alpha\\beta\\test.hpp" // WRONG!
#endif
```

Note that [you should use forward slashes \("/"\) on all your filenames](#), not just on your `#include` files.

Note that your particular compiler might not treat a backslash within a header -name the same as it treats a backslash within a string literal. For instance, your particular compiler might treat `#include "foo\bar\baz"` as if the `'\'` chars were quoted. This is because header names and string literals are different: your compiler will always parse backslashes in string literals in the usual way, with `'\t'` becoming a tab character, etc., but it might not parse header names using those same rules. In any case, you still shouldn't use backslashes in your header names since there's something to lose but nothing to gain.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.8] What are the C++ scoping rules for `for` loops?

Loop variables declared in the `for` statement proper are local to the loop body.

The following code used to be legal, but not any more, since `i`'s scope is now inside the `for` loop only:

```
for (int i = 0; i < 10; ++i) {
    ...
    if ( /* something weird */ )
        break;
```

```
...
}

if (i != 10) {
    // We exited the loop early; handle this situation separately
    ...
}
```

If you're working with some old code that uses a `for` loop variable after the `for` loop, the compiler will (hopefully!) give you a warning or an error message such as "Variable `i` is not in scope".

Unfortunately there are cases when old code will compile cleanly, but will do something different — the wrong thing. For example, if the old code has a global variable `i`, the above code `if (i != 10)` silently change in meaning from the `for` loop variable `i` under the old rule to the global variable `i` under the current rule. This is not good. If you're concerned, you should check with your compiler to see if it has some option that forces it to use the old rules with your old code.

Note: You should avoid having the same variable name in nested scopes, such as a global `i` and a local `i`. In fact, you should avoid globals altogether whenever you can. If you abided by these coding standards in your old code, you won't be hurt by a lot of things, including the scoping rules for `for` loop variables.

Note: If your new code might get compiled with an old compiler, you might want to put `{...}` around the `for` loop to force even old compilers to scope the loop variable to the loop. And *please try to* avoid the temptation to use macros for this. Remember: macros are [evil](#) in 4 different ways: [evil#1](#), [evil#2](#), [evil#3](#), and [evil#4](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.9] Why can't I overload a function by its return type?

If you declare both `char f()` and `float f()`, the compiler gives you an error message, since calling simply `f()` would be ambiguous.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.10] What is "persistence"? What is a "persistent object"?

A persistent object can live after the program which created it has stopped. Persistent objects can even outlive different versions of the creating program, can outlive the disk system, the operating system, or even the hardware on which the OS was running when they were created.

The challenge with persistent objects is to effectively store their member function code out on secondary storage along with their data bits (and the data bits and member function code of all member objects, and of all their member objects and base classes, etc). This is non-trivial when you have to do it yourself. In C++, you have to do it yourself. C++/OO databases can help hide the mechanism for all this.

[39.11] How can I create two classes that both know about each other?

Use a forward declaration.

Sometimes you must create two classes that use each other. This is called a circular dependency. For example:

```
class Fred {
public:
    Barney* foo(); // Error: Unknown symbol 'Barney'
};

class Barney {
public:
    Fred* bar();
};
```

The Fred class has a member function that returns a Barney*, and the Barney class has a member function that returns a Fred. You may inform the compiler about the existence of a class or structure by using a "forward declaration":

```
class Barney;
```

This line must appear *before* the declaration of class Fred. It simply informs the compiler that the name Barney is a class, and further it is a promise to the compiler that you will eventually supply a complete definition of that class.

[39.12] What special considerations are needed when forward declarations are used with member objects?

The order of class declarations is critical.

The compiler will give you a compile-time error if the first class contains an object (as opposed to a pointer to an object) of the second class. For example,

```
class Fred; // Okay: forward declaration

class Barney {
    Fred x; // Error: The declaration of Fred is incomplete
};

class Fred {
    Barney* y;
};
```

One way to solve this problem is to reverse order of the classes so the "used" class is defined before the class that uses it:

```
class Barney; // Okay: forward declaration

class Fred {
    Barney* y; // Okay: the first can point to an object of the second
};

class Barney {
    Fred x; // Okay: the second can have an object of the first
};
```

Note that it is never legal for each class to fully contain an object of the other class since that would imply infinitely large objects. In other words, if an instance of `Fred` contains a `Barney` (as opposed to a `Barney*`), and a `Barney` contains a `Fred` (as opposed to a `Fred*`), the compiler will give you an error.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.13] What special considerations are needed when forward declarations are used with inline functions?

The order of class declarations is critical.

The compiler will give you a compile-time error if the first class contains an inline function that invokes a member function of the second class. For example,

```
class Fred; // Okay: forward declaration

class Barney {
public:
    void method()
    {
        x->yabbaDabbaDo(); // Error: Fred used before it was defined
    }
private:
    Fred* x; // Okay: the first can point to an object of the second
};

class Fred {
public:
    void yabbaDabbaDo();
private:
    Barney* y;
};
```

One way to solve this problem is to move the offending member function into the `Barney.cpp` file as a non-inline member function. Another way to solve this problem is to reverse order of the classes so the "used" class is defined before the class that uses it:

```
class Barney; // Okay: forward declaration
```

```

class Fred {
public:
    void yabbaDabbaDo();
private:
    Barney* y;    // Okay: the first can point to an object of the second
};

class Barney {
public:
    void method()
    {
        x->yabbaDabbaDo();    // Okay: Fred is fully defined at this point
    }
private:
    Fred* x;
};

```

Just remember this: Whenever you use [forward declaration](#), you can use only that symbol; you may not do anything that requires knowledge of the forward-declared class. Specifically you may not access any members of the second class.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.14] Why can't I put a forward-declared class in a `std::vector<>`?

Because the `std::vector<>` template needs to know the `sizeof()` its contained elements, plus the `std::vector<>` probably accesses members of the contained elements (such as the copy constructor, the destructor, etc.). For example,

```

class Fred;    // Okay: forward declaration

class Barney {
    std::vector<Fred> x;    // Error: the declaration of Fred is incomplete
};

class Fred {
    Barney* y;
};

```

One solution to this problem is to change Barney so it uses a `std::vector<>` of Fred *pointers* rather than a `std::vector<>` of Fred *objects*:

```

class Fred;    // Okay: forward declaration

class Barney {
    std::vector<Fred*> x;    // Okay: Barney can use Fred pointers
};

class Fred {
    Barney* y;
};

```

Another solution to this problem is to reverse the order of the classes so Fred is defined before Barney:

```
class Barney; // Okay: forward declaration

class Fred {
    Barney* y; // Okay: the first can point to an object of the second
};

class Barney {
    std::vector<Fred> x; // Okay: Fred is fully defined at this point
};
```

Just remember this: Whenever you use a class as a template parameter, the declaration of that class must be complete and not simply [forward declared](#).

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.15] Why do some people think `x = ++y + y++` is bad?

Because it's undefined behavior, which means the runtime system is allowed to do weird or even bizarre things.

The C++ language says you cannot modify a variable more than once between [sequence points](#). Quoth [the standard](#) (section 5, paragraph 4):

Between the previous and next sequence point a scalar object shall have its stored value modified at most once by the evaluation of an expression. Furthermore, the prior value shall be accessed only to determine the value to be stored.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]

[39.16] What's the deal with "sequence points"?

The C++ standard says (1.9p7),

At certain specified points in the execution sequence called *sequence points*, all side effects of previous evaluations shall be complete and no side effects of subsequent evaluations shall have taken place.

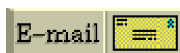
For example, if an expression contains the subexpression `y++`, then the variable `y` will be incremented by the next sequence point. Furthermore if the expression just after the sequence point contains the subexpression `++z`, then `z` will not have yet been incremented at the moment the sequence point is reached.

The "certain specified points" that are called sequence points are (section and paragraph numbers are from [the standard](#)):

- the semicolon (1.9p16)
- the non-overloaded comma-operator (1.9p18)
- the non-overloaded `||` operator (1.9p18)

- the non-overloaded && operator (1.9p18)
- the ternary ?: operator (1.9p18)
- after evaluation of all a function's parameters but before the first expression within the function is executed (1.9p17)
- after a function's returned object has been copied back to the caller, but before the code just after the call has yet been evaluated (1.9p17)
- after the initialization of each base and member (12.6.2p3)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Next section](#) | [Search the FAQ](#)]



[E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)
| [Download your own copy](#)]

Revised Feb 15, 2005

[40] Miscellaneous environmental issues

UPDATED!

(Part of [C++ FAQ Lite](#), [Copyright © 1991-2004](#), [Marshall Cline](#),
cline@parashift.com)

FAQs in section [40]:

- [\[40.1\] How can I generate HTML documentation for my classes? Does C++ have anything similar to javadoc?](#) **UPDATED!**
- [\[40.2\] Is there a TeX or LaTeX macro that fixes the spacing on "C++"?](#)
- [\[40.3\] Are there any pretty-printers that reformat C++ source code?](#)
- [\[40.4\] Is there a C++-mode for GNU emacs? If so, where can I get it?](#)
- [\[40.5\] Where can I get OS-specific questions answered \(e.g., BC++, Windows, etc\)?](#)
- [\[40.6\] Why does my DOS C++ program says "Sorry : floating point code not linked"?](#)
- [\[40.7\] Why does my BC++ Windows app crash when I'm not running the BC45 IDE?](#)

[40.1] How can I generate HTML documentation for my classes? Does C++ have anything similar to javadoc? **UPDATED!**

[Recently updated the link to www.robertnz.net/cpp_site.html#Documentation thanks to [Robert Davies](#) and [Peter Kammer](#) (in 2/05). [Click here to go back to the beginning of the "chain" of recent changes.](#)]

Yes. Here are a few (listed alphabetically by tool name):

- [ccdoc](#) supports javadoc-like syntax with various extensions. It's freely copiable and customizable.



- [doc++](#) generates HTML or TeX. Supports javadoc-like syntax with various extensions. Open Source.
- [doxygen](#) generates HTML, LaTeX or RTF. Supports javadoc-like syntax with various extensions. Open Source.
- [PERCEPS](#) generates HTML, TeX, RTF, man page, plain text, and anything else you'd like (it lets you set up arbitrary output formats). It's freely copiable.

Other documentation tools are listed at www.robertnz.net/cpp_site.html#Documentation.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Search the FAQ](#)]

[40.2] Is there a TeX or LaTeX macro that fixes the spacing on "C++"?

Yes.

Here are two LaTeX macros for the word "C++". They prevent line breaks between the "C" and "++", and the first packs the two "+"s close to each other but the second does not. Try them both and see which one you like best.

```
\newcommand{\CC}{C\nolinebreak\hspace{-
.05em}\raisebox{.4ex}{\tiny\bf ++}\nolinebreak\hspace{-
.10em}\raisebox{.4ex}{\tiny\bf ++}}

\def\CC{C\nolinebreak[4]\hspace{-.05em}\raisebox{.4ex}{\tiny\bf ++}}
```

Here are two more LaTeX macros for the word "C++". They allow line breaks between the "C" and "++", which may not be desirable, but they're included here just in case.

```
\def\CC{C\raise.22ex\hbox{\footnotesize ++}\raise.22ex\hbox{\footnotesize ++}}

\def\CC{C\hspace{-.05em}\raisebox{.4ex}{\tiny\bf ++}}
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Search the FAQ](#)]

[40.3] Are there any pretty-printers that reformat C++ source code ?

In alphabetical order:

- A2PS is a Unix-based pretty-printer. It is available from www.infres.enst.fr/~demaille/a2ps/
- Artistic Style is a reindenter and reformatter of C++, C and Java source code. It is available from astyle.sourceforge.net/
- C++2LaTeX is a LaTeX pretty printer. It is available from www.arnoldarts.de/cpp2latex.html
- C-Clearly by V Communications, Inc. is a Windows program that comes with standard formatting templates and also allows you to customize your own. www.mixsoftware.com/product/ccl.htm

- GNU indent program may help. It's available at www.arceneaux.com/indent.html. You can also find an "official" GNU mirror site by looking at www.gnu.org/order/ftp.html or perhaps the original GNU site, prep.ai.mit.edu/pub/gnu/ (e.g., if the current version is 1.9.1 you could use prep.ai.mit.edu/pub/gnu/indent-1.9.1.tar.gz).
- "HPS Beauty" is reported to be a Windows 95/98/NT4/NT2000 utility that beautifies C/C++ source code based on rules. The interface is entirely GUI, but HPS Beauty may also be run from the command line. It supports style files, which allow you to save and restore groups of settings. HPS Beauty also offers an optional visual results window, that shows both the before and the after file. Optional HTML output allows you to view source code with syntax highlighting in your browser. www.highplains.net.
- "Source Styler for C++" has lots of bells and whistles. It is a commercial product with a free 15-day trial period. It seems to offer control over tons of different features. www.sourcestyler.com/.
- tgrind is a Unix based pretty printer. It usually comes with the public distribution of TeX and LaTeX in the directory "...tex82/contrib/van/tgrind". A more up-to-date version of tgrind by Jerry Leichter can be found on: venus.ycc.yale.edu/pub in [.TGRIND]. *[Note: If anyone has an updated URL for tgrind, please let me know (cline@parashift.com).]*

Finally, you might consider lgrind which is another C++ to LaTeX translator ([check for the closest mirror site of the ctan archive](#)). The following is a grind definition for C++ (but this one doesn't recognize some new keywords such as `bool` or `wchar_t`, and it doesn't recognize a file ending with `.cpp` as C++):

```
C++ | c++ | CC: \
:pb=\p\d? \( :cf:np=\) \d? ; :bb={ :be=} : \
:cb=/* :ce=/* :ab=// :ae=$ :sb=" :se=\e" :lb=' : \
:zb=@ :ze=@ :tb=% :te=% :mb=% \ $ :me=\ $ % :vb=% \ | :ve=\ | % : \
:le=\e' :tl:id=_~\ : \
:kw=asm auto break case cdecl char continue default do double else \
enum extern far float for fortran goto huge if int interrupt long \
near pascal register return short signed sizeof static struct \
switch typedef union unsigned while void \
#define #else #endif #if #ifdef #ifndef #include #undef # define \
endif ifdef ifndef include undef defined #pragma \
class const delete friend inline new operator overload private \
protected public template this virtual:
```

[[Top](#) | [Bottom](#) | [Previous section](#) | [Search the FAQ](#)]

[40.4] Is there a C++-mode for GNU emacs? If so, where can I get it?

Yes, there is a C++-mode for GNU emacs.

The latest and greatest version of C++ -mode (and C-mode) is implemented in the file `cc-mode.el`. It is an extension of Detlef and Clamen's version. A version is included with emacs. Newer version are available from the elisp archives.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Search the FAQ](#)]

[40.5] Where can I get OS-specific questions answered (e.g., BC++, Windows, etc)?

See one of the following:

- MS-DOS issues: comp.os.msdos.programmer
- MS-Windows issues: comp.windows.ms.programmer
- Unix issues: comp.unix.programmer
- Borland C++ issues (e.g., OWL, BC++ compiler bugs, general C++ concepts, windows programming):
 - Using your Web browser: www.cs.rpi.edu/~wiseb/owl-list/
 - To get on the mailing list: send an e-mail message with the word "SUBSCRIBE" in the Subject: line to [\(NOSPAM\)majordomo\(AT\)netlab\(DOT\)cs\(DOT\)rpi\(DOT\)edu](mailto:(NOSPAM)majordomo(AT)netlab(DOT)cs(DOT)rpi(DOT)edu)
 - To get the FAQ: [ftp.netlab.cs.rpi.edu/pub/lists/owl-list-faq/drafts/owl_faq.hlp](ftp://netlab.cs.rpi.edu/pub/lists/owl-list-faq/drafts/owl_faq.hlp)

[[Top](#) | [Bottom](#) | [Previous section](#) | [Search the FAQ](#)]

[40.6] Why does my DOS C++ program says "Sorry: floating point code not linked"?

The compiler attempts to save space in the executable by not including the float -to-string format conversion routines unless they are necessary, but sometimes it guesses wrong, and gives you the above error message. You can fix this by (1) using `<iostream>` instead of `<cstdio>`, or (2) by including the following function somewhere in your compilation (but don't call it!):

```
static void dummyfloat(float *x) { float y; dummyfloat(&y); }
```

See [the FAQ on stream I/O](#) for more reasons to use `<iostream>` VS. `<cstdio>`.

[[Top](#) | [Bottom](#) | [Previous section](#) | [Search the FAQ](#)]

[40.7] Why does my BC++ Windows app crash when I'm not running the BC45 IDE?

If you're using BC++ for a Windows app, and it works OK as long as you have the BC45 IDE running, but when the BC45 IDE is shut down you get an exception during the creation of a window, then add the following line of code to the `InitMainWindow()` member function of your application (`YourApp::InitMainWindow()`):

```
EnableBWCC(TRUE);
```

 [E-mail the author](#)

[[C++ FAQ Lite](#) | [Table of contents](#) | [Subject index](#) | [About the author](#) | [©](#)]

| [Download your own copy](#) |

Revised Feb 15, 2005

